UNIVERSITÀ DI PARMA

IoT Lab

Internet of Things

Prof. Gianluigi Ferrari

Dipartimento di Ingegneria e Architettura – Pal. 2
Internet of Things (IoT) Lab
Università di Parma
gianluigi.ferrari@unipr.it
http://www.tlc.unipr.it/ferrari
http://iotlab. unipr.it/

Tel: 0521 90**6513**

Our Calendar (Spring 2026)

February						
Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
23	24	25	26	27	28	

March						
Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
						1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29
30	31					

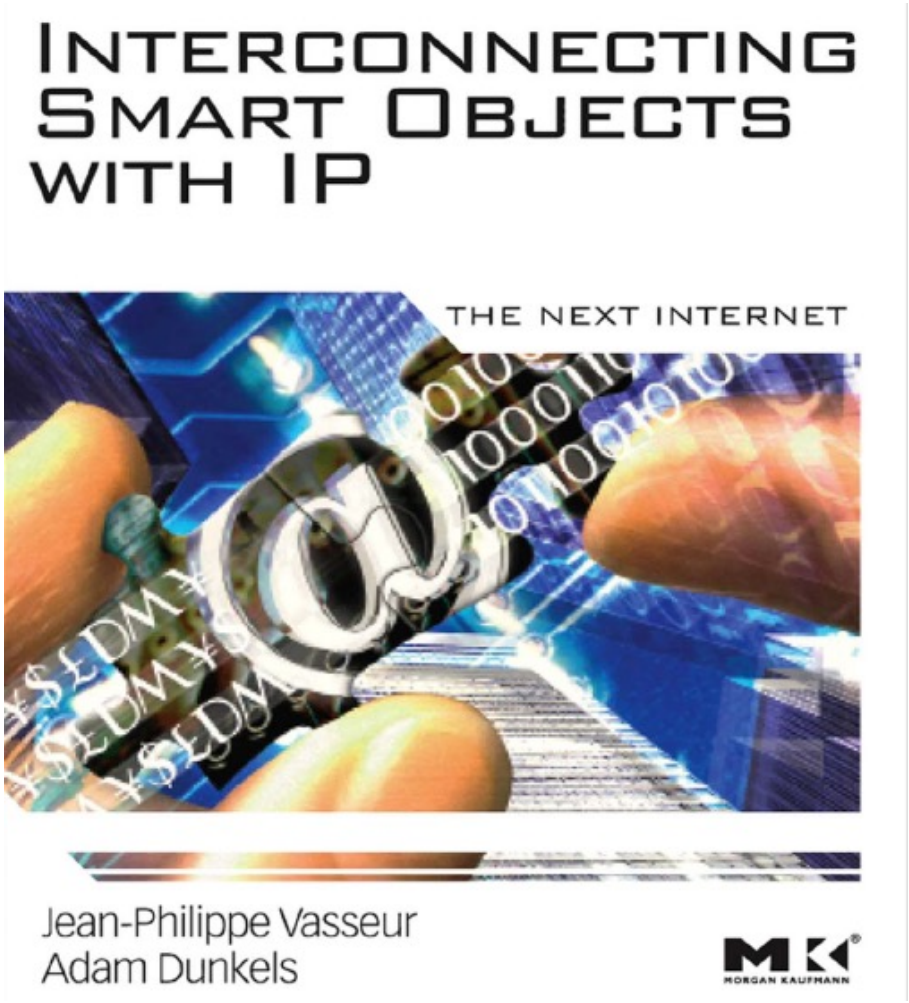
April						
Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30			

May						
Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
				1	2	3
4	5	6 -Midterm	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

June						
Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
1	2	3	4	5		

	theory
	practice
	no lecture
	Midterm (theory)

Our Main Reference



Outline

Theory

- Introduction to IoT
- Transport Layer Protocols: UDP and TCP
- IPv6
- Communication Mechanisms
- Energy-efficient MAC Protocols
- 6LowPAN
- Routing in LLNs & RPL
- Communication paradigms for the IoT
- Operating Systems and Hardware for the IoT
- Constrained Application Protocol (CoAP)
- IoT Security
- LoRa and LoRaWAN

Practice (Seminars, Lab & project)

- CoAP: Introduction to Java, Californium (Cf) Framework **or** Applied project (real devices)
- Lab & project assignment

UNIVERSITÀ DI PARMA

IoT Lab

Introduction to IoT

Main sources

- C. Bormann, “Protocols for the Real “Internet of Things””, keynote speech at the Internet of Things and Smart Cities Ph.D. School 2014 Lerici (SP), Italy - September 8th - 13th, 2014

- J. P. Vasseur and A. Dunkels, `Interconnecting Smart Objects with IP,` Morgan Kaufmann, 2010. Chapter 1.

- M. R. Palattella; N. Accettura; X. Vilajosana; T. Watteyne; L. A. Grieco; G. Boggia; M. Dohler, "Standardized Protocol Stack for the Internet of (Important) Things," *IEEE Communications Surveys & Tutorials*, IEEE , vol.15, no.3, pp.1389,1406, Third Quarter 2013. doi: 10.1109/SURV.2012.111412.00158

Definition and Preliminaries

The Internet of Things (IoT)

- Extending the current Internet and providing connection, communication, and inter-networking between devices and physical objects, or "Things," is a growing trend that is often referred to as the *Internet of Things*.

- “The technologies and solutions that enable integration of real world data and services into the current information networking technologies are often described under the umbrella term of the Internet of Things (IoT)”

Other Definitions (1/2)

(1) The Internet of Things, also called The Internet of Objects, refers to a wireless network between objects, usually the network will be wireless and self-configuring, such as household appliances.

----- Wikipedia

(2) By embedding short-range mobile transceivers into a wide array of additional gadgets and everyday items, enabling new forms of communication between people and things, and between things themselves.

----- WSIS 2005

Other Definitions (2/2)

(3) The term "Internet of Things" has come to describe a number of technologies and research disciplines that enable the Internet to reach out into the real world of physical objects.

----- IoT 2008

(4) “Things having identities and virtual personalities operating in smart spaces using intelligent interfaces to connect and communicate within social, environmental, and user contexts”.

------ IoT in 2020

A brief History of IoT (RFID-oriented)

1997 "The Internet of Things" is the seventh in the series of ITU Internet Reports originally launched in 1997 under the title "Challenges to the Network" → RFID: bringing an object's identity to the Internet

1999 Auto-ID Center founded in MIT

2003 EPC Global founded in MIT

2005 Four important technologies of the internet of things was proposed in WSIS (World Summit on Information Society) conference.

2008 First international conference of internet of things: The IOT 2008 was held at Zurich.

Another History of IoT (Internet-oriented)

Year	Event
1967	REMBASS Remotely Monitored Battlefield Sensor System
1978	Distributed Sensor Networks for Aircraft Detection Lincoln Labs - Lacoss
1992	RAND Workshop - Future Technology Driven Revolutions in Military Conflict. Concepts behind Smart Dust emerge.
1993-1996	DARPA ISAT studies - many WSN ideas and applications discussed. Deborah Estrin leads one of the studies.
1994	LWIM - Low Power Wireless Integrated Microsensors - Bill Kaiser (UCLA)
1997	Smart Dust proposal written, Kris Pister (Berkeley)
1998	Seth Hollar makes wireless mouse collars
1999	Endeavour project proposed by Randy Katz, David Culler (Berkeley) PicoRadio project started by Jan Rabaey (Berkeley)
2000	Crossbow begins selling 'Berkeley motes'
2001	Multiple demos proving viability
2002	Dust, Ember, Millennial, Sensicast founded
2003	IEEE802.15.4-2003 standard Moteiv (now Sentilla) founded
2004	ZigBee 1.0 standard ratified TSMP 1.1 shipping
2005	Arch Rock founded
2006	ZigBee 2006 standard ratified IEEE802.15.4-2006 standard
2007	WirelessHART standard ratified IETF 6LoWPAN's RFC4944 published WirelessHART shown to achieve 99.999% reliability [20]
2008-2009	IETF workgroup Routing Over Low-power Lossy links (ROLL) created. IEEE802.15.4e work group created
2010-2011	IEEE802.15.4e's MAC protocol ratified IEFT 6LoWPAN's RFC4944 updated IETF ROLL's RPL routing protocol ratified

Internet of Things and Smart Cities Ph.D. School (2013,2014)

International speakers from academia and industry gave lectures tailoring their research field for an interdisciplinary audience. A dedicated discussion panel focused on the interaction and the collaboration between academia and industry in order to depict the future vision of Smart Cities and IoT

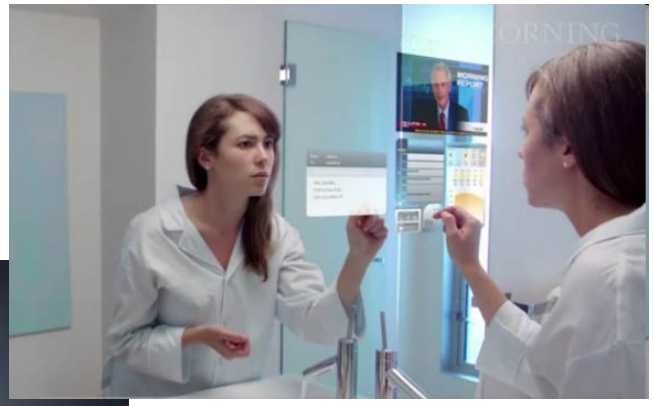
Some numbers:
36 students from 11 countries in Europe
17 speakers from Universities and Industry

2018 This Is What Happens In An Internet Minute
facebook
973,000 Logins
18 Million Text Messages
YouTube
4.3 Million Videos Viewed
375,000 Apps Downloaded
174,000 Scrolling Instagram
481,000 Tweets Sent
1.1 Million Swipes
tinder
187 Million Emails Sent
936,073 Views
twitch
67 Voice-First Devices Shipped
amazon echo
38 Million Messages
25,000 GIFs Sent via Messenger
2.4 Million Snaps Created
$862,823 Spent Online
NETFLIX
266,000 Hours Watched
3.7 Million Search Queries
Google
60 SECONDS
Created By:
@LoriLewis
@OfficiallyChadd

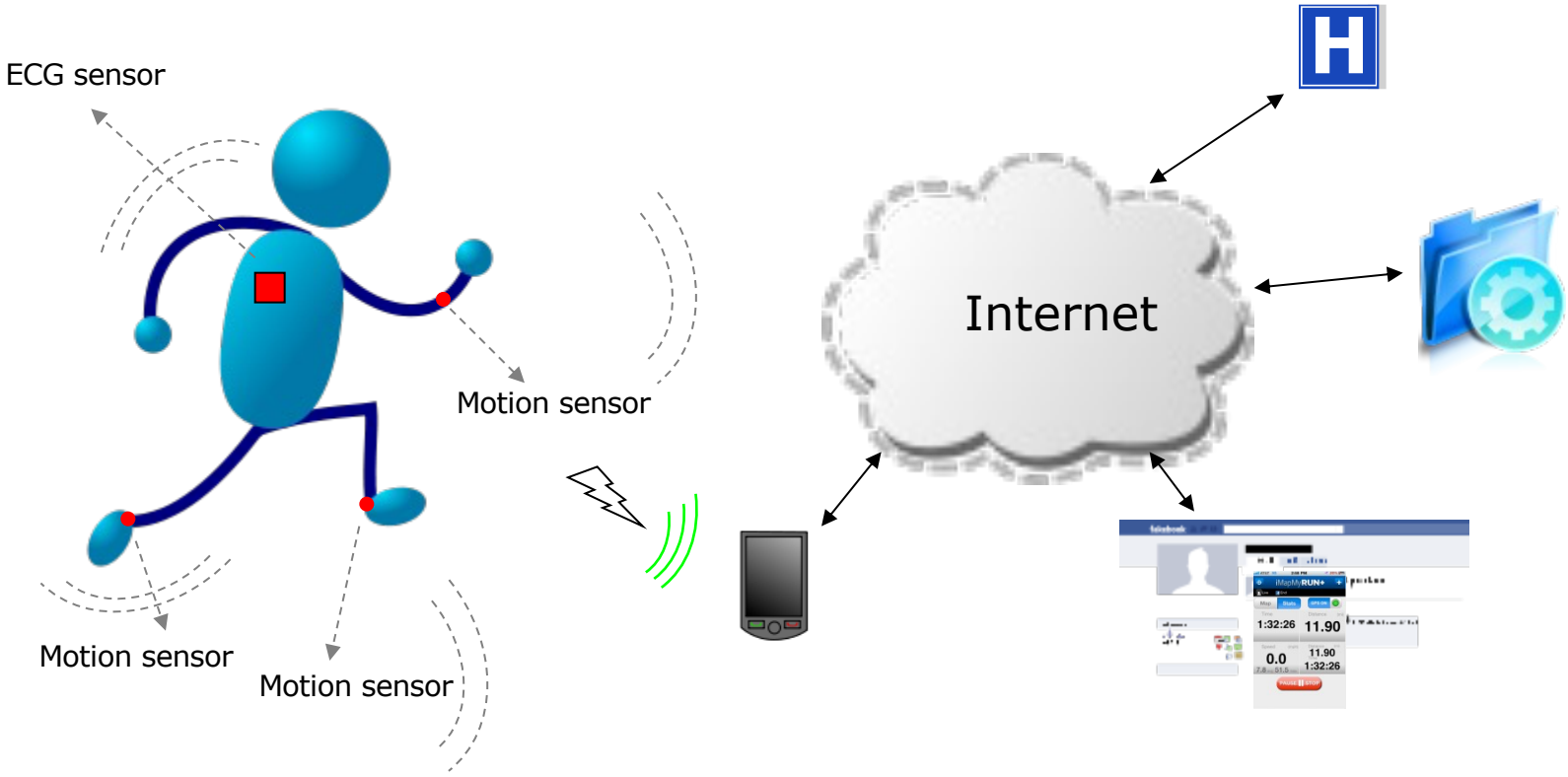
2019 This Is What Happens In An Internet Minute
facebook
1 Million Logging In
18.1 Million Texts Sent
YouTube
4.5 Million Videos Viewed
390,030 Apps Downloaded
347,222 Scrolling Instagram
87,500 People Tweeting
1.4 Million Swipes
tinder
188 Million Emails Sent
1 Million Views
twitch
41 Music Streaming Subscriptions
180 Smart Speakers Shipped
amazon echo
4.8 Million Gifs Served
GIPHY
41.6 Million Messages Sent
2.1 Million Snaps Created
$996,956 Spent Online
694,444 Hours Watched
NETFLIX
3.8 Million Search Queries
Google
60 SECONDS
Created By:
@LoriLewis
@OfficiallyChadd

More “Things” are being connected

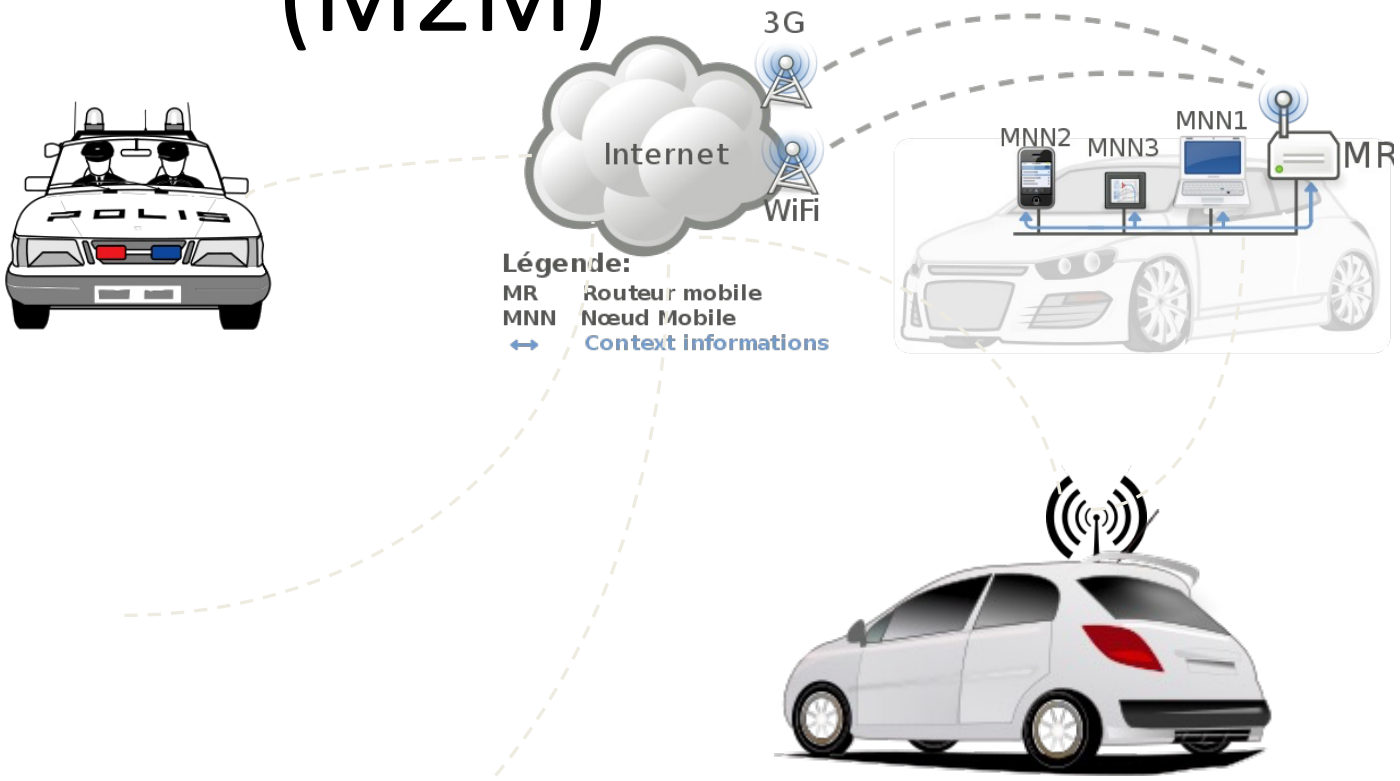
Home/daily-life devices
Business and
Public infrastructure
Health-care
…



People Connecting to Things



Things Connecting to Things (M2M)

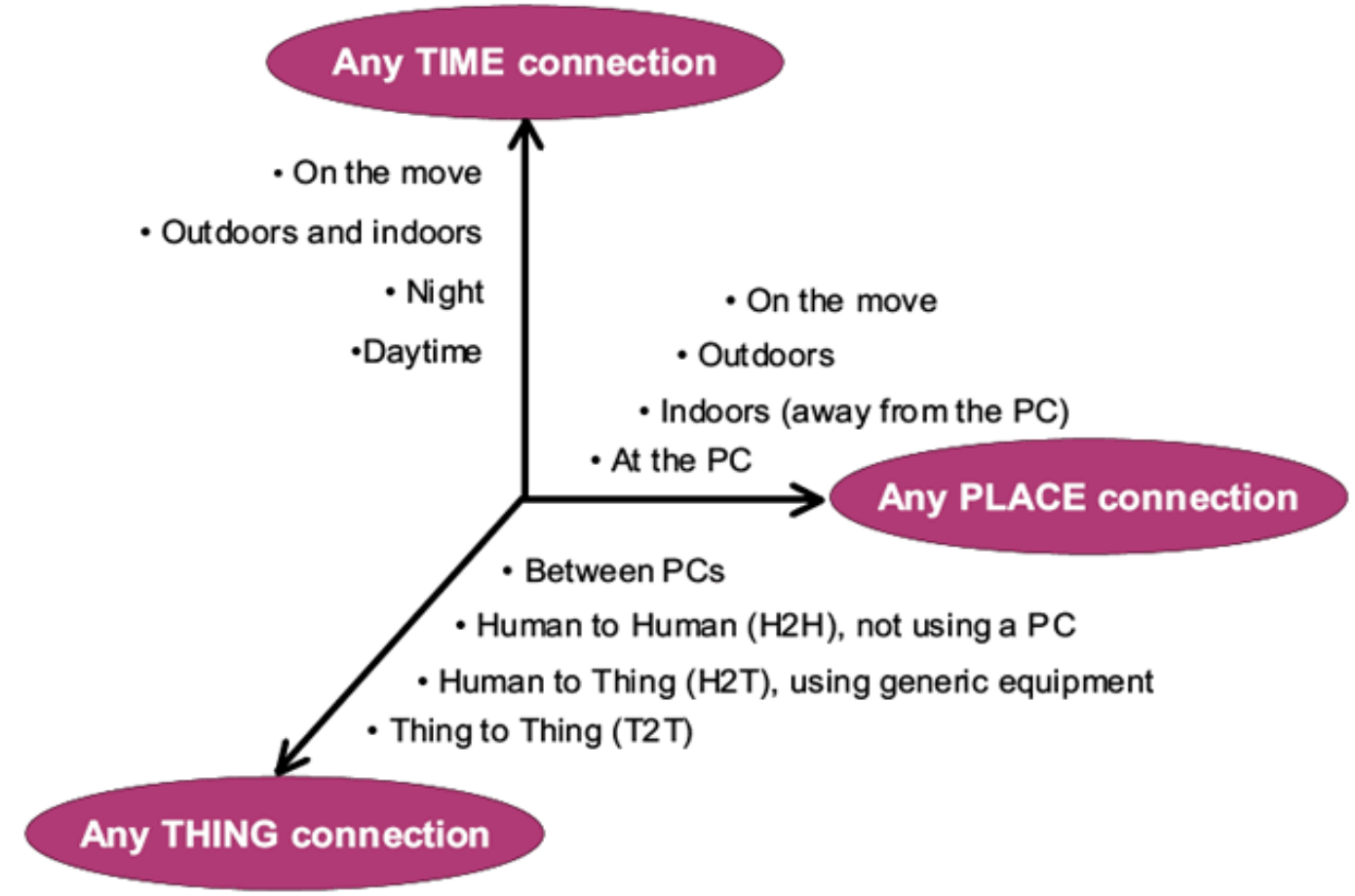


- Complex and heterogeneous resources and networks

How are the networks changing?

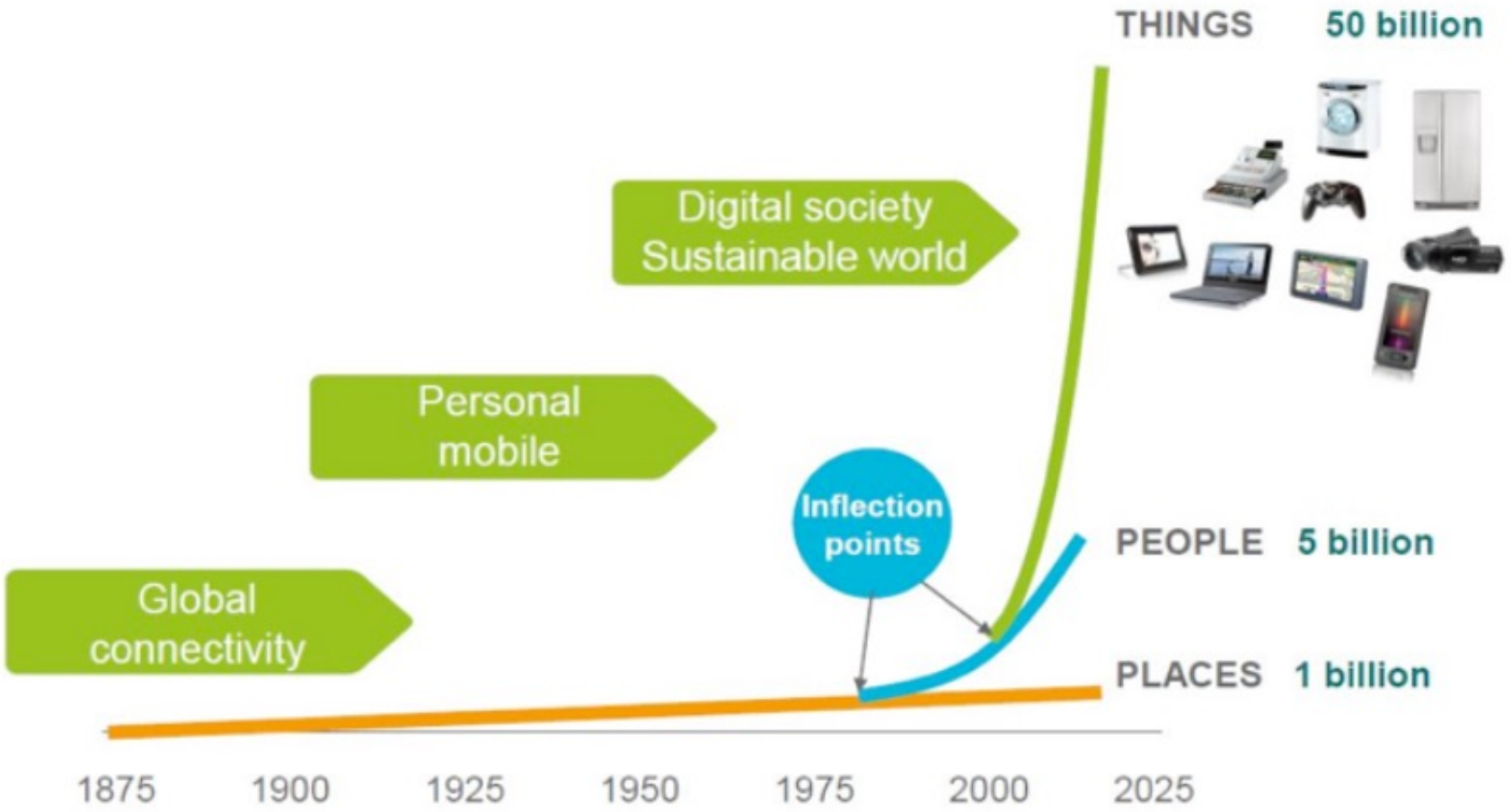
- Extensions
  - More nodes, more connections, IPv6, 6LowPan,...
  - Any TIME, Any PLACE + Any THING
  - M2M, IoT
    - Billions of interconnected devices,
    - Everybody connected.
- Expansions
  - Broadband
- Enhancements
  - Smart networks
  - Data-centric and content-oriented networking
  - Context-aware (autonomous) systems

Future Networks



Source: ITU adapted from Nomura Research Institute

Connecting:
Places → People → Things



A New Perspective

- Scale up:
  - Number of nodes (50 billion by 2020)
- Scale down:
  - Node
  - Cost
  - Complexity
  - Reuse: cent, kylobyte, megahertz

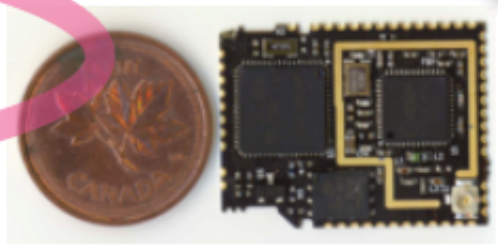
Constrained nodes: orders of magnitude

10/100 vs. 50/250

- There is not just a single class of "constrained node"

RFC 7228

- Class 0: too small to securely run on the Internet
  - "too constrained"
- Class 1: ~10 KiB data, ~100 KiB code
  - "quite constrained", "10/100"
- Class 2: ~50 KiB data, ~250 KiB code
  - "not so constrained", "50/250"

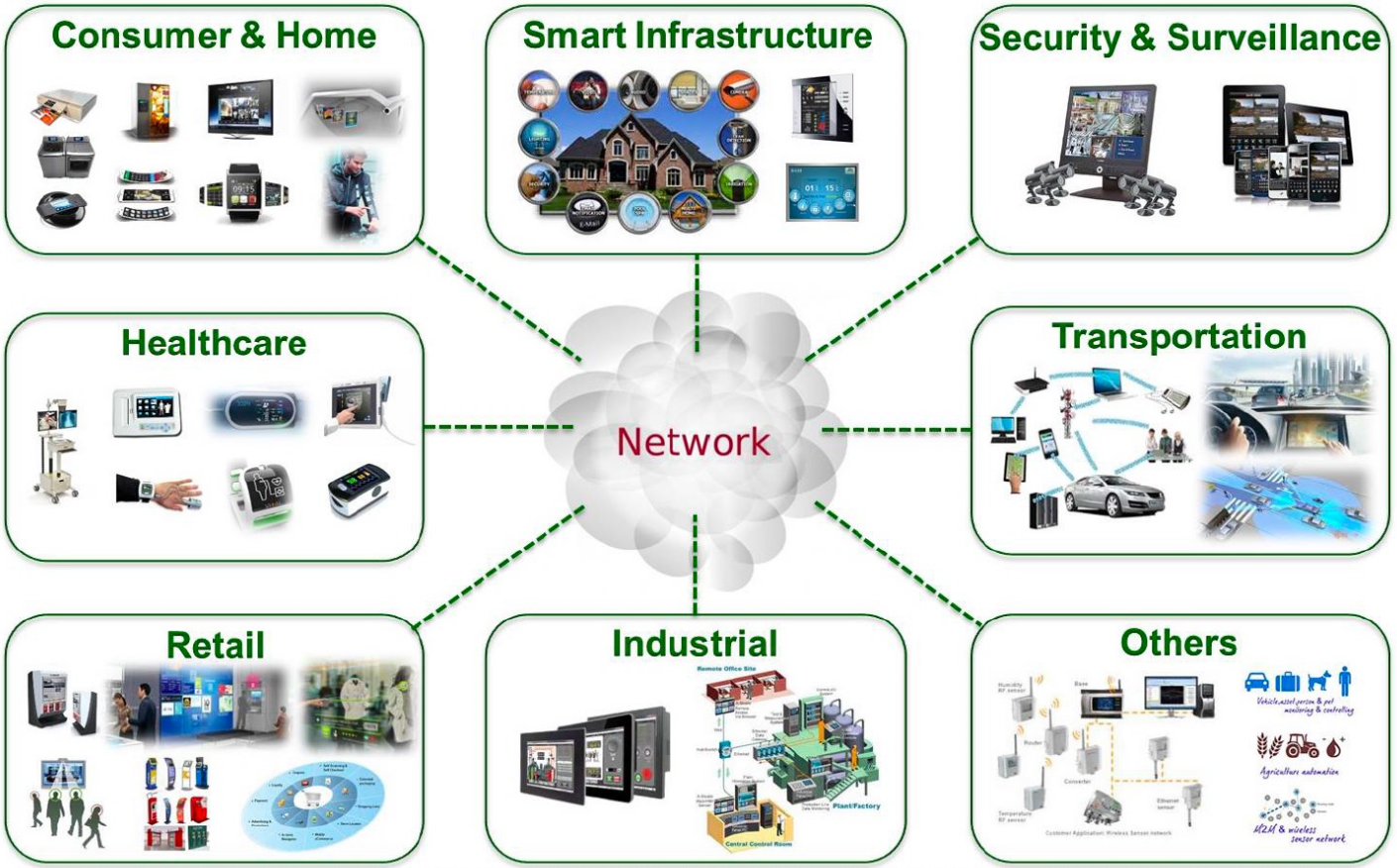


- These classes are not clear-cut, but may structure the discussion and help avoid talking at cross-purposes

Smart agriculture

EXPO 2015: UK Pavillion
Accelerometers to understand the status of a beehive

IoT Applications



Vivante and the Vivante logo are trademarks of Vivante Corporation. All other product, image or service names in this presentation are the property of their respective owners. © 2013 Vivante Corporation

Today's Smart Objects (1)

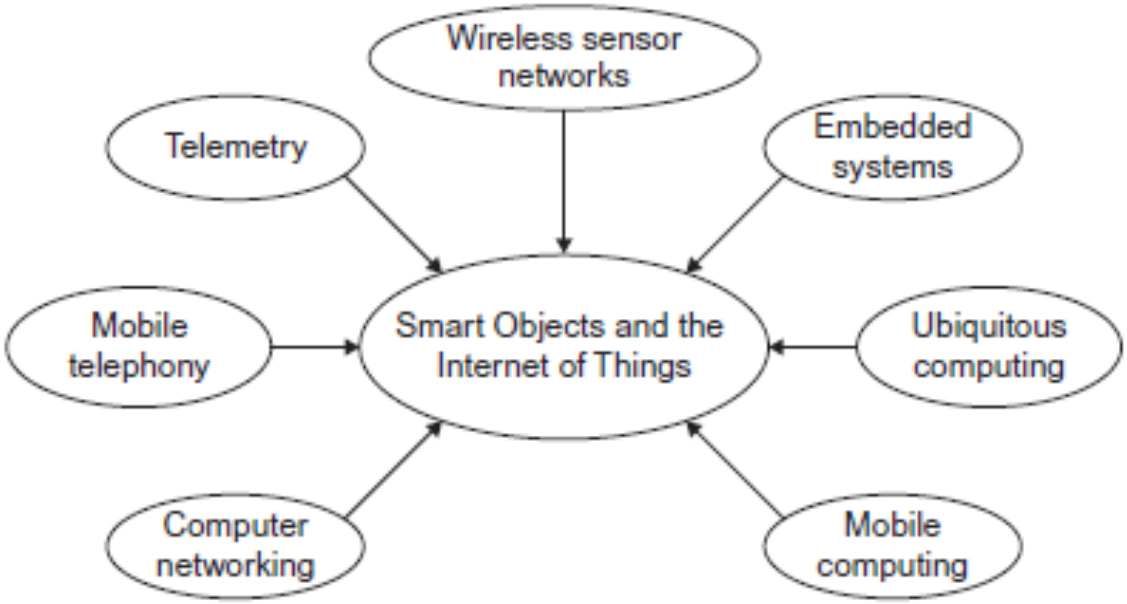


FIGURE 1.1

Smart objects are the intersection of embedded systems, ubiquitous computing, mobile telephony, telemetry, wireless sensor networks, mobile computing, and computer networking.

Embedded Systems (1)

- A computer (microprocessor) embedded in something other than a computer
- Traditionally: real-time systems (strict time response requirements, industry) → Real-Time Operating System (RTOS)
- Commonalities (emb. sys. - smart obj.):
  - Hardware
  - Constraints in terms of computational power and memory
  - Same microcontrollers
- Differences:
  - Communication is not core for embedded systems (e.g., a car engine which cannot communicate can still work as car engine)
  - Communication is crucial for a smart object

Embedded Systems (2)

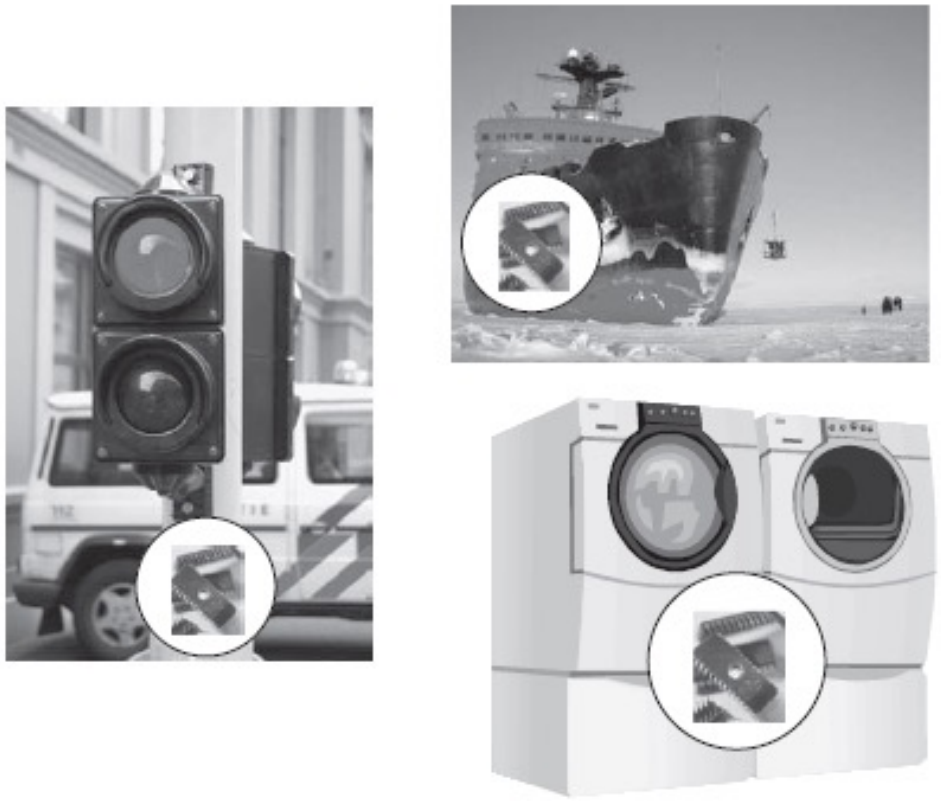


FIGURE 1.2

Embedded systems are microprocessor-equipped systems and devices that interact with the physical world. Examples include traffic lights, a ship's rudder controllers, and washing machine controllers.

Ubiquitous and Pervasive Computing (1/2)

- Field of study based on concept of what happens when computers move away from the desktop and immerse in the environment (MIT, 1988): “computing will no longer be done by desktop pcs, but will move into our daily environment”
- Ubiquitous computing: successful measure is its invisibility
- Examples:
  - Active badges (AT&T@ Cambrige, UK, late 80’s-early 90’s) with readers
  - AmbientROOM project (MIT, 1996)
  - Wearable computing: computing infrastructure moves on the body (**our research**)

Ubiquitous and Pervasive Computing (2/2)

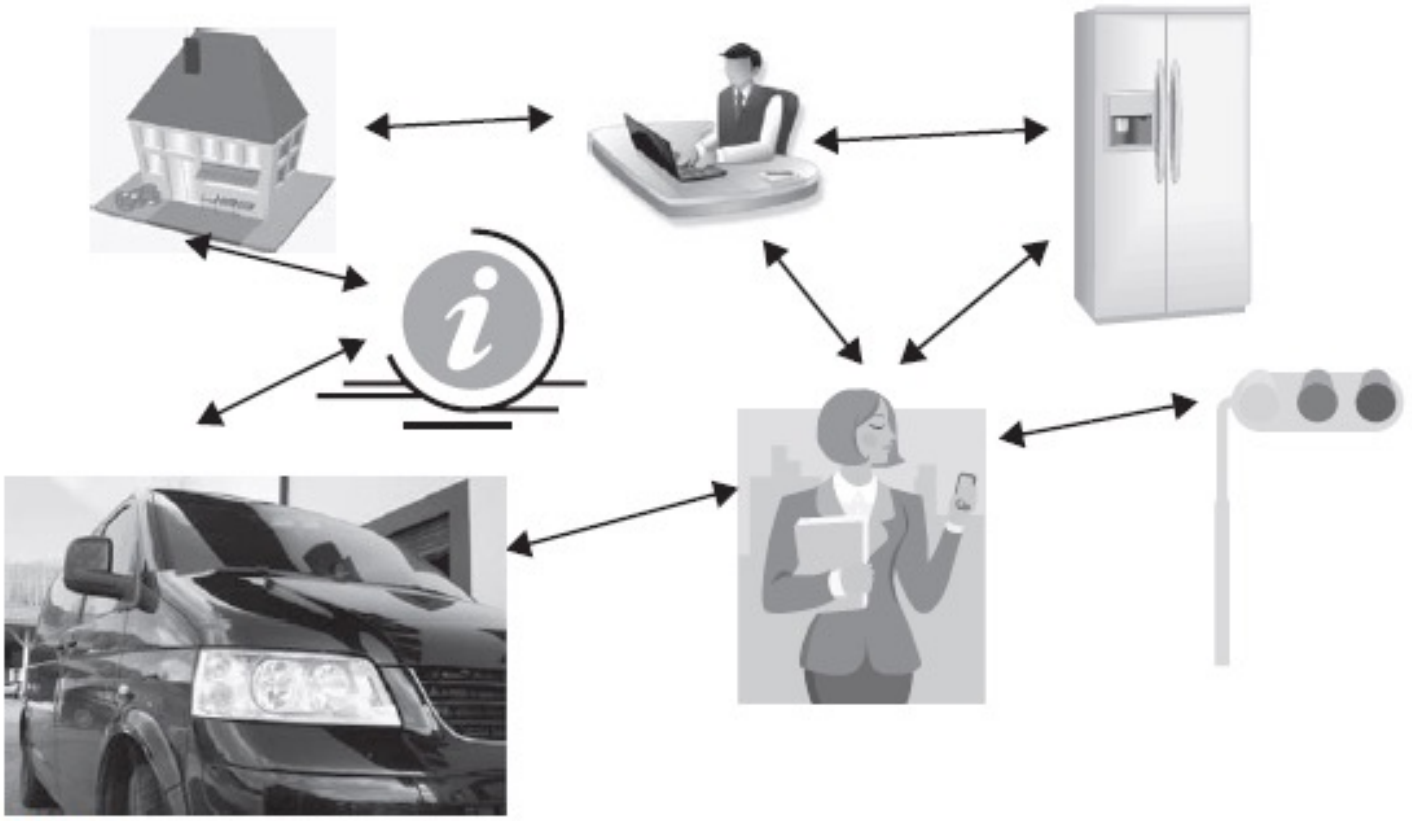


FIGURE 1.3

Ubiquitous computing is a vision for the future of computers where computing moves into everyday objects.

Mobile Telephony

- Grew from telephony industry with the promise of ubiquitous access to telephony
- Typically thought of as cellular telephony
- Smartphones: far more than cellular phones
  - Several other wireless interfaces: WiFi, Bluetooth, NFC
  - A large number of on-board sensors
- Mobile telephony has revolutionized access to the Internet → really seamless
- So, smartphones are likely to play a key role in IoT

Telemetry and M2M

- Telemetry: remote measurements
- M2M: generalization of telemetry, which implies autonomic communication between non-human operated machines and central to the concept of telemetry
- Telemetry:
  - Long-range: temperature, humidity, wind from remote weather stations (cellular network or Inmarsat satellite network); Formula 1
  - Short-range: pacemaker (telemetry coil, to measure the activity without surgery: low-power communication)

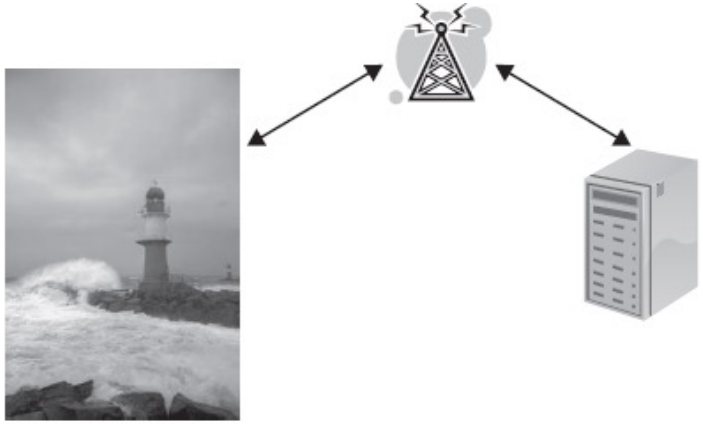


FIGURE 1.4

Telemetry allows reading measurements from remote systems such as weather stations. Data are typically transported using mobile telephony systems.

Wireless Sensor Networks

- Derive from the idea that wireless sensors can collect information from the physical world in a large number of situations (wild fire tracking, agriculture management, etc.)
- Significant research activity in the 2000's:
  - Smart Dust: Berkeley, 1999
  - First prototypes with larger sizes
- Idea: try to be different from the Internet → cross-layer optimization → no scalability
- Several results (energy savings, algorithms, protocols) have been moved to smart objects (not only data gathering, but actuation/control)

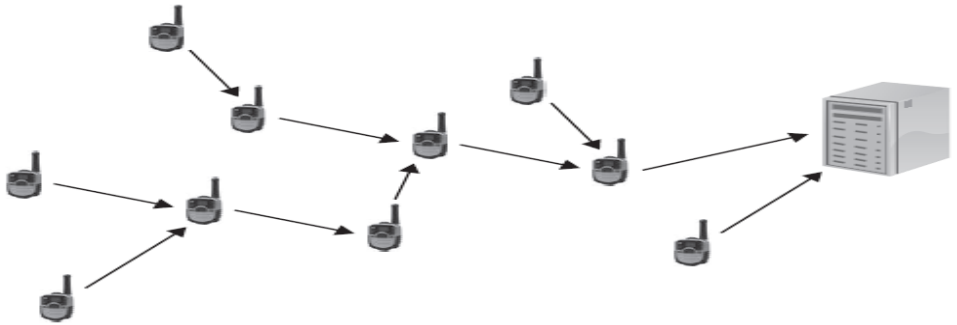
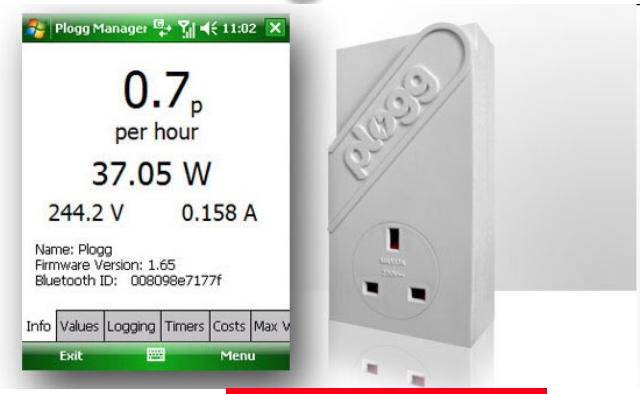


FIGURE 1.5
Wireless sensor networks provide large-scale measurements of physical properties using large amounts of sensors that transport their data wirelessly to a base station.

Sensor devices are becoming widely available

- Programmable devices
- Off-the-shelf gadgets/tools



Linker Intel Group

Image Sensor Device

Challenges for Smart Objects

- **Node-level** internals: power consumption and physical size
- **Network-level** mechanisms and structures: determined by the entire ensemble of smart objects
- Aspects are highly interrelated:
  - power consumption (node-level) is highly influenced by communication patterns (network-level);
  - a routing protocol (network-level) should take into account the energy statuses of the nodes (node-level)
- Internet Protocol for Smart Objects (IPSO) Alliance: set up to spread awareness of the technology around smart objects → smart objects need evolvable technology → technology around IP is the right one

Node-level Challenge: Power Consumption

- Power consumption: battery-power or external low-power energy source (physical vibrations)
- Design implications for hardware, software, network protocols and architecture
  - Hardware designer: low-power components, minimal current leakage, power-efficient sleep mode
  - Software designer: make the most out of hardware
  - Operating systems for smart objects: natively provide mechanisms for low-power operation
- Power efficiency significantly affects network architectures and protocols design
  - Communication patterns→ use efficiently resources (sleeping)
  - Hardware and software keep track of energy spent and communicate to network layer

Node-level Challenges: Physical Size and Cost

- Physical size: size and form factor determine the potential applications for a given smart object system
- Hardware designer
  - Hardware small
  - Number of components small
  - Each component small and inexpensive
- Software designer
  - Microprocessor smaller → Reduced computational speed and memory size
  - Each “line of code” (protocol “feature”) counts: a few kBytes (e.g., efficient multicast in 6LoWPANs)
- Network protocols: designed minimizing the amount of information a node keeps about the network and other nodes
- Memory constraints: network level ←→ node level

Network-level Challenge: Network Size

- Large networks of small objects (e.g., thousands of nodes) → large amount of generated data
  - Example: Smart object network that samples the temperature inside a building. Temperature is generally a slow-moving phenomenon, so the nodes do not need to sample very often: sample the temperature twice every minute → 737,280 readings per year. If the system is designed to work for ten years, there will be over seven million readings, from each node, during the lifetime of the system
- Network size → impact on routing protocol design
- Routing protocol is related to:
  - amount of data the network can sustain
  - speed at which these data can be successfully transported through the network
  - achievable lifetime of the network as a whole
- Routing protocol must make well-informed choices: information on the network as a whole and on the neighbors

Network-level Challenge: Lossy Communications

- Unreliable nature of smart object networks
- Lossyness: inherent property of smart object networks → design for lossy networks, so that the communication protocol is robust for both lossy and non-lossy
- Lossy links → impact on routing protocol design
- Lossyness: illusive property, particularly in wireless networks.
  - Affected by environmental factors (e.g., temperature and humidity)
  - Electromagnetic interference
- Routing protocols for smart object networks must be prepared for this illusiveness.

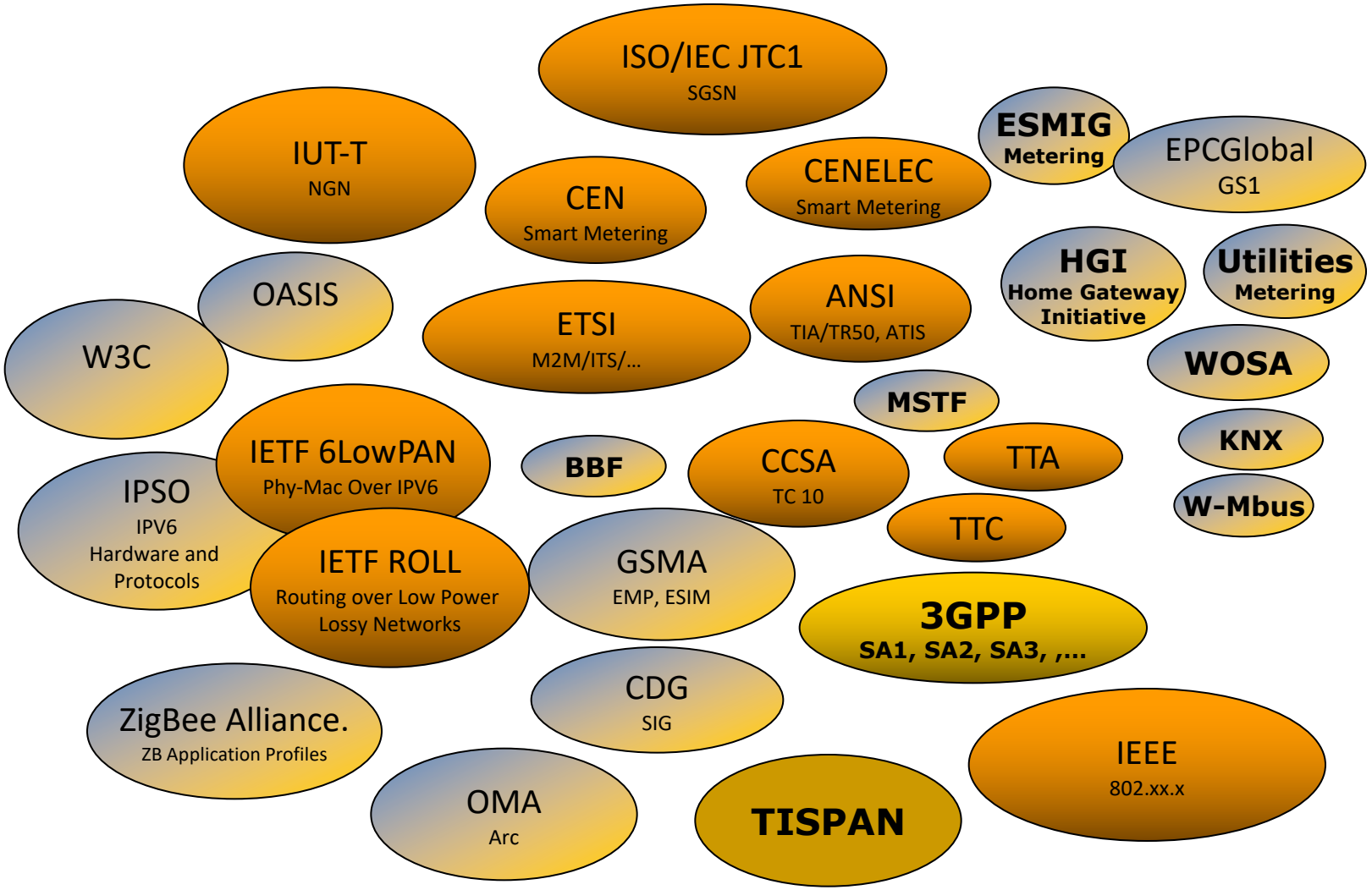
Other Network-level Challenges

- Node addressing: each smart object must be addressable uniquely
  - Within a single (even if large) network
  - With virtually any other SO of any other network → exponential growth → IPv6
- Network management: very challenging (no manual intervention; self-organization)
- Scale of smart object networks: amount of different environments and types of systems in which smart objects are used → various communication technologies
- Smart object network: provide mechanisms for external access to itself

Other relevant aspects: Standardization

- Different technology manufacturers have different specializations
  - For example, environment control system is controlled by an advanced IT system that receives its input from the humidity sensors
- With standardized technology, the technology is independent of its vendors, producers, and users → high acceptance
- Open standards produced through established practices of international standardization organizations
  - Patents for technology standardized must be freely accessible (protection against “submarine” patents)
- Standardization of smart object technology → challenge with several organization (different levels of protocol stack)

Standards Development Organizations (SDOs) and Industrial Groups



M2M: Vertical Fragmentation

- **Fragmented industry verticals:**
  - most industries are solving their M2M needs on their own;
  - often converging upon similar architectures and elements;
  - the M2M “wheel” is reinvented from industry to industry.
- **Customized solutions abound:**
  - since solutions are on an industry-by-industry basis, each solution requires its own design, production, and implementation cycle;
  - unique solutions often result in sole-sourcing of customized hardware;
  - software design is limited to a small group of developers who understand the architecture and API’s for a particular industry or platform, thus leading to high development costs and high costs for support.
- **Lack of scale for components:**
  - economies of scale are difficult to achieve in any given industry since solutions are so varied and unique.

Examples of Verticals (1/2)

Security	Surveillance applications, alarms, object / people tracking
Transportation	Fleet management, emission control, toll payment, road safety (e.g. eCall stow pipe)
Health care	e-Health, and personal security
Utilities	Smart Metering and Grid such as oil, water, electricity, heat, and others
Manufacturing	Production chain monitoring and automation

Examples of Verticals (2/2)

Supply and provisioning	Freight supply and distribution monitoring, Vending Machines
Facility Management	Home / building / campus automation

. . .

Future	Multitude of new M2M applications based on standardized Service Capabilities and API [Ericsson CEO vision of 50 Billion Connected Objects by 2020]

Industrial Standardization for M2M

- M2M is here, and has massive potential for growth to be unleashed by standards.
- M2M first step towards future Internet of things (IoT) and will support Smart Metering/Grids.
- Market value increases with the “square” of interoperable/ accessible services.
- Operators and vendors see ETSI as a good place to do the work, however, progress requires wider consensus. (IETF, OMA and BBF contributes parts)

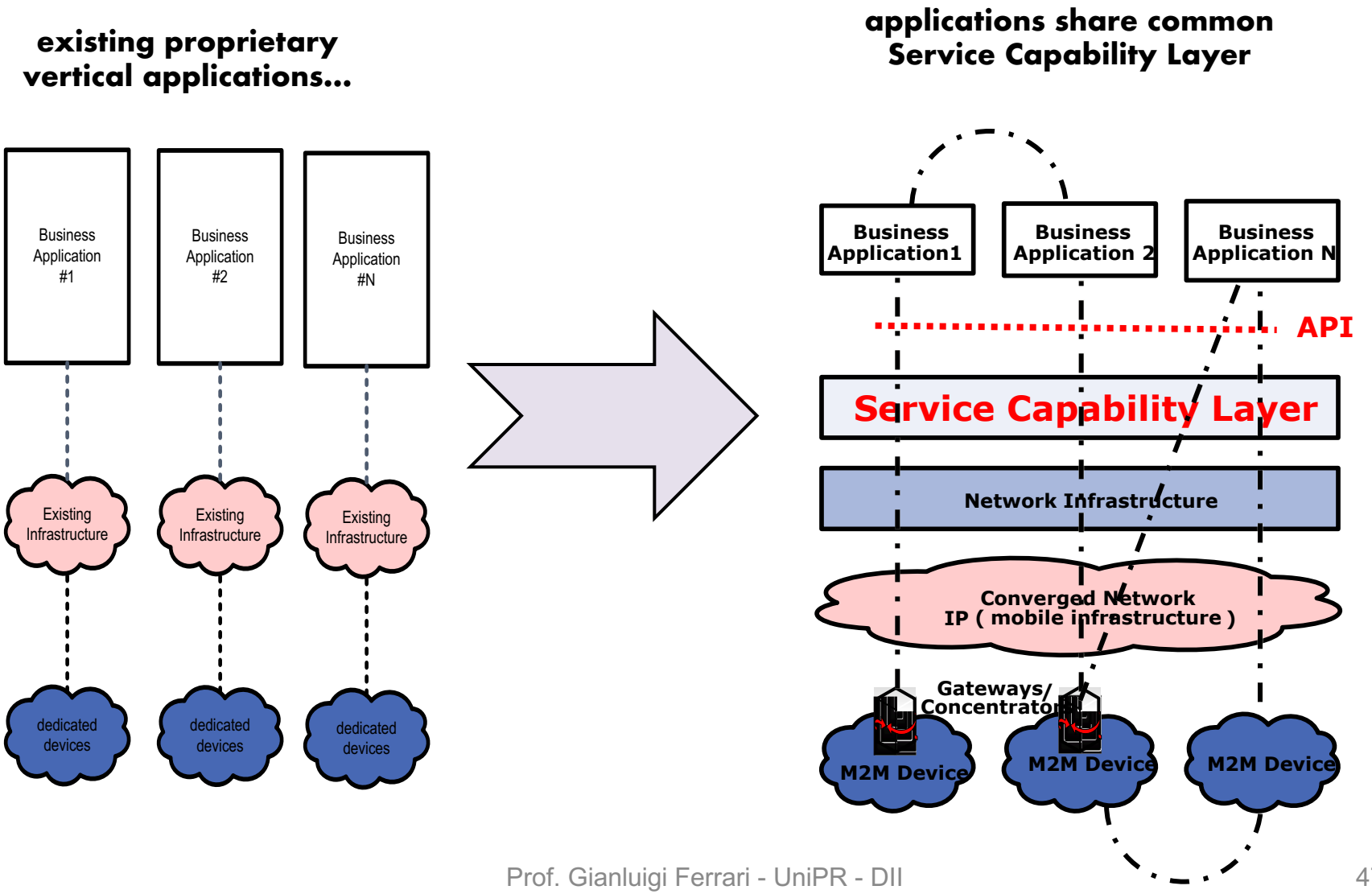
ITU-T M2M Service Layer Focus Group established in Jan. 2012

- M2M is included under the larger umbrella of the “Internet of Things” (IoT).
- The group will study and evaluate the M2M landscape and M2M work currently being undertaken by regional and national standards development organizations (SDOs), with a view to identifying a common set of requirements.
- The Focus Group will initially focus on the APIs and protocols to support e-health applications and services.
- The Focus Group will work closely with all ITU-T Study Groups and with other relevant SDOs, forums/consortia, regulators, policy makers, industry and academia. Within the ITU, the group will work particularly closely with the Internet of Things Global Standards Initiative (IoT-GSI). http://www.itu.int/en/ITU-T/gsi/iot/Pages/default.aspx.

Merits of Service Capabilities and API

- Service Capabilities provide functions that are to be shared by different applications in an interoperable manner.
- Service Capabilities expose functionalities through open interfaces (e.g. an API).
  - This will support interoperable ubiquitous services, however, area-specific data-models may be required (e.g. for Smart Metering /AMR).
- Service Capabilities simplify and optimize applications development and deployment and hide technology specifics.
- Service Capabilities may be M2M specific or generic, i.e. providing support to other than M2M applications.

Needs to invert the stove pipes



IETF (Academy-friendly)

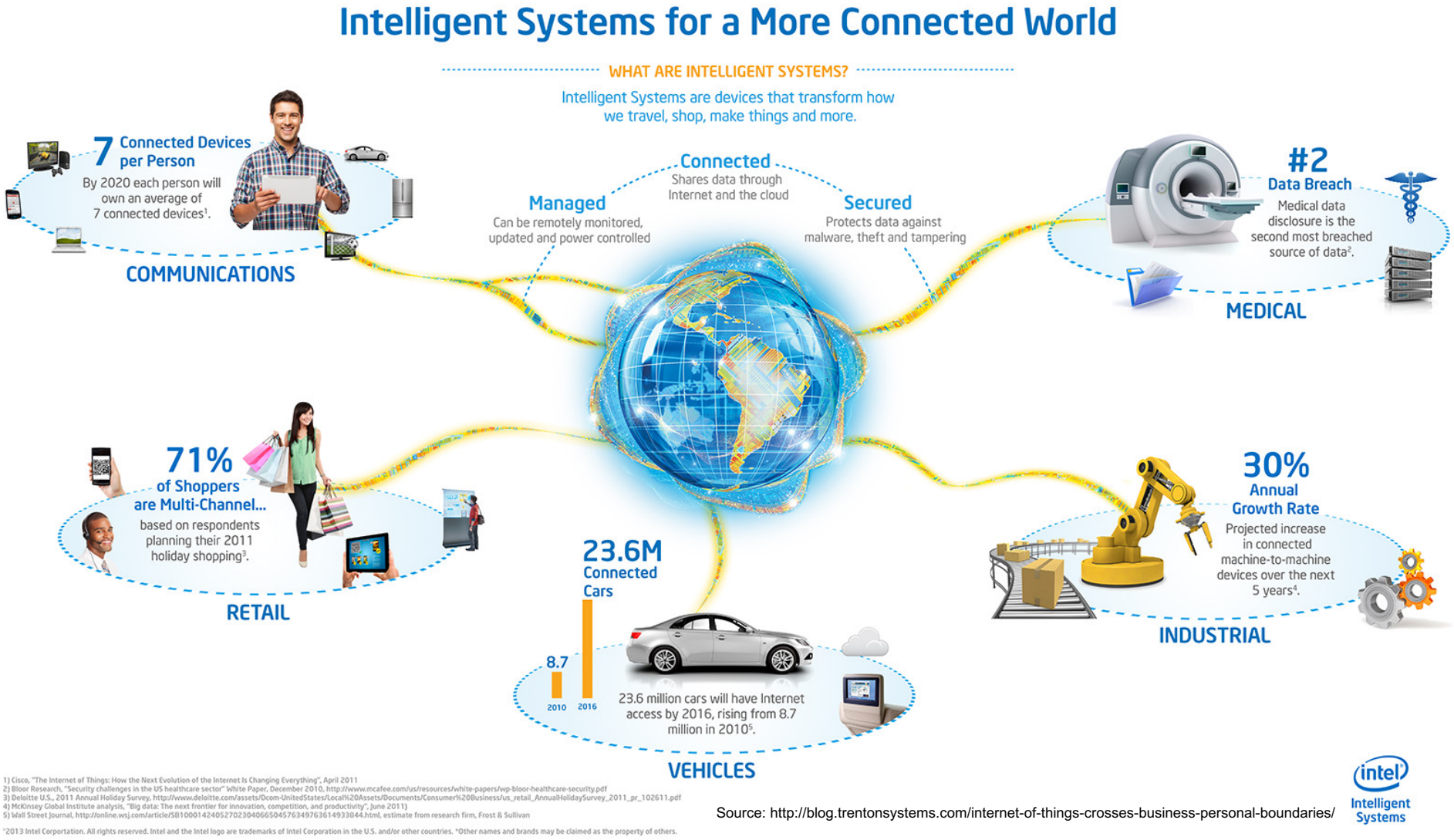
- The IETF work of the 6LoWPAN, ROLL and CoRE Working Groups (WGs) is of particular relevance for IoT/M2M.
- The CoRE WG is providing a framework for resource-oriented (i.e. RESTful) applications intended to run on constrained IP networks.
- The CoRE WG has defined a Constrained Application Protocol CoAP for the manipulation of Resources on a Device. The protocol supports the communications aspects of the M2M service layer.
- CoAP and related protocols will continue to be developed by the IETF.
- **CoAP is adopted by ETSI M2M**.
- The CoRE infrastructure includes a Link format (discovery mechanism), an event subscription mechanism and a scalable device proxy. Devices are viewed as RESTFul resources and can directly be interacted with using HTTP or CoAP protocol operations (PUT, GET, POST, DELETE, etc.).

Another relevant aspect: Interoperability

- Interoperability: the ability of equipment and systems from different vendors to operate together
- Smart objects must interoperate from the physical layer up to the application or integration layer
  - At physical level: agreement on physical frequencies, signal modulation, information rate
  - At network level: agreement on the format of the information that is sent and received over the physical channel; how nodes are addressed; how messages should be transported through a network of smart objects
  - At application or integration level: share a common view on how data should be entered or extracted from a smart object network, as well as how the smart objects should be reached from outside systems
- Imperative: technical architecture of smart objects defined to ease interoperability → IP-based architecture

Smart Things/Objects – Smart Environments

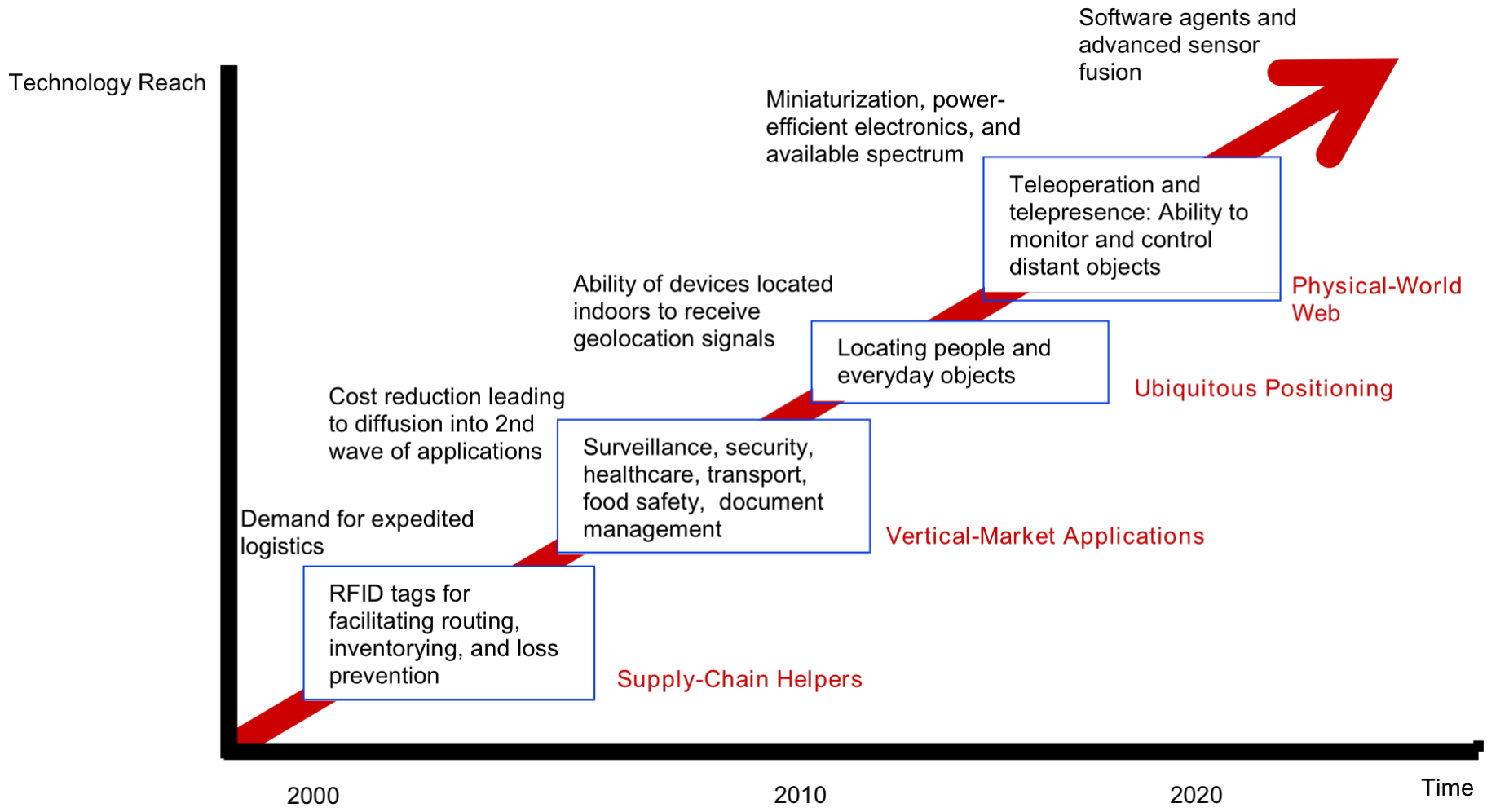
Opportunities



Source: http://blog.trentonsystems.com/internet-of-things-crosses-business-personal-boundaries/

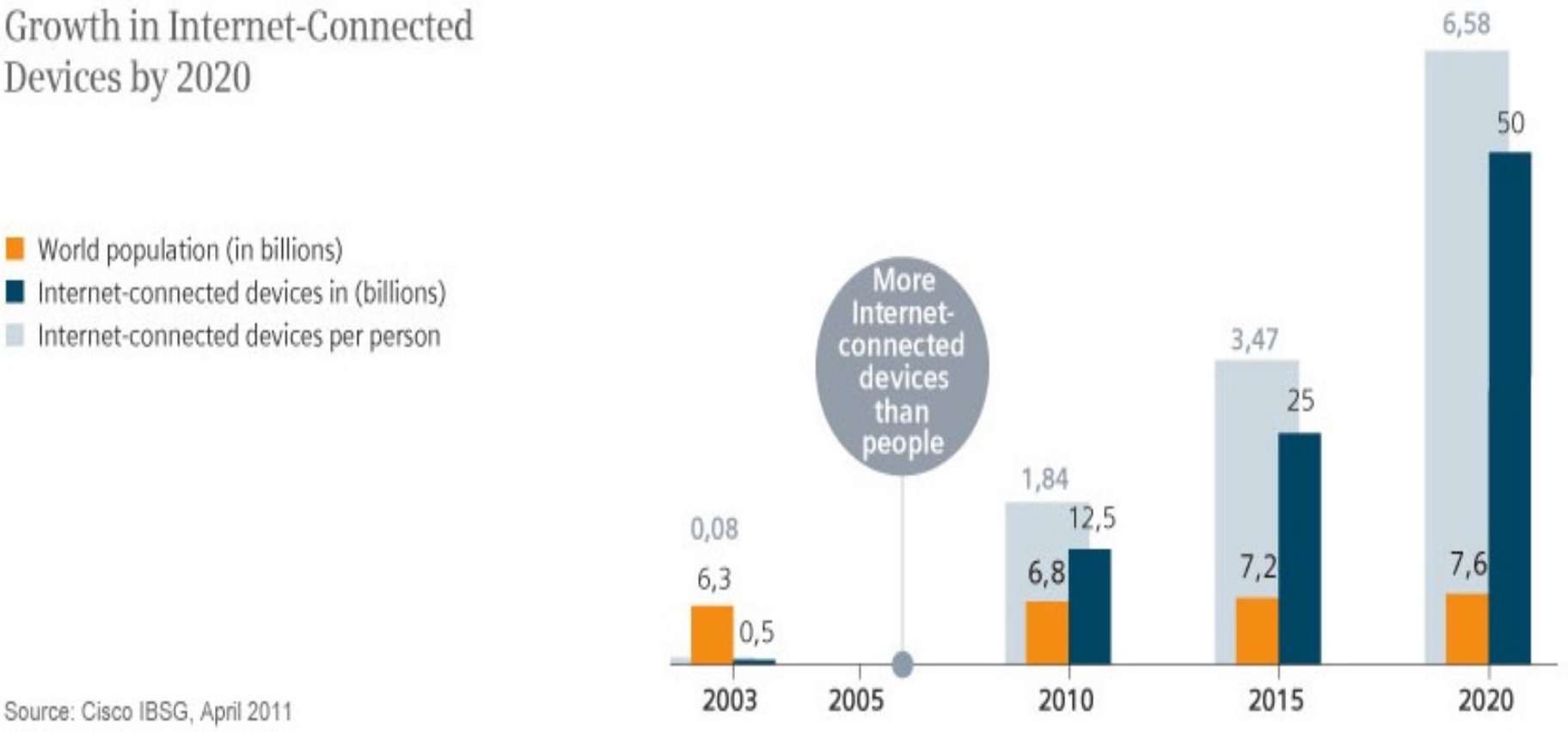
Technology trend

TECHNOLOGY ROADMAP: THE INTERNET OF THINGS



Source: SRI Consulting Business Intelligence

Internet-connected devices



Source: Siemens, http://www.siemens.com/innovation/apps/pof_microsite/_pof-fall-2012/_html_en/facts-and-forecasts-growth-market-of-the-future.html

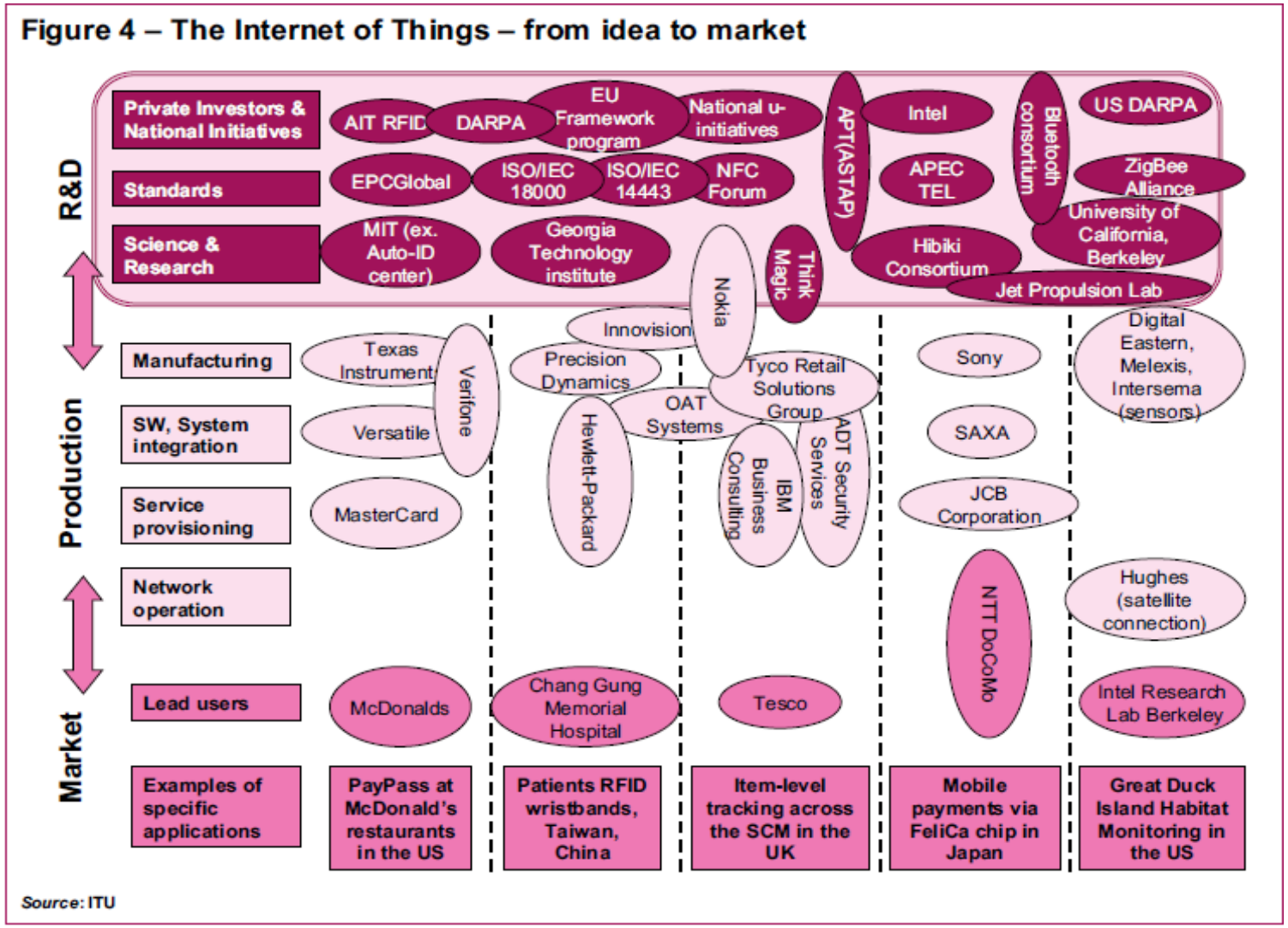
Global Data Generation

- Everyday around 20 quintillion (10^18) bytes of data are produced (Source: http://www-01.ibm.com/software/data/bigdata/).

- This data includes textual content (unstructured, semi-structured, structured) to multimedia content (images, video and audio), on a variety of platforms (enterprise, social media, and sensors).

- Big data!

State of the Art of IoT



An Outlook on Hardware

Main Hw Components

- The hardware of smart objects consists of four main components
  - [mouth/ears] Communication device (either a wireless transceiver with antenna or wired connection)
  - [brain] Microcontroller: defines the smart object's behavior
  - [hands] Set of sensors or actuators: interact with the physical world
  - [food] Power source: needed because the smart object contains electrical circuits

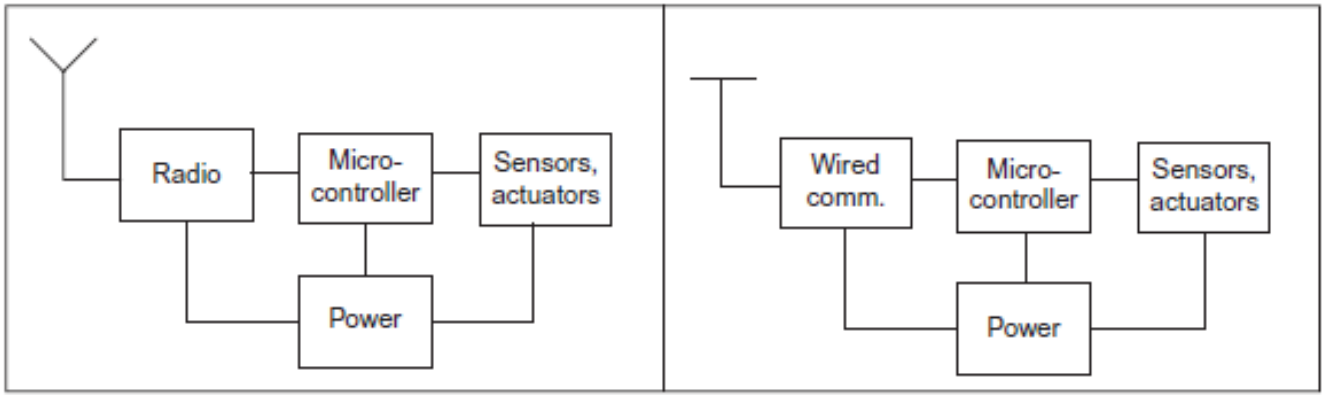


FIGURE 11.1

The hardware architecture of two smart objects: a radio-equipped, wireless smart object (left) and a smart object with wired communication (right).

An example: MicaZ

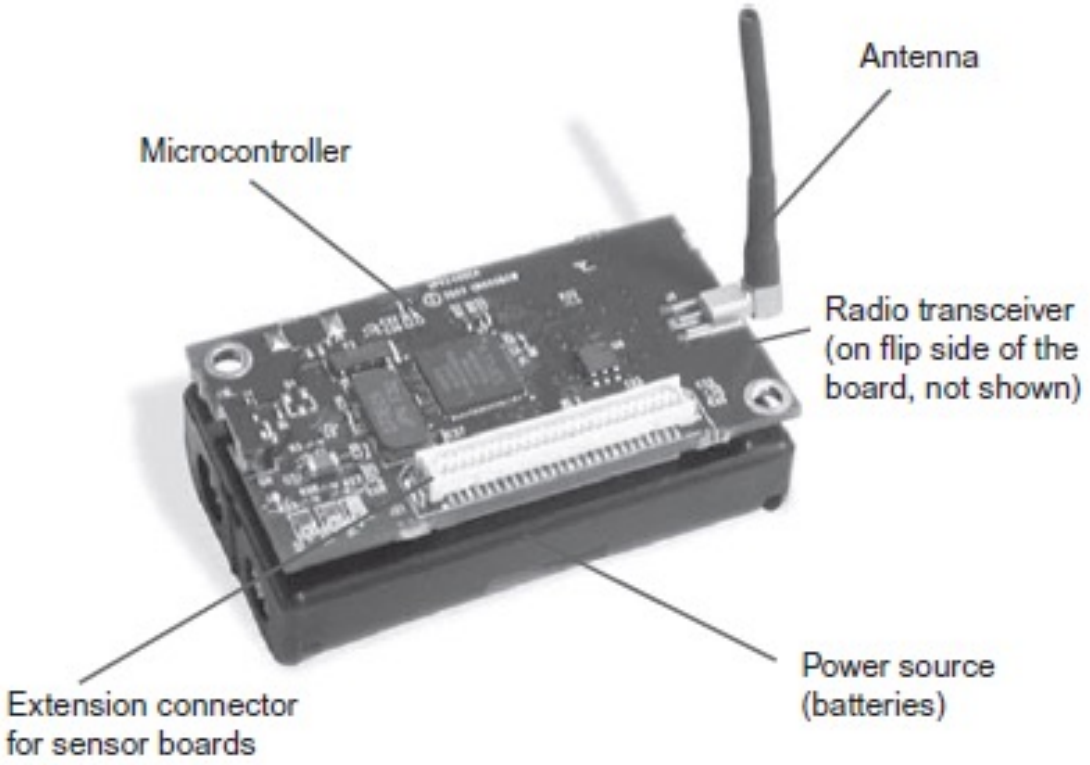


FIGURE 11.2

A MicaZ prototype board with a microcontroller, power source, communication device, and sensor connectors.

Photo courtesy of Crossbow Technology.

Communication Device

- Different types of radio transceivers have different amounts of built-in processing capabilities: (simple) send and receive individual bits of information into the air; (more powerful) package the information into packets, form headers, and even encrypt and decrypt the data using secure encryption methods
- Radio is usually the most power-consuming component (10 times the power consumption of the microcontroller or the sensors): modulation (tx) and demodulation (rx) –> idle listening as expensive as transmission
- Saving energy: duty cycling → need synchronization (see later)

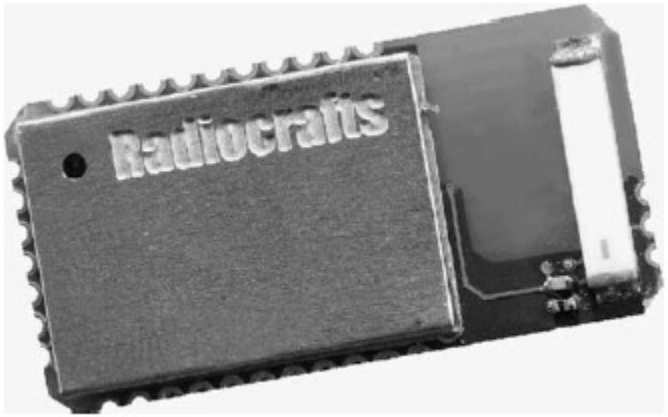


FIGURE 11.3

Texas Instruments CC2430 single-chip radio transceiver with integrated 8051 microcontroller and on-board antenna manufactured by Radiocrafts. The size of the board is $1.2 \times 1.0\,cm^2$.

Photos courtesy of Radiocrafts.

Example of system-on-chip (SoC)

Microcontroller

- Runs the software of the smart object & connects radio with sensors/actuators
- Microcontroller: microprocessor with built-in memory, timers, and hw for external connections
- Microcontrollers used in smart objects (cost lower than 10$) much smaller than those used in PCs
  - Smart object: a few Kbytes of on-chip memory ; clock rate of a few MHz
  - PC: Gbytes of memory; clock rate of several GHz
- Memory: ROM (behaviour) and RAM (temporary sw for tasks)

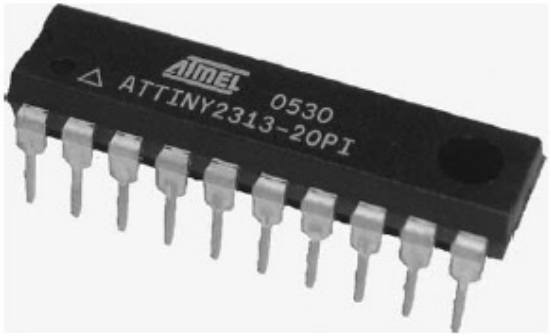


FIGURE 11.4

An Atmel ATTINY 2313 smart object microcontroller with 20 pins. The ATTINY 2313 has 2 kB of ROM and 128 bytes of RAM. This represents the low end of smart object microcontrollers.

Table 11.1 Microcontrollers Used in Smart Objects

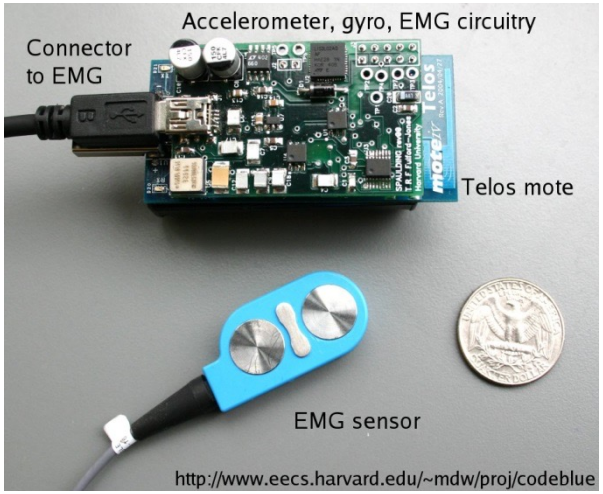
Name	Manufacturer	RAM (kB)	ROM (kB)	Current consumption (active/sleep), mA
MSP430xF168	Texas Instruments	10	48	2/0.001
AVR ATmega128	Atmel	8	128	8/0.02
8051	Intel	0.5	32	30/0.005
PIC18	Microchip	4	128	2.2/0.001

Note: Each manufacturer has several models of each device. This table lists only one example from each manufacturer.

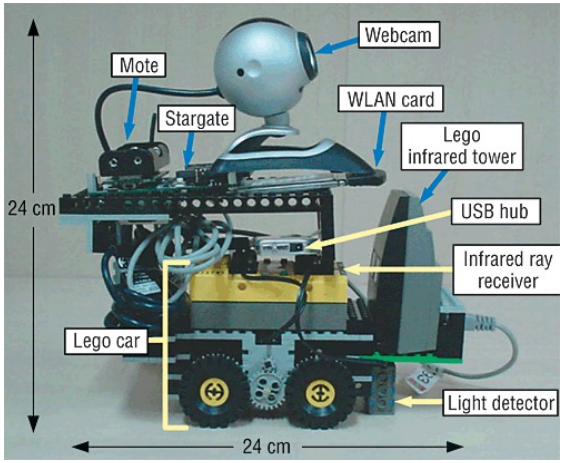
Sensors and Actuators

- Sensors: sense the environment
- Actuators: affect or change the environment
- From very simple (temperature sensor) to complex (ultrasonic range device or camera for fence monitoring)

Video recording



ElectroMioGraphy



Power Sources

- Most common power source: battery (Lithium, non rechargeable)
  - (+) lifetime of many years on a single standard AA-size battery (low-power electronics and power-saving software)
  - (-) difficult to recycle
- Power scavenging: harvest from the environment (solar cells, piezoelectricity, electromagnetic energy-RFID)

FIGURE 11.5
PowerCast P2100 module converts radio energy from the air into electricity that powers an electrical circuit. The module requires an external antenna to pick up the radio signals (not shown).

Table 11.2 Different Power Sources for Smart Objects, Their Maximum Current Draw, and the Amount of Charge They Store

Power source	Typical maximum current (mA)	Typical charge (mAh)
CR2032 button cell	20	200
AA alkaline battery	20	3000
Solar cell	40	Limitless
RF power	25	Limitless

Energy Consumption

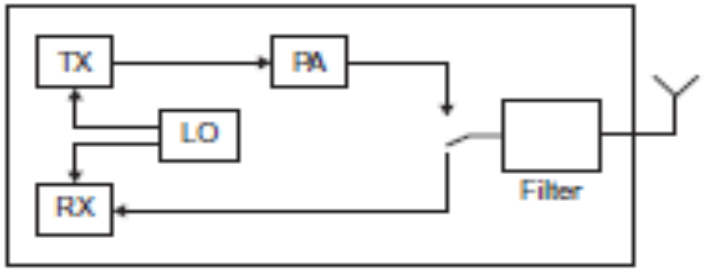


FIGURE 11.17

Schematic drawing of a radio transceiver.

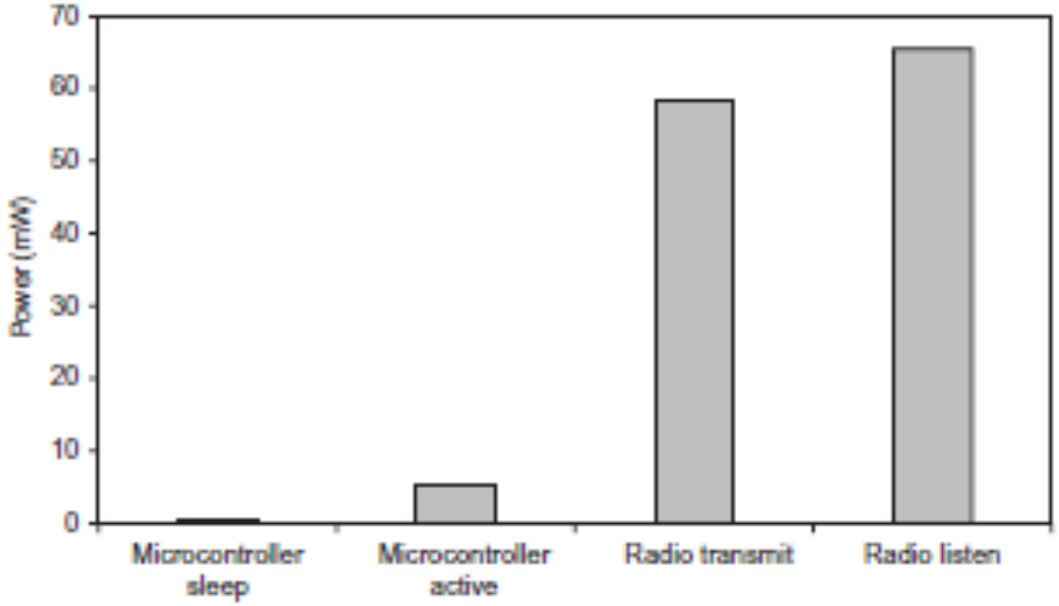


FIGURE 11.16

Power consumption of the microcontroller and the radio on the Tmote Sky smart object prototyping board. The power consumption of the microcontroller sleep mode is so low that it is not visible in the graph.

An Outlook on Software

Programming a Smart Object

- Programs written in a high-level programming language (e.g., C, maybe Java) and compiled with a compiler to machine code for the microcontroller
- Machine code written into ROM, run when the smart object is powered on
- Operating systems are used by smart objects (see next slide)
- Memory footprints makes programming (e.g., communication protocol stack) challenging (see later)

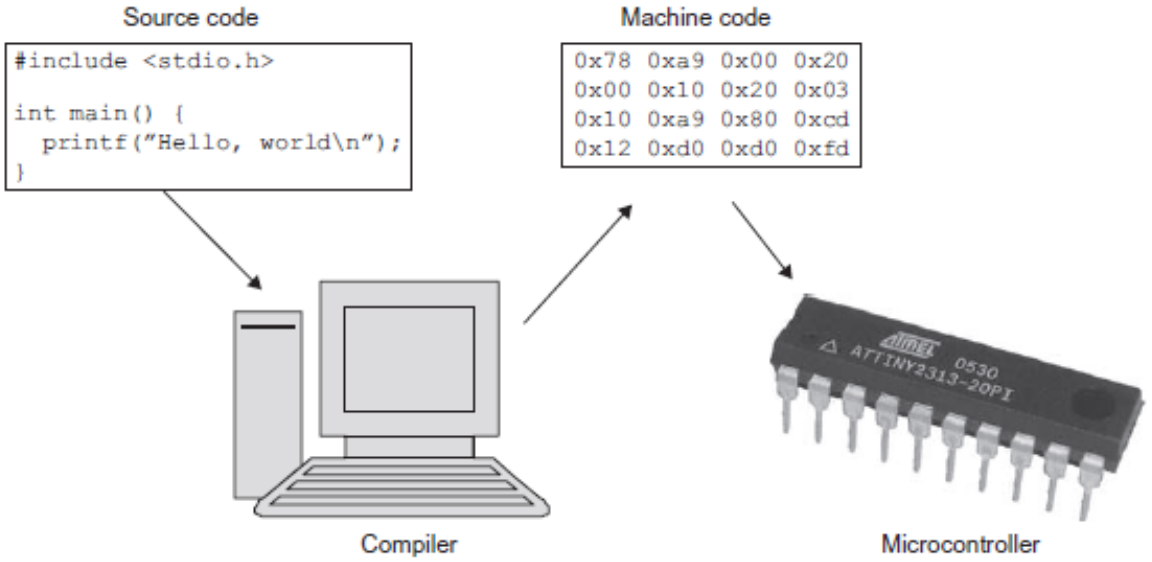


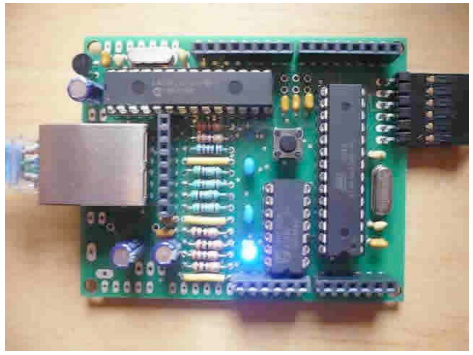
FIGURE 11.8

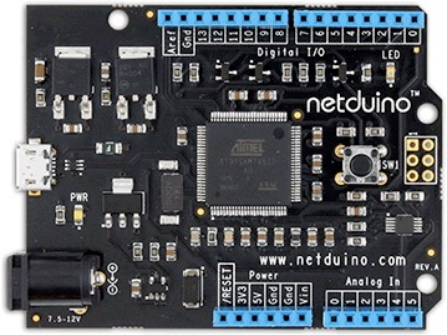
The process of software development for a smart object. Source code is compiled to machine code that is written to the ROM of the smart object microcontroller.

Operating Systems

- Very different from general-purpose operating systems
  - Tailored to specific requirements of smart objects and hardware constraints (memory constraints and processing speed)
  - No graphical interface (hidden deeply within the microcontroller)
- Main operating systems (all open-source)
  - **Contiki** (2003-, SICS-Thingsquare): first OS which guarantees IP communication with uIP TCP/IP stack (5 kB), incorporating also uIPv6 (11 kB) in 2008; useful simulator (Cooja); many components used in industry (uIP TCP/IP, protothread programming abstraction); written in C (ported to more than 12 microprocessors/controllers); direct communication with other IP-based application and web services (e.g., Twitter, mail)
  - **TinyOS** (2000-, Berkeley-Stanford): focus on wireless sensor networks and academia; used uIP for IP communication and has recently incorporated independent IPv6 implementation; written in nesC; event-driven programs
  - **FreeRTOS**: designed for embedded systems, providing real-time guarantees (control applications); provides TCP/IP support (uIP and lwIP); uses a preemptive, multi-threaded (multiple programs simultaneously running-large memory) programming model; ported to over 50 microprocessors/controllers

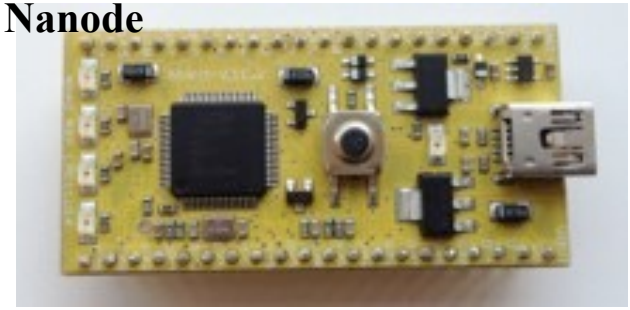
IoT Devices (“homegrown”)





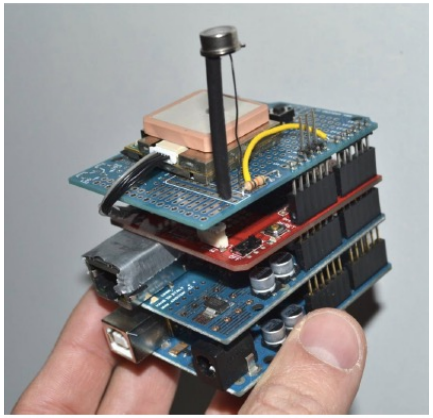
Netduino

Nanode



mbed.org ARM® Cortex™-M0

- Open source electronic platforms
- Must add sensors, comms, positioning
- Prices dropping rapidly



52North SenseBox light

Arduino + Ethernet + GPS ~ 100 €

Twine

WiFi, temperature and vibration sensors, $100+ plus cloud service

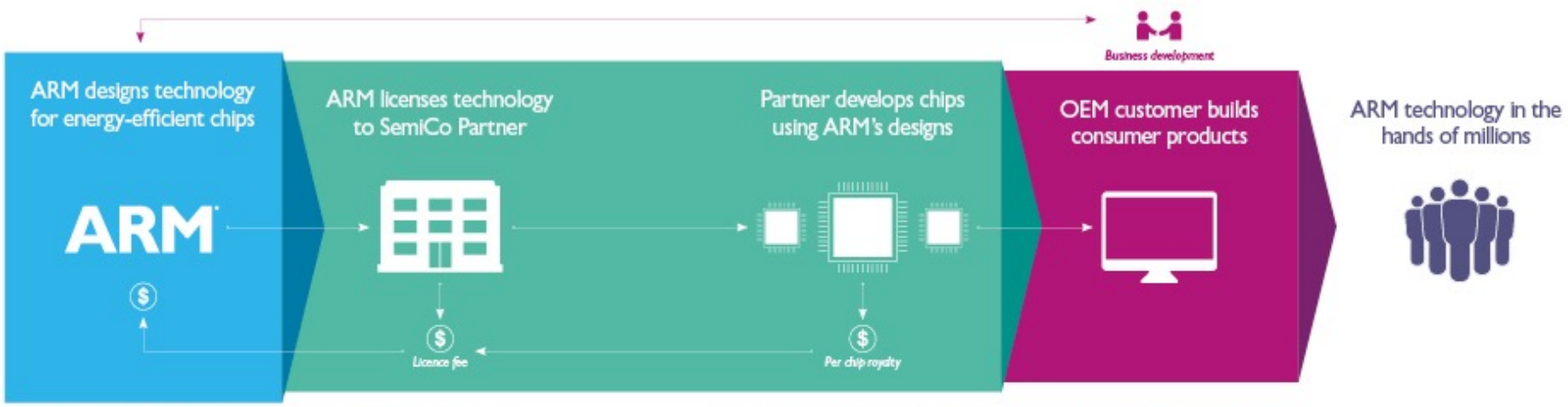
© 2012 Open Geospatial Consortium

Illustrative Spectrum of Embedded Resources: Cortex

MCU capability	Class	System Cost $ (IC + required crystals, memory, etc)	RAM (B)	Flash (B)	OS	Connectivity	Application	Capability
High	Low A	12	>16M	>64M	Linux, Android	Full IP stack	Gateway, local hub, smart pen	Full internet stack, rich OS (Linux, Android), typical gateway device
Med High	M4	8	1M	1M				
Med	M3, M4	5	48k	256k	RTOS, Embedded OS	6LowPAN	Smart watch, washing machine controller	COAP + DTLS, application developed using rich ecosystem of frameworks and libraries, capable of some audio and GUI
Low	M0, low M3	2	10k	64k	Embedded OS, none	6LowPAN	Light fitting, motion sensor	COAP (+DTLS?), 6LowPAN (just), quite constrained application
Ultra low	M0+	<1	<4k		none	Proprietary low-rate / short range	Single use temperature sensor, mm^3	Highly constrained, carefully optimised code, uplink via an application gateway device

The Architecture for the Digital World® ARM®

ARM Business Model



1. ARM designs technology to go into energy-efficient chips for a broad range of end markets. ARM licenses each design to multiple semiconductor companies. Every company pays an upfront licence fee to gain access to the design.

2. The semiconductor company will incorporate the ARM technology design into their chip. It can take 2–3 years to build a chip and a further year for an OEM to build their product, such as a digital TV or mobile phone, containing the chip. When the chip starts to ship, ARM receives a royalty on every chip that uses the design. Typically our royalty is based on the price of the chip.

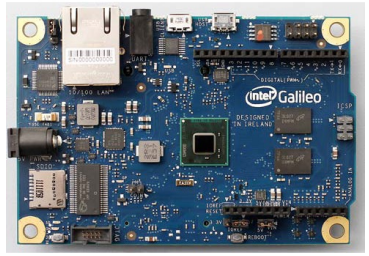
3. Each ARM design is suitable for a wide range of end applications and so can be re-used in different chip families addressing multiple markets. Each new chip family generates a new stream of royalties. An ARM design may be used in many different chips and may ship for over 20 years.

4. ARM's flexible business model has proven to be suitable for different technologies, end products, licensing strategies and a broad range of ecosystem Partners.

Deployment of an IoT testbed

150+ nodes

(3/4G, WiFi, 802.15.4, SubGHz, UWB, BLE,NFC,...)



intel Galileo

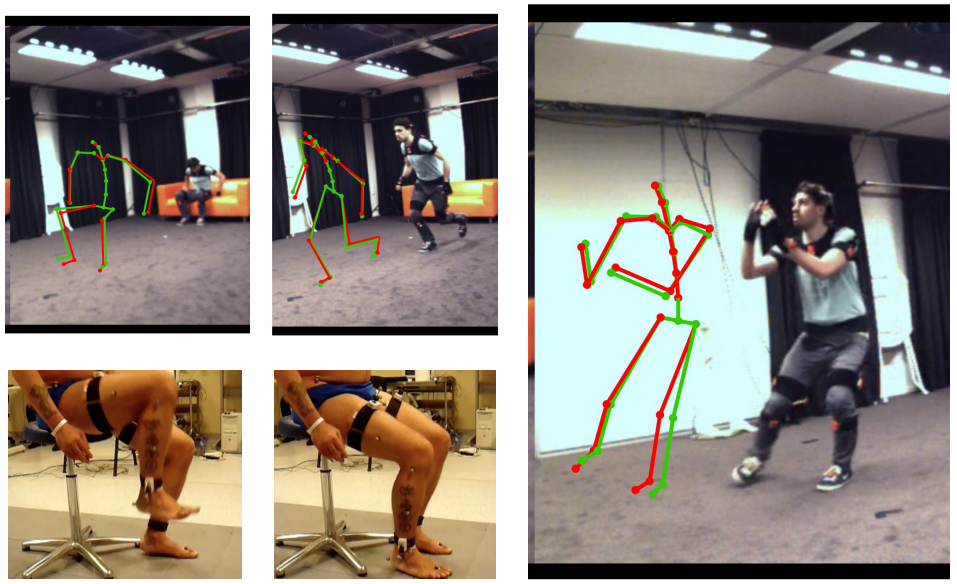
Affiliated with the Intel Galileo University Program

Body Sensor Networks

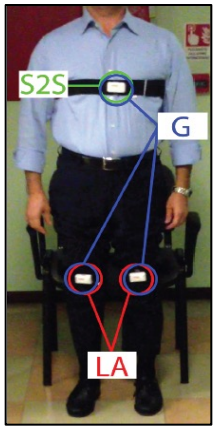
A Body Sensor Network (BSN) is a sensor network composed by nodes placed on a user body.The network nodes are, typically, devices supplied with inertial (i.e., accelerometers and gyroscopes) and magnetic (i.e., magnetometers) sensors.

Applications:

- motion capture
- posture recognition
- activity classification
- tele-rehabilitation (Parkinson's Disease)
- rehabilitation monitoring (post-stroke)



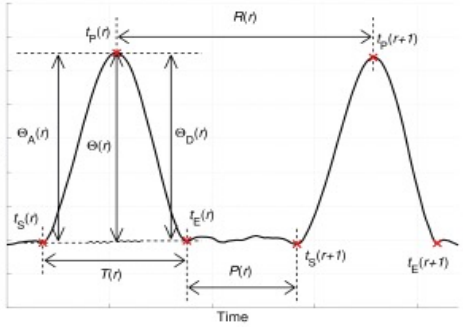
BSN-based Evaluation of PD Motor Tasks

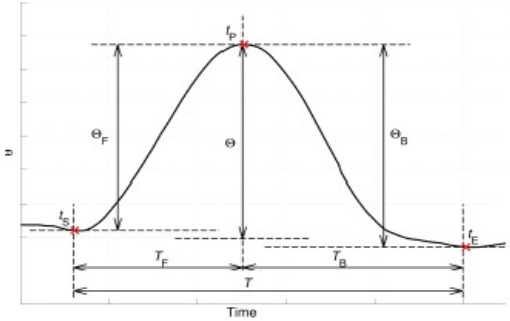


- **Main goal**: provide an automatic assessment of PD motor tasks
  - 3 UPDRS tasks: Leg Agility (LA), Sit-to-Stand (S2S), and Gait (G)
  - More objective and continuous evaluation of the symptoms
- **Hardware:** low-complexity BSN (3 IMUs - Shimmer)
- **Kinematic Characterization:** extraction of task-relevant features in both time and frequency domains

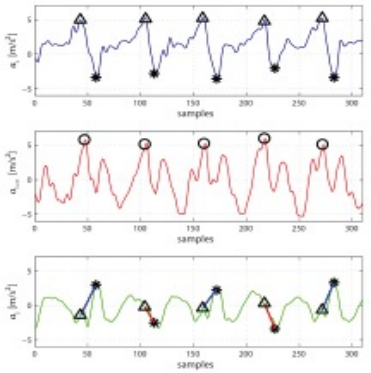
ISTITUTO AUXOLOGICO ITALIANO
Istituto di ricovero e cura a carattere scientifico

LA

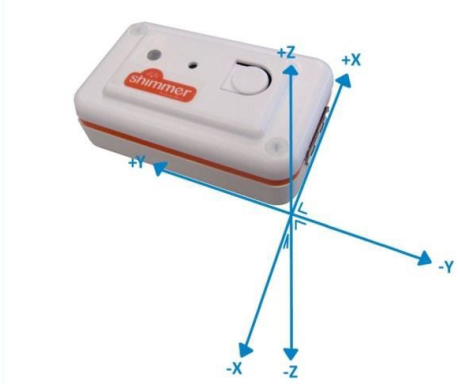




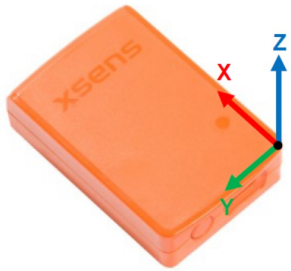
S2S



IMUs @ IoT Lab



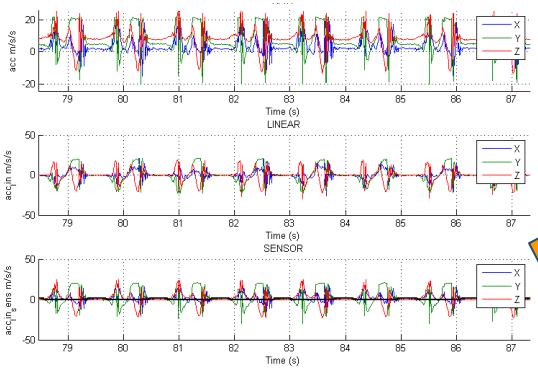
- Shimmer 2r node
- Small-sized, low-power, Bluetooth interface
- 9 DOF: 3-axial accelerometer, 3-axial gyroscope, 3-axial magnetometer
- Sampling Rate: 102.4 Hz

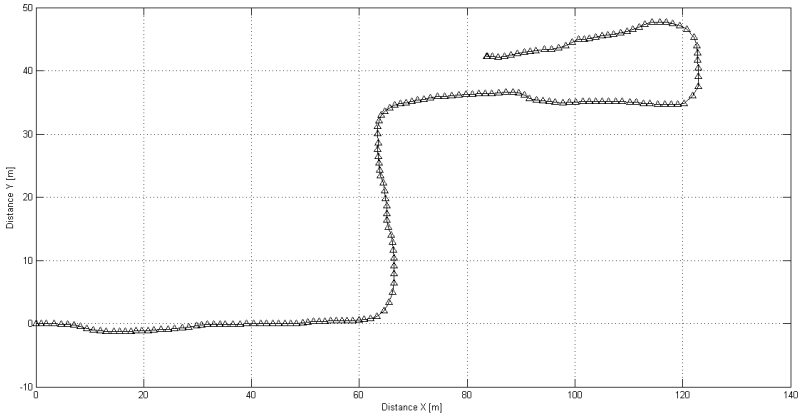


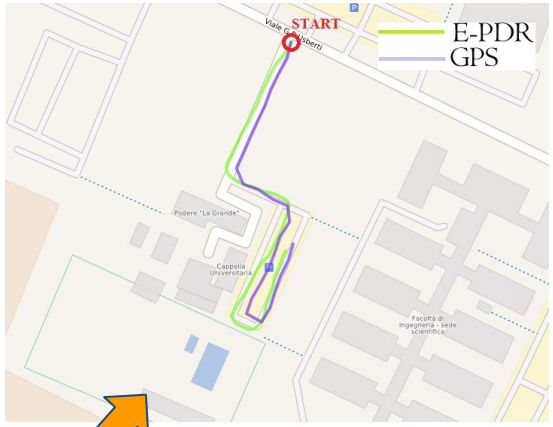
- 6 MTw Awinda IMU (accelerometer, gyroscope, magnetometer, barometer)
- Awinda Full Body straps set
- Awinda USB receiving dongle
- 6 slot Xsens docking station

- LPMS-B2
- Size: 39 x 39 x 8 mm
- Bluetooth Low Enery (LE) 4.1
- Communication distance < 20 m
- Orientation range Roll: ±180°; Pitch: ±90°; Yaw: ±180°
- Resolution < 0.01°
- Accuracy < 0. 5°(static), < 2° RMS (dynamic)
- Accelerometer 3 axes, ±2 / ±4 / ±8 / ±16 g, 16 bits
- Gyroscope 3 axes, ±125 / ±245 / ±500 / ±1000 / ±2000 °/s, 16 bits
- Magnetometer 3 axes, ± 4 / ± 8 / ± 12 / ± 16 gauss, 16 bits

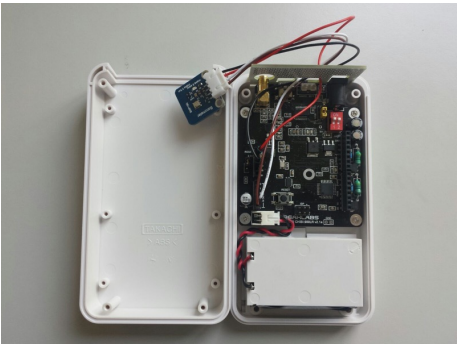
Inertial navigation and tracking: outdoor

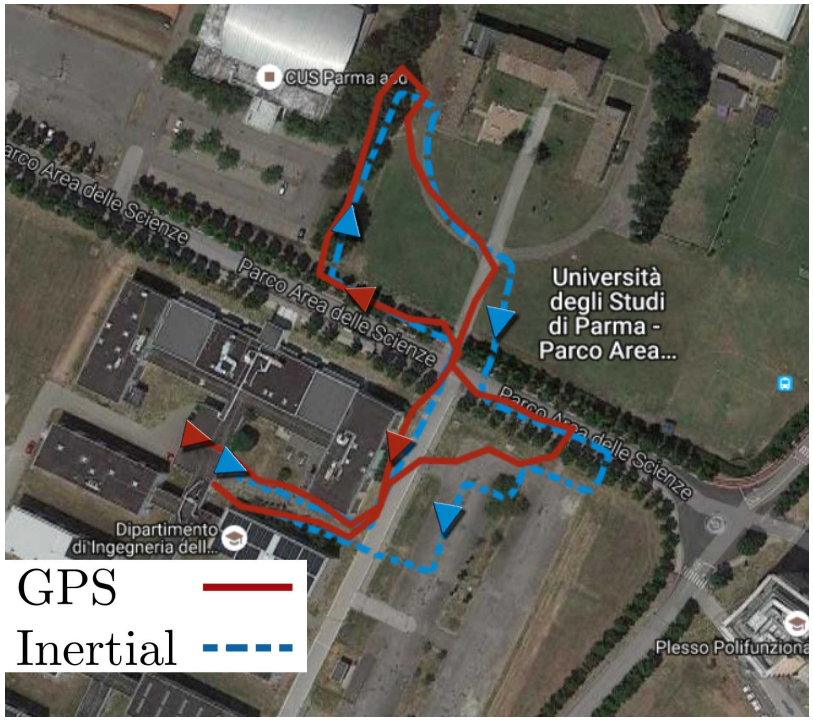






Evolutions of Inertial Navigation





- Hybrid inertial/barometric sensor → multi-floor navigation
- Navigation and tracking on non-flat surfaces (e.g., cross training)

A Demo (IPIN 2016, October 2016)

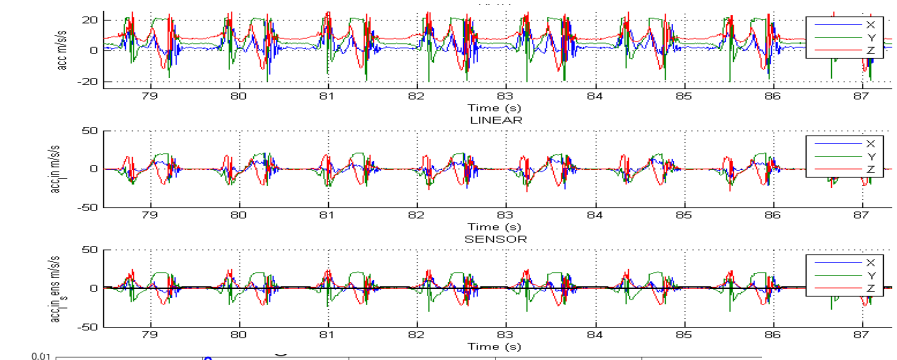
IoT Lab @ IPIN 2016
SEVENTH INTERNATIONAL CONFERENCE ON
INDOOR POSITIONING AND INDOOR NAVIGATIO
October 4-7, 2016 / Alcalá de Henares, Madrid, Spain

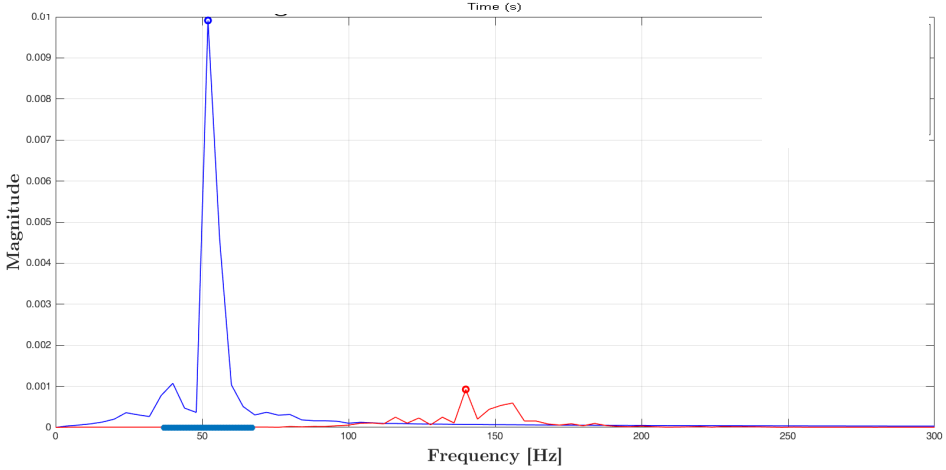
Track 2 Competition
3-rd Place Classified

- IPIN 2016 indoor localization competition (Alcalá de Henares, Madrid)

- IPIN tracks
  - Track 1: Smartphone-based
  - Track 2: Pedestrian dead reckoning positioning
  - Track 3: Smartphone-based (off-site)
  - Track 4: Indoor mobile robot positioning

Vibration Analysis



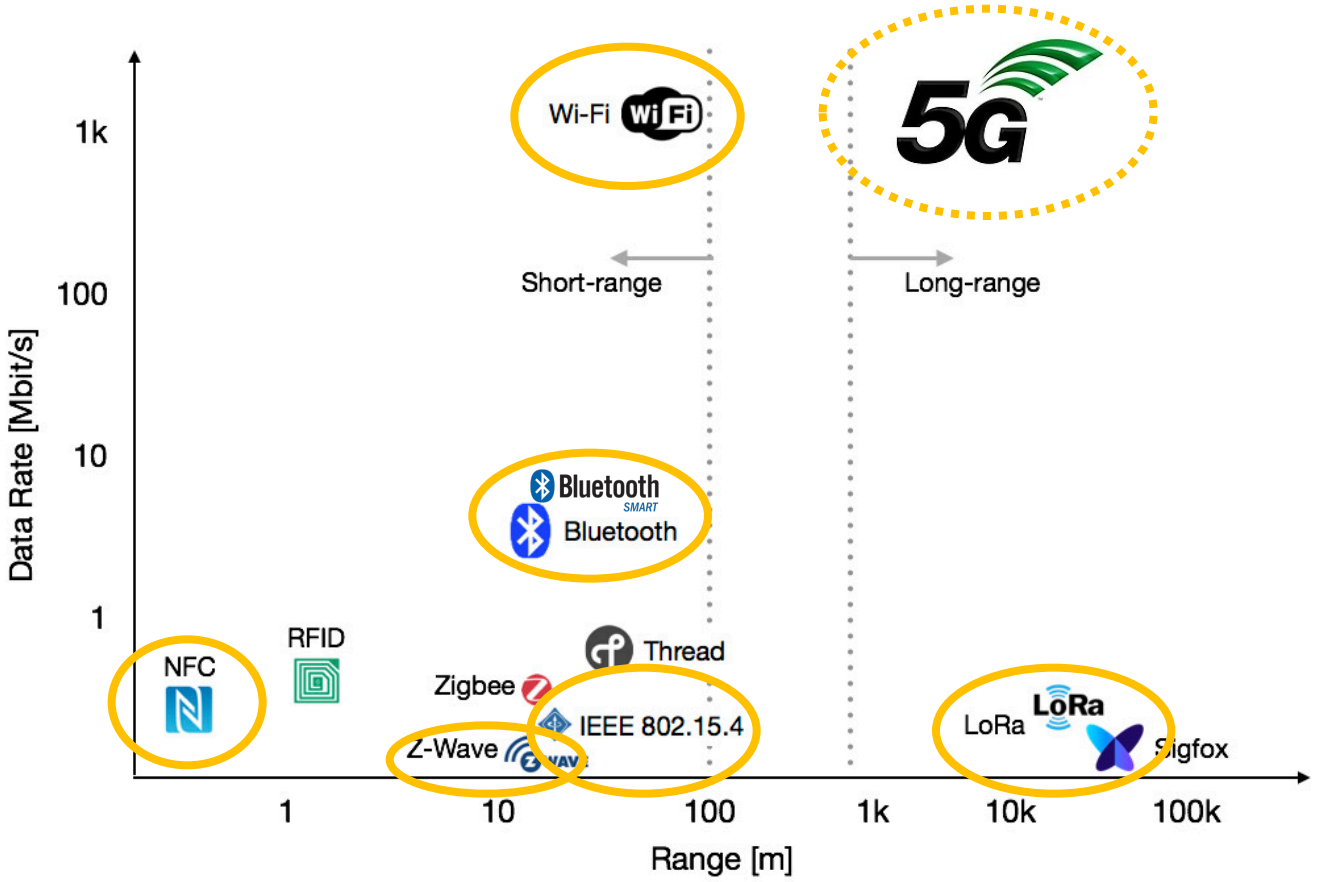


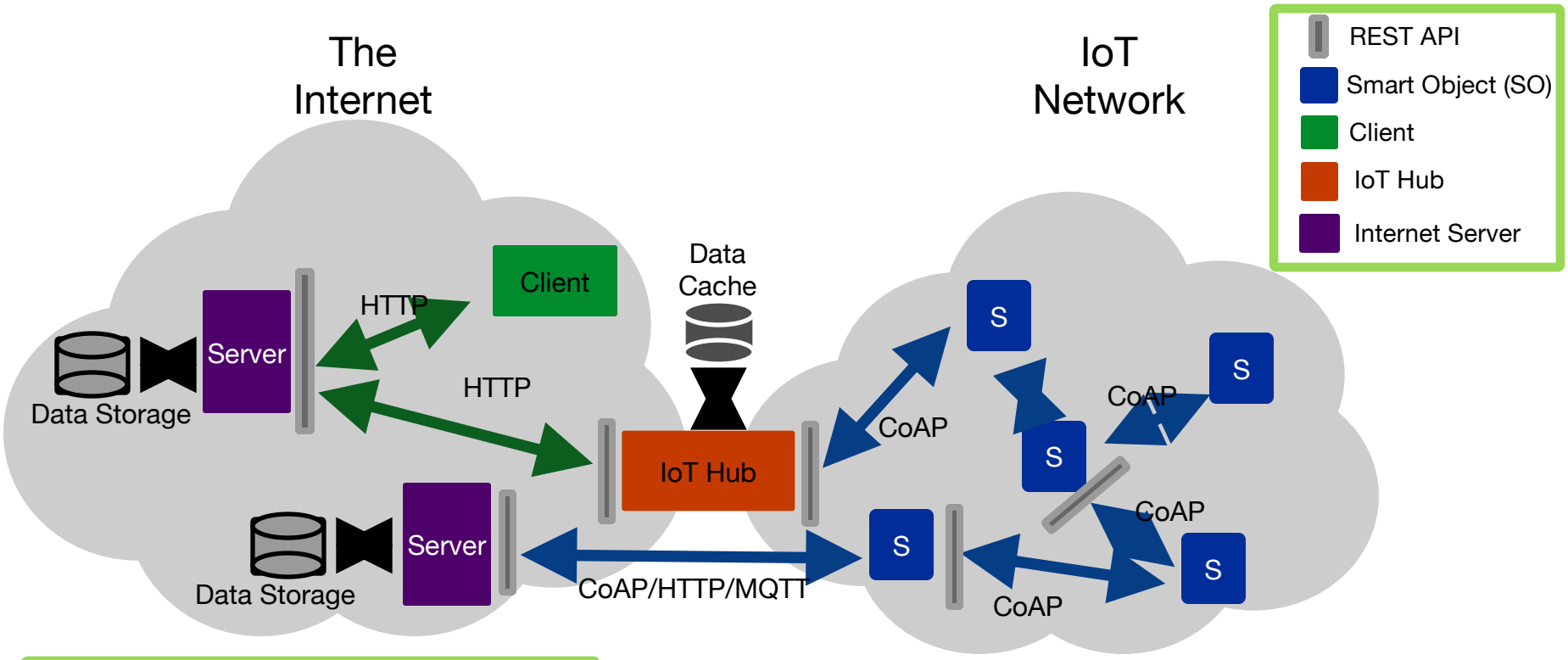
- Simple and non-invasive
- Combine an inertial sensor with an embedded system
- Extension: combine with audio sensor

Application: real-time RPM estimation

- Hybrid time/frequency processing

“Our” Wireless Communication Technologies

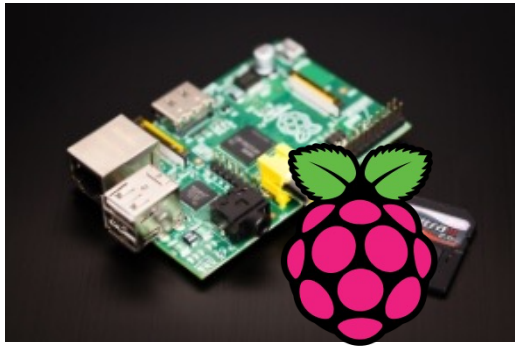




Data Storage, Processing, Aggregation and Distribution

Data Source/Stream, manage Incoming and Outgoing requests/responses SO

Infrastructure nodes



RaspberryPi

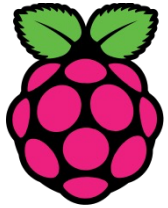
- 802.11
- Ethernet
- Camera

Arduino Yùn

- 802.11
- Ethernet

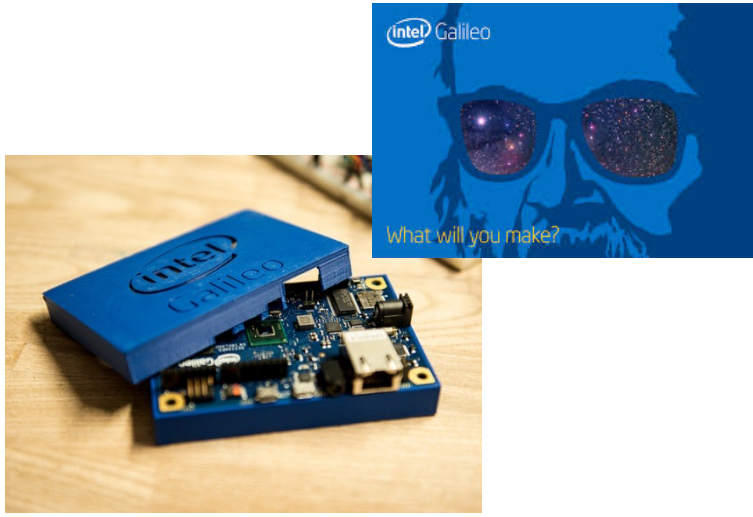
12 AP Cisco

- 802.11
- Ethernet



Gateway = RaspeberryPi & Telos B

- 802.15.4
- 802.11
- Ethernet



INTEL Galileo

- Ethernet
- WiFi (dongle)

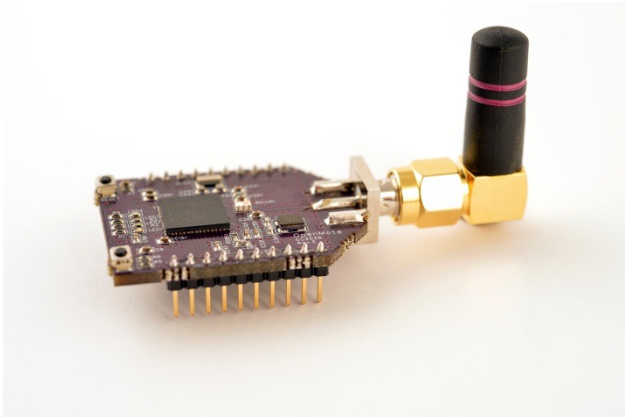
Mobile nodes

Telos B

- 802.15.4
- Contiki

OpenMote

- 802.15.4
- Contiki



Zolertia Z1

- 802.15.4
- Contiki

UNIVERSITÀ DI PARMA

IoT Lab

Transport Layer Protocols: UDP and TCP

Main sources

- J. P. Vasseur and A. Dunkels, `Interconnecting Smart Objects with IP,` Morgan Kaufmann, 2010. Chapters 2 and 6.

- Princeton (COS 461: Computer Networks, Spring 2006) and Virginia Tech (Internet Teaching Lab, http://www.cs.virginia.edu/~itlab/) lectures

Original TCP/IP Goals (70's)

- Internet communication must continue despite loss of networks or gateways (" in the presence of link or node failures " in today's terms).

- The Internet must support multiple types of communication services.

- Internet architecture must:
  - accommodate a variety of networks (" networks " means link and physical layers)
  - permit distributed management of its resources
  - be cost-effective
  - permit host attachment with little effort

- Resources used in the Internet architecture must be accountable .

Scalability of TCP/IP

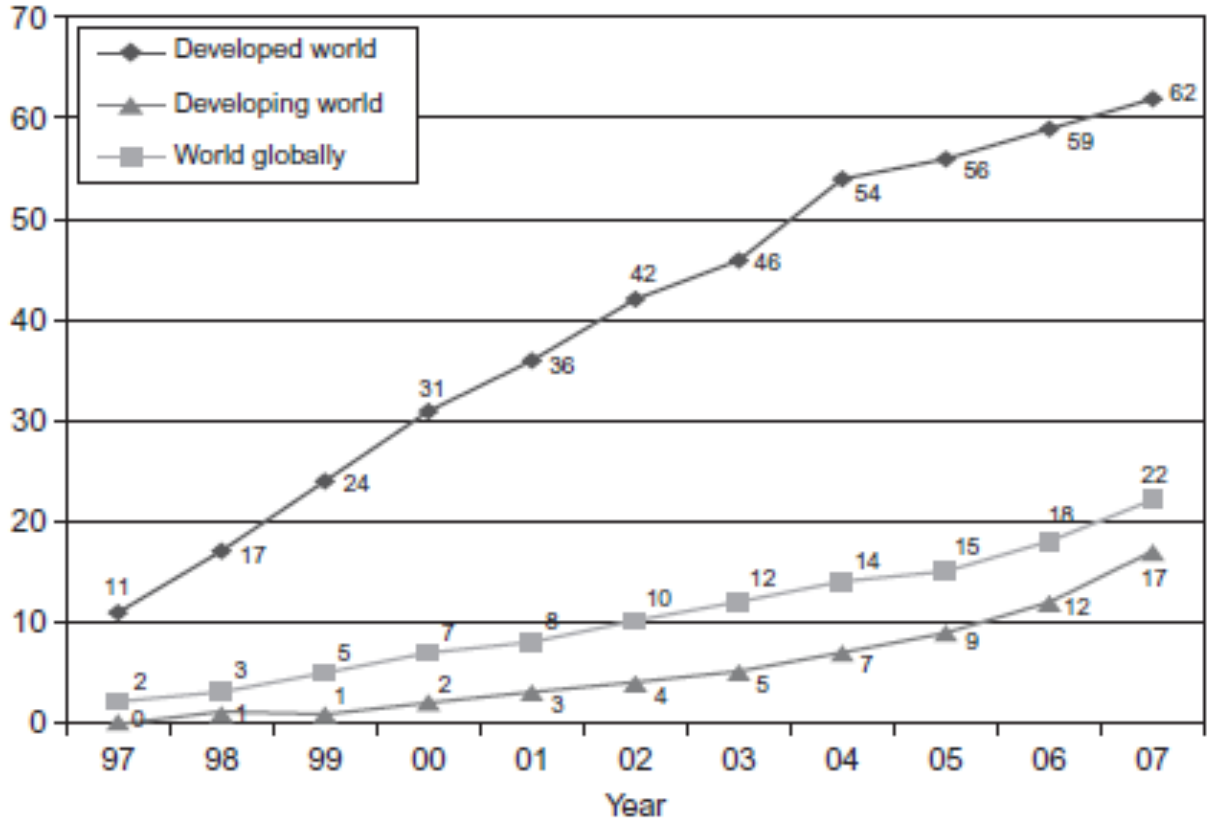


FIGURE 2.1

Level of penetration of the Internet regarding user number.

Protocol Stack (layers)

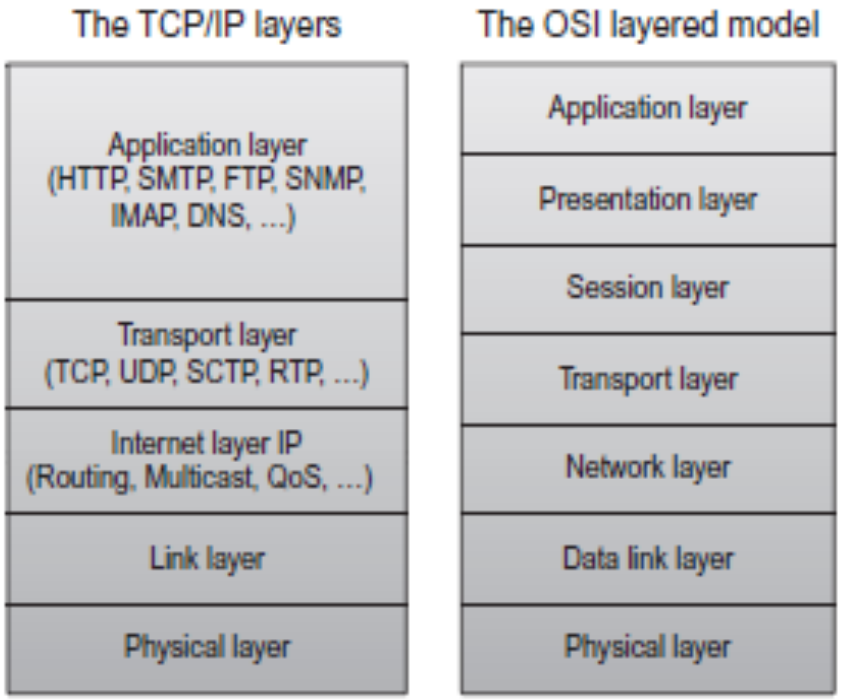


FIGURE 2.2

The concept of layering in the TCP/IP and OSI architectures.

Link Layer

- Usually refers to the physical and data link layers (the use of the PHY/MAC acronym is fairly common).

- Responsible for forwarding the IP packet on a link between two devices. This involves several functions such as: medium access control (MAC), error detection and (sometimes) retransmission, and flow control.

- Link layer protocol information is added in the form of a frame that carries the IP packet.

- Some links provide a very limited set of functions whereas others implement fairly sophisticated services that often include a link layer “ routing ” function.

Internet/Network Layer (IP)

- Responsible for providing an unreliable service for sending a packet between a source and a destination across the network, where host and routers are uniquely identified by their IP (IPv4 or IPv6) addresses, using a hierarchical addressing scheme.

- Routing is one of the main tasks accomplished by the IP layers

- Protocols such as ICMP (Internet Control Message Protocol) and IGMP (Internet Group Management Protocol) for multicast traffic are both considered part of the IP layer.

Transport Layer

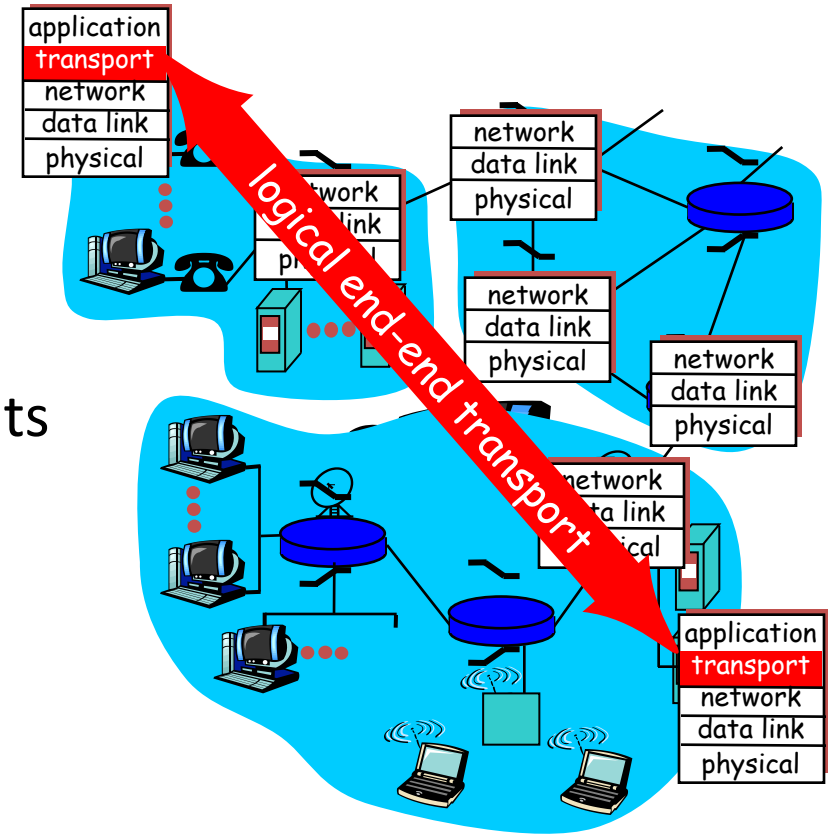
- Responsible for end-to-end communication between two devices.
- TCP (states are maintained at the end devices) provides a reliable transport mechanism with error detection and retransmission, flow control using dynamic windowing techniques, security mechanisms, and so forth.
- UDP (stateless) is mostly used for application addressing and optional error detection (done by an optional checksum in IPv4; note that the UDP checksum is mandatory in IPv6).
- Other transport protocols:
  - the message stream transport protocol SCTP (Stream Control Transmission Protocol) offers additional capabilities such as the bundling of multiple user messages in a single SCTP packet, the support of multi-homing, and so forth.
  - Real-time Transport Protocol (RTP) is another transport protocol designed for realtime applications such as streaming audio traffi c and video .

Application Layer

- Refers to higher level protocol(s) that supports the applications.
- The list of application layer protocols is fairly long, but few well known include:
  - File Transfer Protocol (FTP)
  - Trivial File Transfer Protocol (TFTP)
  - Simple Network Management Protocol (SNMP)
  - HypertextTransport Protocol (HTTP)
  - Network News Transfer Protocol (NNTP)
  - Telnet

Transport Protocols

- Provide *logical communication* between application processes running on different hosts
- Run on end hosts
  - Sender: breaks application messages into segments, and passes to network layer
  - Receiver: reassembles segments into messages, passes to application layer
- Multiple transport protocol available to applications
  - Internet: TCP and UDP



Internet Transport Protocols

- Datagram messaging service (UDP)
  - No-frills extension of “best-effort” IP
- Reliable, in-order delivery (TCP)
  - Connection set-up
  - Discarding of corrupted packets
  - Retransmission of lost packets
  - Flow control
  - Congestion control
- Other services not available
  - Delay guarantees
  - Bandwidth guarantees

Transport Protocols in the Internet

- The Internet supports 2 transport protocols

UDP - User Datagram Protocol

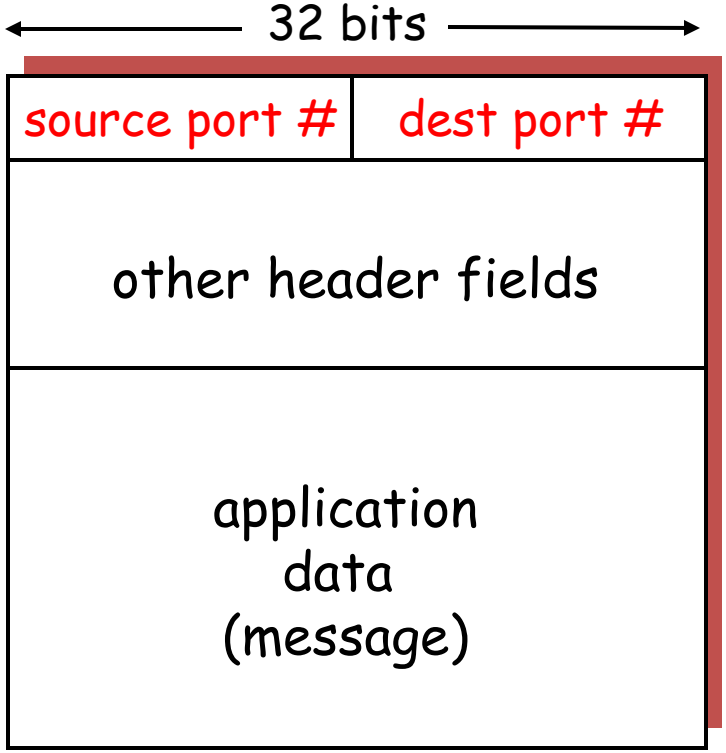
- datagram oriented
- unreliable, connectionless
- simple
- unicast and multicast
- useful only for few applications, e.g., multimedia applications
- used a lot for services
  - network management (SNMP, Simple Network Management Protocol), routing (Routing Information Protocol, RIP), naming (DNS), etc.

TCP - Transmission Control Protocol

- stream oriented
- reliable, connection-oriented
- complex
- only unicast
- used for most Internet applications:
  - web (http), email (smtp), file transfer (ftp), terminal (telnet), etc.

Multiplexing and Demultiplexing

- Host receives IP datagrams
  - Each datagram has source and destination IP addresses
  - Each datagram carries one transport-layer segment
  - Each segment has source and destination port number
- Host uses IP addresses and port numbers to direct the segment to appropriate socket



TCP/UDP segment format

User Datagram Protocol (UDP)

- UDP is the simplest protocol in the TCP/IP suite
- Lightweight communication between processes
  - Avoid overhead and delays of ordered, reliable delivery
  - Send messages to and receive them from a socket
- UDP in a nutshell:
  - IP plus port numbers to support (de)multiplexing
  - Optional error checking on the packet contents

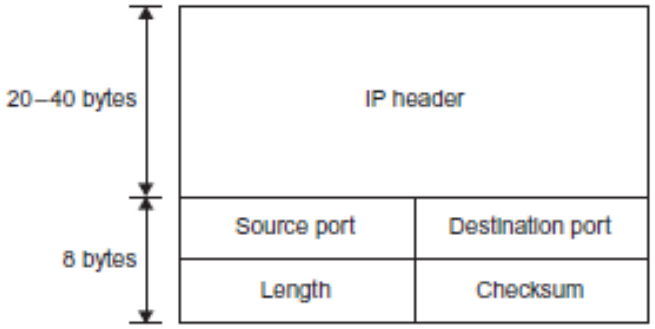


FIGURE 6.1

The UDP header consists of four fields: source port, destination port, a length field, and a checksum.

• **Port numbers** identify sending and receiving applications (processes). Maximum port number is 2^{16}-1= 65,535

• **Message Length** is at least 8 bytes (I.e., Data field can be empty) and at most 65,535

• **Checksum** is for header (of UDP and some of the IP header fields)

•A globally unique address at the transport layer (for both UDP and TCP) is a tuple **<IP address, port number>**

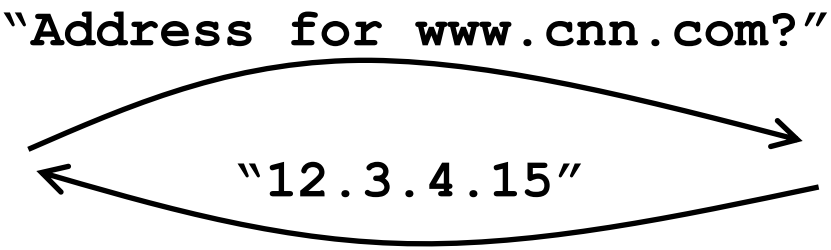
Why Would Anyone Use UDP?

- Finer control over what data is sent and when
  - As soon as an application process writes into the socket
  - … UDP will package the data and send the packet
- No delay for connection establishment
  - UDP just blasts away without any formal preliminaries
  - … which avoids introducing any unnecessary delays
- No connection state
  - No allocation of buffers, parameters, sequence #s, etc.
  - … making it easier to handle many active clients at once
- Small packet header overhead
  - UDP header is only eight-byte long
- **All excellent reasons for IoT scenarios**

Popular Applications That Use UDP

- Multimedia streaming
  - Retransmitting lost/corrupted packets is not worthwhile
  - By the time the packet is retransmitted, it's too late
  - E.g., telephone calls, video conferencing, gaming

- Simple query protocols like Domain Name System
  - Overhead of connection establishment is overkill
  - Easier to have application retransmit if needed



Transmission Control Protocol (TCP)

- TCP provides a reliable byte stream on top of the best-effort packet service provided by the IP layer
- TCP hides the packet-oriented IP network beneath a virtual circuit abstraction, in which each virtual circuit is called a connection
- TCP provides a reliable stream transport service on top of the best-effort IP layer with three mechanisms:
  - Acknowledgments and retransmissions: All data sent with TCP are acknowledged by the receiver.
  - Sequence numbers: Every byte in the TCP byte stream is given a sequence number. The sequence numbers are used when matching acknowledgments with the corresponding data.
  - Sliding window (flow control): The receiver advertises how many bytes it is currently able to receive and the sender sends only as much data as the receiver can receive. As the receiver receives the data, it is able to receive more data.

An Analogy: Talking on a Cell Phone

- Alice and Bob on their cell phones
  - Both Alice and Bob are talking
- What if Alice couldn't understand Bob?
  - Bob asks Alice to repeat what she said
- What if Bob hasn't heard Alice for a while?
  - Is Alice just being quiet?
  - Or, have Bob and Alice lost reception?
  - How long should Bob just keep on talking?
  - Maybe Alice should periodically say "uh huh"
  - … or Bob should ask "Can you hear me now?" ☺

Some Take-Aways from the Example

- Acknowledgments from receiver
  - Positive: “okay” or “ACK”
  - Negative: “please repeat that” or “NACK”
- Timeout by the sender (“stop and wait”)
  - Don’t wait indefinitely without receiving some response
  - … whether a positive or a negative acknowledgment
- Retransmission by the sender
  - After receiving a “NACK” from the receiver
  - After receiving no feedback from the receiver

Segmented TCP Stream

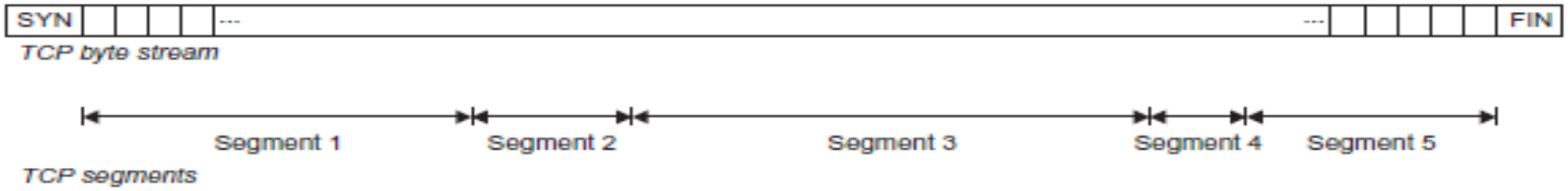
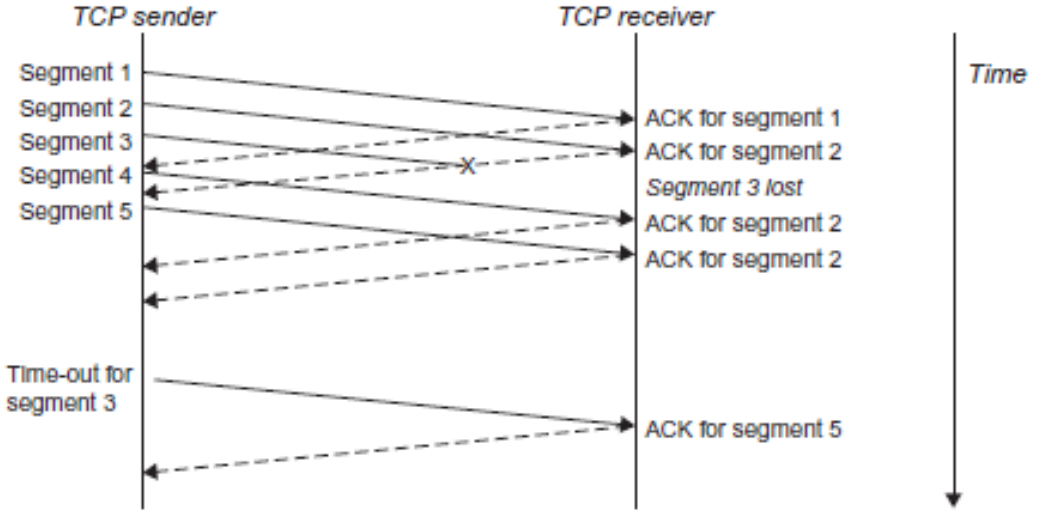


FIGURE 6.2

A segmented TCP byte stream.



(Fast retransmission: see p. 124)

FIGURE 6.4

Loss of a TCP segment and the corresponding time-out.

Challenges of Reliable Data Transfer

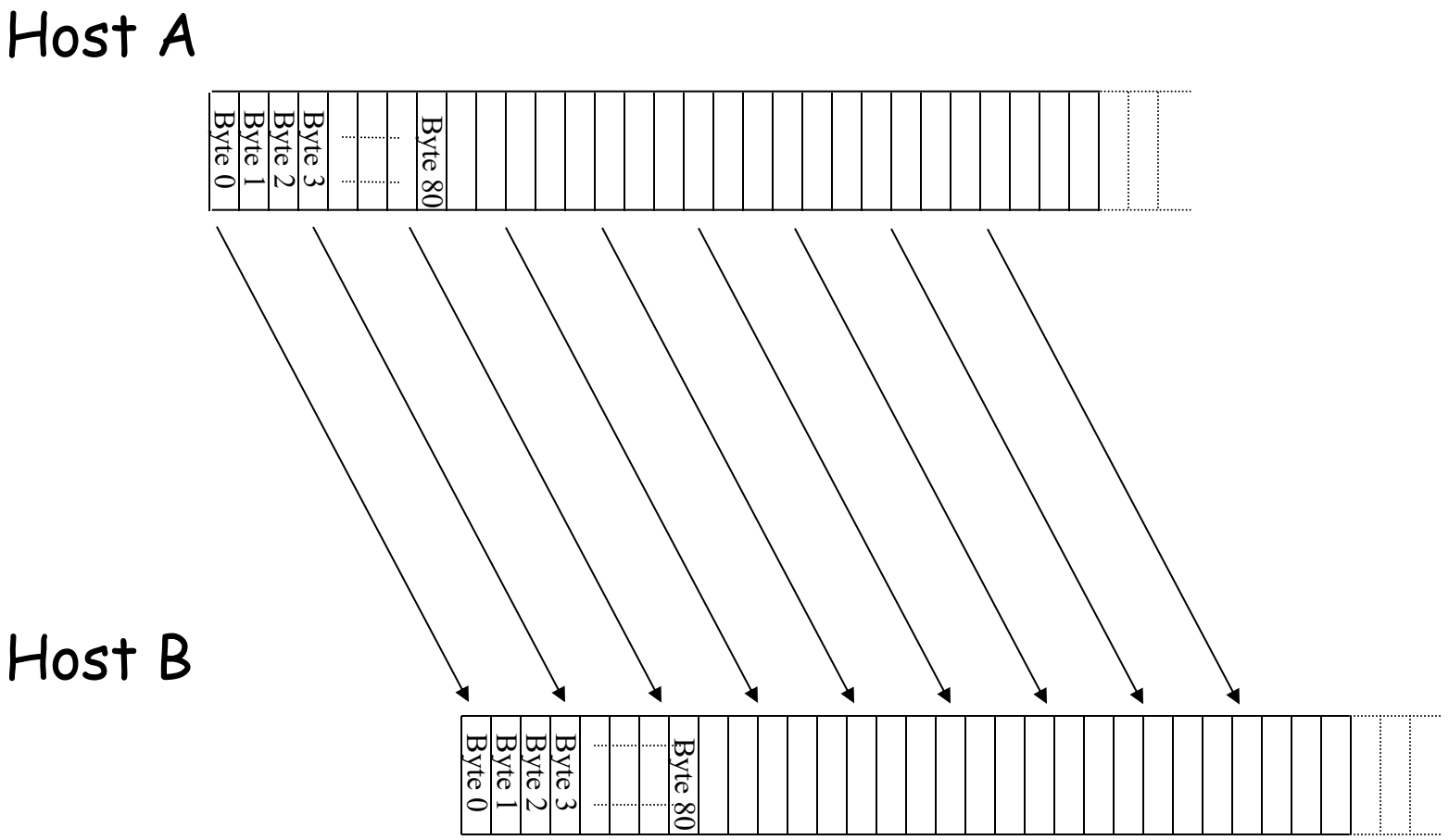
- Over a perfectly reliable channel
  - All of the data arrives in order, just as it was sent
  - Simple: sender sends data, and receiver receives data
- Over a channel with bit errors
  - All of the data arrives in order, but some bits corrupted
  - Receiver detects errors and says “please repeat that”
  - Sender retransmits the data that were corrupted
- Over a lossy channel with bit errors
  - Some data are missing and some bits are corrupted
  - Receiver detects errors but cannot always detect loss
  - Sender must wait for acknowledgment (“ACK” or “OK”)
  - … and retransmit data after some time if no ACK arrives

TCP Support for Reliable Delivery

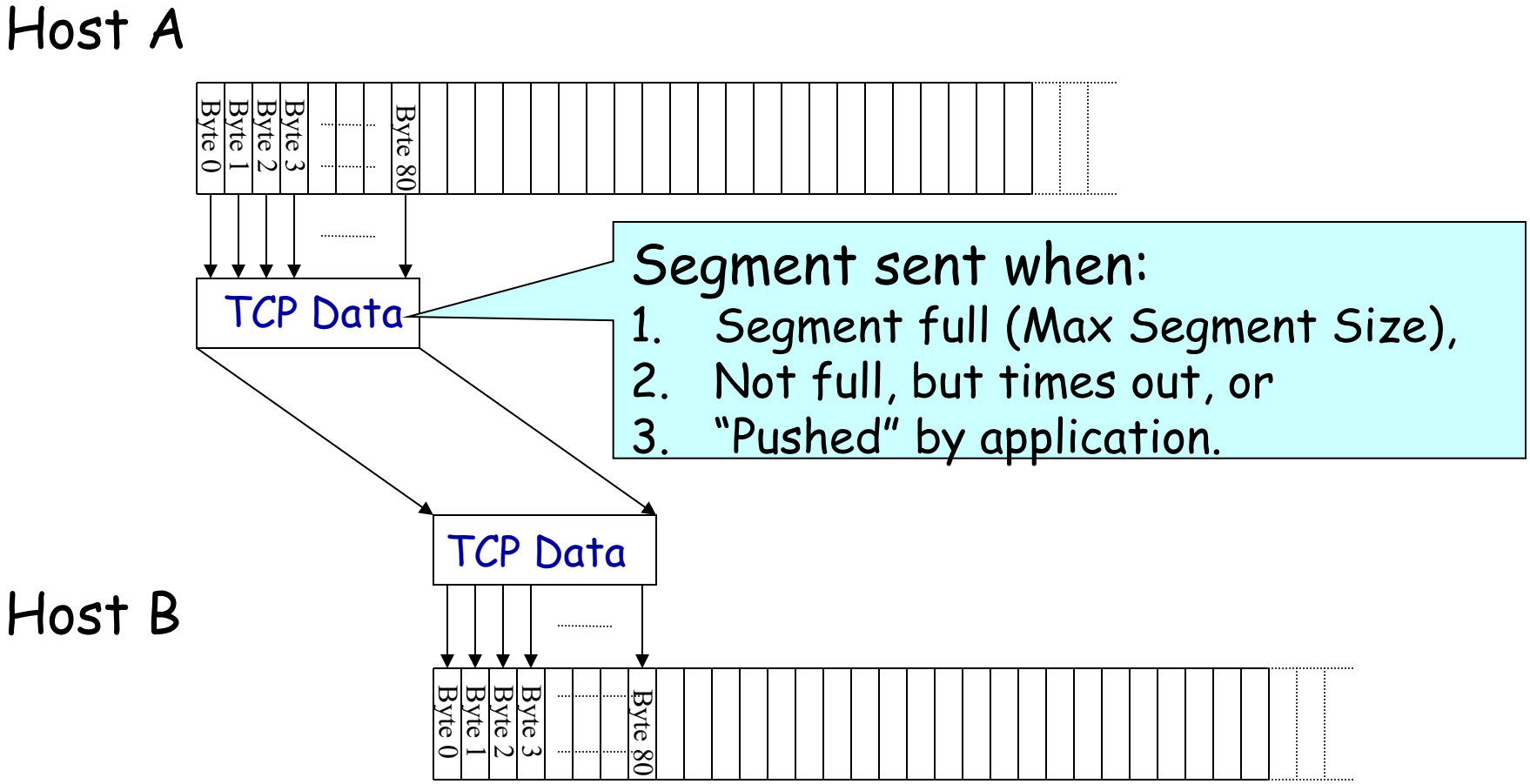
- Checksum
  - Used to detect corrupted data at the receiver
  - …leading the receiver to drop the packet
- Sequence numbers
  - Used to detect missing data
  - … and for putting the data back in order
- Retransmission
  - Sender retransmits lost or corrupted data
  - Timeout based on estimates of round-trip time
  - Fast retransmit algorithm for rapid retransmission

TCP Segments

TCP "Stream of Bytes" Service



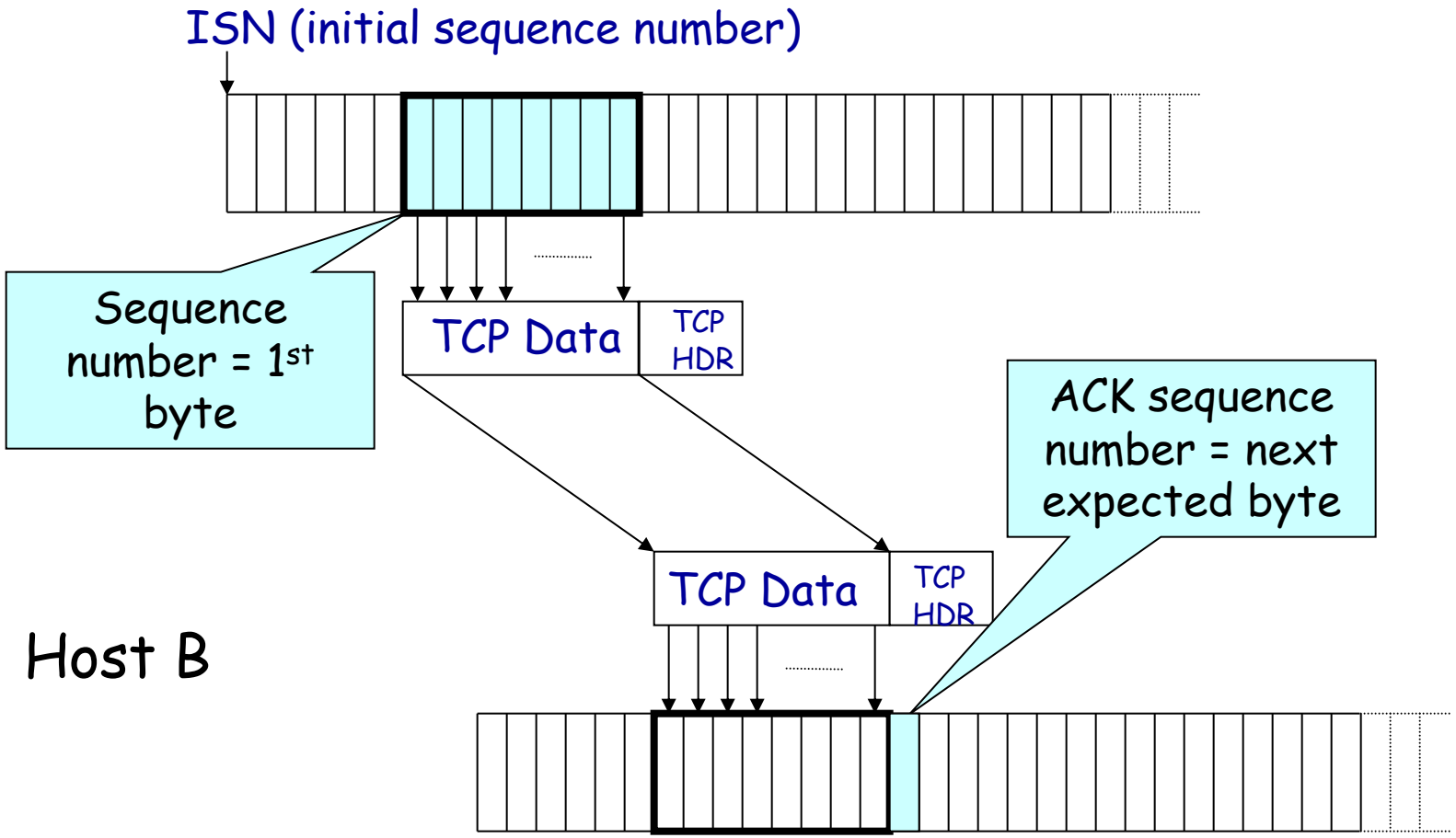
…Emulated Using TCP "Segments"



TCP Segment

- IP packet
  - No bigger than Maximum Transmission Unit (MTU)
  - E.g., up to 1500 bytes on an Ethernet
- TCP packet
  - IP packet with a TCP header and data inside
  - TCP header is typically 20 bytes long
- TCP segment
  - No more than Maximum Segment Size (MSS) bytes
  - E.g., up to 1460 consecutive bytes from the stream

Sequence Numbers

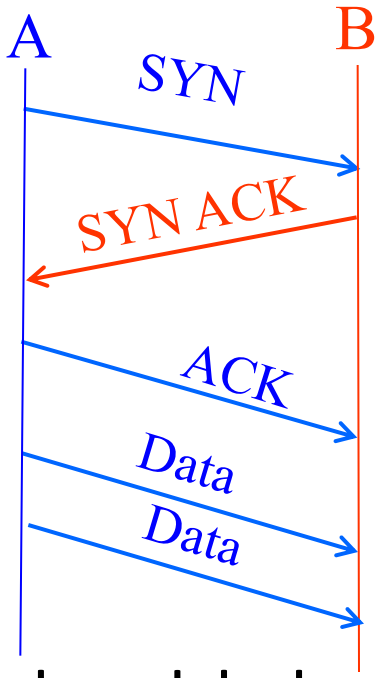


Initial Sequence Number (ISN)

- Sequence number for the very first byte
  - E.g., Why not a de facto ISN of 0?
- Practical issue
  - IP addresses and port #s uniquely identify a connection
  - Eventually, though, these port #s do get used again
  - … and there is a chance an old packet is still in flight
  - … and might be associated with the new connection
- So, TCP requires changing the ISN over time
  - Set from a 32-bit clock that ticks every 4 microseconds
  - … which only wraps around once every 4.55 hours!
- But, this means the hosts need to exchange ISNs

TCP Three-Way Handshake

Establishing a TCP Connection



Each host tells its ISN to the other host.

- Three-way handshake to establish connection
  - Host A sends a **SYN** (open) to the host B
  - Host B returns a SYN acknowledgment (**SYN ACK**)
  - Host A sends an **ACK** to acknowledge the SYN ACK

TCP Header (20 bytes)

Flags: SYN
FIN
RST
PSH
URG
ACK

Source port			Destination port
Sequence number			
Acknowledgment			
HdrLen	0	Flags	Advertised window
Checksum			Urgent pointer
Options (variable)			
Data			

Step 1: A's Initial SYN Packet

Flags: SYN
FIN
RST
PSH
URG
ACK

A's port			B's port
A's Initial Sequence Number			
Acknowledgment			
20	0	Flags	Advertised window
Checksum			Urgent pointer
Options (variable)			

A tells B it wants to open a connection…

Step 2: B's SYN-ACK Packet

Flags: SYN
FIN
RST
PSH
URG
ACK

B's port			A's port
B's Initial Sequence Number			
A's ISN plus 1			
20	0	Flags	Advertised window
Checksum			Urgent pointer
Options (variable)			

B tells A it accepts, and is ready to hear the next byte…

… upon receiving this packet, A can start sending data

Step 3: A's ACK of the SYN-ACK

Flags: SYN
FIN
RST
PSH
URG
ACK

A's port			B's port
Sequence number			
B's ISN plus 1			
20	0	Flags	Advertised window
Checksum			Urgent pointer
Options (variable)			

A tells B it wants is okay to start sending

… upon receiving this packet, B can start sending data

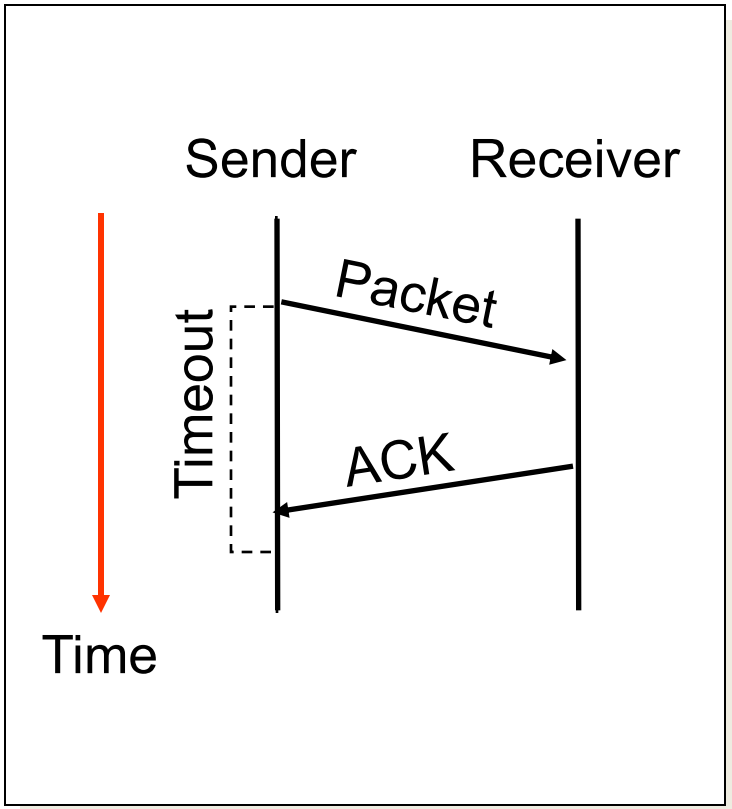
What if the SYN Packet Gets Lost?

- Suppose the SYN packet gets lost
  - Packet is lost inside the network, or
  - Server rejects the packet (e.g., listen queue is full)
- Eventually, no SYN-ACK arrives
  - Sender sets a timer and wait for the SYN-ACK
  - … and retransmits the SYN-ACK if needed
- How should the TCP sender set the timer?
  - Sender has no idea how far away the receiver is
  - Hard to guess a reasonable length of time to wait
  - Some TCPs use a default of 3 or 6 seconds

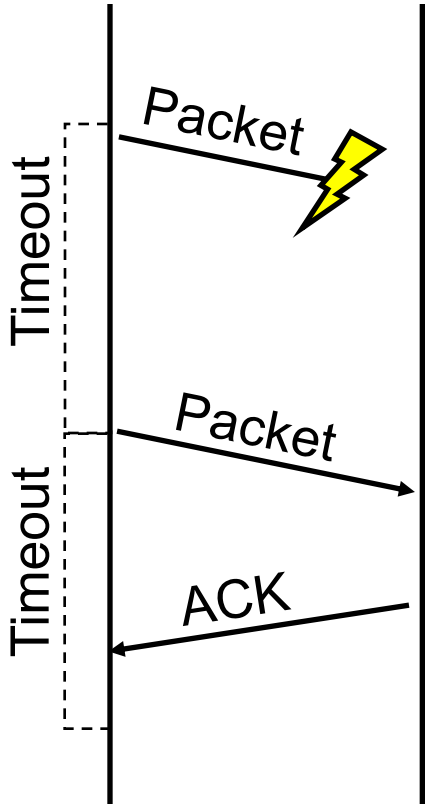
TCP Retransmissions

Automatic Repeat reQuest (ARQ)

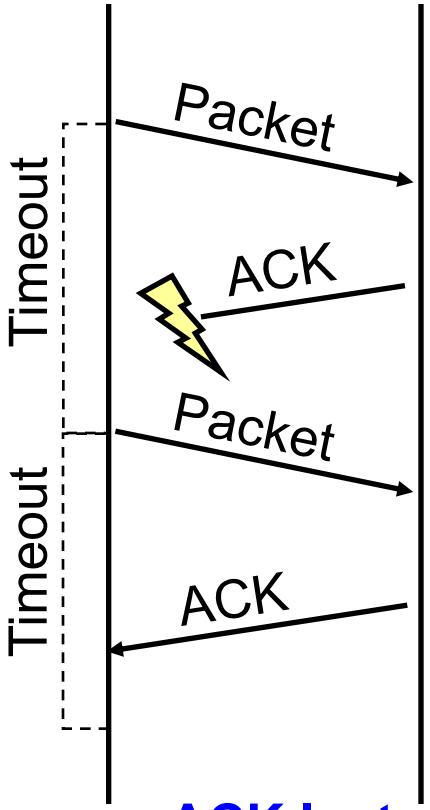
- Automatic Repeat Request
  - Receiver sends acknowledgment (ACK) when it receives packet
  - Sender waits for ACK and timeouts if it does not arrive within some time period
- Simplest ARQ protocol
  - Stop and wait
  - Send a packet, stop and wait until ACK arrives



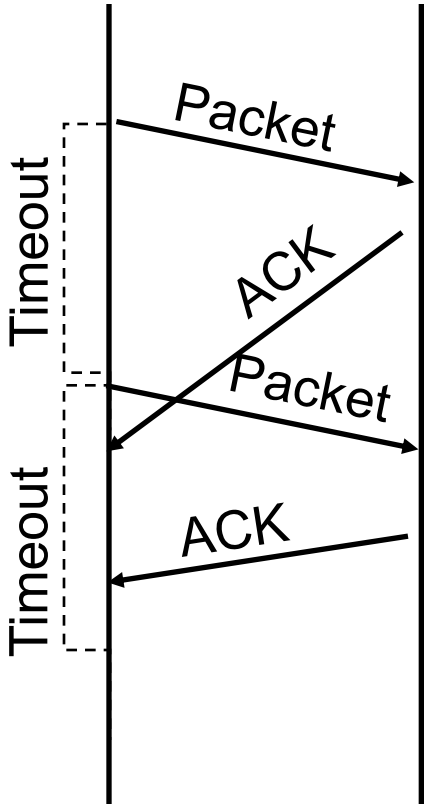
Reasons for Retransmission



Packet lost



ACK lost
DUPLICATE PACKET

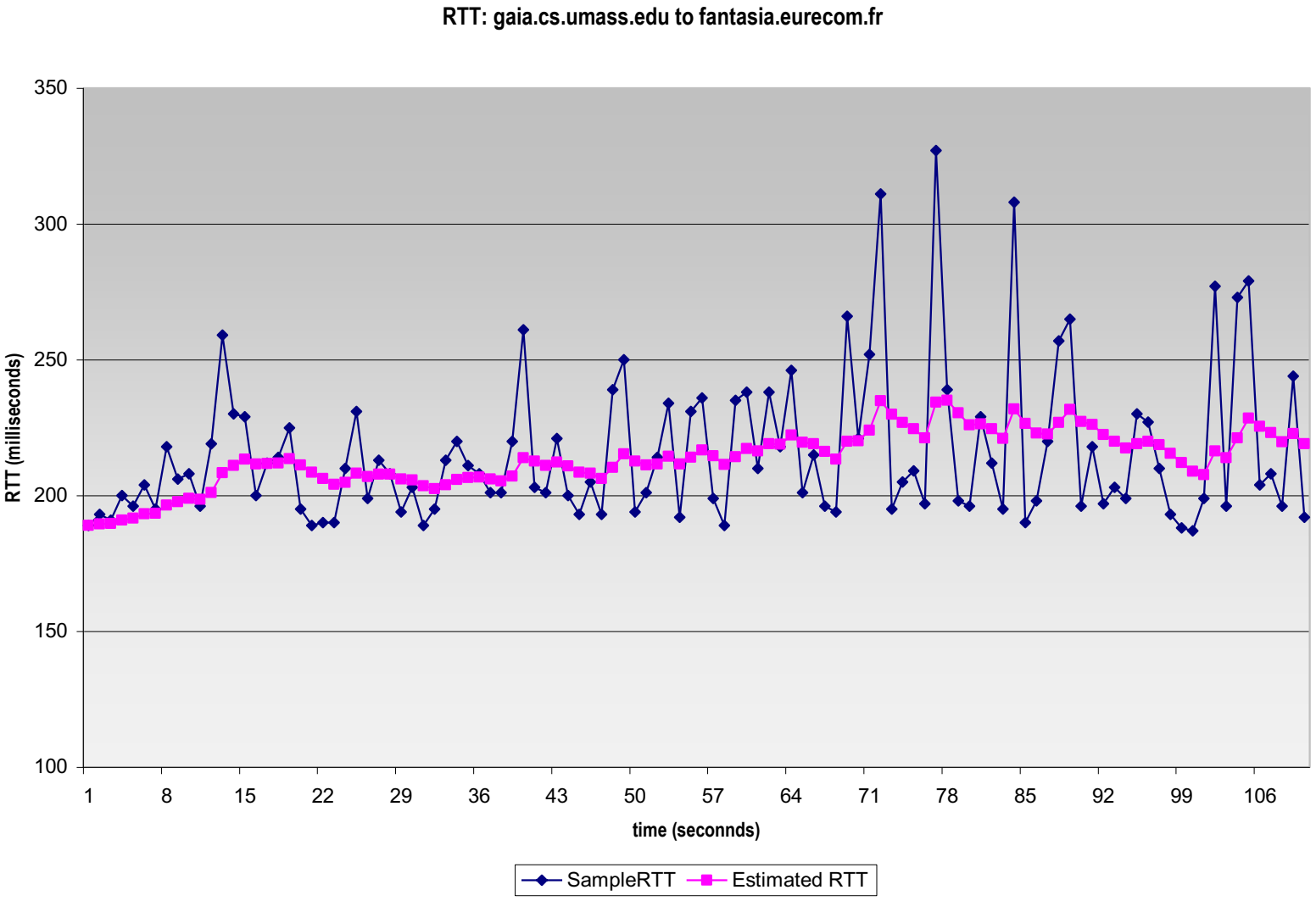


Early timeout
DUPLICATE PACKETS

How Long Should Sender Wait?

- Sender sets a timeout to wait for an ACK
  - Too short: wasted retransmissions
  - Too long: excessive delays when packet lost
- TCP sets timeout as a function of the RTT (Round Trip Time)
  - Expect ACK to arrive after an RTT
  - … plus a fudge factor to account for queuing
- But, how does the sender know the RTT?
  - Can estimate the RTT by watching the ACKs
  - Smooth estimate: keep a running average of the RTT
    - EstimatedRTT = a * EstimatedRTT + (1 –a) * SampleRTT
  - Compute timeout: TimeOut = 2 * EstimatedRTT

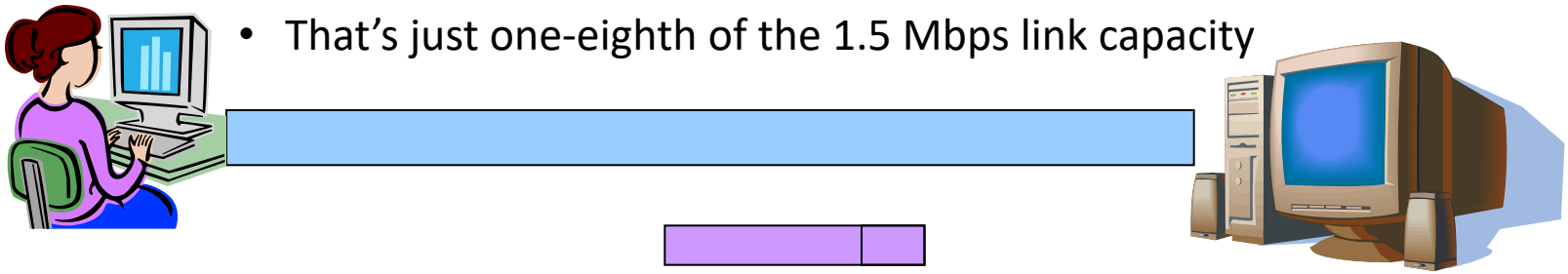
Example RTT Estimation



TCP Sliding Window

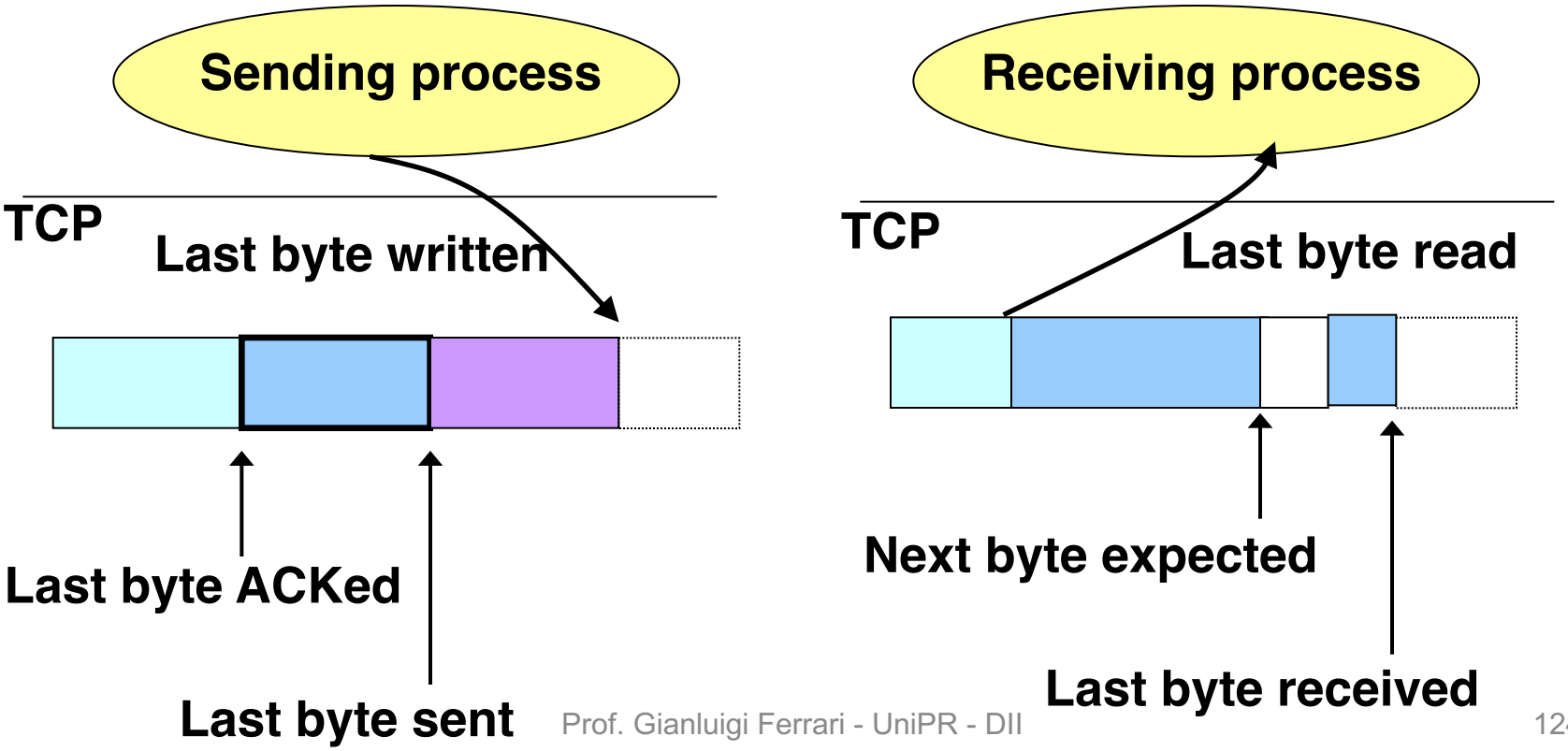
Motivation for Sliding Window

- Stop-and-wait is inefficient
  - Only one TCP segment is “in flight” at a time
  - Especially bad when delay-bandwidth product is high
- Numerical example
  - 1.5 Mbps link with a 45 msec round-trip time (RTT)
    - Delay-bandwidth product is 67.5 Kbits (or 8 KBytes)
  - But, sender can send at most one packet per RTT
    - Assuming a segment size of 1 KB (8 Kbits)
    - … leads to 8 Kbits/segment / 45 msec/segment ➔ 182 Kbps
    - That’s just one-eighth of the 1.5 Mbps link capacity



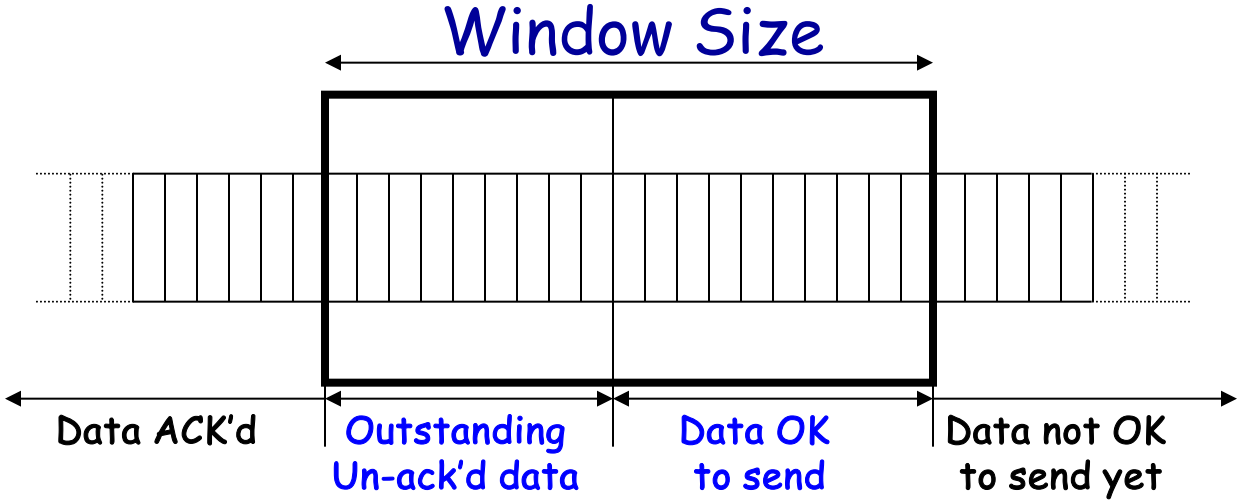
Sliding Window

- Allow a larger amount of data “in flight”
  - Allow sender to get ahead of the receiver
  - … though not *too far* ahead



Receiver Buffering

- Window size
  - Amount that can be sent without acknowledgment
  - Receiver needs to be able to store this amount of data
- Receiver advertises the window to the receiver
  - Tells the receiver the amount of free space left
  - … and the sender agrees not to exceed this amount



TCP Header for Receiver Buffering

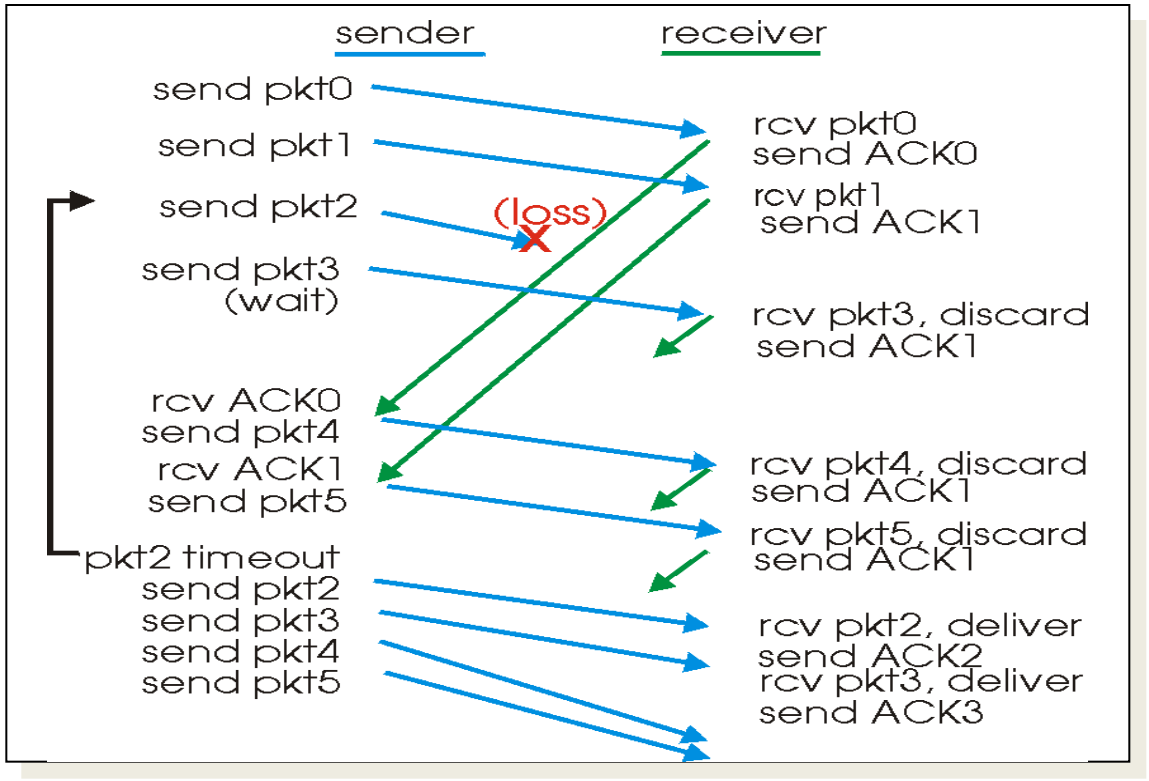
Flags: SYN
FIN
RST
PSH
URG
ACK

Source port			Destination port
Sequence number			
Acknowledgment			
HdrLen	0	Flags	**Advertised window**
Checksum			Urgent pointer
Options (variable)			
Data			

Fast Retransmission

Timeout is Inefficient

- Timeout-based retransmission
  - Sender transmits a packet and waits until timer expires
  - … and then retransmits from the lost packet onward



Fast Retransmission

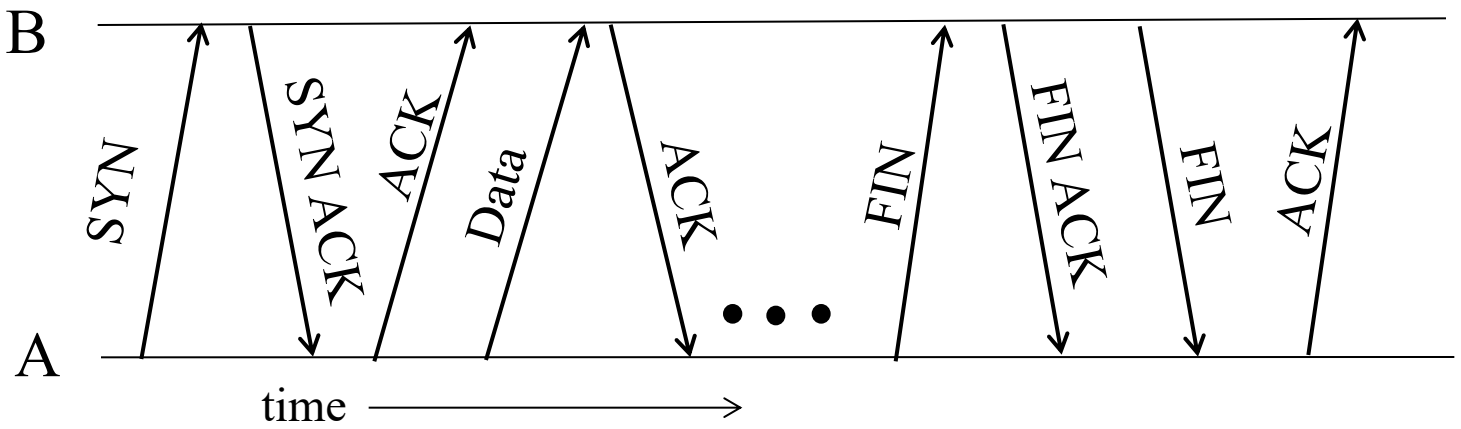
- Better solution possible under sliding window
  - Although packet n might have been lost
  - … packets n+1, n+2, and so on might get through
- Idea: have the receiver send ACK packets
  - ACK says that receiver is still awaiting n^{th} packet
    - And *repeated* ACKs suggest later packets have arrived
  - Sender can view the "duplicate ACKs" as an early hint
    - … that the n^{th} packet must have been lost
    - … and perform the retransmission early
- Fast retransmission
  - Sender retransmits data after the triple duplicate ACK

Effectiveness of Fast Retransmit

- When does Fast Retransmit work best?
  - Long data transfers
    - High likelihood of many packets in flight
  - Large window size
    - Support for many packets in flight
  - Low burstiness in packet losses (only one pck lost at a time)
    - Higher likelihood that later packets arrive successfully
- Implications for Web traffic
  - Most Web transfers are short (e.g., 10 packets)
    - Short HTML files or small images
  - So, often there aren't many packets in flight
  - … making fast retransmit less likely to "kick in"
  - Forcing users to like "reload" more often… ☺

Tearing Down the Connection

Tearing Down the Connection



- Closing the connection
  - Finish (FIN) to close and receive remaining bytes
  - And other host sends a FIN ACK to acknowledge
  - Reset (RST) to close and not receive remaining bytes

Sending/Receiving the FIN Packet

- Sending a FIN: close()
  - Process is done sending data via the socket
  - Process invokes “close()” to close the socket
  - Once TCP has sent all of the outstanding bytes…
  - … then TCP sends a FIN
- Receiving a FIN: EOF
  - Process is reading data from the socket
  - Eventually, the attempt to read returns an EOF

TCP State Diagram

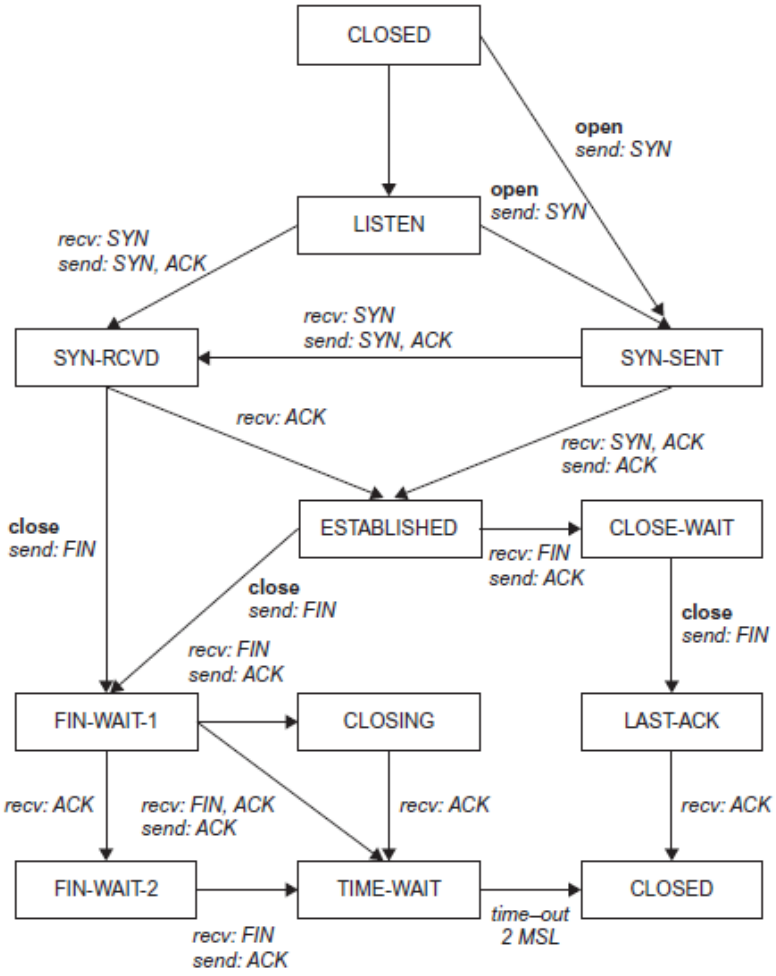


FIGURE 6.6

TCP state diagram.

UDP for Smart Objects

- Advantages
  - Very low overhead (header size and protocol logic) → low energy consumption; more room in each packet for application layer data; memory footprint
  - Simplicity of UDP fits well with several applications (e.g., temperature monitoring: losing a reading is not crucial) → in general, low reliability demands
  - Well suited to applications which require their own routing (application overlay mechanism)
  - UDP maps well to underlying multicast mechanisms
- Drawbacks
  - Loss of packets and no recovery mechanism → application has to recover (increase complexity)
  - Smart object networks have small packet sizes and UDP does not provide mechanisms to split their data → application layer decides proper packet size

TCP for Smart Objects

- Advantages
  - Smart object networks are typically lossy → reliable transport
  - TCP has problems in high-throughput applications over wireless links (not designed for wireless links), but high throughput is typically not relevant
  - Smart object networks interact with existing systems using TCP
  - TCP MSS (Maximum Segment Size) option is very useful (e.g., for 802.15.4 packets)
  - Core of TCP is simple (many of the complex mechanisms, e.g., sliding window and congestion control, aim at achieving high throughput)
  - Even though TCP header is larger than UDP header, it can be compressed
- Drawbacks
  - Reduced TCP implementations for smart objects do not achieve high throughput (as full-blown TCP stack): (i) no sliding window mechanism (only one TCP packet active at a time); (ii) delayed ACKs reduce throughput

UNIVERSITÀ DI PARMA

IoT Lab

IPv6

Main sources

- Slides by Prof. Luca Veltri (see following slides)
- J. P. Vasseur and A. Dunkels, `Interconnecting Smart Objects with IP,` Morgan Kaufmann, 2010. Chapters 6 and 15.
- Ansar Pasha and Patrick Grossetete, “IPv6 @ Cisco”

UNIVERSITA' DEGLI STUDI DI PARMA
Dipartimento di Ingegneria dell'Informazione

IPv6

Luca Veltri

(mail.to: luca.veltri@unipr.it)

Course of Internet of Things, 2015/2016

IPv4

- RFC 791 del 1981
- More or less the same in 30 years
- Main issues:
  - **Limited addressing space**
  - **Growing of Routing Tables**
  - **Host configuration**
  - **Security**
  - **Quality of service**
  - **Mobility**

IPv4 issues: Addressing

- IPv4 addresses: 32-bit
  - **4,294,967,296 IPv4 "theoretical" addresses, but the exact number is far below**
- Depletion of IPv4 address
  - **early predictions (made at the time IPv6 was in its early phase of design, end of 90's): IPv4 address depletion as early as 2002**
- Mainly due to:
  - **growing countries (e.g. China)**
  - **Mobile phones**
  - **Always-on devices**
  - **IoT**

IPv4 issues: Addressing

- Several mitigation strategies were developed:
  - **better address allocation and subnetting**
  - **Classless inter-domain routing (CIDR)**
    - it also reduces the routing table sizes
  - **Network Address Translation (NAT)**
    - same public address shared by several hosts with private addresses
    - Some problems
      - introduce states in the network between end points
      - Some application protocols require specific application level gatways (ALG)
        - » e.g. FTP, SIP, DNS, ICMP
        - » need to maintain various protocol states
    - Incompatible with some security mechanism
      - e.g. IPSec

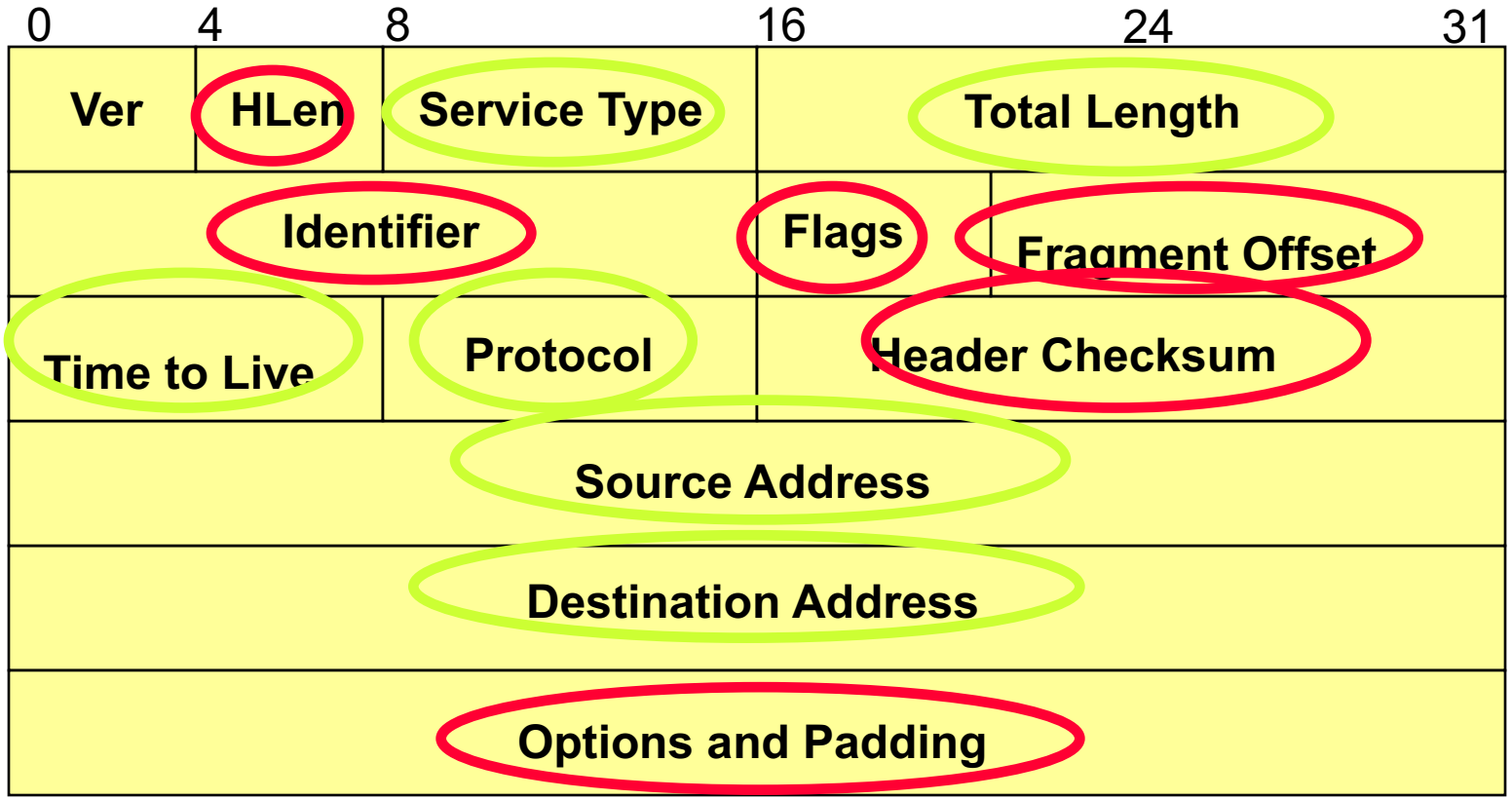
Internet Protocol v6

- IPv6 (RFC 2460) = the next generation Internet Protocol
  - **Complete redesign of IP addressing**
  - **Hierarchical 128-bit address with decoupled host identifier**
  - **Stateless auto-configuration**
  - **Simple routing and address management**
- Majority of traffic not yet IPv6 but..
  - **Most PC operating systems already have IPv6**
  - **Most routers already have IPv6 support**
  - **IoT is coming**

IPv6 main characteristics

- Larger address space
  - **quadrupled to 16 bytes=128 bits**
  - **required for large-scale networks (urban networks, smart grids, industrial automation networks,..)**
- Larger number of levels of addressing hierarchy
  - **routing table efficiency**
- Autoconfiguration
  - **each node should be able to generate its own unique IPv6 address**
- Mobility support
  - **Mobile IPv6**
- Security
  - **IPSec/AH/ESP/IKE**

IPv4 vs IP6 Header format



- 20 octects + options
- 13 mandatory fields
  - **one of them is also formed by 3 bits of flags**

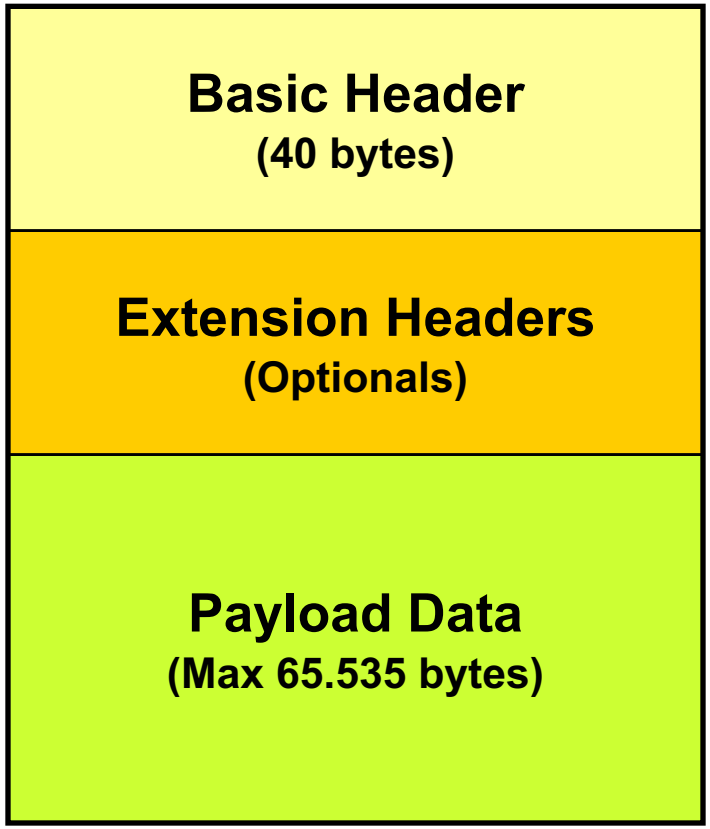
Modified

Removed

IPv4 vs IP IP6 Header format

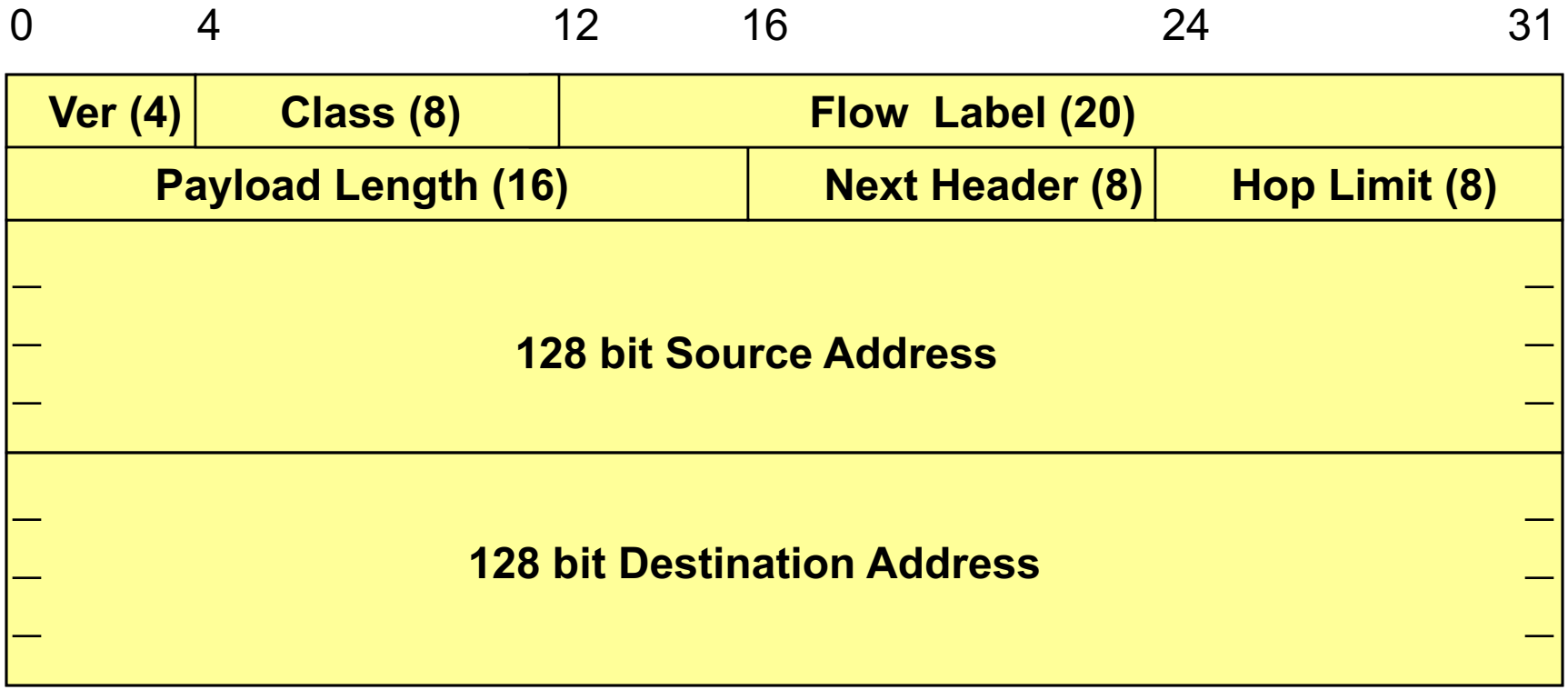
- No checksumming at the IP network layer → transport layer needs to take care
- No hop-by-hop segmentation: path MTU (Maximum Transmission Unit) discovery
- Header change
  - **Unused IPv4 header fields have been removed**
  - **Simpler structure with potentially daisy-chained headers**
  - **IPv6 header is twice as long (40 bytes) as IPv4 header without options (20 bytes)**
  - **64 bits aligned (various fields can be grouped in groups of 64 bits) → 64-bit processors!**

IPv6 packet general format



- Basic header
  - **contiene le informazioni comuni a tutti i datagrammi**
- Extension Headers
  - **contengono le opzioni utilizzate dai router intermedi e/o dall'host di destinazione**
- Payload Data
  - **sono i bit informativi elaborati dall'host di destinazione**

IPv6 Basic Header



40 octects
8 header fiels

IPv6 Basic Header

- Version (4 bits): IP versions number 6
- Traffic class (8 bits): used to indicate the Class of Service (CoS) of the packet
- Flow label (20 bits): a label may be used by a source node to refer to a sequence of packets identifying a flow that requires specific handling of the packet by routers along the path to its destination
- Payload length (16 bits): indicates the length of the payload (excluding the basic header)
- Next header (8 bits): identifies the header that follows the IPv6 packet header. It provides a flexible way to add optional extension headers
- Hop limit (8 bits): decremented each time the packet is forwarded by a node. When the hop limit field is equal to 0 (2^8 hops), the packet is discarded
- Source address: 128-bit IPv6 source address of the packet
- Destination address: 128-bit IPv6 destination address of the packet

IPv6 Basic Header

- Payload Length (16 bit)
  - **indicates the length of the payload (excluding the basic header)**
  - **normal maximum length is 65.535 byte**
  - **It is possible to extend it using "jumbo payload" (hop-by-hop options header)**
- Next Header (8 bit)
  - **identifies the header that follows the IPv6 packet header**
  - **Some values:**

0	Hop-by-hop options header	46	Resource Reservation Protocol
4	Internet Protocol	50	Encapsulating Security Payload
6	Transmission Control Protocol	51	Authentication Header
17	User Datagram Protocol	58	Internet Control Message Protocol
43	Routing Header	59	No Next Header
44	Fragment Header	60	Destination Options Header

IPv6 Extension Headers

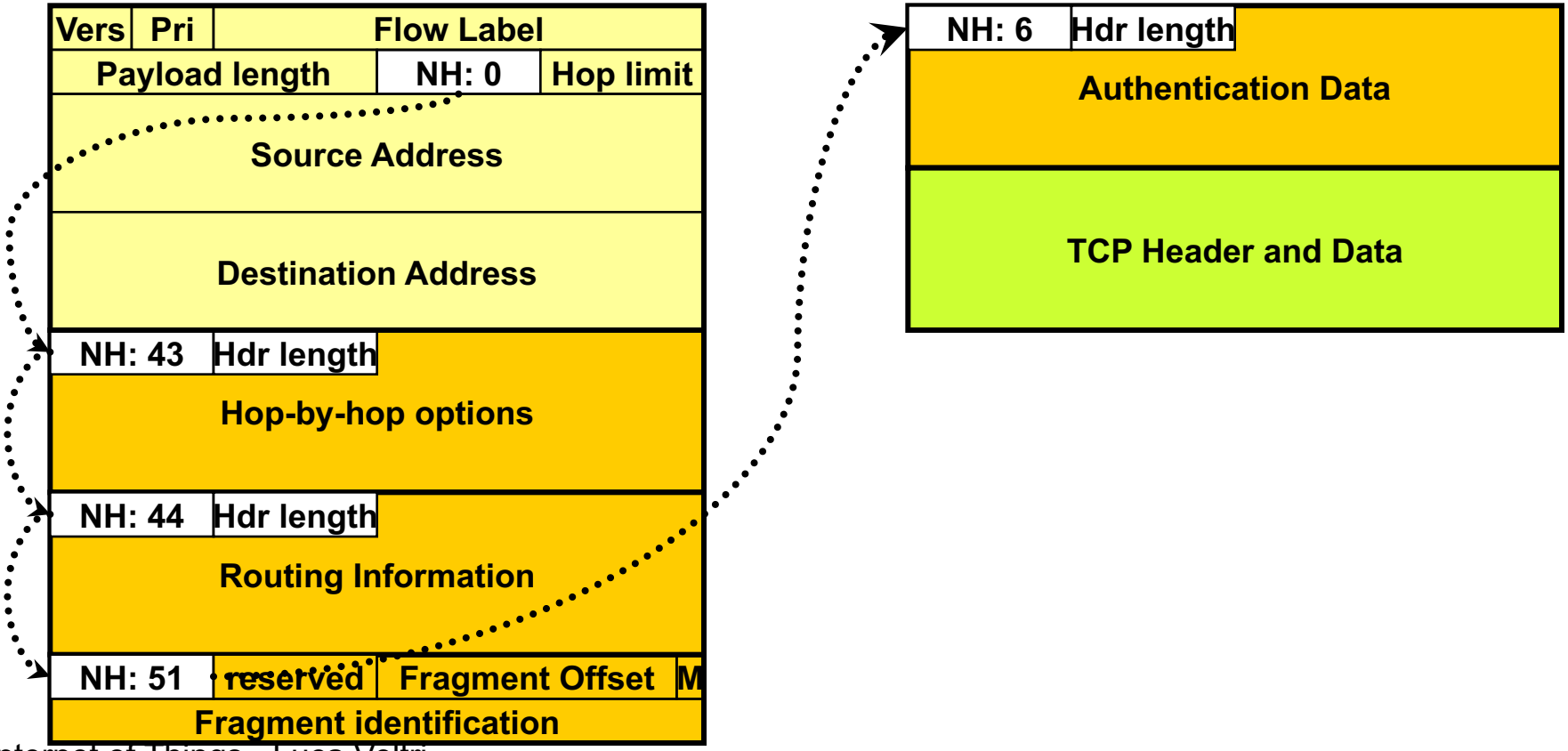
- Extension headers are inserted between the IPv6 basic header and the payload (upper layer data, e.g. TCP PDU)

- IPv6 “normal” packet

- Examples of di IPv6 packets with “Extension Headers”

IPv6 Extension Headers (cont.)

- Every EH has a length of 8*(1+n) byte, where *n* is the Header Extension Length value
- Every EH indicates the type of the next EH (Next header field)



IPv6 Extension Headers (cont.)

- Basic standard EH (in the right order):
  - **Hop-by-hop Options header**
  - **Destination Options header**
    - **only in prsence of Routing header**
  - **Routing header**
  - **Fragment header (only destination)**
  - **Authentication header (only destination)**
  - **Encapsulation security payload header (only destination)**
  - **Destination Options header (only destination)**
- Every EH may occur only 1 time
  - **The only exception is the Destination Options (at most 2 times)**

IPv6 Extension Headers (cont.)

- Hop-by-hop Options (Type=0)
  - **It includes the options which involve all traversed routers: es. Jumbo Option (for packets with >65.575 bytes)**
- Routing (Type=43)
  - **It specifies a list of routers to be traversed (Strict o Loose)**
- Fragment (Type=44)
  - **It is useful to send fragmented packets (only from the source)**
  - **It a packets exceeds the link MTU (MTU min = 576 in IPv6!) it has to be discared and an ICMP msg is sent to the source, which can**
    - i) reduce the length of the subsequent packets
    - ii) send packets with the EH Fragment
- Destination Options (Type=60)
  - **It includes the options which need to be processed only at the destination node/s**
- Authetication (Type=51)
- Encapsulating Security Payload (ESP) (Type=50)

Hop-by-Hop Options

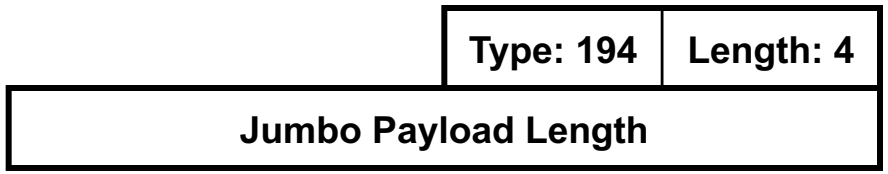
- It contains the options for each intermediate system over the path of the datagram
  - **It is processed also by intermediate nodes**
- It is constitute by a vsariable number of options codified as triplets TLV (Type-Length-Value)
  - **Type (8 bit): indicates the type of option**
  - **Length (8 bit): indicates the length of the field Value**
  - **Value: it transports the option and some indications for the router useful to process the option**
- Defined options:

Type	Option	Size	Allineamento
0	Pad 1	1 byte	nessuno
1	Pad N	2+n bytes	nessuno
194	Jumbo Payload Length	2+4 bytes	4·n + 2

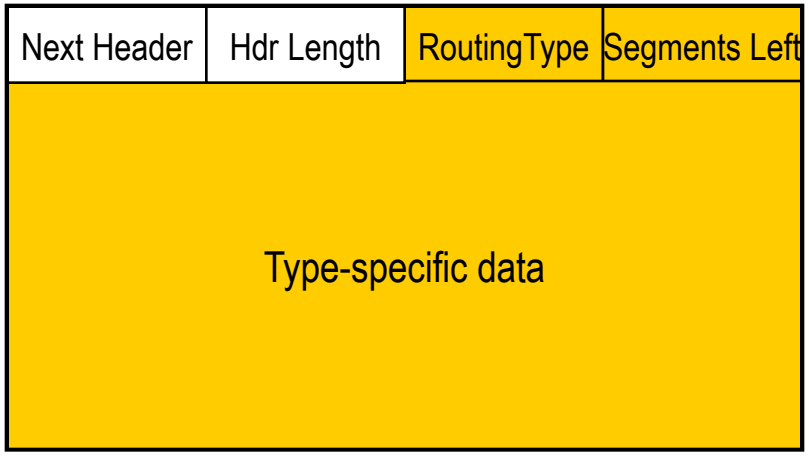
Hop-by-Hop Options Header

- Pad1 e PadN options
  - **Needed to insert filling octets in the header for alignment puroposes**
  - **Allow to increase the router processing speed**
    - Pad1: 1 byte: [type=0]
    - Pad2: 2+n bytes: [type=1 (1 byte), len (1 byte), N bytes]

- Jumbo Payload Length Option
  - **Allows to increase the maximum length of the datagram with respect to the value allowed by the basic header**
  - **If this option is used, the field payload length of the basic header has to be set to the value 0**

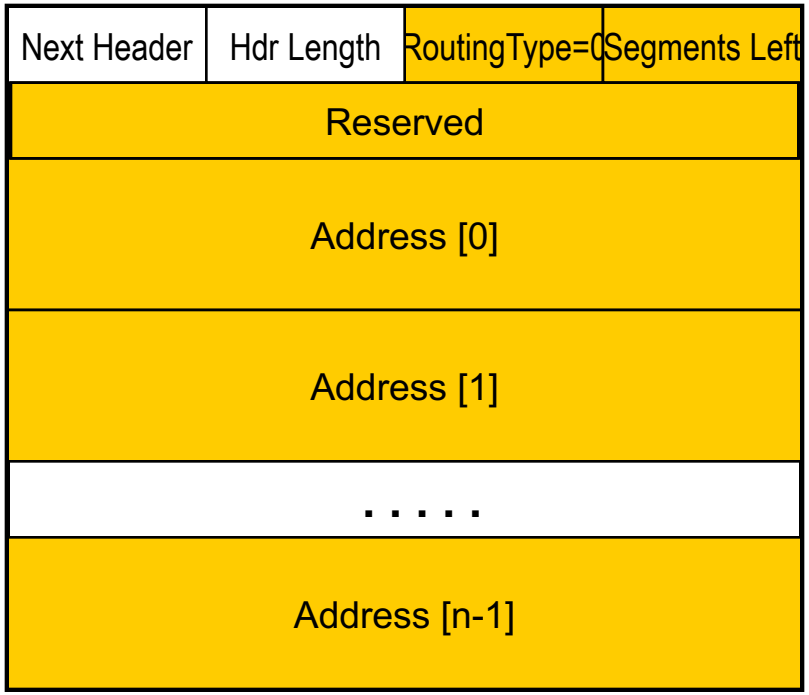


Routing Header



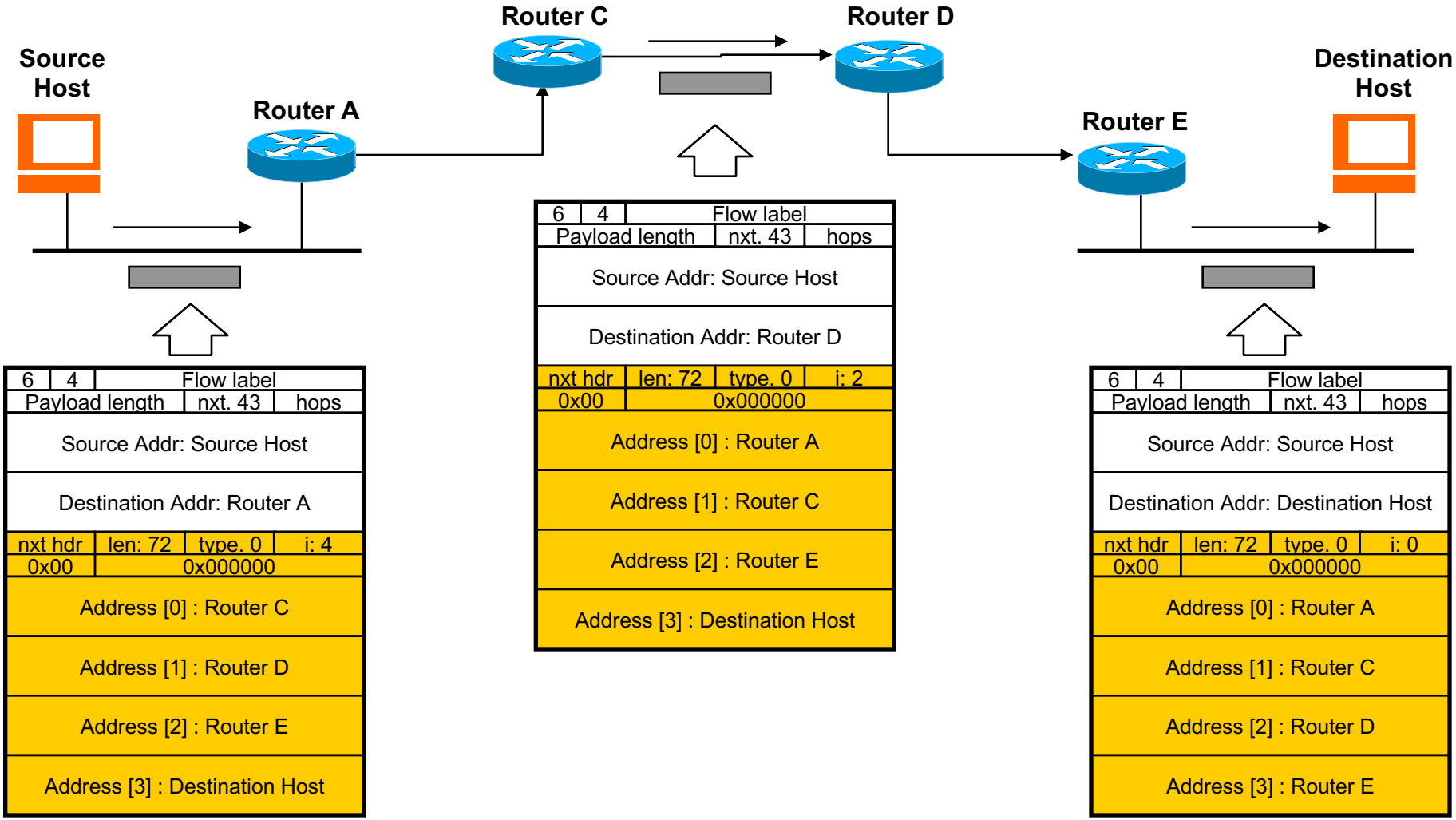
- Used by the source node to control the path followed by a packet
  - **similar to the Option *Loose Source* e *Record Route* of IPv4**
  - **processed only at the nodes with address in the *Dest Address***
- Format:
  - **Routing Type**
    - Identifies the type of routing
  - **Segments left**
    - It indicates the number of addresses which still need to be processed
    - Decremented by every node which processes it
  - **Type-specific data**
    - Field of variable length, multiple of 8 bytes
    - Depends on Routing Type

Routing Type 0 - Loose Source Route



- Address [0..n-1]
  - **Vector of *n* IPv6 addresses (*n* x 128 bit)**
    - **Adress[i] = (*i+1*)-th router along the path**
    - **At each hop, the index *i*=(*n-SegmentLeft*) is computed; the address *Address[i]* is exchanged with the *DA* (del basic Header), and the *Segment Left* field is decremented**
      - At the beginning:
        - DA= first node
        - Address[0]= second node
        - Address[n-1]= last nodo
      - At the end one has:
        - DA= last node
        - Address[0]= first node
        - Address[n-1]= before than last node

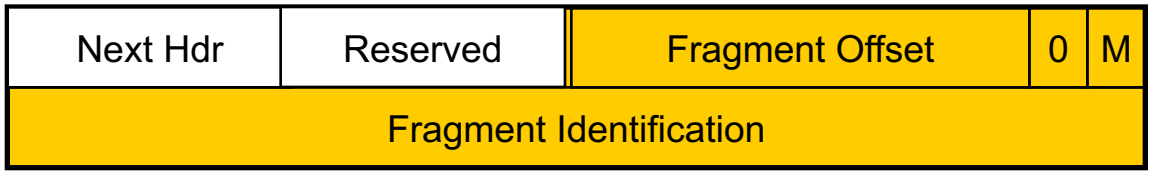
Loose Source Route - Example



Fragment Header

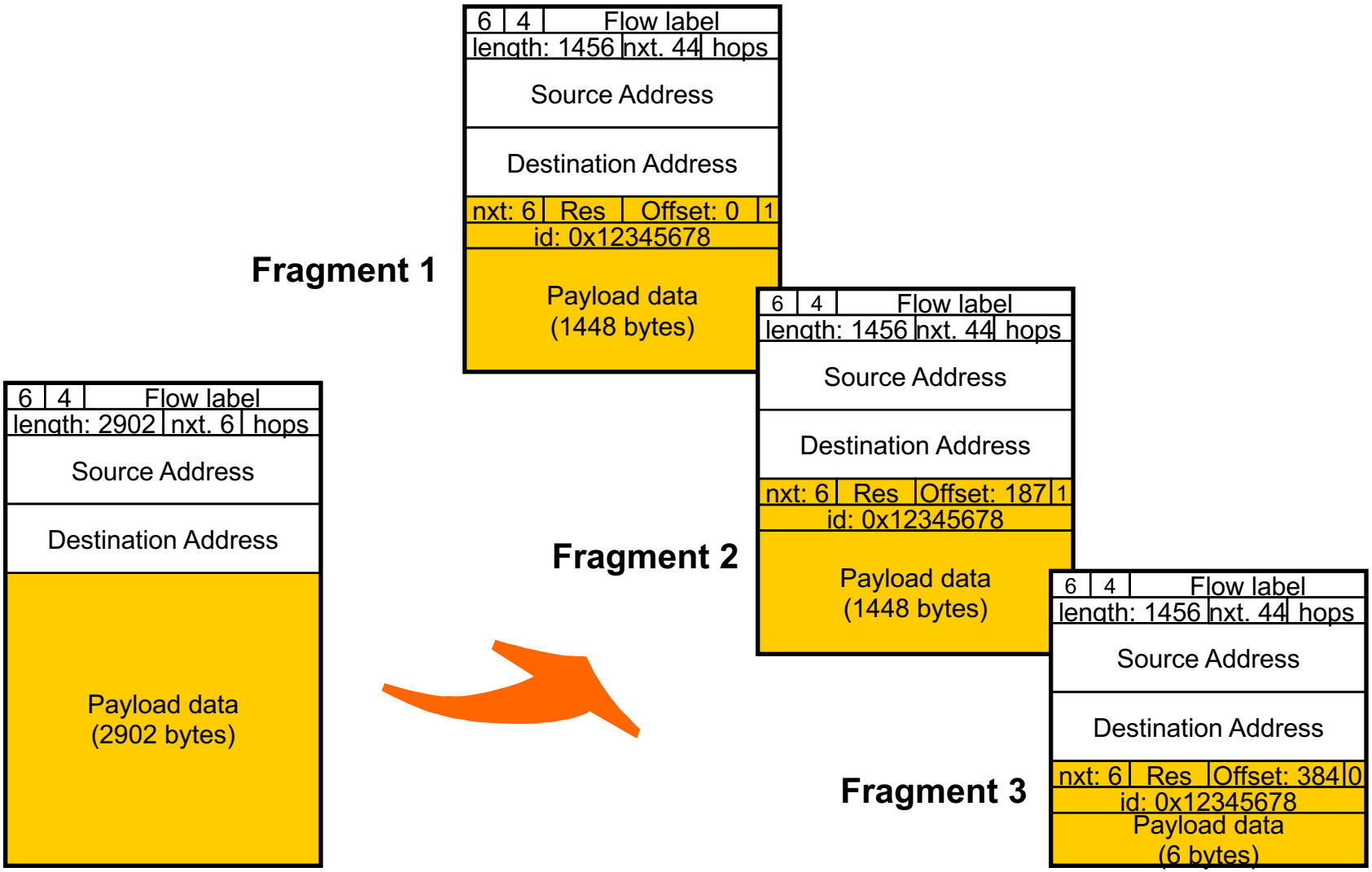
- Header of fixed length equal to 64bit
- Used for segmentation and reconstruction of datagrams
- In IPv6 only the source can fragment a datagram
  - **When an intermediate node receives a datagram with length longer than the MTU, it discards it and notifieds the error to the sender through ICMP**
- Every datagram is composed by a non-fragmentable and a fragmentable part
  - **The non-fragmentable part is formed by the header which need to be processed also in intermediate nodes (Basic Header, Hop by Hop Header e Routing Header)**
- All fragments but the last one have the same length multiple of 8 bytes

Fragment Header



- Fragment Offset
  - **Specifies the position (in units multiple of 8 bytes) of the first byte of the fragment in the original datagram**
- Bit M (More Fragments)
  - **Its value is set to 1 in all fragments of the datagram but in the last one, where it is placed to 0**
- Fragment identification
  - **Identifies the datagram which the fragment belongs to**

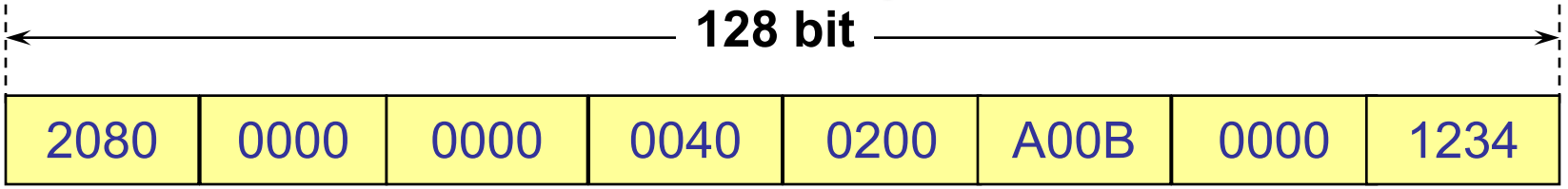
Fragment Header



IPv6 Addressing

- The designers of IPv6 had the benefit of IPv4 large experience and realized there would be tremendous advantages to designing the unicast address structure to reflect the overall topology of the Internet. These include:
  - **Easier allocation of address blocks at various levels of the Internet topological hierarchy**
  - **IP network addresses that automatically reflect the hierarchy by which routers move information across the Internet, allowing routes to be easily aggregated for more efficient routing**
  - **Flexibility for organizations like ISPs to subdivide their address blocks for customers**
  - **Flexibility for end-user organizations to subdivide their address blocks to match internal networks, much as subnetting did in IPv4**
  - **Greater "meaning" to IP addresses. Instead of just being a string of 128 bits with no structure, it would become possible to look at an address and know certain things about it**

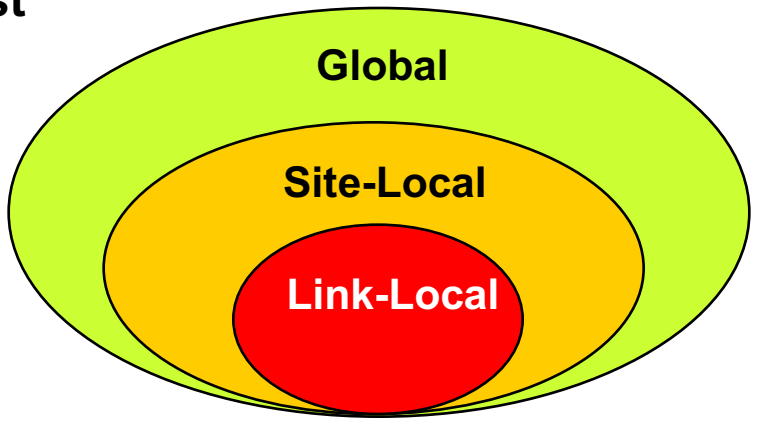
IPv6 Address Representation



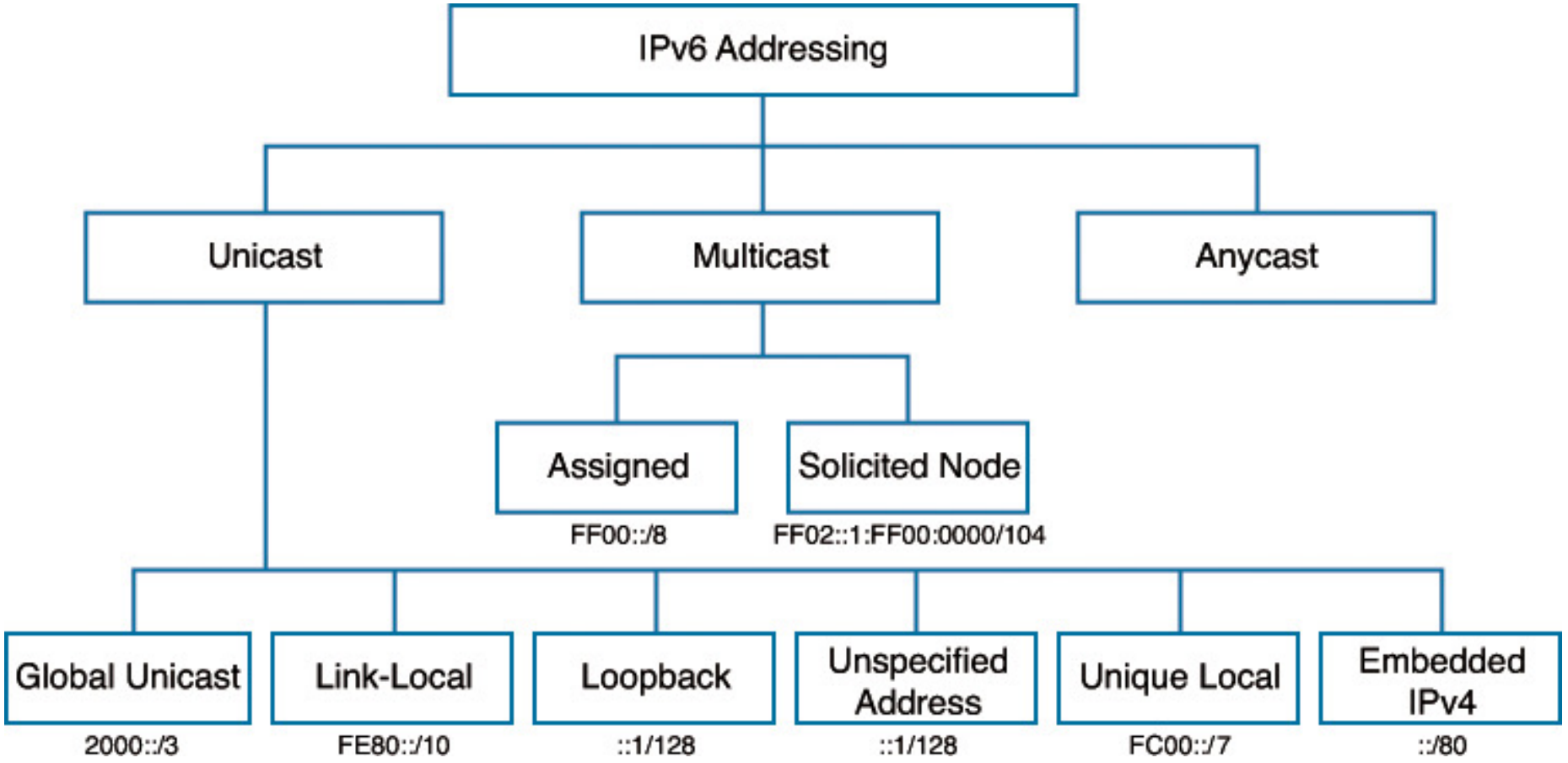
- 16-bit fields in case-insensitive colon hexadecimal (0,…,9,A,…,F) representation
  - 2080:0000:0000:0040:0200:A00B:0000:1234
- Some simplifications:
  - **leading zeroes in a field are optional and can be omitted**
    - 2080:0:0:40:200:A00B:0:1234
  - **successive fields of 0 represented as ::, but only once**
    - 2080::40:200:A00B:0:1234
    - 0:0:0:0:0:0:0:1 → ::1
    - 0:0:0:0:0:0:0:0 → ::
- IPv4-compatible address representation:
  - **0:0:0:0:0:0:10.0.0.1 = 0:0:0:0:0:0:A00:1 = ::A00:1 = ::10.0.0.1**

IPv6 Addressing

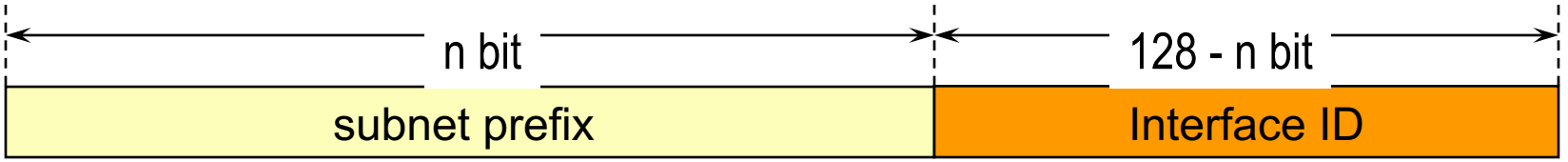
- IPv6 Addressing rules are covered by multiples RFC's
- Address Types are:
  - **Unicast : One-to-One (Global, Link local, Site local, Compatible)**
  - **Anycast : One to Nearest (Allocated from Unicast)**
  - **Multicast : One to Many**
  - **Reserved**
- A single interface may be assigned multiple IPv6 addresses of any type (unicast, anycast, multicast)
  - **No Broadcast Address → Use Multicast**
- Different address visibility
  - **Global**
  - **Site-Local (deprecated)**
  - **Link-Local**



IPv6 Addressing



IPv6 Addresses: Subnet Prefix



- Subnet prefix identifies:
  - **Type of address**
  - **Network/subnetwork that the address belongs to**
- The prefix length is specified by adding "/N" where N is the lenght (in bits)
  - **example:**
    - FEDC:0123:8700::1:2:3:4 /36
- Interface ID identifies:
  - **a node interface (within the given network)**

IPv6 Addresses

- IPv6 addresses start with a variable length “Prefix Format” (PF) indicating the address type

Address type	*Binary prefix*	*IPv6 notation*
Unspecified	00...0 (128 bits)	::/128
Loopback	00...1 (128 bits)	::1/128
Multicast	1111 1111	FF00::/8
Link-local unicast	1111 1110 10	FE80::/10
Site-local unicast	11111110 11	FEC0::/10
Global unicast	001	2000::/3

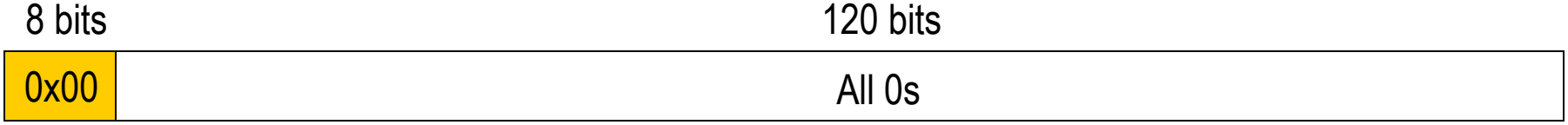
- Anycast addresses are taken by unicast addresses spaces

IPv6 Addresses (cont.)

Binary prefix	*Address type*
0000 0000	Reserved
0000 0001	Unassigned
0000 001	Reserved for NSAP (non-IP addresses used by ISO)
0000 010	Reserved for IPX (non-IP addresses used by IPX)
0000 011	Unassigned
0000 1	Unassigned
0001	Unassigned
001	Unicast Address Space
010	Unassigned
011	Unassigned
100	Unassigned
101	Unassigned
110	Unassigned
1110	Unassigned
1111 0	Unassigned
1111 10	Unassigned
1111 110	Unassigned
1111 1110 0	Unassigned
1111 1110 10	Link Local Use addresses
1111 1110 11	Site Local Use addresses
1111 1111	Multicast addresses

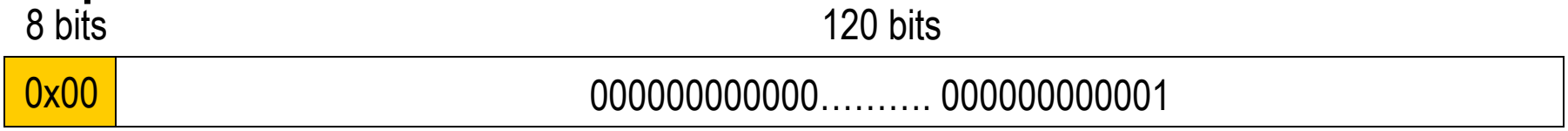
Reserved Addresses with prefix 0x00

Unspecidfied address (only as source address)



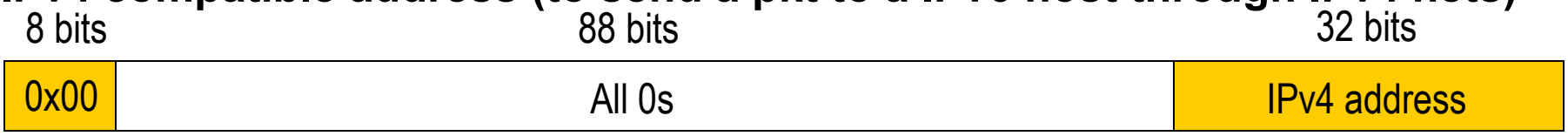
::0

Loopback address



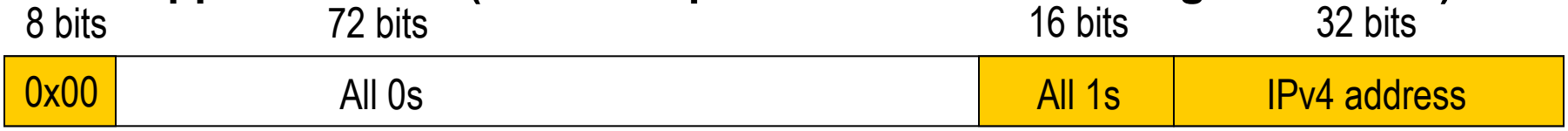
::1

IPv4 compatible address (to send a pkt to a IPv6 host through IPv4 nets)



::x.y.z.w

IPv4 mapped address (to send a pkt to a IPv4 host through IPv6 nets)



::FFFF:x.y.z.w

Unspecified and Loopback Addresses

- Unspecified address 0:0:0:0:0:0:0:0
  - **::**
  - **It must never be assigned to any node**
  - **It indicates the absence of an address**
  - **e.g. as src addr of any pkts sent by an initializing host**
  - **must not be used as the dest addr of pkts**
  - **a packet with a src addr :: must never be forwarded by an IPv6 router**
- Loopbackaddress 0:0:0:0:0:0:0:1
  - **::1**
  - **It may be used by a node to send an IPv6 packet to itself**
  - **It must not be assigned to any physical interface**
  - **It is treated as having link-local scope (of a virtual interface called "loopback interface")**
  - **must not be used as src addr in pkts sent outside of a single node**
  - **a packet with dest addr of loopback must never be sent outside**

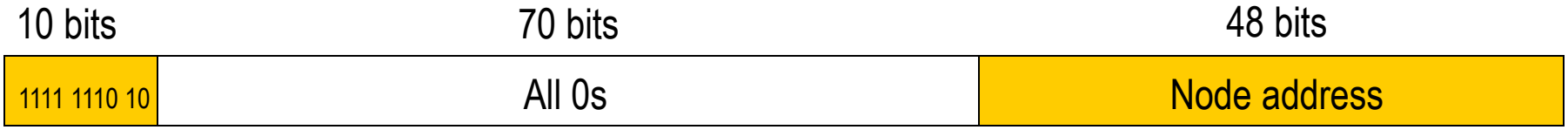
IPv6 Addresses with Embedded IPv4 Addresses

- IPv4-Compatible IPv6 Address
  - **was defined to assist in the IPv6 transition (to send a pkt to a IPv6 host through IPv4 nets)**
  - **is now deprecated because the current IPv6 transition mechanisms no longer use these addresses**

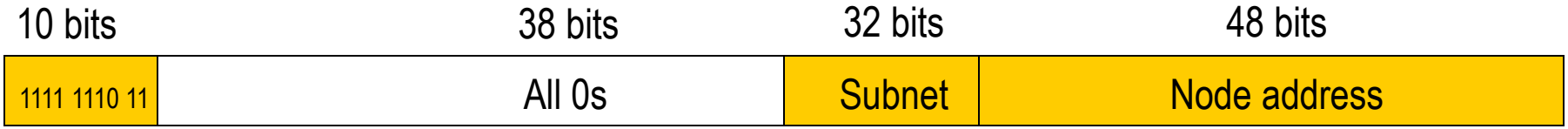
- IPv4-Mapped IPv6 Address
  - **IPv6 address that holds an embedded IPv4 address is defined**
  - **it is used to send a pkt to a IPv4 host through IPv6 nets**

Local Addresses

- Link local address (only within a subnet, e.g. LAN)



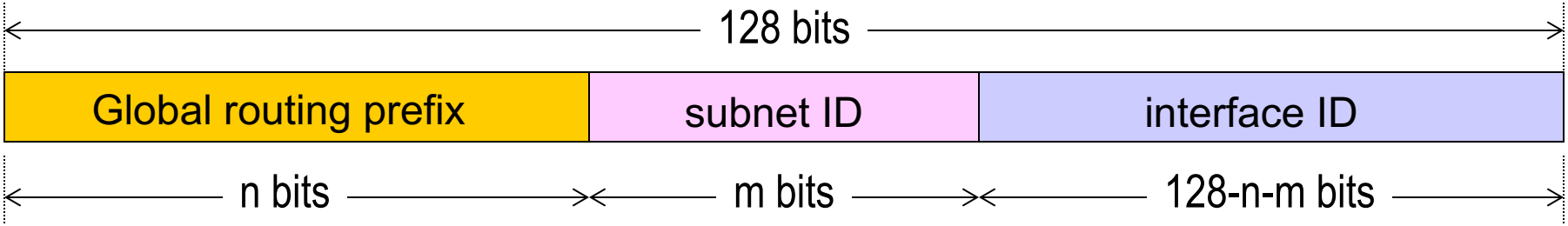
- Site local address (only though private subnets)



- Note:
  - **Site-local addresses were originally designed to be used for addressing inside of a site without the need for a global prefix**
  - **Site-Local addresses are now deprecated**

Global Unicast Addresses

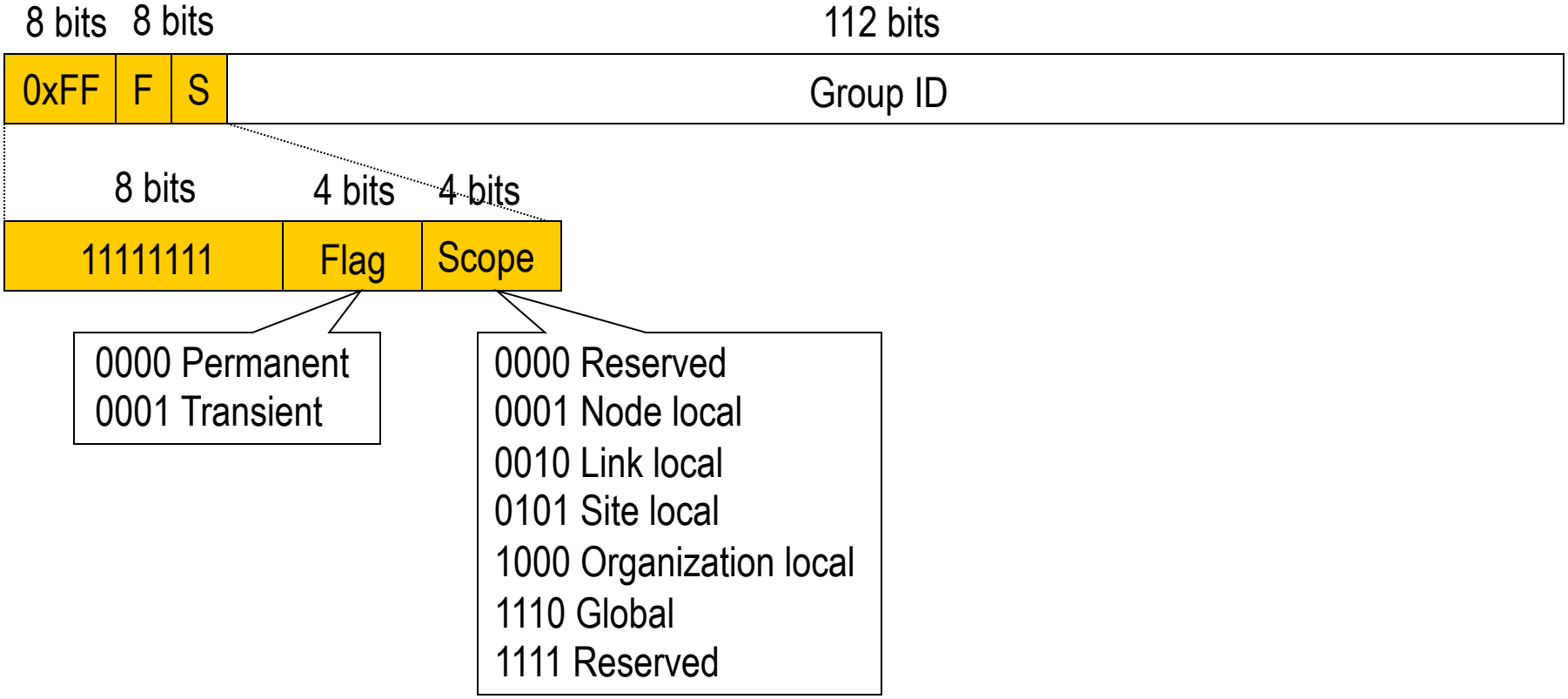
- The general format for IPv6 global unicast addresses is as follows



- where
  - **global routing prefix is a (typically hierarchically-structured) value assigned to a site (a cluster of subnets/links)**
  - **subnet ID is an identifier of a link within the site**
  - **interface ID uniquely identify interfaces on a link; it can be derived directly from that interface's link-layer address**
- All global unicast addresses other than those that start with binary 000 have a 64-bit interface ID field (e.g. n=48, m=16)
  - **the 64 bits interface ID is constructed in Modified EUI-64 format**
- Global unicast addresses that start with binary 000 have no such constraint on the size or structure of the interface ID field

Multicast Addresses

Multicast addresses



ICMPv6

- Internet Control Message Protocol per IPv6, used for:
  - **error reporting**
  - **diagnostic**
  - **neighbor discovery**
  - **management of multicast groups**
- It replaces some functions previously done (for IPv4) by:
  - **ICMP**
  - **ARP (Address Resolution Protocol)**
  - **IGMP (Internet Group Membership Protocol)**
- Two classes of messages:
  - **Error messages**
  - **Informational messages**
    - diagnostic
    - neighbor discovery
    - management of multicast groups

ICMPv6 messages

- Error messages (types from 0 to 127)
  - **Destination Unreachable**
  - **Packet Too Big**
  - **Time Exceeded**
  - **Parameter Problem**
- Informational messages (types from 128 to 255)
  - **Echo Request**
  - **Echo Reply**
  - **Router Solicitation**
  - **Router Advertisement**
  - **Neighbor Solicitation**
  - **Neighbor Advertisement**
  - **Redirect**
  - **Group management (Multicast Listener Discovery Protocol)**

Neighbor Discovery protocol

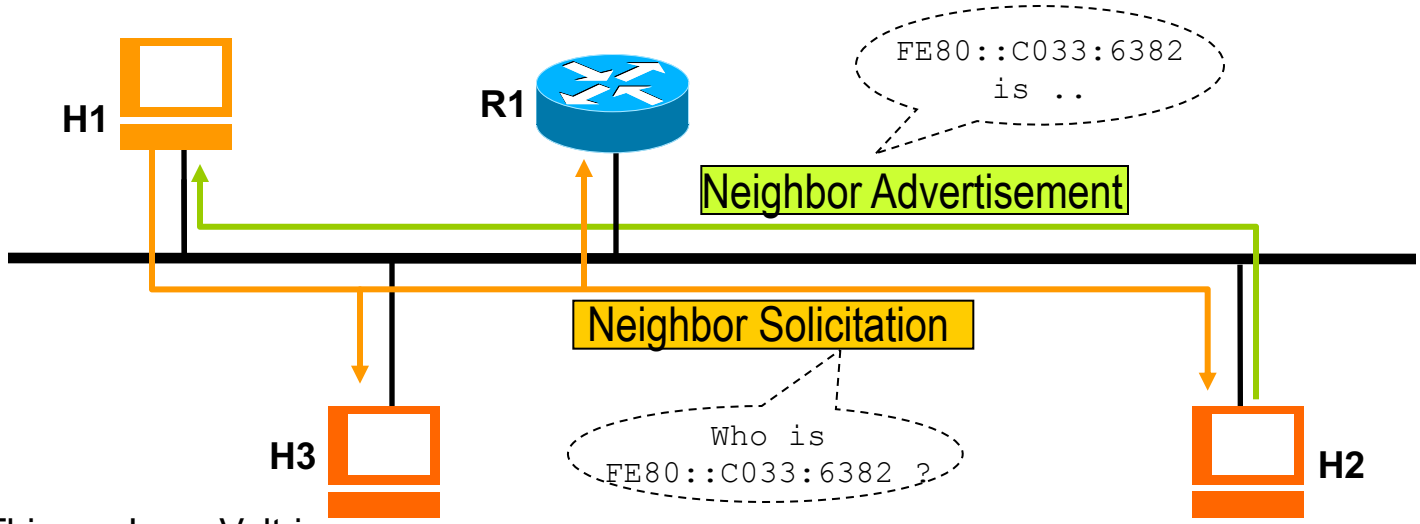
- It solves a set of problems related to the interaction between nodes attached to the same link
  - **Router Discovery**
    - how hosts locate routers that reside on an attached link
  - **Prefix Discovery**
    - how hosts discover the set of address prefixes that define which destinations are on-link for an attached link
  - **Parameter Discovery**
    - How a node learns such link parameters as the link MTU or such Internet parameters as the hop limit value to place in outgoing packets
  - **Address Autoconfiguration**
    - Introduces the mechanisms needed in order to allow nodes to automatically configure an address for an interface

Neighbor Discovery protocol (cont.)

- **Address resolution**
  - How nodes determine the link-layer address of an on-link destination (e.g., a neighbor) given only the destination's IP address
- **Next-hop determination**
  - The algorithm for mapping an IP destination address into the IP address of the neighbor to which traffic for the destination should be sent
  - The next-hop can be a router or the destination itself
- **Neighbor Unreachability Detection**
  - How nodes determine that a neighbor is no longer reachable
  - For neighbors used as routers, alternate default routers can be tried
- **Duplicate Address Detection**
  - How a node determines that an address it wishes to use is not already in use by another node
- **Redirect**
  - How a router informs a host of a better first-hop node to reach a particular destination

Address Resolution

- The process through which a node determines the link-layer address of a neighbor given only its IP address
- Neighbor Solicitation
  - **Sent by a node when it has a unicast packet to send to a neighbor, but does not know the neighbor's link-layer address**
  - **Used also for Duplicate Address Detection**
- Neighbor Advertisement
  - **In reply to Neighbor Solicitation**
  - **Sent also for notifying a change of address**



Host Autoconfiguration

- IPv6 supports different type of host auto-configurations
- Two “plug-and-play” ways for host configuration:
  - **Stateful**
    - DHCPv6 (Dynamic Host Configuration Protocol)
    - Similar to DHCP for IPv4
  - **Stateless**
    - Message exchange between Host and Router
    - Network prefix from ICMP Router Solicitation/Advertisement
    - The host forms its own address by appending its PH/DL interface address (e.g. MAC address)
- The two modes may coexist

Multicast Listener Discovery Protocol (MLDv2)

- Multicast Listener Discovery Protocol (MLD)
- Used by IPv6 routers to
  - **discover the presence of multicast listeners (i.e., nodes that wish to receive multicast packets) on their directly attached links, and**
  - **discover specifically which multicast addresses are of interest to those neighboring nodes**
- MLDv2 is a translation of the IGMPv3 protocol for IPv6 semantics

Transition mechanisms

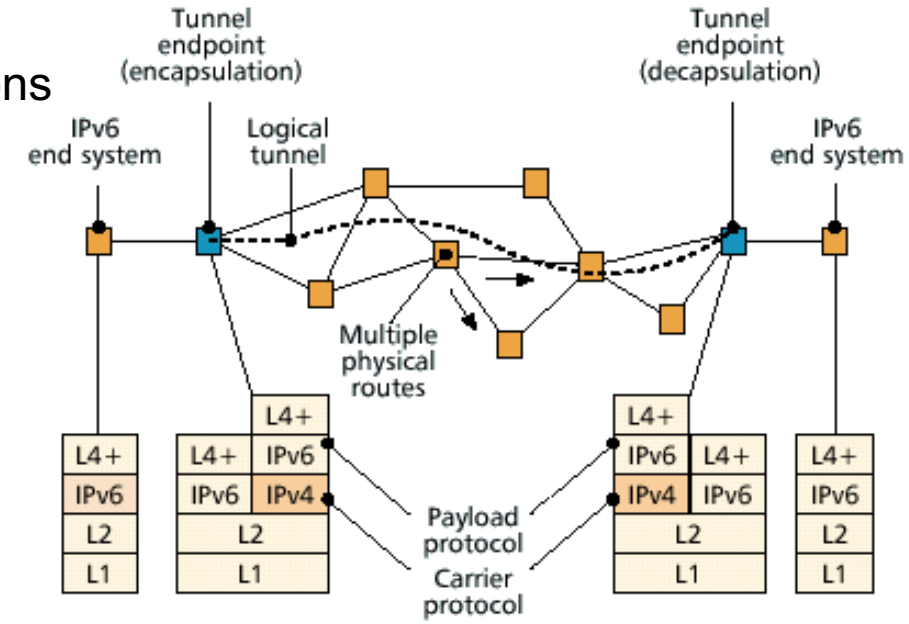
Transition mechanisms

- Transition mechanisms generally come in one one three forms
  - **Dual stack**
  - **Translation**
  - **Tunneling**

- Dual stack
  - **two protocol stacks are separately maintained and operate in parallel**
  - **the device can operate via either protocols**
  - **can be implemented in both end systems and network nodes**
    - in end systems: it enables both IPv4 and IPv6 applications to operate on the same node
    - in network nodes: it allows handling of both IPv4 and IPv6 packets

Transition mechanisms

- Tunneling
  - **used to bridge compatible nodes (e.g. IPv6) across incompatible networks (e.g. IPv4)**
  - **IPv6-over-IPv4 or IPv4-over-IPv6**
  - **main problem is the configuration of tunnels (on tunnel endpoints)**
  - **tunnel endpoint addresses are generally attained:**
    - manually or by tools (e.g. tunnel broker)
    - through existing services such as DNS or DHCP options
    - by embedding information in the link layer addresses or IP addresses



UNIVERSITÀ DI PARMA

IoT Lab

Communication Mechanisms

Main sources

- J. P. Vasseur and A. Dunkels, "Interconnecting Smart Objects with IP,` Morgan Kaufmann, 2010. Chapter 12.

Communication Patterns

- Smart object communication: one-to-one, one-to-many, and many-to-one.
- Each communication pattern is used in different situations. Many applications use a combination of the patterns.
- Smart objects often communicate over unreliable communication channels: (i) interference (shared medium and free spectrum band); (ii) obstacles on the path
- Need for reliable communication protocols (best-effort may be sufficient in some cases).
- Different requirements: static (e.g., industrial WSN) vs dynamic (BSN).
- Power consumption: switch off as much as possible.

One-2-one Communication

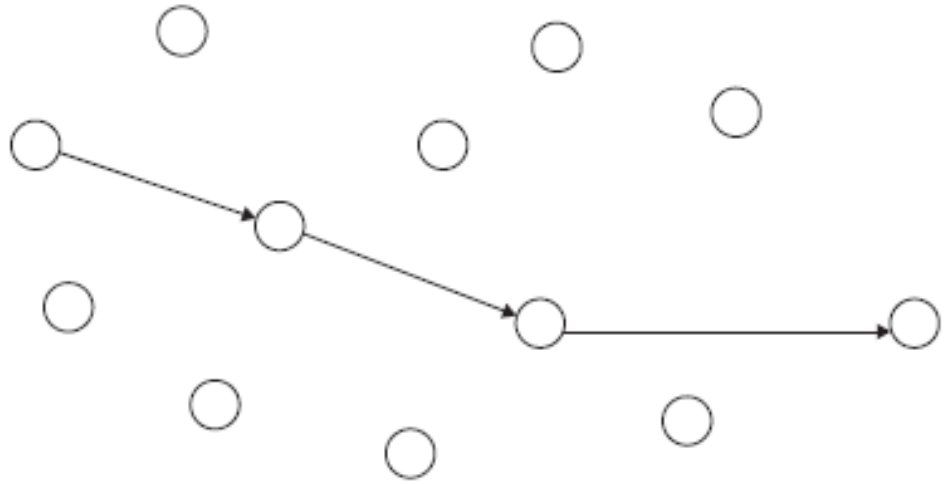


FIGURE 12.1
One-to-one communication in a smart object network.

- One source and one destination, with possibly more relays in the middle (ad-hoc network)
- Could rely on the extensive body of knowledge on ad-hoc wireless networks

One-2-many Communication

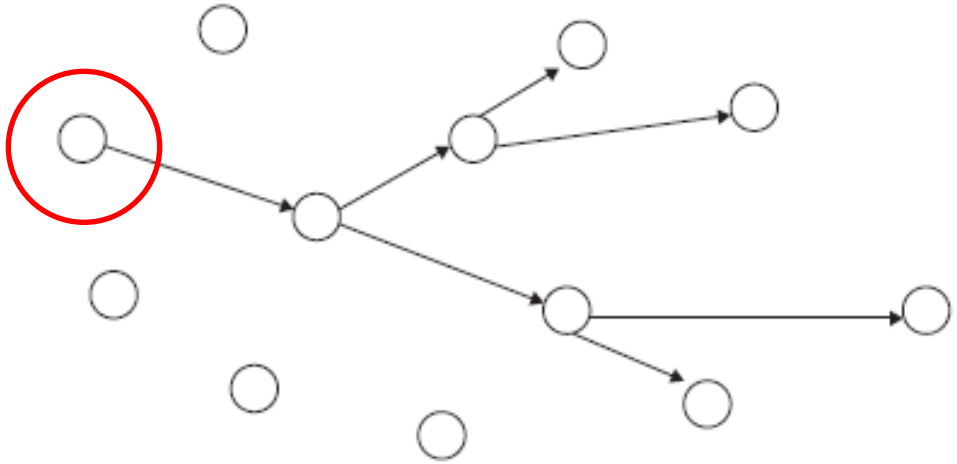


FIGURE 12.2
One-to-many communication in a smart object network.

- Need to reach all nodes (e.g., sending a command)
- Various mechanisms: flooding; irresponsible forwarding (IF); trickle (explicitly designed for low-power networks→RPL)
- Used in routing protocols for one-2-one communications (e.g., Ad-hoc On-demand Distance Vector, AODV) → iAODV

Many-2-one Communication

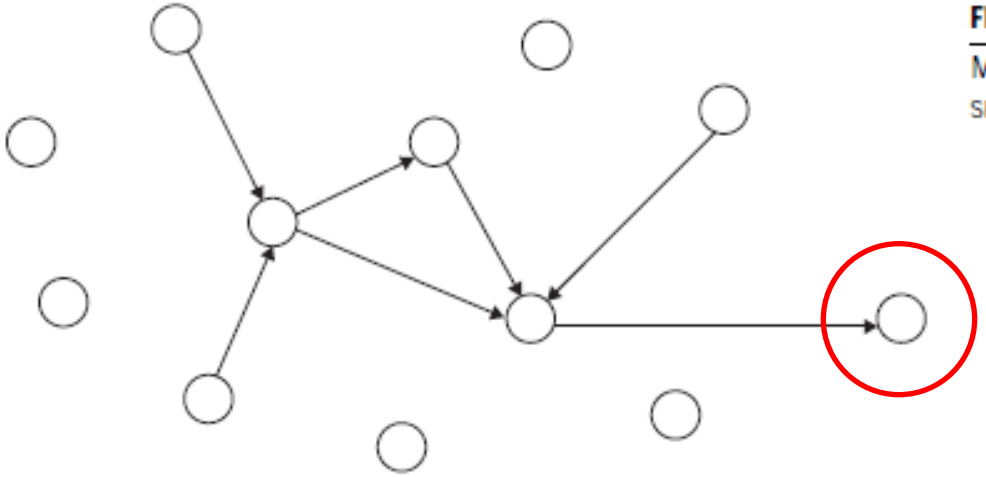


FIGURE 12.3
Many-to-one communication in a smart object network.

- Collect at a *sink*: sensor data or network health status (e.g., periodic status reports)
- There may be more than one sink inside the network.
- Set-up: build a tree structure with its root at the sink (iterative process from sink to all nodes) → hop count-based routing
- To account for radio quality in addition to hop count, several cost metrics can be considered
  → expected transmissions (ETX) is a good metric
- Collection Tree Protocol (CTP) based on ETX

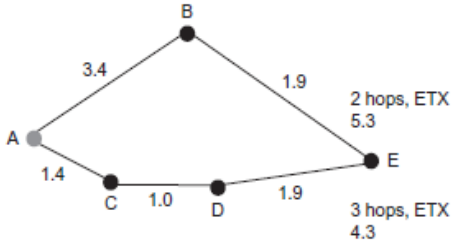


FIGURE 12.4
ETX in a five-node network.

Physical Communication Standards

- Of interest: IEEE 802.15.4 (wireless), IEEE 802.11 (wireless), Power Line Communications (PLC, wired)
- Transmission ranges:
  - IEEE 802.15.4: short range (a few m)
  - IEEE 802.11: medium range (up to several hundred m)
  - PLC: potentially long range (depends on eletric loads connected to the cable)
- Network formation
  - PLC: all nodes connected to the same cable (same connectivity level)
  - IEEE 802.15.4: topology change (mobile nodes or environment change)
  - IEEE 802.11: similar to IEEE 802.15.4 (problems less pronounced)
- Routing
  - IEEE 802.15.4: routing is crucial (nodes act often as relays)
  - PLC: no real need for routing (broacast in the cable)
  - IEEE 802.11: for extended network coverage, APs are connected

IEEE 802.15.4: Basics

- Developed by the 802.15 personal area network (PAN) Working Group within the Institute of Electrical and Electronics Engineers (IEEE).
  - Maximum data rate: 250 kb/s
  - Maximum transmit power: 1 mW
  - Nominal transmit range: a few tens of meters
- Focus: to allow low-cost and low-complexity transceivers
- Low-power radio stacks are built on IEEE 802.15.4: WirelessHART, ISA100a, IPv6, and ZigBee.
- The IEEE 802.15.4 standard specifies two layers: PHY and MAC (several mechanisms are proposed, but not all are used)
- Maximum PHY packet (payload) size: 127 bytes → data for upper layers is between 86 and 116 bytes (besides the MAC header) → upper layers need to fragment large data portions into 802.15.4 frames
- IEEE 802.15.4 is typically implemented as a combination of hw (PHY and part of MAC) and sw (MAC logic)

IEEE 802.15.4: Topologies

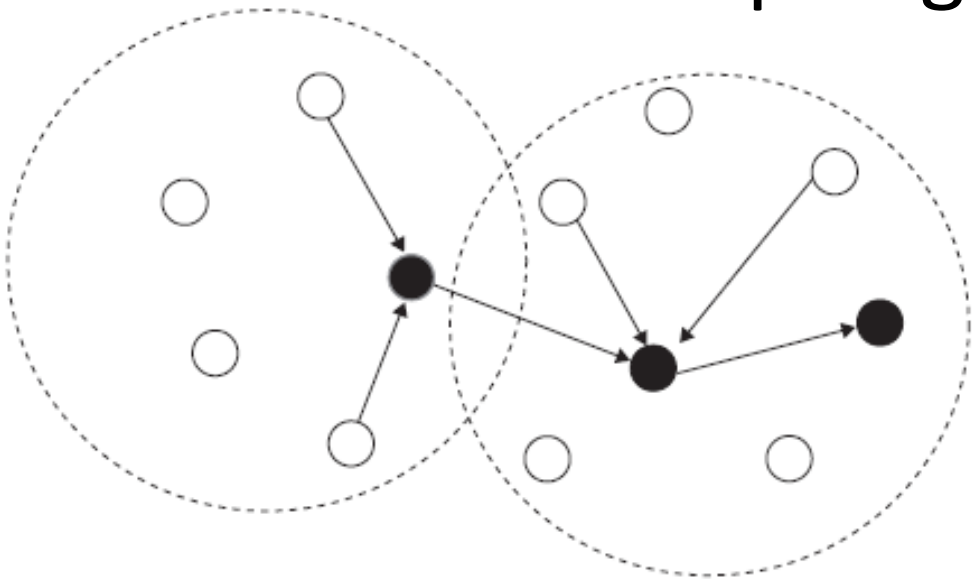


FIGURE 12.5

An IEEE 802.15.4 network with FFDs shown as dark dots and RFDs shown as hollow dots. Two of the FFDs are PAN coordinators in the two PANs, shown as dotted circles. The right PAN contains two FFDs, but only one is the PAN coordinator.

- Each personal area network (PAN) has a PAN coordinator and packets carry a 16-bit identifier.
- A device can participate in one PAN as the PAN coordinator and simultaneously participate as a PAN member in another PAN.
- Two types of nodes: fully functional devices (FFDs, can be coordinators) and reduced function devices (RFDs, cannot be coordinators)
- Three types of network topology: star, mesh, and cluster tree
- Routing is minor → protocols on top of 802.15.4 build their own topology and routing protocols

IEEE 802.15.4: Addresses

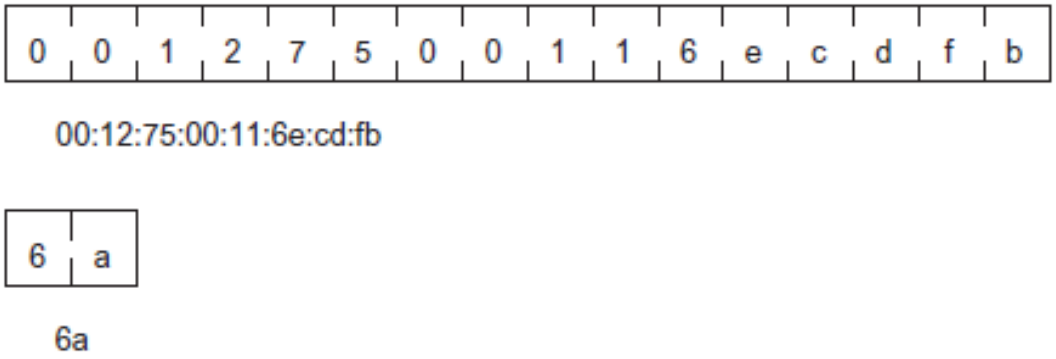


FIGURE 12.6
IEEE 802.15.4 supports two addressing formats: long (64-bit) addresses and short (16-bit) addresses.

- Each node in an 802.15.4 network has a 64-bit address that uniquely identifies the device: very long!
- Long addresses are globally unique and each 802.15.4 device is assigned an address when manufactured (24-bit for organizational unique identifier (OUI) from the IEEE; 40 bit assigned by the manufacturer)
- Short addresses (16-bit) are assigned (run-time) by the PAN coordinator and are valid only within the context of a PAN

IEEE 802.15.4: PHY layer (1/2)

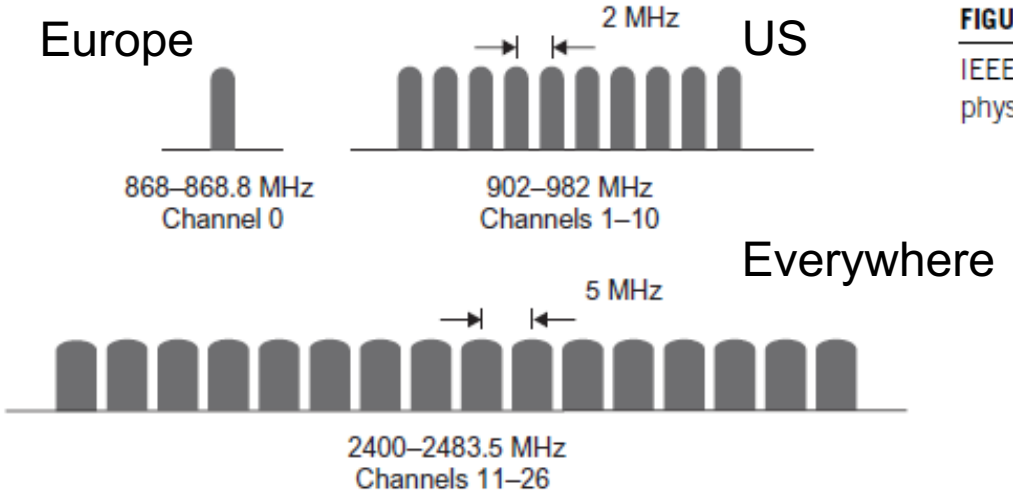


FIGURE 12.7

IEEE 802.15.4 defines 26 physical radio channels.

- **Modulations:**
  - Channels 0-10: BPSK
  - Channels 11-26: QPSK
- **Bit rates:**
  - ch. 0: 20 kb/s
  - ch. 1-10: 40 kb/s
  - ch. 11-26: 250 kb/s

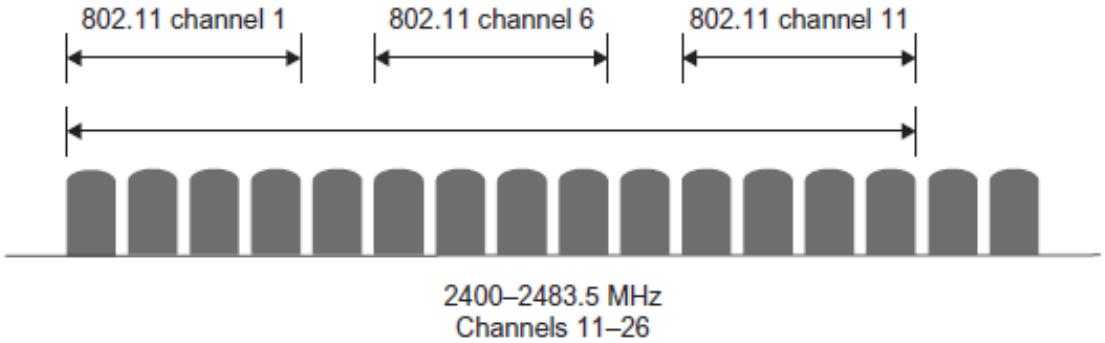


FIGURE 12.8

IEEE 802.15.4 channels 11–24 overlap the 802.11 channels. Channels 25 and 26 are not covered by 802.11 channels. When the non-overlapping 802.11 channels 1, 6, and 11 are used, two additional 802.15.4 channels are undisturbed by 802.11.

IEEE 802.15.4: PHY layer (2/2)

- Interference from IEEE 802.11 devices (WiFi) is a serious issue → lossy links
- PHY layer: provides mechanisms to measure the radio energy for a given radio channel: clear channel assessment (CCA) → used by MAC layer
- CCA uses three mechanisms:
  - measure the radio energy and compare it with a predefined energy threshold
  - demodulate the incoming radio signal to see if it is a valid 802.15.4 signal
  - a combination of the two mechanisms above

IEEE 802.15.4: MAC layer

- MAC layer: control access to the radio medium → control when the medium is idle (CCA) and how to coordinate access by multiple nodes
- IEEE 802.15.4 MAC layer provides: channel access management, validation of incoming frames, and acknowledgment of frame reception.
- Optional mechanisms for a time-division multiple access (TDMA) mechanism for medium access: the PAN coordinator assigns time slots to PAN devices (beacon mode: typically not of interest)
- Validation of incoming frames by computing: 16-bit cyclic redundancy check (CRC) of the entire frame
- If an incoming frame has the acknowledgment bit set →MAC layer broadcasts an acknowledgment frame → interference!

IEEE 802.15.4: Frame formats

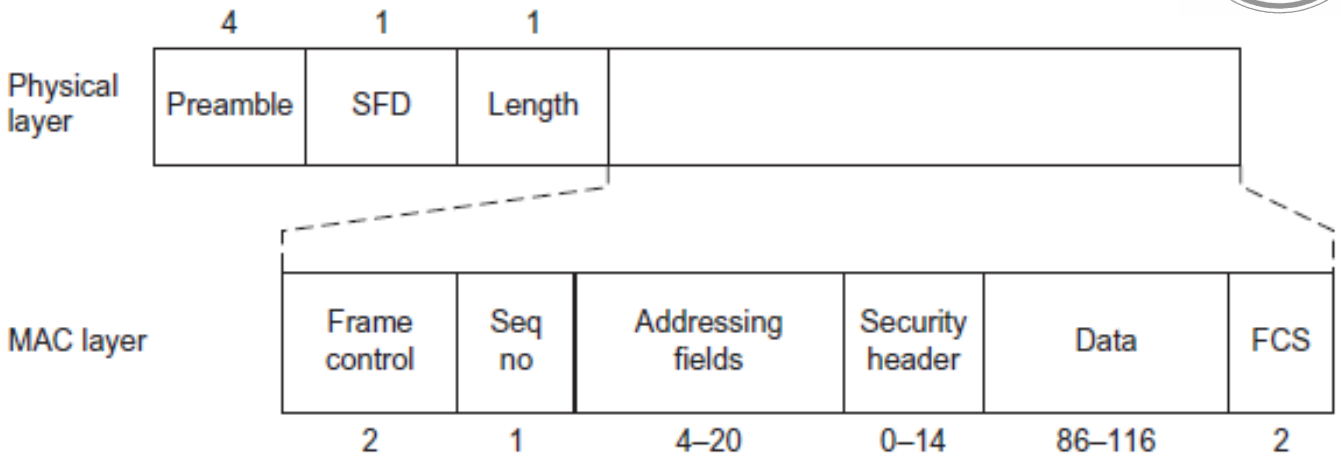


FIGURE 12.9

The IEEE 802.15.4 physical layer and MAC layer header formats.

- **PHY frame**:
  - Preamble: used to synchronize
  - Start of Frame Delimiter (SFD): indicates end of preamble
  - Length: between 0 and 2^7-1=127
- **MAC frame**:
  - Frame control: flags to interpret the rest of the header and request of ack
  - Sequence number: used by ack
  - Addressing fields: optional (sender/receiver, PAN)
  - Frame check sequence (FCS): contains the CRC to be checked

IEEE 802.15.4: Power consumption

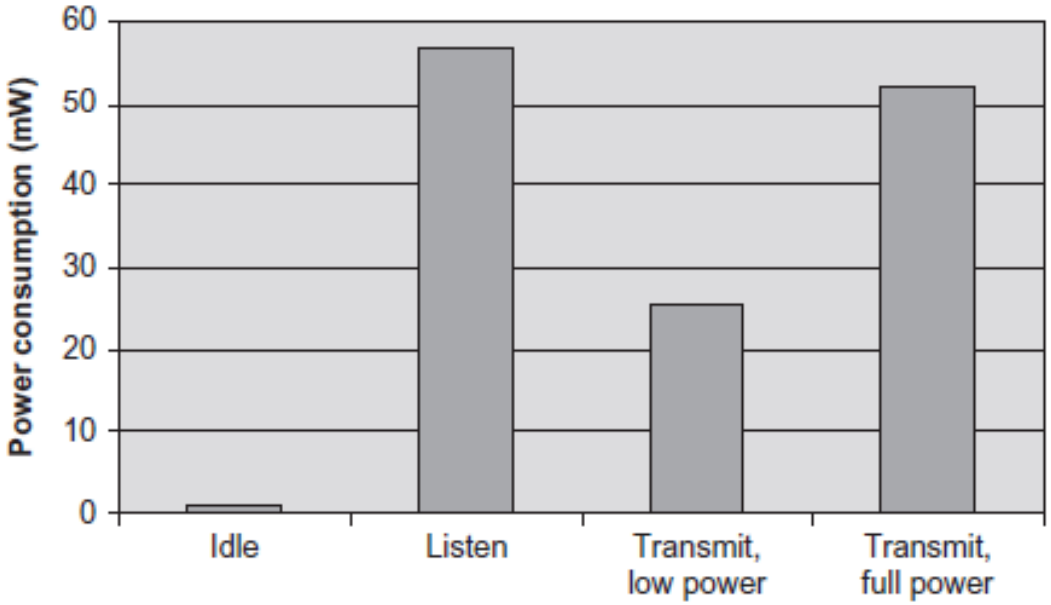
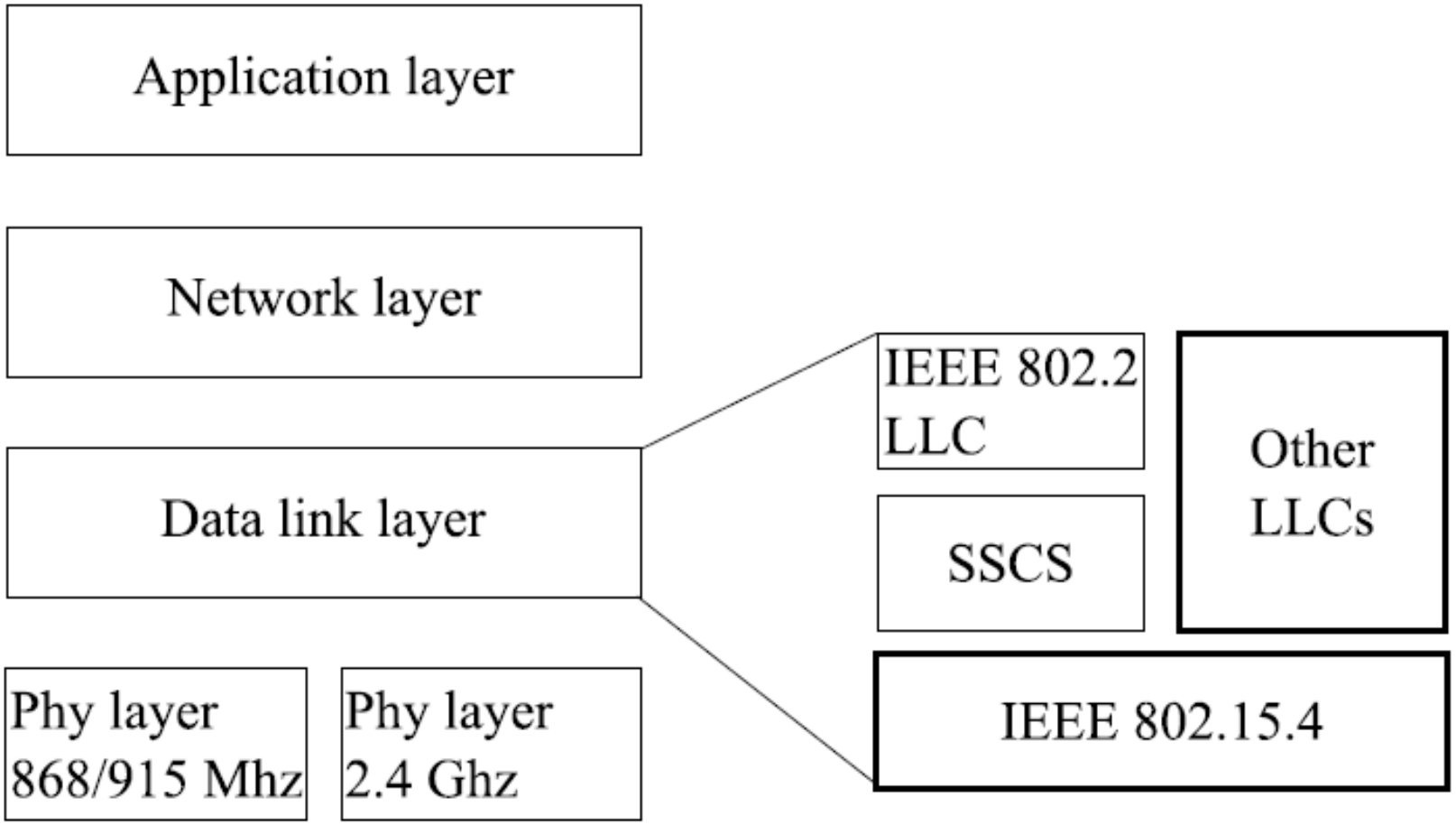


FIGURE 12.10

The power consumption of the CC2420 IEEE 802.15.4 radio transceiver.

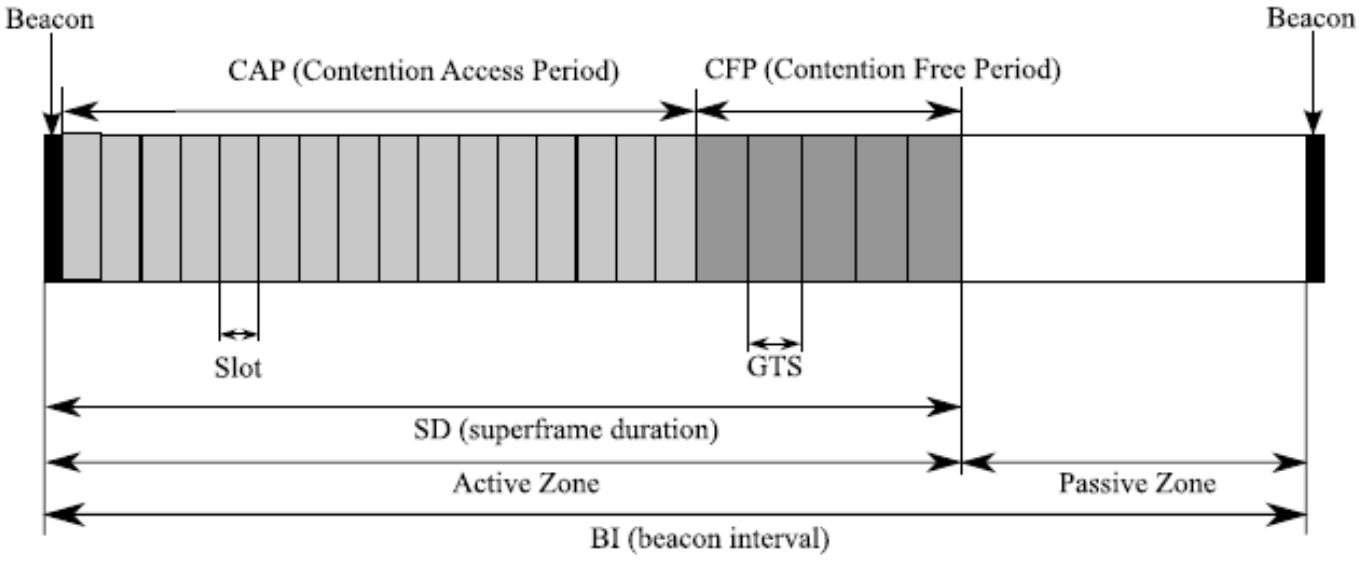
- Listen power consumption higher than transmit power consumption (needs to demodulate even if no useful signal is present)
- Transmit power can be set on a per-packet basis (between low and full)

The ISO/OSI Stack of IEEE 802.15.4



Becon-Enabled Mode

- The use of beacon packets allows to synchronize tne nodes.
- The time is divided into superframes.
- The nodes have, therefore, higher complexity, but duty cycle and channel access are better managed (GTSs).



Beaconless Mode

- No synchronization between nodes.
- Nodes' structures are simpler, but, obviously the performance of beacon-enabled networks is better.
- The MAC protocol differs in terms of time management, synchronization, and packet retransmissions.

CSMA/CA MAC Protocol: Unslotted Version

- The source node waits for a number of slots randomly generated in the interval $[0, 2^{BE_i} - 1]$.
- Once elapsed this time, the node verifies the channel status through a Clear Channel Assessment (CCA), i.e., the energy on the RF interface is measured over $8T_{\text{sym}}$.
- If the energy is above a threshold, the channel is busy; otherwise, it is assumed idle.
- If channel is idle, the node can transmit a packet in its queue. Otherwise, *NB* (number of backoff cycles for this packet) and *m* (maximum number of backoff cycles, denoted as backoff stage) are compared.
- If $NB = m$, the CSMA/CA is declared as failed. Otherwise, *BE* is incremented by 1. If $BE < BE_{\max}$, the procedure is repeated.

CSMA/CA MAC Protocol: Slotted Version

- The main difference is that all operations must be performed in the correspondence of a backoff boundary.
- CCAs of different nodes can collide. Therefore, two CCAs should be performed to prevent from collisions. However, this leads to waste in channel utilization.
- Obviously, in all cases collisions are still possible!!!

IEEE 802.11: Overview

- IEEE 802.11: wireless communication standard originally designed (late 90's) as a high-speed, short-range communication mechanism for laptops and general purpose PCs.
- WiFi is a brand name of the WiFi Alliance: identify equipment and software that is compatible with other WiFi and 802.11 systems
- Number of 802.11 devices worldwide: billions
- The widespread adoption of 802.11 makes deployment of smart objects easy
- Availability of 802.11 chipsets, routers, and network access cards reduces the cost of hardware for 802.11-enabled smart objects.
- Power consumption: typically one order of magnitude higher than that of 802.15.4 devices → new generation of low-power 802.11 transceivers (significant improvement of power consumption during sleeping periods and low power mode: turn off most of the functions, but resume quickly)

IEEE 802.11: standards

IEEE 802.11	Release Date	Frequency (GHz)	Max Data Rate (Mbps)	Range (m)		Status and Comments
				Indoor	Outdoor	
-1997	1997	2.4	2	20	100	Obsolete
a	1999	5/3.7	54	35/-	120/5k	Legacy systems
b	1999	2.4	11	35	140	Legacy systems
g	2003	2.4	54	38	140	Legacy systems
n	2009	2.4/5	600	70	250	Current systems
ac	2013	2.4/5	450/7,000	35		Next generation – just starting to be deployed.
ad	2012	60	7,000	10		Known as WiGig. Short-reach high data rate data transfers.
af	Est. 2016	0.470-0.710	568		6,000	Being called White-Fi because it uses unused TV spectrum.
ah		0.9	40			In development.
aj		45/60	7,000	10		Modification of 802.11ad for 45GHz band for use in China
ax	Est. 2019	2.4/5	450/10,000	35		Revision to 802.11ac to increase efficiency.

IEEE 802.11: foci of the standards

Task Group	Purpose	Status
PHY	Develop Three Physical Layer Options at 1–2 Mbps	Standard
MAC	Develop One Common MAC Layer	Standard
802.11a	High-Speed (54 Mbps) Physical Layer Extension in the 5.8 GHz Band	Standard
802.11b	High-Speed (11 Mbps) Physical Layer in the 2.4 GHz Band	Standard
802.11e	MAC Enhancements for QoS	Standard
802.11g	Even Higher Speed (54 Mbps) Physical Layer Extension in the 2.4 GHz Band	Standard
802.11i	MAC Enhancements for Enhanced Security	Standard
802.11n	Very High-Speed (100 Mbps) Physical Layer	Draft
802.11r	Mobility Handoffs	Draft
802.11s	Mesh Networks	Draft

IEEE 802.11: Low-Power

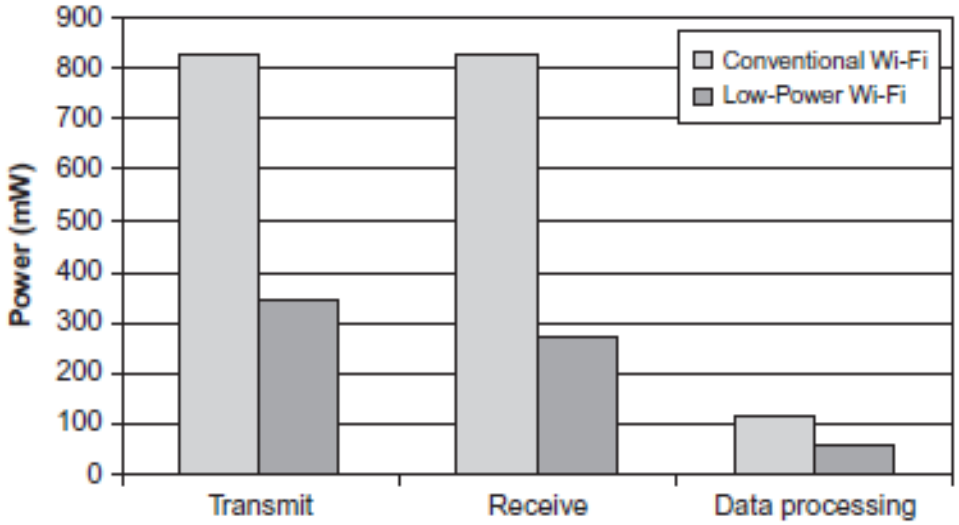


FIGURE 12.15

Comparison of power consumption in transmit, receive, and processing mode for conventional 802.11 and low-power 802.11.

Table 12.1 Power Consumption of Standby and Sleep Modes of a Conventional 802.11 Transceiver and of a Low-power 802.11 Transceiver

	Conventional 802.11 (mW)	Low-power 802.11 (mW)
Standby mode	N/A	0.018
Sleep mode	13	0.2

Note: Conventional 802.11 transceivers do not have a standby mode.

IEEE 802.11: network topology

- Two operational modes: infrastructure (access points) and ad-hoc (direct communications)

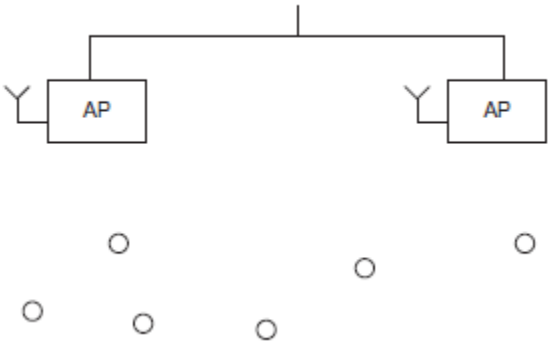


FIGURE 12.11
A network topology of 802.11 in access point mode. Each 802.11 transceiver is associated with an access point. A network may be served by more than one access point. The access points typically have wired network access.

- A node must associate itself with the AP → a group of nodes with an AP is called Basic Service Set (BSS) → with more than one AP, Extended Service Set (ESS) → Service Set IDentifier (SSID)
  - Node probes for a known or unknown SSID
  - AP replies with a probe response packet, with configuration parameters
  - If the network is ok, the node sends an authentication request frame
  - AP replies with association reply frame

IEEE 802.11: PHY layer

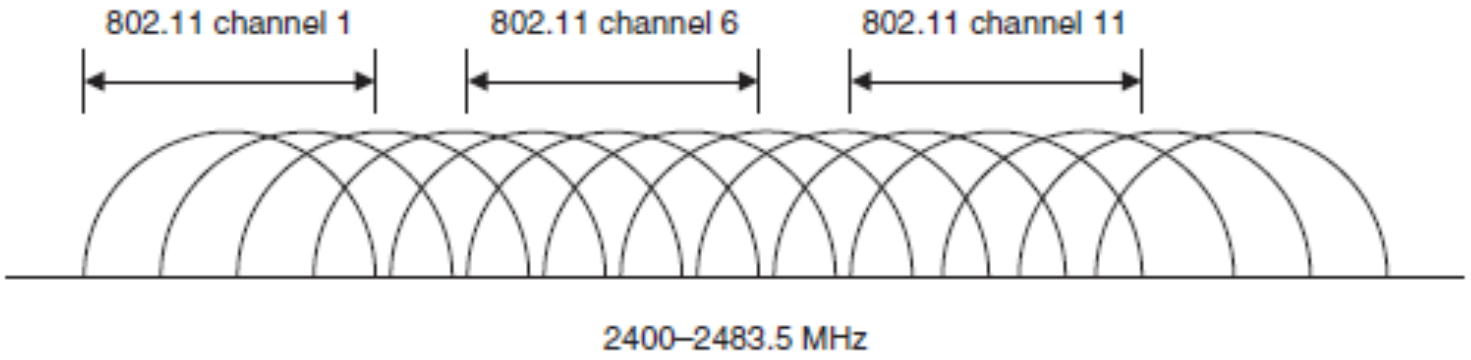


FIGURE 12.12

IEEE 802.11 operates on 14 different channels in the 2.4 GHz band. Channels 1, 6, and 11 do not overlap. Channels 12, 13, and 14 are not available in every country due to radio frequency licensing restrictions.

- Each 802.11 packet is sent with a fixed bit rate, but the bit rate may differ between packets (higher rate with a good communication channel)
- Broadcast packets are sent with a lower bit rate than packets addressed to a specific host to be compatible with older versions

IEEE 802.11: MAC layer

- MAC layer of 802.11: based on Carrier Sense Multiple Access with Collision Avoidance (CSMA/CA)

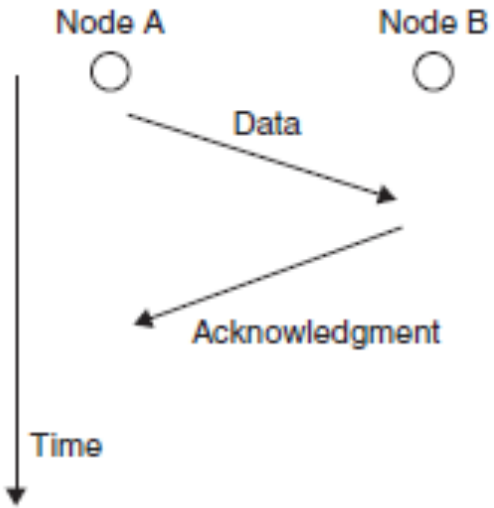


FIGURE 12.13

The IEEE 802.11 MAC layer uses acknowledgment packets that inform the sender that a packet was successfully received by the receiver.

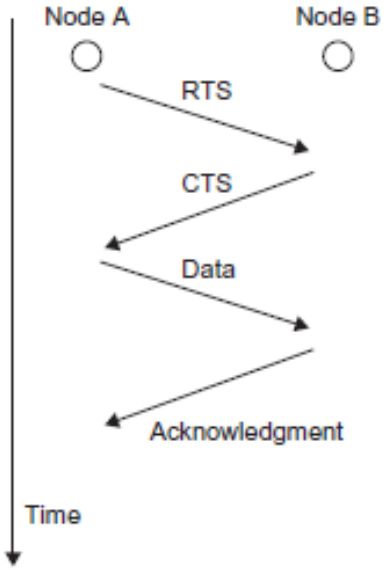


FIGURE 12.14

The RTS/CTS mechanism of the 802.11 MAC layer. To send a packet, a node first sends an RTS packet. If the access point grants the request, it sends a CTS packet and the node can send its packet.

- Mechanism of ready-to-send (RTS) and clear-to-send (CTS) solves the hidden and exposed terminal problems (see later)

PLC: a Quick Overview (1/2)

- Power line communications: eletrical lines are everywhere
- PLC: provides smart objects with data and energy (no need for batteries)

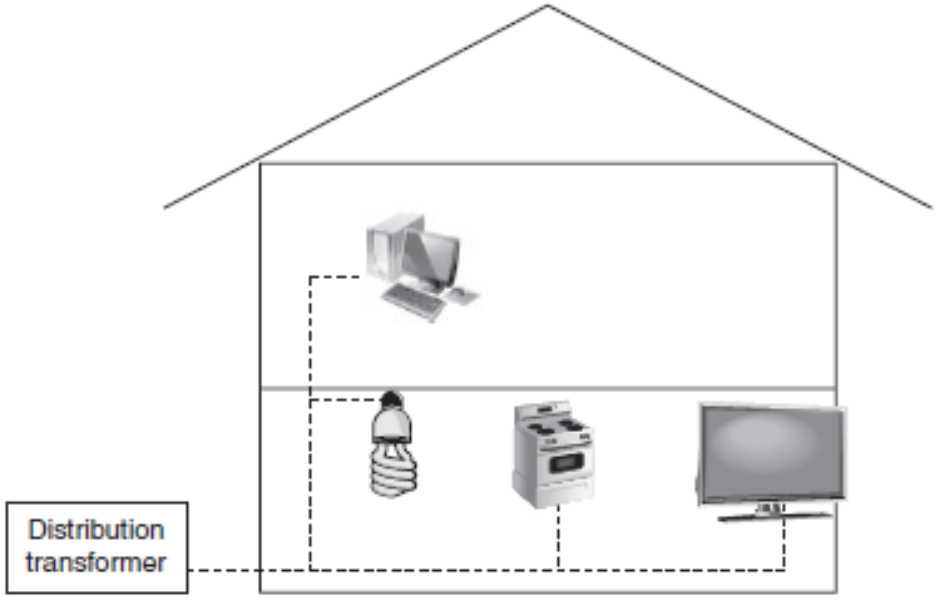


FIGURE 12.16

PLC network within a home.

PLC: a Quick Overview (2/2)

- Data transported across the 50 Hz or 60 Hz electrical distribution network: electrical equipment may add a significant noise
- PLC for short-range communication: only within one electrical domain such as a home or an office building (modulated data signal cannot traverse the electrical voltage transformers outside the electrical domain)
- PHY: copper wires (carry high-power AC signal→ noisy; add interferences from plugged-in devices) → very robust communication mechanisms → data rates from kb/s to Mb/s
- MAC: CSMA/CA (unlike transceivers for Ethernet cables, a node transmitting over an electrical line cannot receive)
- Power savings still important: (i) reduce overheating and (ii) avoid compromising overall power savings

UNIVERSITÀ DI PARMA

IoT Lab

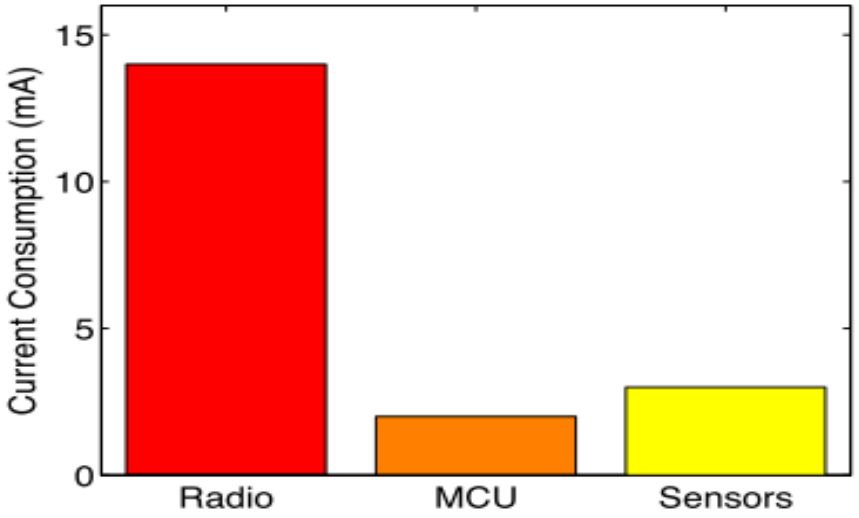
Energy-efficient MAC Protocols

Main sources

- A. Bachir, M. Dohler, T. Watteyne, K. K. Leung, "MAC Essentials for Wireless Sensor Networks," *IEEE Communications Surveys & Tutorials,* vol.12, no.2, pp.222-248, Second Quarter 2010. doi: 10.1109/SURV.2010.020510.00058
- [MACA] P. Karn,"MACA - a New Channel Access Method for Packet Radio", in ARRL/CRRL Amateur Radio 9tt,~ Computer Networking Conference, pp. 134-140, 1990.
- [PAMAS] Suresh Singh and C. S. Raghavendra. 1998. PAMAS—power aware multi-access protocol with signalling for ad hoc networks. *SIGCOMM Comput. Commun. Rev.* 28, 3 (July 1998), 5-26. DOI=10.1145/293927.293928 http://doi.acm.org/10.1145/293927.293928
- [STEM] C. Schurgers, V. Tsiatsis, S. Ganeriwal, M. Srivastava, "Optimizing sensor networks in the energy-latency-density design space," *IEEE Transactions on Mobile Computing*, vol.1, no.1, pp.70,80, Jan-Mar 2002. doi: 10.1109/TMC.2002.1011060
- [B-MAC] J. Polastre, J. Hill, and D. Culler, "Versatile low power media access for wireless sensor networks." Proc. 2nd international conference on Embedded networked sensor systems (SenSys '04), pp. 95-107, Baltimore, MA, USA November 2014. doi>10.1145/1031495.1031508
- [S-MAC] W. Ye, J. Heidemann, D. Estrin, "An energy-efficient MAC protocol for wireless sensor networks," *Proc. 21st Annual Joint Conference of the IEEE Computer and Communications Societies* (INFOCOM 2002) , pp.1567-1576, 2002. doi: 10.1109/INFCOM.2002.1019408
- [X-MAC] Michael Buettner, Gary V. Yee, Eric Anderson, and Richard Han. 2006. X-MAC: a short preamble MAC protocol for duty-cycled wireless sensor networks. In *Proceedings of the 4th international conference on Embedded networked sensor systems* (SenSys '06). ACM, New York, NY, USA, 307-320. DOI=10.1145/1182807.1182838
- [ContikiMAC] A. Dunkels, "The ContikiMAC Radio Duty Cycling Protocol", SICS Technical Report T2011:13. ISSN 1100-3154. December 2011

Power consumption

- Power consumption is one of the biggest challenges because:
  - nodes typically have a limited source of energy (finite battery);
  - battery difficult to replace (and to waste).



(a) Active Mode

Radio mode	Power consumption (mW)
Transmit (T_x)	14.88
Receive (R_x)	12.50
Idle	12.36
Sleep	0.016

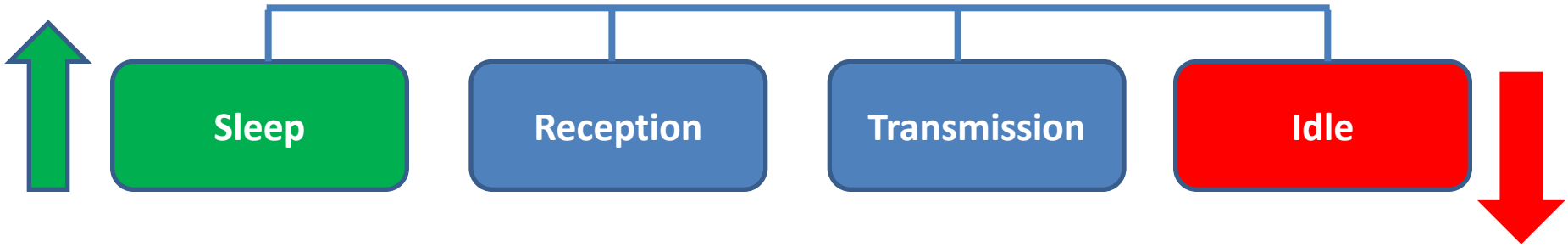
Sources of power consumption

- Useful power consumption:
  - Transmitting or receiving data.
  - Processing queries requests.
  - Forwarding queries and data to the neighbours.
- Wasteful power consumption:
  - Idle listening to the channel “waiting for possible traffic”.
  - Retransmitting because of collisions
  - Overhearing: a node receives (and processes) a packet which was not destined to it.
  - Generating and handling control packets (overhead).

Power consumption in WSNs

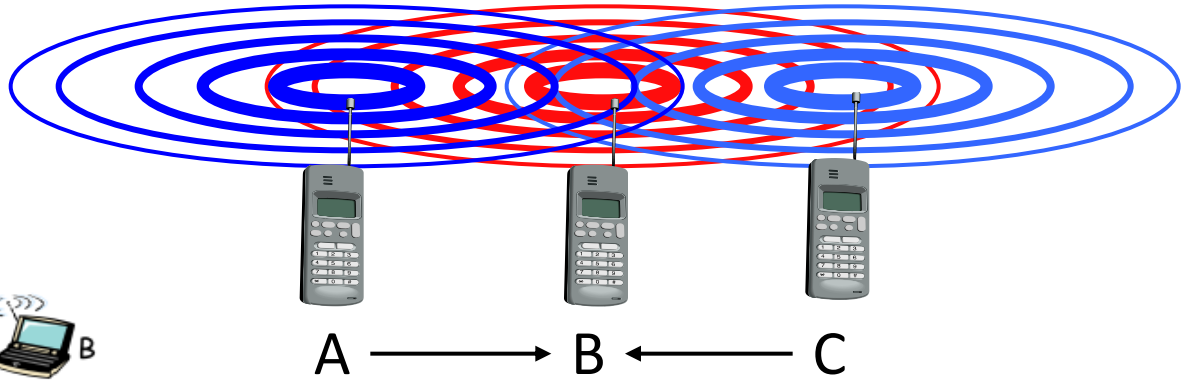
How to minimize the energy consumption of sensor nodes while meeting the application requirements?

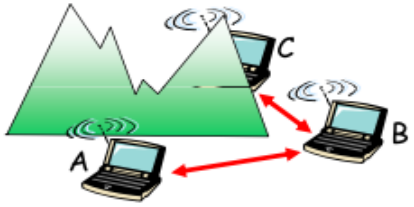
- Use Protocols that aim mainly to increase the sleep periods as much as possible



Hidden Terminal Problem

- A sends to B, C cannot receive A
- C wants to send to B, C senses a “free” medium (CS=carrier sensing fails)
- collision at B, A cannot receive the collision (CD=collision deteciont fails in wireless systems; works in wired systems)
- A is “hidden” for C



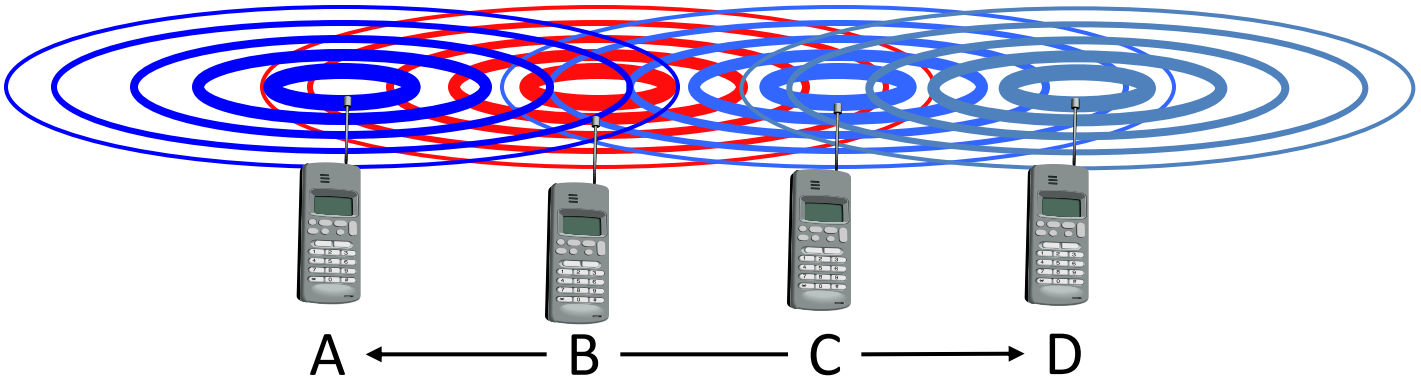


Hidden terminal problem
- B, A hear each other
- B, C hear each other
- A, C can not hear each other

means A, C unaware of their interference at B

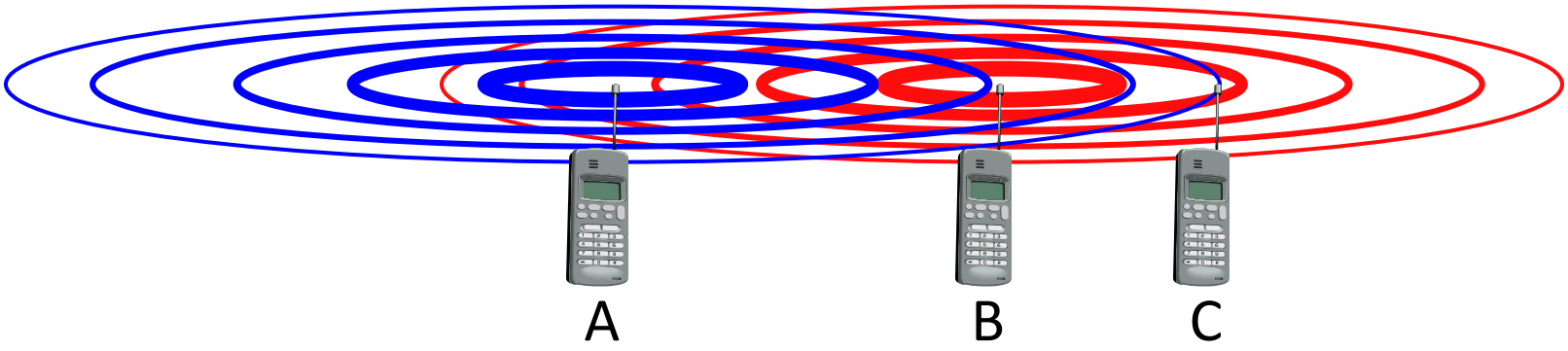
Exposed Terminal Problem

- B sends to A, C wants to send to D
- C has to wait, as it senses a medium in use
- since A is outside the radio range of C waiting is not necessary
- C is “exposed” to B



Near-Far Problem

- Terminals A and B send, C receives
  - the signal of terminal B hides A's signal
  - C cannot receive A



- This is also a severe problem for CDMA networks
- Precise power control required

Centralized Access Methods (Cellular)

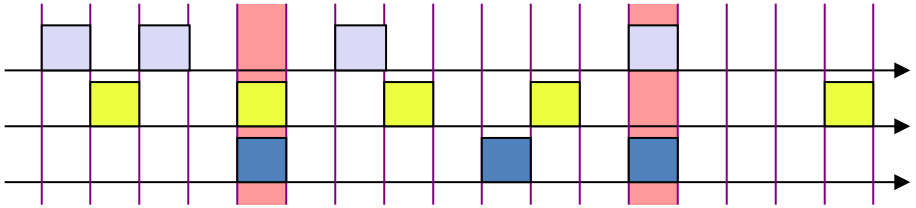
- SDMA (Space Division Multiple Access)
  - Segment space into sectors, use directional antennas
  - Use cells to reuse frequencies
- FDMA (Frequency Division Multiple Access)
  - Assign a certain frequency to a transmission channel for the entire transmission duration
  - Permanent (radio broadcast), slow hopping (GSM), fast hopping (FHSS, Frequency Hopping Spread Spectrum → bluetooth)
- TDMA (Time Division Multiple Access)
  - Assign a fixed sending frequency (possibly the entire spectrum) for a certain amount of time
- CDMA (Code Division Multiple Access)
- Combinations!

Traditional MAC Protocol Classification

- Centralized/Single-Hop Protocols (see previous page)
  - A base station/controller coordinates all traffic (SDMA, FDMA, TDMA, CDMA)
- Contention Protocols (Aloha, Carrier Sense Multiple Access-CSMA)
  - Transmit when you feel like transmitting (Aloha)
  - Retry if collision, try to minimize collisions, additional reservation modes (CSMA)
  - Problem: Receiver must be awake as well
- Scheduling Protocols
  - Use a “pre-computed” schedule to transmit messages
  - Distributed (non-centralized) and adaptive solutions are difficult
- Hybrid protocols
  - E.g. contention with reservation → scheduling (e.g., IEEE 802.15.4 with beacons)
  - Specific (“cross-layer”) solutions

Slotted Aloha

- We assume that the stations are perfectly synchronous
- In each time slot each station transmits with probability *p*.



$$P_1 = \Pr[\text{Station 1 succeeds}] = p(1-p)^{n-1}$$

$$P = \Pr[\text{any Station succeeds}] = nP_1$$

$$\text{maximize } P: \frac{dP}{dp} = n(1-p)^{n-2}(1-pn) \overset{!}{=} 0 \quad \Rightarrow \quad pn = 1$$

$$\text{then, } P = (1-\frac{1}{n})^{n-1} \geq \frac{1}{e}$$

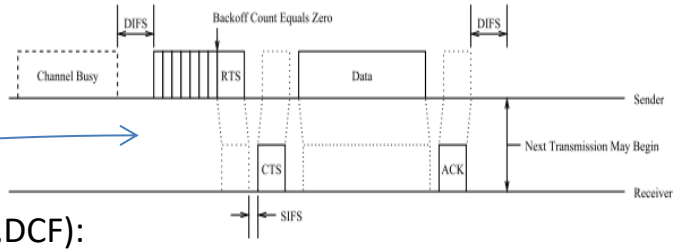
- In Slotted Aloha, a station can transmit successfully with probability at least 1/*e*, i.e., about 36% of the time.
- In pure Aloha (unslotted), a station can transmit successfully with probability at least 1/2e, i.e., about 18% of the time.

Adaptive Slotted Aloha

- Idea: change the access probability as a function of the number of stations
- How can we estimate the current number of stations in the system?
- Assume that stations can distinguish whether 0, 1, or more than 1 stations transmit in a time slot.
- Model:
  - Number of stations that want to transmit: n.
  - Estimate of n: $\hat{n}$
  - Transmission probability: $p = 1/\hat{n}$
- Idea:
  - If you see that nobody transmits, increase p.
  - If you see that more than one transmits, decrease p.

Carrier Sense Multiple Access (CSMA)

- Listen before transmit: if the channel is free, then transmit
- Various versions:
  - non-persistent: if the device detects activity on the channel, it performs a back off by waiting before attempting to transmit.
  - *p*- persistent: if it detects activity on the channel, it keeps on sensing the channel instead of delaying.
- "Basic" CSMA requires devices to remain in the receive state (not idle) when not transmitting → the transceiver consumes energy too quickly.
- Various variations
  - CSMA/CD: collision detection (Ethernet, IEEE 802.3)
  - CSMA/CA: collision avoidance, LIFO (IEEE 802.15.4, IEEE 802.11-PCF,DCF): combine with Ready-to-send (RTS) and Clear-to-Send (CTS) mechanisms → RTS messages may collide
  - IEEE 802.11 uses Network Allocation Vector (NAV), virtual carrier-sensing mechanism: the MAC frame header contains the packet transmission duration



Backoff

- Backoff protocols rely on acknowledgements.

- Binary exponential backoff:
  - If a packet has collided k times (no ACK for k times), we set $p = 2^{-k}$
  - Or alternatively: wait from random number of slots in $[1,...,2^k]$

- It has been shown that binary exponential backoff is not stable for any arrival rate $\lambda > 0$ if there are infinitely many potential stations

- Interestingly when there is a finite number of stations, binary exponential backoff becomes unstable with $\lambda > 0.568$;
  Polynomial backoff however, remains stable for any $\lambda < 1$.

Network Performance

Moving towards WSNs

- Traditional MAC protocols
  - High throughput
  - Low latency
  - Fairness
  - Mobility
  - Energy? → neglected

- MAC protocols for WSNs:
  - Primary goals
    - Energy conservation
    - Scalability and adaptivity
  - Secondary goals
    - Throughput
    - Latency
    - Fairness

Energy-efficient MAC Protocols

- In WSNs, energy is often more critical than throughput.
  - The radio transceiver should be turned off as much as possible.
- Energy management considerations have a big impact on MAC protocols.
  - Idle listening costs about as much energy as transmitting
- In the following: a few ideas, taken from some known protocols that try to balance throughput and energy consumption:
  - MAC,PAMAS, STEM, B-MAC, X-MAC, WiseMAC, S-MAC, **ContikiMAC**
  - Many of the hundreds of MAC protocols that were proposed have similar ideas…

Unscheduled MAC protocols

- "Evolutions" of CSMA
- Advantages:
  - they can adapt for changes "in the node density, traffic load or the topology" better than scheduled protocols.
  - The nodes do not have to be synchronized together.
- Disadvantages:
  - All nodes need to listen to the channel
- Examples:
  - MACA (1990)
  - PAMAS (1998)
  - STEM (2002)
  - B-MAC (2004), WiseMAC (
  - X-MAC (2006)

MACA (Multiple Access with Collision Avoidance) - 1990

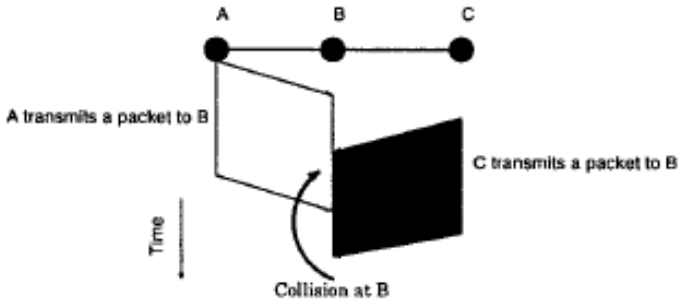
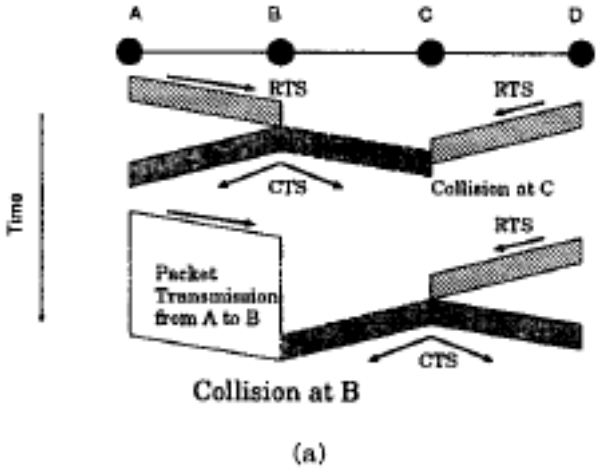


Figure 2: The hidden terminal problem.

- Hidden terminal problem: partially solved with the RTS/CTS mechanism introduced by MACA



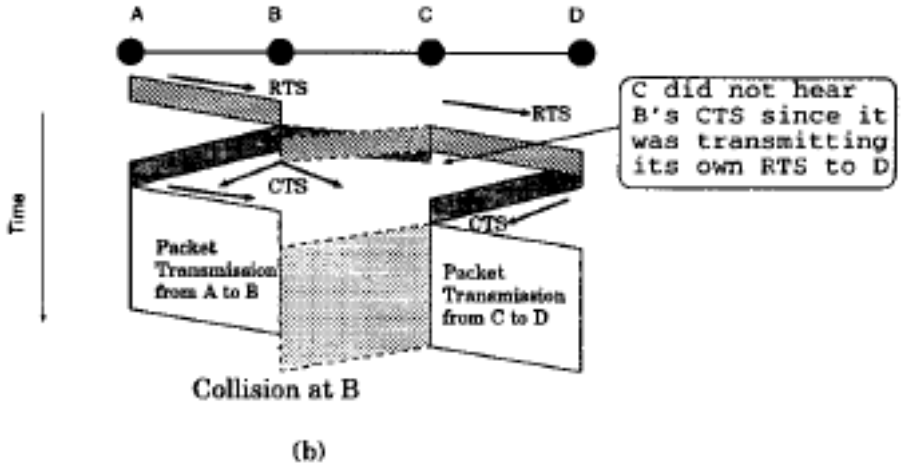


Figure 3: Hidden terminal problem in MACA.

PAMAS (Power Aware Multi-Access protocol with Signaling for Ad-Hoc Networks) - 1998

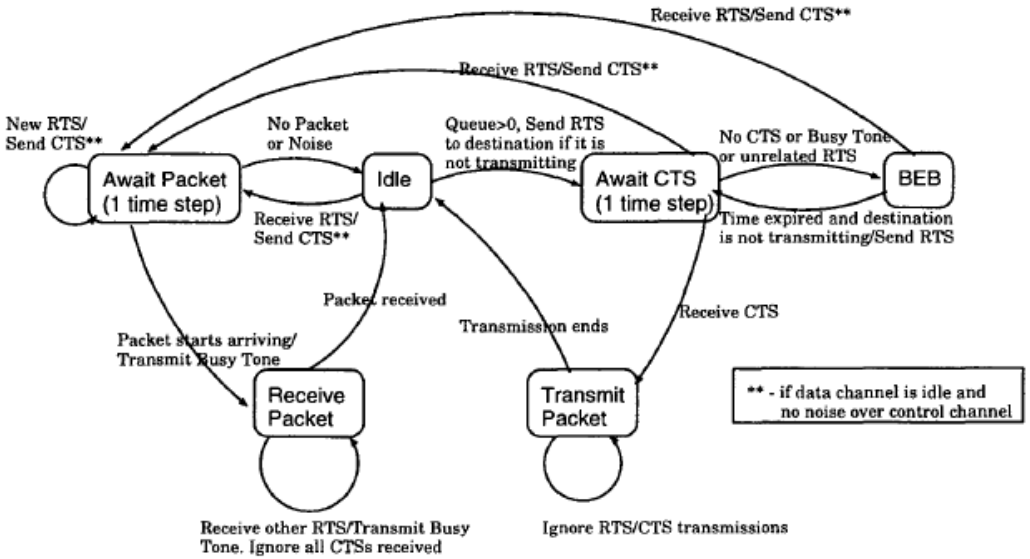
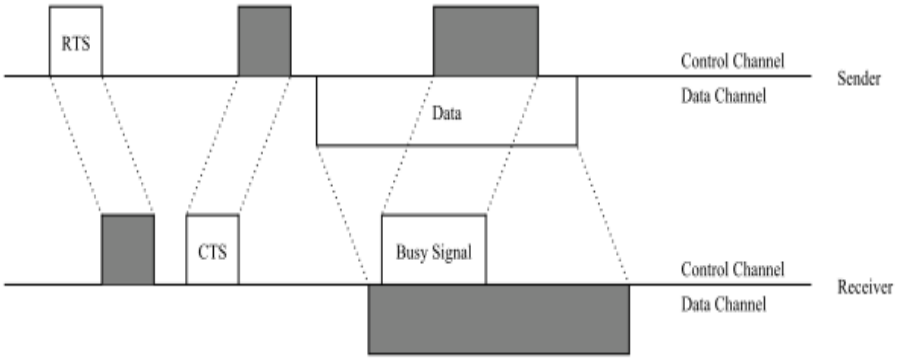


Figure 4: The PAMAS Protocol.



- Basic idea: use a *signaling channel* separated from data channel
- Advantages:
  - Prevent collisions
- Disadvantages:
  - Multiple radio requirement
  - Increase energy consumption
  - Increase device complexity and cost

STEM (Sparse Topology and Energy Management) - 2002

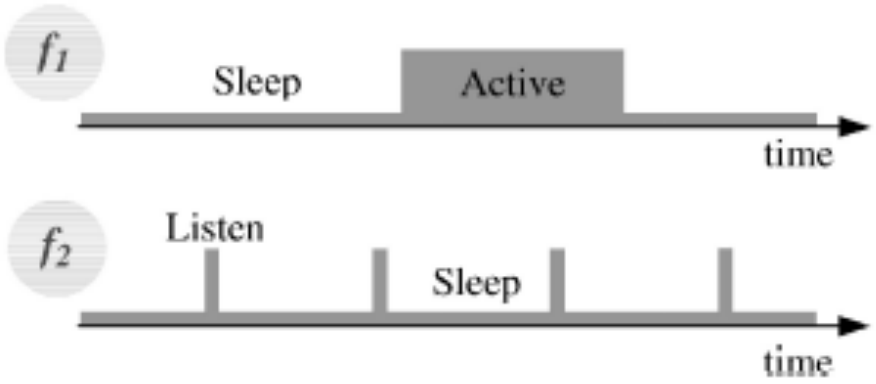


Fig. 3. Separate data and wakeup using two radios.

- Basic idea: uses two different channels: the wakeup channel (f_2) and the data channel (f_1)
- Requires two transceivers in each node

Fig. 4. Separate data and wakeup in time.

- Could "mix" the two channels → synchronization is required

STEM-B and STEM-C - 2002

- STEM-B: a sensor node wakes up a neighbor by transmitting a beacon (no RTS/CTS)
  - Advantages:
    - Lower Latency
  - Disadvantages:
    - More complex
    - High energy consumption
- STEM-T: a sensor node wakes a neighbor by transmitting a busy tone (contains no destination address) of sufficient length so that the destination will have a high probability of sensing
  - Disadvantages:
    - High latency
    - Results in overhearing (stops the transmission from nodes which would not interfere)

“Berkeley MAC” (B-MAC) – 2004

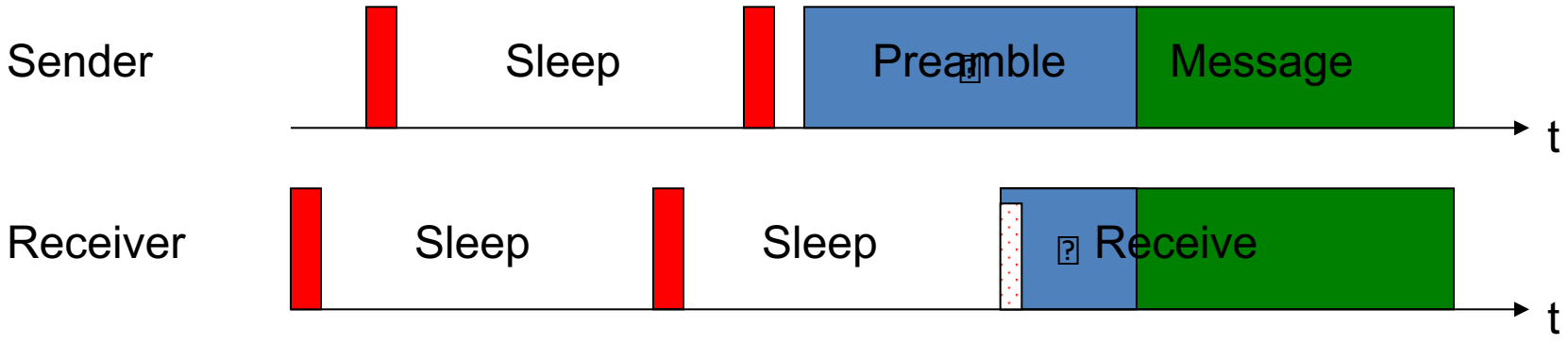
(Versatile low power media access for wireless sensor networks)

- Goals
  - Low power operation
  - Effective collision avoidance
  - Simple implementation (small code)
  - Energy-efficient at both low and high data rates
  - Reconfigurable by upper layers
  - Tolerant to changes on the network
  - Scalable to large number of nodes
- Strategy
  - Nodes wake up for a short period and check for channel activity.
  - Return to sleep if no activity is detected.
  - If a sender wants to transmit a message, it sends a long preamble to make sure that the receiver is listening for the packet (STEM-T) → preamble has the size of a sleep interval
- Very robust
  - No synchronization required
  - Instant recovery after channel disruption

B-MAC's Features

- Clear Channel Assessment (CCA)
- Low Power Listening (LPL) using preamble sampling
- Hidden terminal and multi-packet mechanisms not provided: they should be implemented, if needed, by higher layers

- CCA on/off
- Acknowledgements on/off
- Initial and congestion backoff on a per-packet basis
- Configurable check interval and preamble length



B-MAC's Lifetime Model

Operation	Time (s)		I (mA)	
Initialize radio (b)	350E-6	$t_{r_{init}}$	6	$c_{r_{init}}$
Turn on radio (c)	1.5E-3	t_{ron}	1	c_{ron}
Switch to RX/TX (d)	250E-6	$t_{rx/tx}$	15	$c_{rx/tx}$
Time to sample radio (e)	350E-6	t_{sr}	15	c_{sr}
Evaluate radio sample (f)	100E-6	t_{ev}	6	c_{ev}
Receive 1 byte	416E-6	t_{rxb}	15	c_{rxb}
Transmit 1 byte	416E-6	t_{txb}	20	c_{txb}
Sample sensors	1.1	t_{data}	20	c_{data}

Table 2: Time and current consumption (I) for completing primitive operations of a monitoring application using the Mica2 mote and CC1000 transceiver. Identifiers on each operation map back to the activities of acquiring a radio sample in Figure 3.

Notation	Parameter	Default
c_{sleep}	Sleep Current (mA)	0.030
C_{batt}	Capacity of battery (mAh)	2500
V	Voltage	3
$L_{preamble}$	Preamble Length (bytes)	271
L_{packet}	Packet Length (bytes)	36
t_i	Radio Sampling Interval (s)	100E-3
n	Neighborhood Size (nodes)	10
r	Sample Rate (packets/s)	1/300
t_l	Expected Lifetime (s)	-

Table 3: Parameters for a monitoring application running B-MAC. The first three parameters are specific to Mica2 motes; the next three are default values for B-MAC parameters on the Mica2; the remaining parameters are application semantics affecting B-MAC's performance. Each parameter affects the node's total energy consumption, E.

$$t_{tx} = r \times (L_{preamble} + L_{packet}) t_{txb}$$
$$E_{tx} = t_{tx} c_{txb} V$$

$$t_d = t_{data} \times r$$
$$E_d = t_d c_{data} V$$

$$t_{sleep} = 1 - t_{rx} - t_{tx} - t_d - t_{listen}$$
$$E_{sleep} = t_{sleep} c_{sleep} V$$

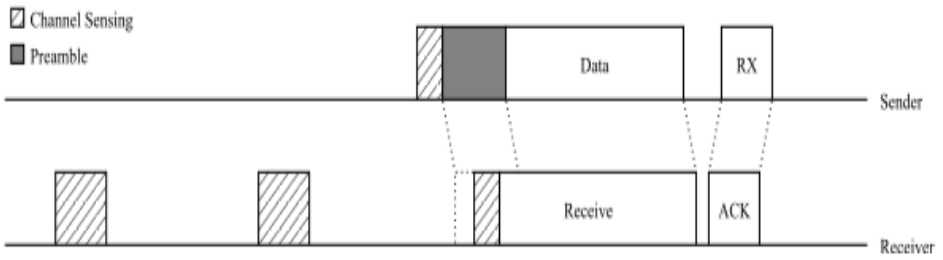
$$E = E_{rx} + E_{tx} + E_{listen} + E_d + E_{sleep}$$

$$t_{rx} \leq nr(L_{preamble} + L_{packet}) t_{rxb}$$
$$E_{rx} = t_{rx} c_{rxb} V$$

$$E_{sample} = 17.3\mu\text{J}$$
$$t_{listen} = (t_{r_{init}} + t_{ron} + t_{rx/tx} + t_{sr}) \times \frac{1}{t_i}$$
$$E_{listen} \leq E_{sample} \times \frac{1}{t_i}$$

B-MAC's Issues

- Problem: All nodes in the vicinity of a sender wake-up and wait for the packet → overhearing
  - Solution 1: Send wake-up packets instead of preamble, wake-up packets tell when data is starting so that receiver can go back to sleep as soon as it received one wake-up packet.
  - Solution 2: Just send data several times such that receiver can tune in at any time and get tail of data first, then head.
- Communication costs are mostly paid by the sender.
  - The preamble length can be much longer than the actual data length.
- Idea: Learn wake-up schedules from neighboring nodes.
  - Start sending preamble just before intended receiver wakes up.
  - **WiseMAC** → encode wake-up pattern in the ACK message

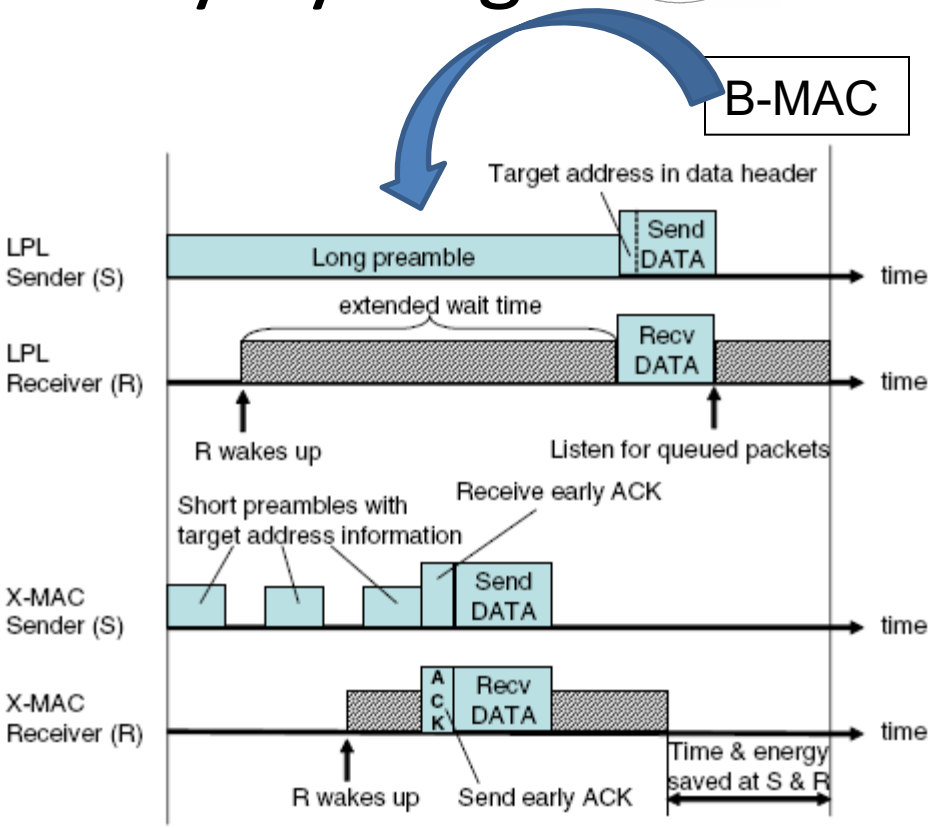


X-MAC: Principles – 2006

- Short preamble
  - Reduce latency and reduce energy consumption
- Target (destination) in preamble
  - Minimize overhearing problem.
- Strobed preamble
  - Reduces latency for the case where destination is awake before preamble completes.
- Reduced per-hop latency and energy
- Dynamic duty-cycling (depending on network conditions)

X-MAC: Asynchronous Duty Cycling

- Overhearing energy loss is proportional to number of receivers in range.
- Short preamble packets.
- Non-receivers return to sleep more quickly.
- **Bad in high density sensor networks** (collisions between strobing packets)



Scheduled MAC protocols

- Idea: try to reduce the energy consumption by *coordinating* sensor nodes with a common schedule
- Advantages:
  - Saving the power from being wasted by turning off the radio outside the allocated time slot.
  - Limits collisions, idle listing, and overhearing
- Disadvantages:
  - When a sensor node enters the network, it must wait till it learns the schedule → some delay exist
  - Cost of increased number of exchanged messages
  - No flexibility to changes in sensor nodes' density or movements.
  - All sensors have to be synchronized.
- Example:
  - S-MAC (2002)
  - ContikiMAC (from 2008 on)

Sensor MAC (S-MAC) - 2002

- Tries to reduce waste of energy from all four sources of energy inefficiency
  1. Collisions – by using RTS and CTS
  2. Overhearing – by switching the radio off when transmission is not meant for that node
  3. Control Overhead – by message passing
  4. Idle listening – by periodic listen and sleep
- At the cost of:
  - some reduction in per-hop fairness and latency
  - but does not reduce end-to-end fairness and latency

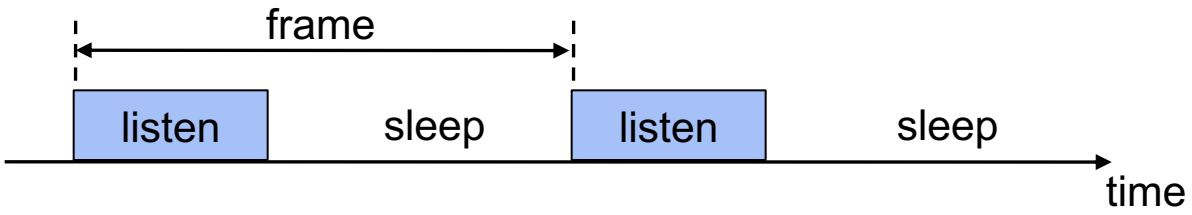
S-MAC: Assumptions

- Network assumptions:
  - Composed of many small nodes deployed in ad hoc fashion (random)
  - Most communications will be between nodes as peers, rather than to/from a single base station
  - Nodes must self-configure
- Application assumptions:
  - Dedicated to a single application or a few collaborative applications
  - Involves in-network processing to reduce traffic and increase life time
  - Applications have long idle periods and can tolerate some latency

S-MAC: Approach

- [Periodic listening and sleeping] Coarse-grained TDMA-like sleep/awake cycles:
  - the sensor node periodically goes to the fixed listen/sleep cycle.
  - A time frame in S-MAC is divided into two parts: one for a listening session and the other for a sleeping session.

increased latency



- [Message passing] All nodes choose and announce awake schedules.
  - synchronize to awake schedules of neighboring nodes.
- [Collision and overhearing avoidance] Uses RTS/CTS to resolve contention during listen intervals.
  - And allows interfering nodes to go to sleep during data exchange.

S-MAC: Possible Scenarios

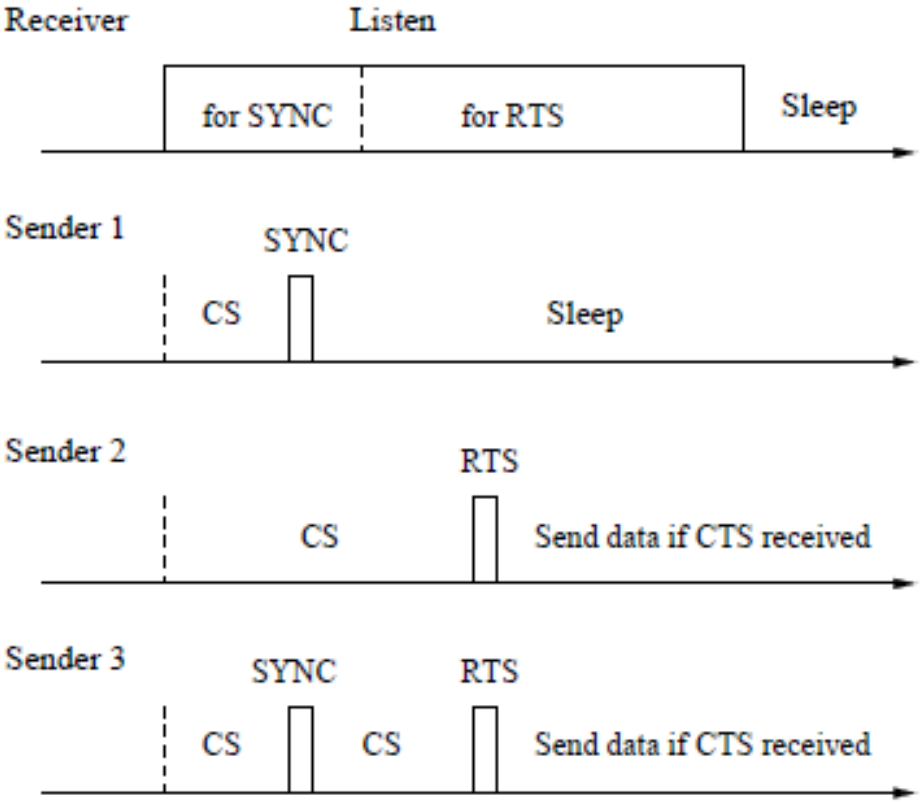
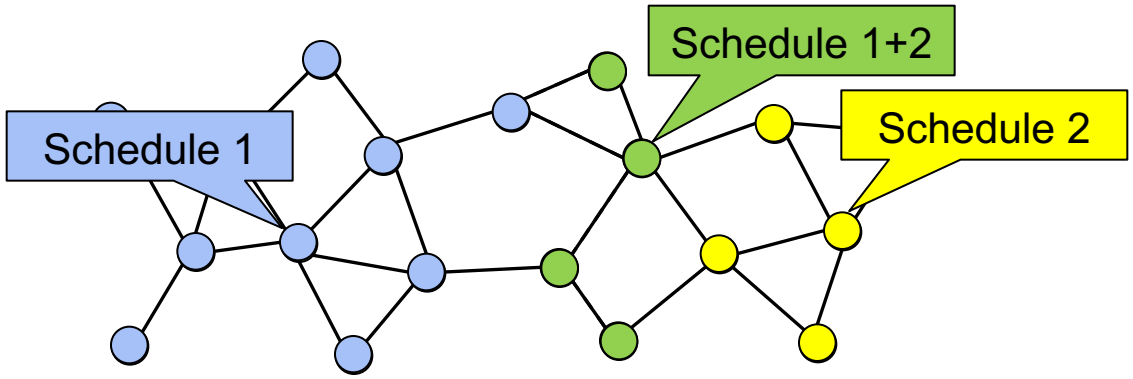


Fig. 3. Timing relationship between a receiver and different senders. CS stands for carrier sense.

Figure 3 also shows the timing relationship of three possible situations that a sender transmits to a receiver. CS stands for carrier sense. In the figure, sender 1 only sends a SYNC packet. Sender 2 only wants to send data. Sender 3 sends a SYNC packet and a RTS packet.

S-MAC: A Problem

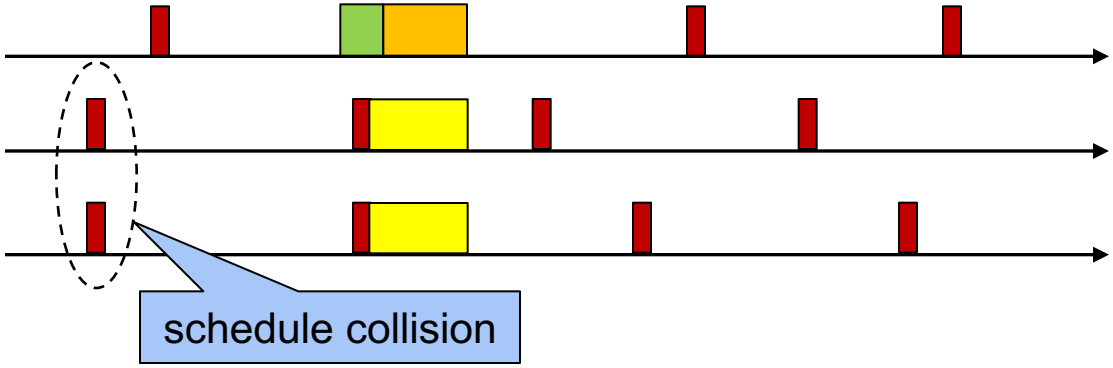
- Nodes may have to follow multiple schedules to avoid network partition.



- A fixed sleep/awake ratio is not always optimal.
  - Variable load in the network.
- Idea: Adapt listen interval depending on the current network load → T-MAC

Hybrid MAC Protocols

- Protocols may use information from upper layers to further improve their performance.
  - Information about neighborhood
  - Routing policies
- Minimize costly overhearing of neighboring nodes
  - Inform them to change their channel sniff patterns



- Use randomization to resolve schedule collisions
- Example: I. Rhee, A. Warrier, M. Aia, and J. Min, “Z-MAC: a Hybrid MAC for Wireless Sensor Networks”, Sensys 2005.

ContikiMAC – 2011 (Contiki 2.5)

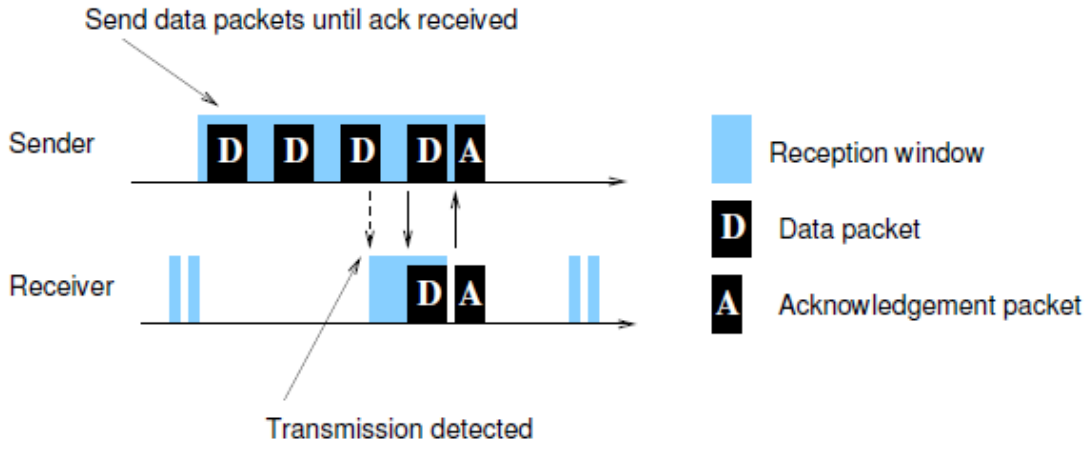


Figure 1: ContikiMAC: nodes sleep most of the time and periodically wake up to check for radio activity. If a packet transmission is detected, the receiver stays awake to receive the next packet and sends a link layer acknowledgment. To send a packet, the sender repeatedly sends the same packet until a link layer acknowledgment is received.

ContikiMAC: Broadcast

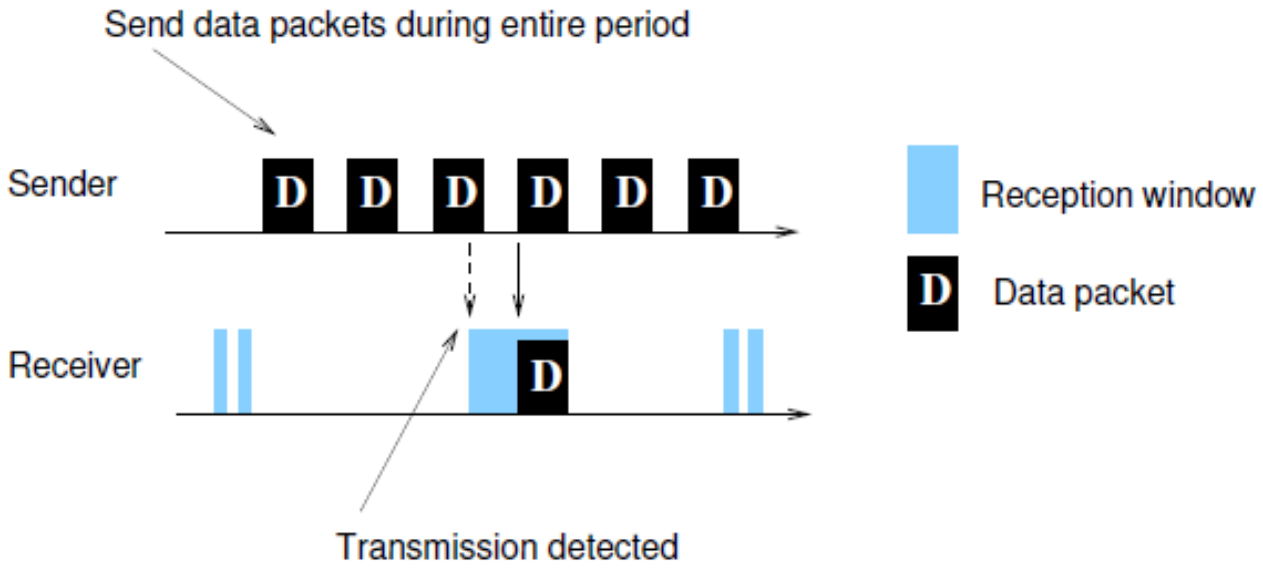


Figure 2: Broadcast transmissions are sent with repeated data packets for the full wake-up interval.

ContikiMAC: CCA timing

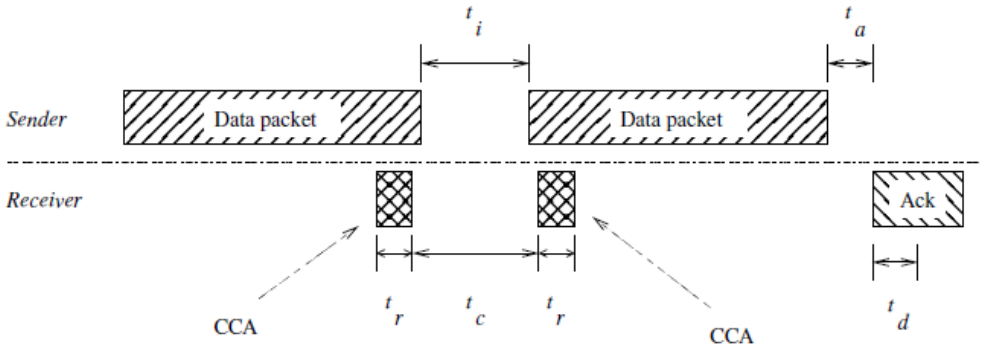


Figure 3: The ContikiMAC transmission and CCA timing.

The ContikiMAC timing is shown in Figure 3. The timing requirements from the figure are:

t_i: the interval between each packet transmission.

t_r: the time required for a stable RSSI, needed for a stable CCA indication.

t_c: the interval between each CCA.

t_a: the time between receiving a packet and sending the acknowledgment packet.

t_d: the time required for successfully detecting an acknowledgment from the receiver.

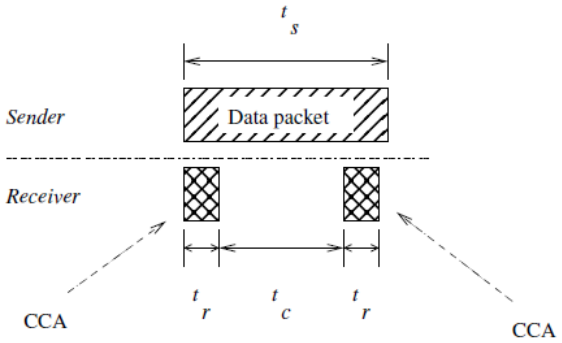


Figure 4: A packet transmission must be long enough so that it does not fall between to subsequent CCAs.

Timing constraints

$$t_a + t_d < t_i < t_c < t_c + 2t_r < t_s.$$

ContikiMAC: Transmission Phase Lock

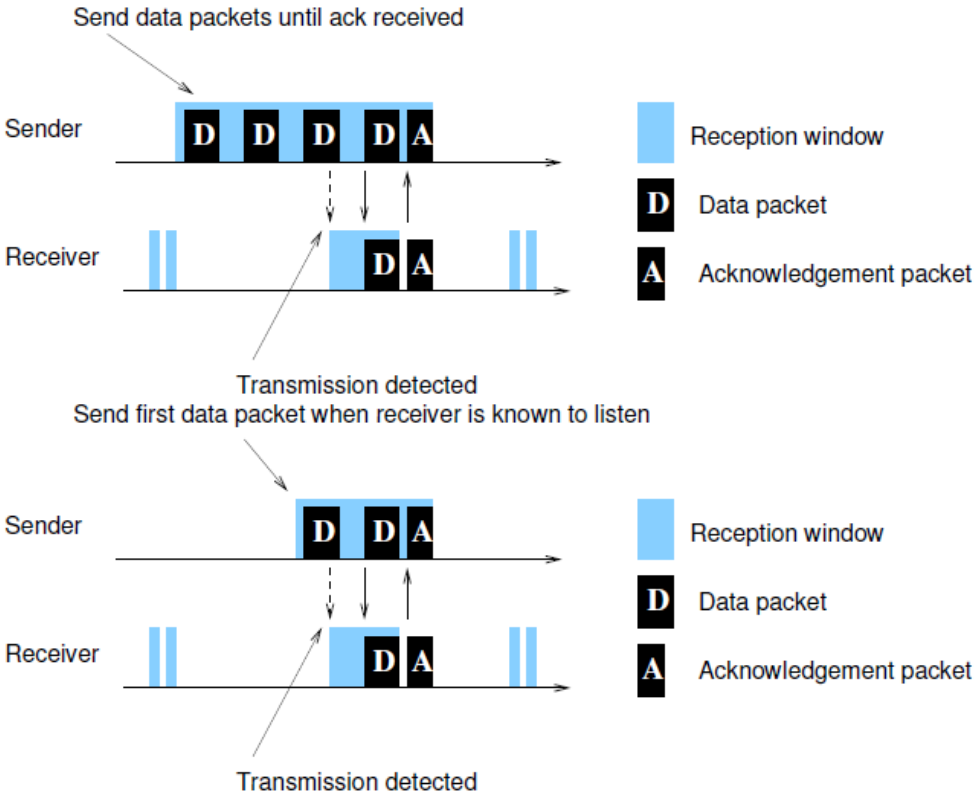


Figure 6: Transmission phase-lock: after a successful transmission, the sender has learned the wake-up phase of the receiver and subsequently needs to send fewer transmissions.

UNIVERSITÀ DI PARMA

IoT Lab

6LoWPAN

Main source

- Shelby & Bormann, *6LoWPAN: The Wireless Embedded Internet*, ISBN: 978-0-470-74799-5, (c) 2009 John Wiley & Sons Ltd
- J. P. Vasseur and A. Dunkels, "Interconnecting Smart Objects with IP,` Morgan Kaufmann, 2010. Chapter 16.

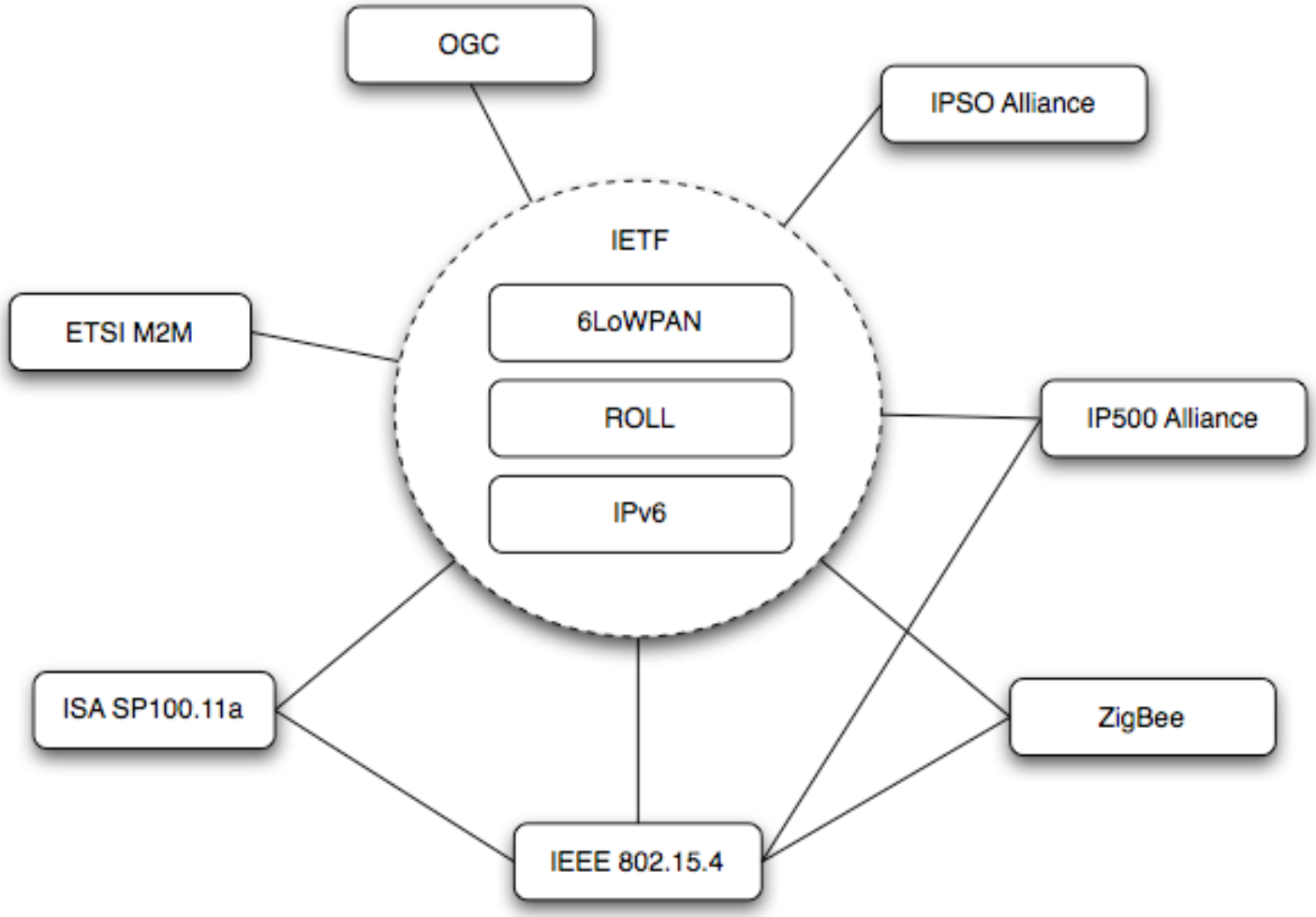
A LoWPAN

- LoWPAN: Low-power Wireless Personal Area Network composed of devices conforming to the IEEE 802.15.4-2003 standard
- Summary of characteristics of IEEE 802.15.4:
  - Smal packet size (MTU of 127 bytes)
  - Support for both 16-bit (short) and 64-bit (EUI, from MAC address) addresses
  - Low-rate: from 20 kb/s to 250 kb/s
  - Support of star and mesh topologies
  - Constrained devices
  - Large number of deployed devices
  - There may be mobility
  - IEEE 802.15.4 links are unreliable
  - Nodes can be sleeping for long periods of time
- A LoWPAN is a Low-power and Lossy Network (LLN, more general concept) where the links connecting the nodes are IEEE 802.15.4 links
- IETF 6LoWPAN working group (started in 2004): IPv6 protocol extensions for a LoWPAN (with IEEE 802.15.4 links)

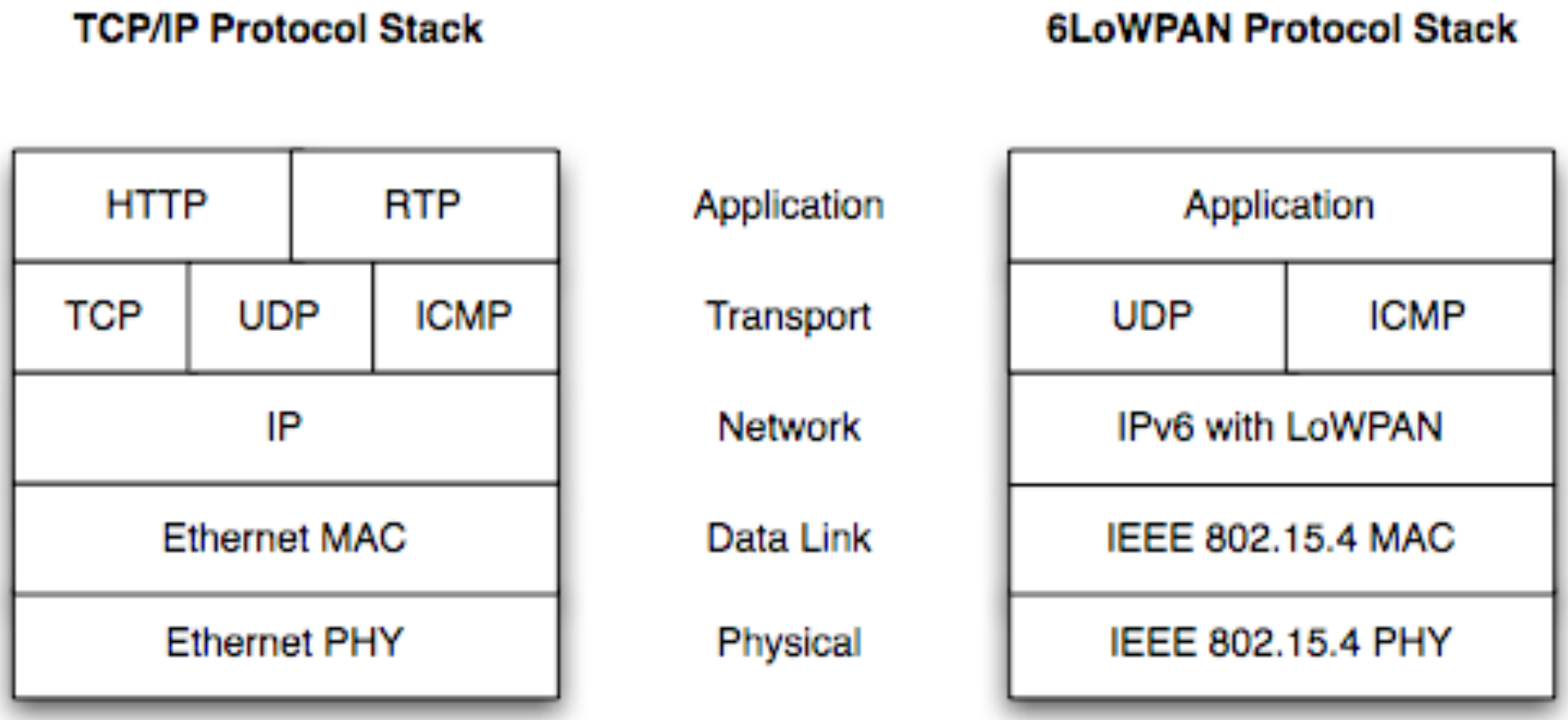
IETF Documents on 6LowPAN

Title		Reference
Problem Statement		RFC4919("Problems Statement, Assumptions and Goals for IPv6 for LoWPANs", Define the problem statement and goals of 6lowpan networks)
6LoWPAN Format		RFC4944 (Defines basic packet formats and sub-IP adaptation layer for transmission over 6LoWPAN.Includes framing, adaptation, header compression, address generation, and packet delivery in mesh topology)
Routing Requirements		Problem Statement and Requirements for 6LoWPAN Routing draft-ietf-6lowpan-routing-requirements-00
Routing Protocols	Load Routing Protocol	draft-daniel-6lowpan-load-adhoc-routing-02.txt
	Dymo Routing Protocol	draft-montenegro-6lowpan-dymo-low-routing-02
	HiLow Routing Protocol	draft-daniel-6lowpan-hilow-hierarchical-routing-00.txt
Header Compression		Compression Format for IPv6 Datagrams in 6LoWPAN Networks draft-ietf-6lowpan-hc-04
Neighbor Discovery		Neighbor Discovery for 6LoWPAN draft-ietf-6lowpan-nd-00
Use Cases		Design and Application Spaces for 6LoWPANs draft-ietf-6lowpan-usecases-01

Relationship of Standards



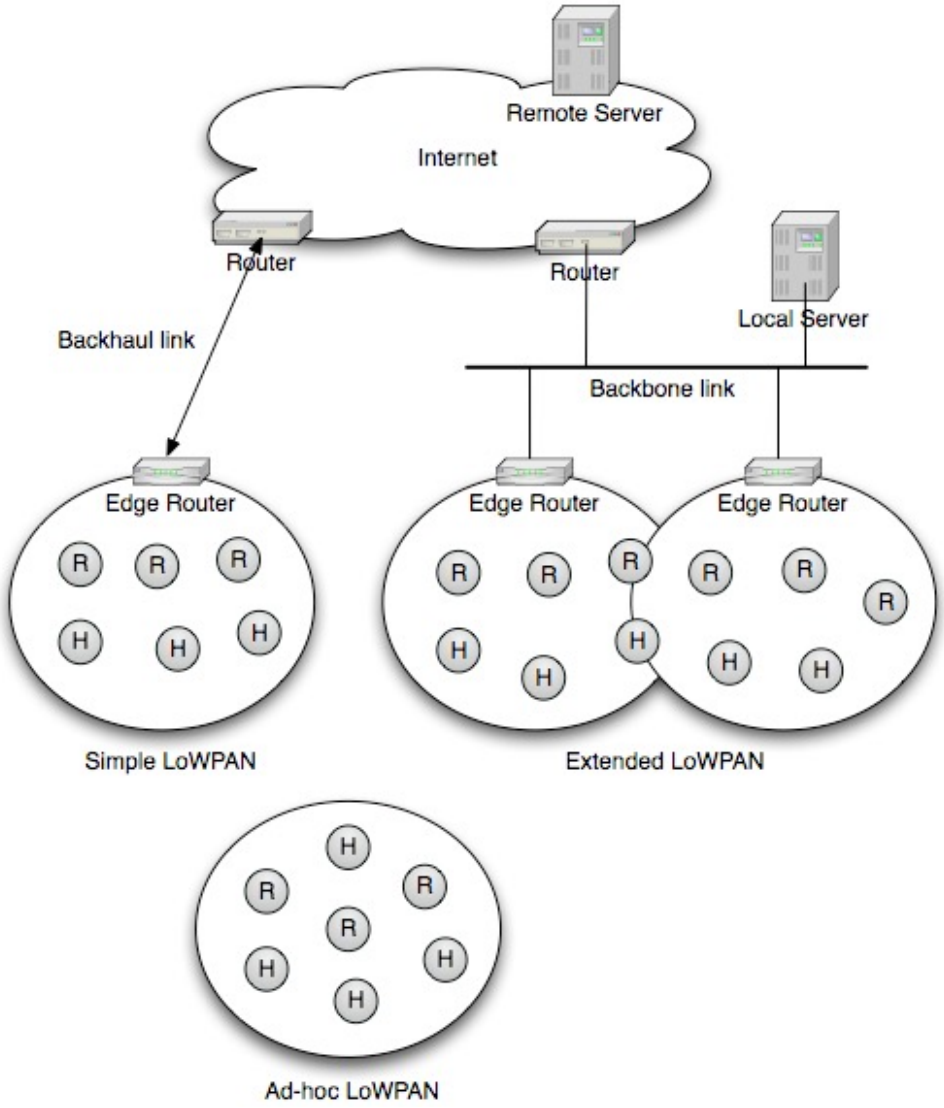
Protocol Stack



Features of 6LoWPAN in a Nutshell

- Fragmentation
  - 1280 byte IPv6 MTU -> 127 byte 802.15.4 frames
- Support for 64-bit and 16-bit 802.15.4 addressing
- Efficient header compression
  - IPv6 base and extension headers, UDP header (transport layer)
- Network autoconfiguration using neighbor discovery
- Unicast, multicast and broadcast support
- Support for IP routing (e.g., LOAD, DYMO-LOW, HIGH-LOW, **IETF RPL**)

Architecture



- LoWPANs are stub networks (i.e., networks with a single point of access towards Internet)
- Simple LoWPAN
  - Single Edge Router
- Extended LoWPAN
  - Multiple Edge Routers with common backbone link
- Ad-hoc LoWPAN (no stub)
  - No route outside the LoWPAN

6LoWPAN Adaptation Layer (1/3)

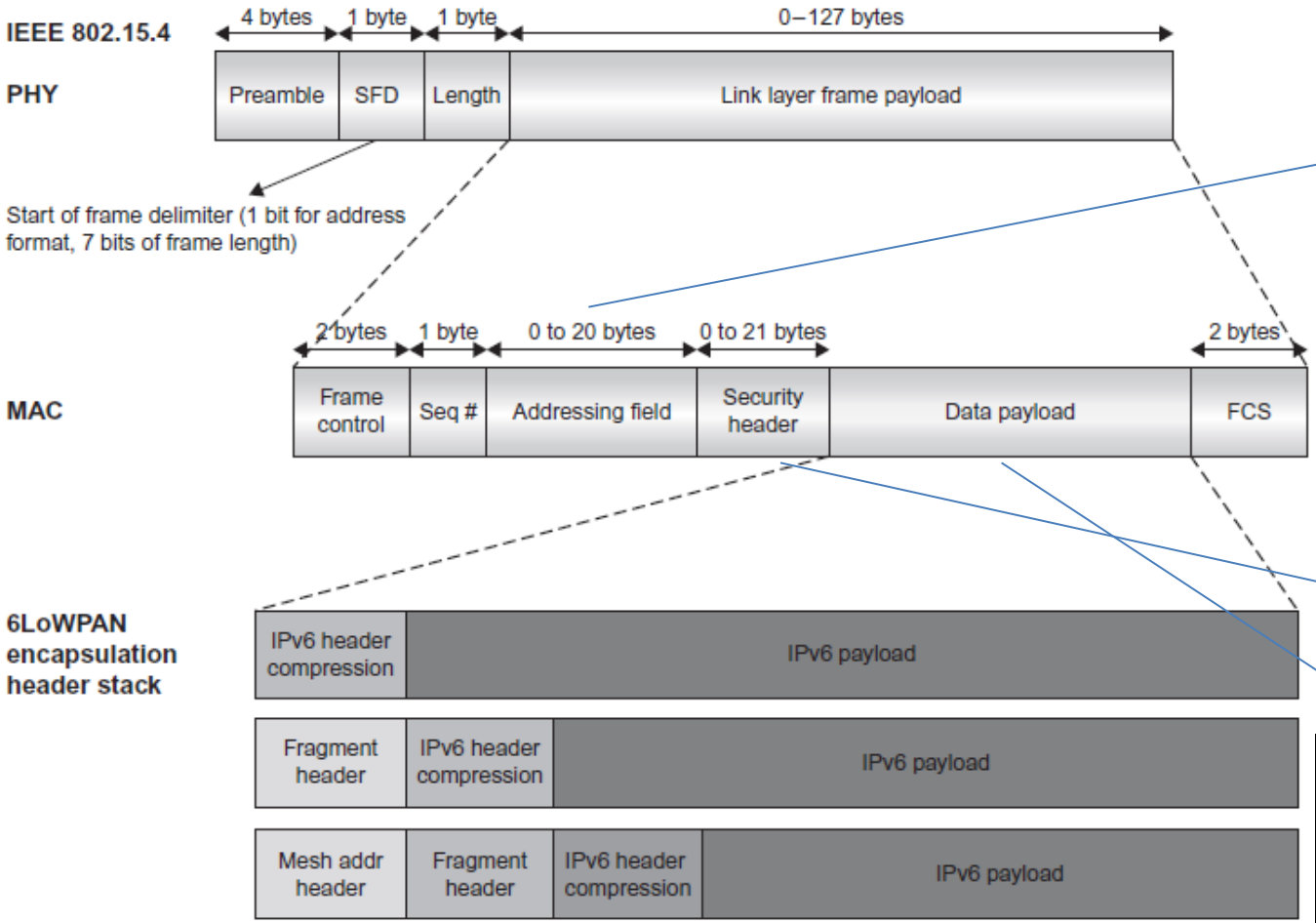


FIGURE 16.1

6LoWPAN encapsulation header stack.

Addressing: 20 bytes with: source and destination 16-bit PAN IDs; and source/destination 64-bit addresses

Security header: 21 bytes (AES-128), 13 bytes (AES-64), and 9 bytes (AES-32)

Payload: in the worst case 127-46=81 bytes → 40 bytes for IPv6 header → 41 bytes left → take out 8 bytes for UDP header or 20 bytes for TCP header

→21 bytes for TCP?

6LoWPAN Adaptation Layer (2/3)

- Adaptation layer is needed:
  - To comply with IPv6 requirement to support a minimum MTU of 1280 bytes (use of efficient compression techniques allows most applications to send their data within a single IPv6 packet)
  - To support compression techniques to reduce protocol overhead
- The 6LoWPAN adaptation layer provides three main services:
  - Packet fragmentation and reassembly
  - Header compression
  - Link layer (layer 2) forwarding when multi-hop is used by the link layer

6LoWPAN Adaptation Layer (3/3)

- 6LoWPAN makes use of header stacking (headers are added only when needed), as IPv6
- Three headers are supported (encapsulation header stack, see p. 284)
  - Mesh addressing header
  - Fragment header
  - IPv6 compression header

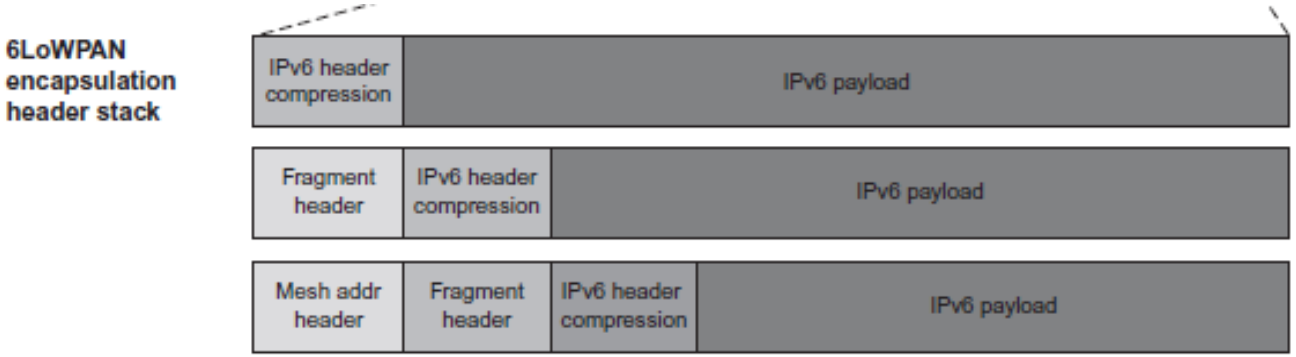
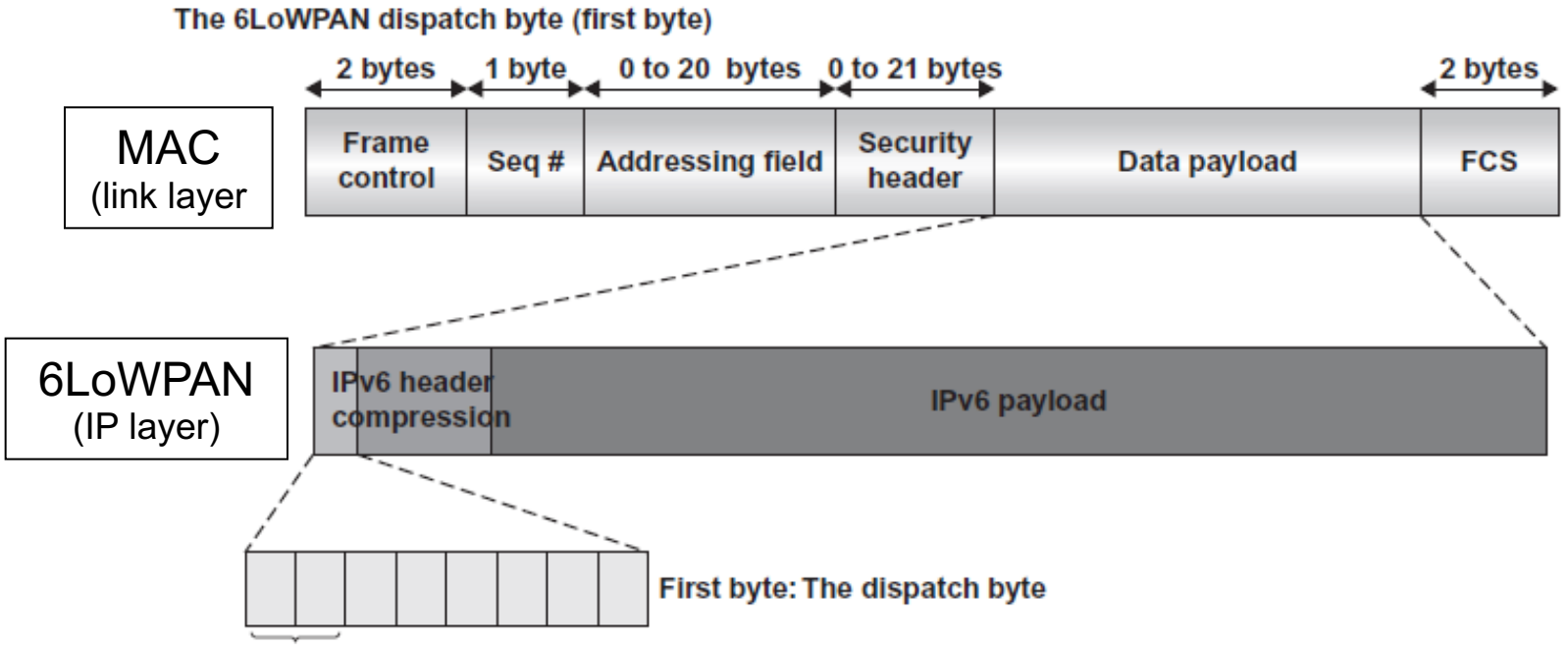


FIGURE 16.1

6LoWPAN encapsulation header stack.

6LoWPAN Dispatch Byte (1/2)



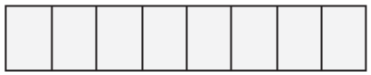
00	Not a 6LoWPAN frame
01	IPv6 addressing header
10	Mesh header
11	Fragmentation header (6 lower bits are 100xxx)

FIGURE 16.2

Dispatch byte of the IPv6 header compression header.

6LoWPAN Dispatch Byte (2/2)

The 6LoWPAN dispatch byte (first byte)



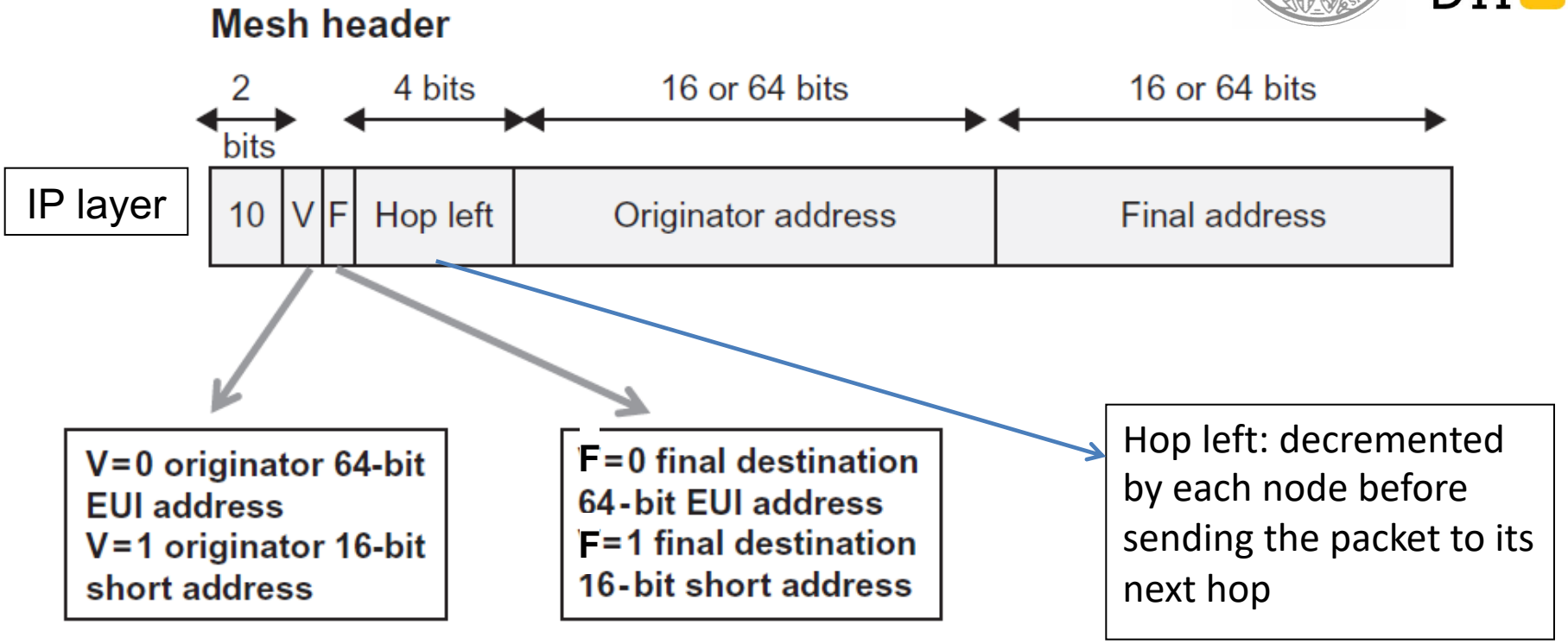
First byte: The dispatch byte

Pattern	Header type
00 xxxxxx	NALP - not a LoWPAN frame
01 000001	IPv6 - uncompressed IPv6 addresses
01 000010	LOWPAN_HC1-LOWPAN_HC1 compressed IPv6
01 000011	reserved - reserved for future use
...	reserved - reserved for future use
01 001111	reserved - reserved for future use
01 010000	LOWPAN_BCO - LOWPAN_BCO broadcast
01 010001	reserved - reserved for future use
...	reserved - reserved for future use
01 111110	reserved - reserved for future use
01 111111	ESC - additional dispatch byte follows
10 xxxxxx	MESH - Mesh header
11 000xxx	FRAG1 - fragmentation header (first)
11 001000	reserved - reserved for future use
...	reserved - reserved for future use
11 011111	reserved - reserved for future use
11 100xxx	FRAGN - fragmentation header (subsequent)
11 101000	reserved - reserved for future use
...	reserved - reserved for future use
11 111111	reserved - reserved for future use

FIGURE 16.3

Value of the 6LoWPAN dispatch byte.

6LoWPAN Mesh Header (1/2)

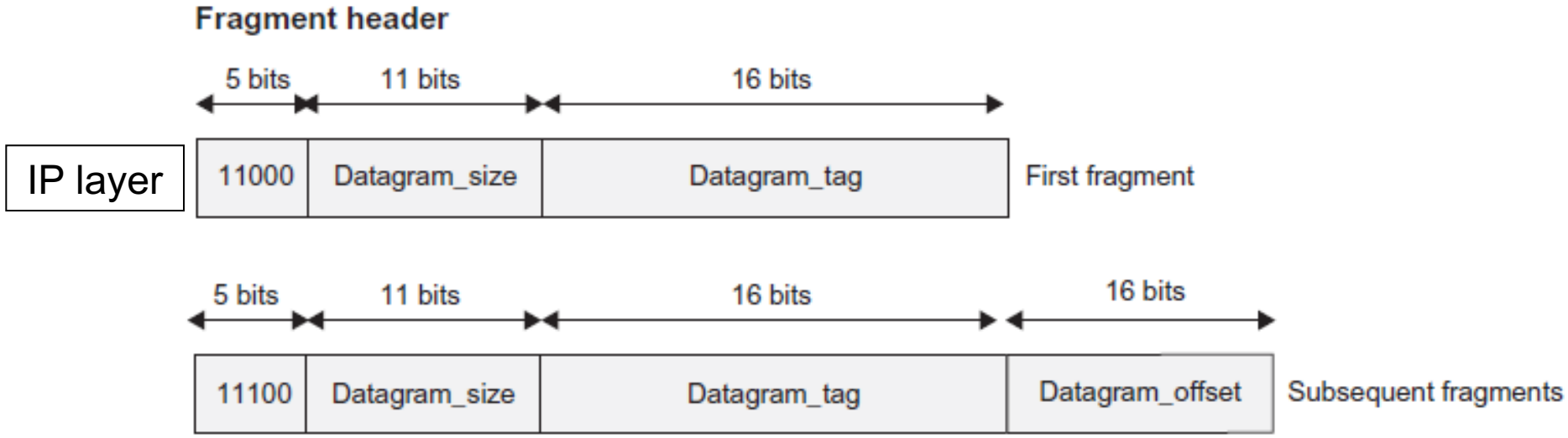


- The mesh addressing header: used in conjunction with a mesh-under “routing ” approach where nodes that are not in direct communication make use of multi-hop “routing ” at the link layer using link layer addresses.
- According to IEEE 802.15.4, only full function devices (FFDs) perform mesh-under operation. Reduced function devices (RFDs) systematically send all of their traffic to FFDs.
- With mesh-under routing it is necessary to provide the originator and final destination as well as the hop-by-hop source and destination addresses.

6LoWPAN Mesh Header (2/2)

- The set of link layer addresses is as follows. When a node A sends a frame to a final destination C via the node B:
  - [IP layer] The originator address of the mesh header is set to the link layer address of A.
  - [IP layer] The final destination address of the mesh header is set to the link layer address of C.
  - [link layer] The source address of the IEEE 802.15.4 frame (link layer) is the address of the node sending the frame (A).
  - [link layer] The destination address of the IEEE 802.15.4 frame is the link layer address of the next-hop node as determined by the mesh-under routing protocol (B in this example).
- Upon receiving the frame, B performs the following process:
  - [IP layer] The hop left field is decremented.
  - [IP layer] If the hop left field is not equal to 0 (if equal to 0, the frame is discarded), then B determines that the next hop is C.
  - [IP layer] The originator and final destination addresses of the mesh header are unchanged.
  - [link layer] The source address of the IEEE 802.15.4 frame is set to the link layer address of B.
  - [link layer] The destination address of the IEEE 802.15.4 frame is set to the link layer address of C.

6LoWPAN Fragment Header



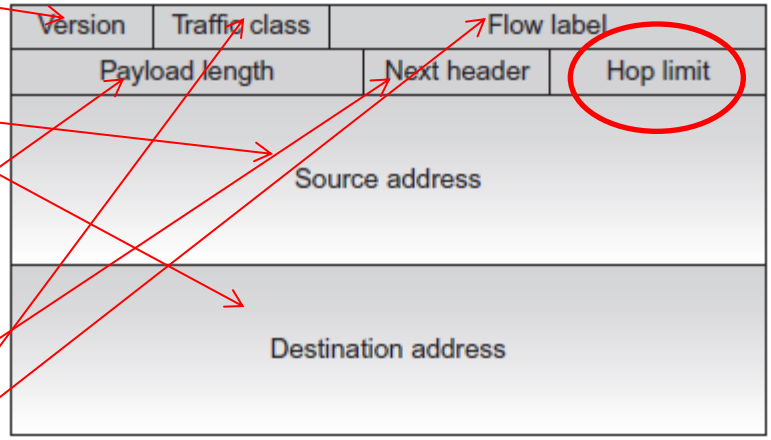
- Fragmentation needed when IPv6 payload cannot be carried in a single IEEE 802.15.4 frame → link frame broken into multiple link fragments
- First fragment does not have the Datagram offset field
- Datagram_size: fragment sizes are expressed in units of 8 bytes (64 bits); could be used only in the first fragment (problem! in the reassembly, as receiver may not know how much memory is needed)
- Datagram_tag: used in conjunction with the IEEE 802.15.4 source address (or originator address if a mesh header is present), the IEEE 802.15.4 destination address (or the final destination address if a mesh header is present), and the datagram size to uniquely identify the fragmented frame and must be identical for all link fragments. It is recommended to increment the datagram_tag for successive groups of fragmented frames.
- datagram_offset:offset (in 8-byte units) from the beginning of the payload datagram

6LoWPAN Header Compression

- IP compression techniques (e.g., RObust Header Compression , ROHC): rely on stateful flow-based compression optimized for long-lived flows → avoid information redundancy across IP packets that belong to the same long-lived flow
- 6LoWPAN have typically short-lived flows (nodes send a few packets and go to sleep) → avoid information redundancy across layers
- General ideas of 6LoWPAN header compression:
  - Derive the IP address from link layer addresses (e.g., EUI) to avoid needless information duplication
  - Suppress IPv6 headers that have common values (typically elide the fields that have a value of 0).
  - Use shared contexts, such as the use of a common network prefix for the LoWPAN, to allow address compression of IPv6 global addresses.
- 6LoWPAN header compression can either be stateless or stateful and is flow-independent.
- Several of the IPv6 headers have common values and are easily compressed
  - IP version (v6)
  - the flow label, traffic class, and so on
  - IPv6 interface ID: can be derived from the link layer frame when using extended 64-bit 802.15.4 addresses
- What we are going to see: HC1 (efficient link-local addresses) and IPHC (more general)

HC1 Compression (1/2)

- IP version is always 6.
- Since HC1 is optimized for link-local addresses, the IPv6 interface ID (bottom 64 bits of the IPv6 address) can be inferred from the link layer MAC address
- The packet length can be inferred from the frame length field of the IEEE 802.15.4 frame (or from the datagram size field of the fragment header when present)
- Common value for the TC and flow label is 0
- Next header is UDP, TCP, or ICMP
- Hop limit field: needs to remain 1 byte



40-byte IPv6 header

3-byte header

- Byte 1: dispatch byte (to 01000010) – see p. 288
- Byte 2: HC1 byte
- Byte 3: hop limit field

HC1 Compression (2/2)

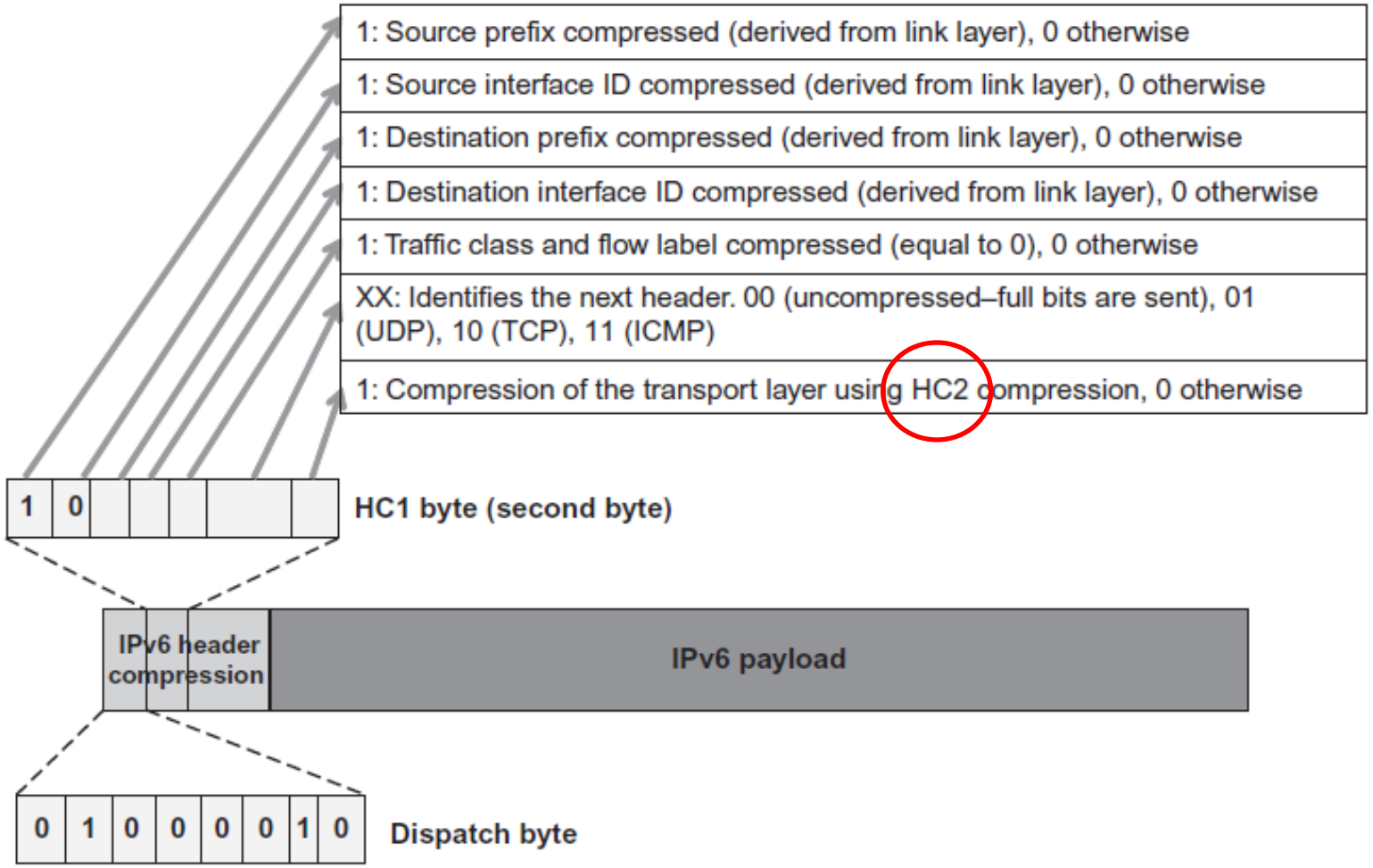


FIGURE 16.5

6LoWPAN HC1 byte.

HC2 Byte: HC_UDP Compression

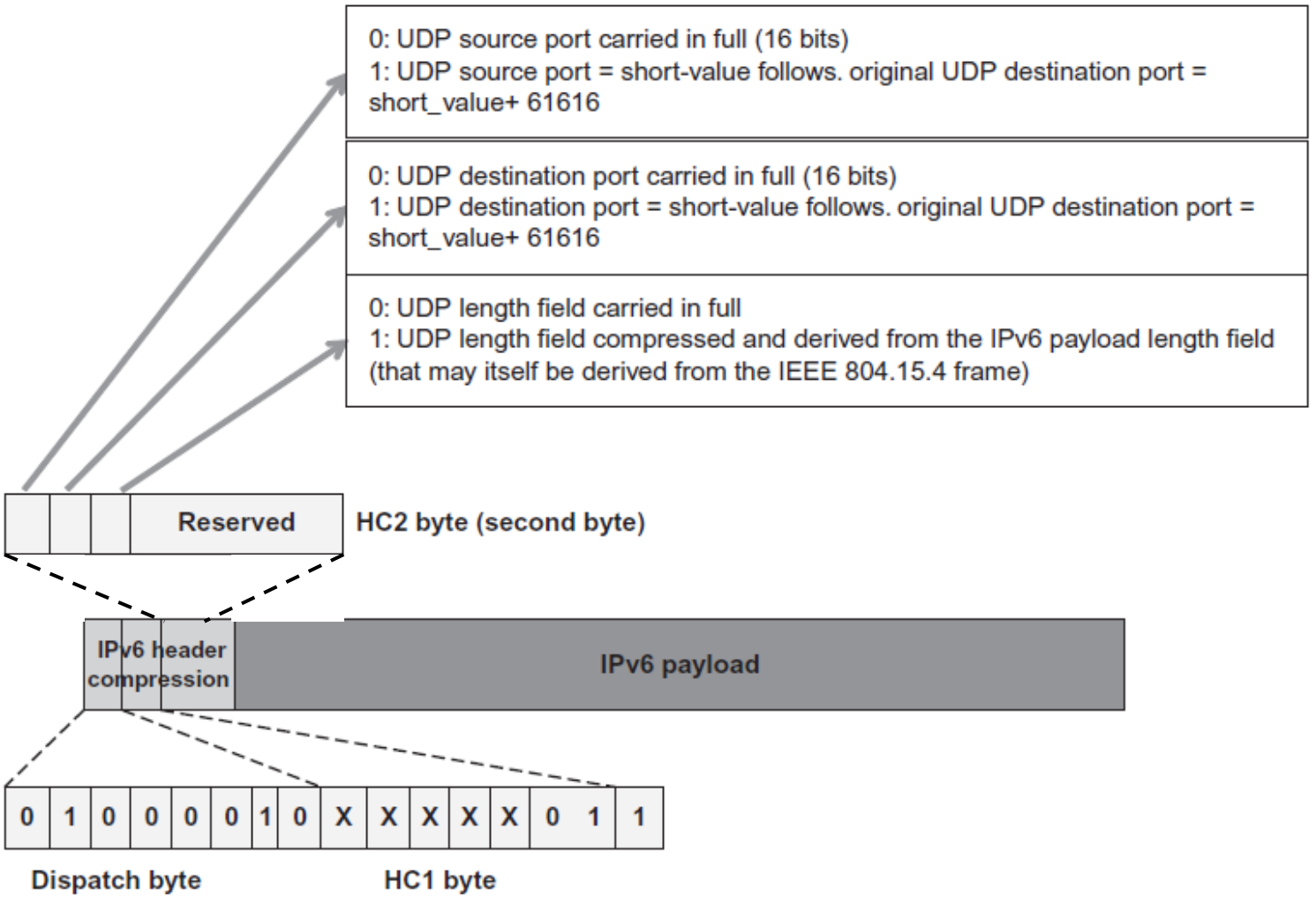


FIGURE 16.6

Header compression using HC1 and HC2 bytes.

HC1 without UDP Header Compression

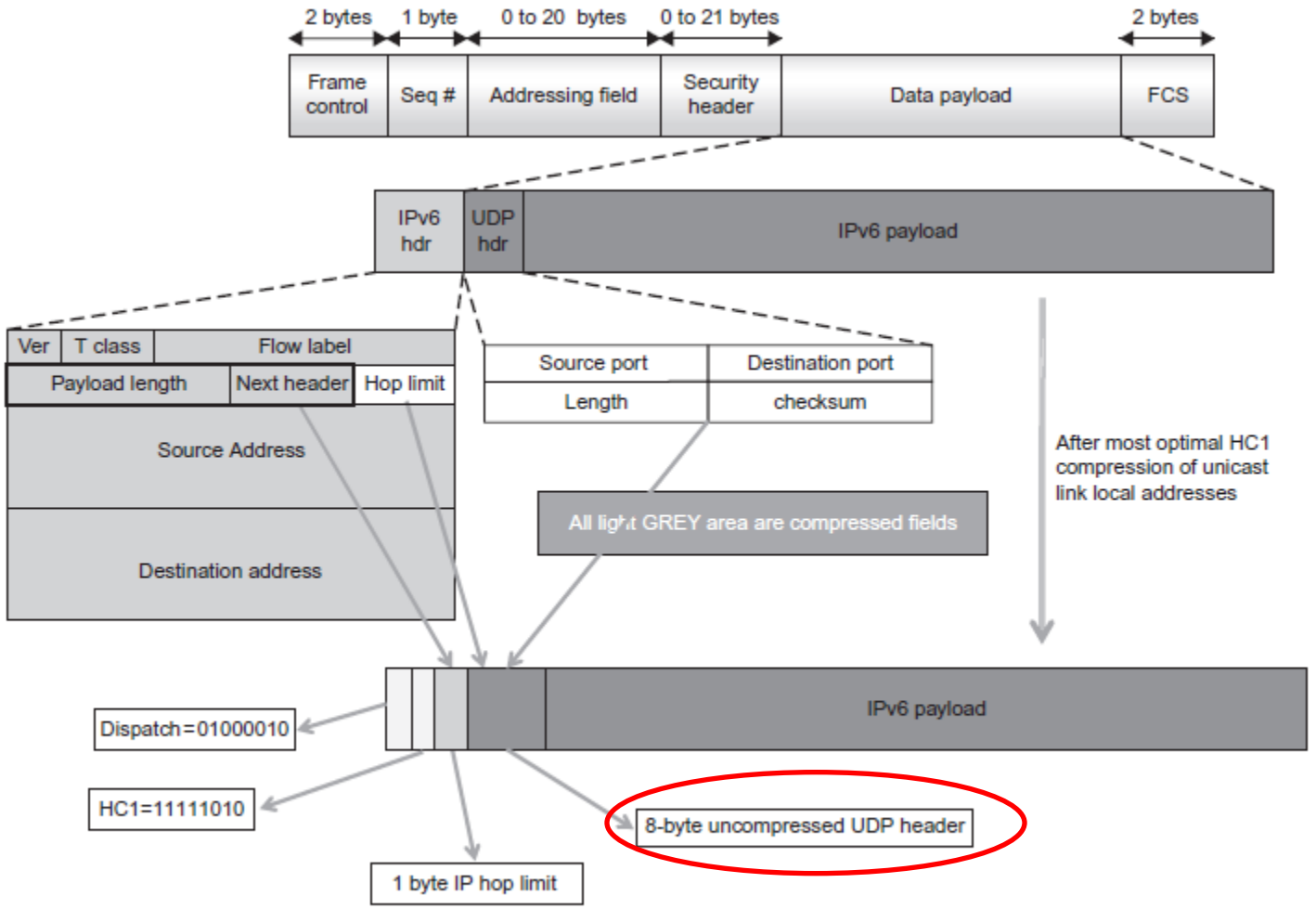


FIGURE 16.7

Compression of the IPv6 HC1 link-local IPv6 addresses without UDP header compression.

HC1 with UDP Header Compression (HC2)

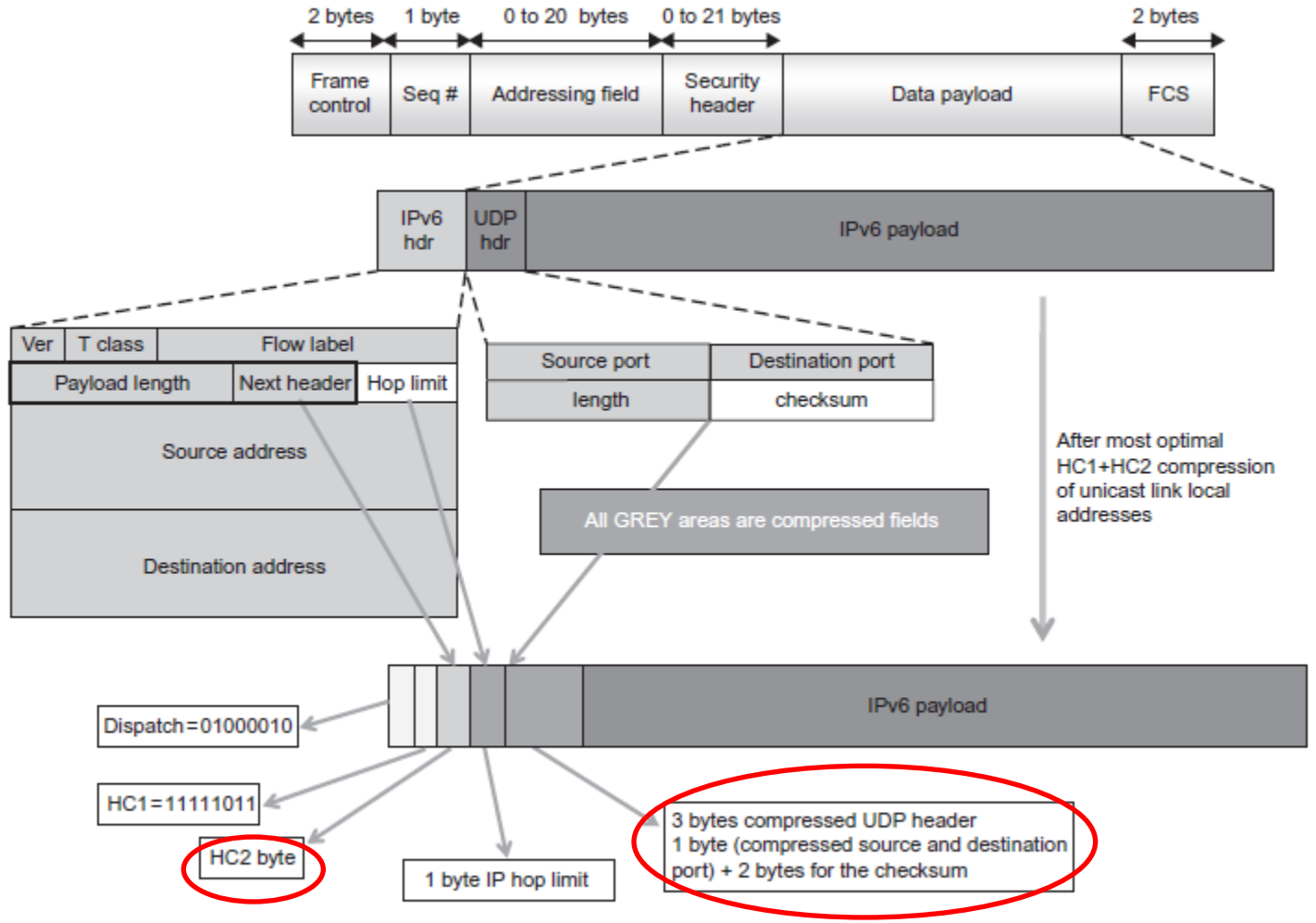


FIGURE 16.8

Maximized compression of IPv6 and UDP headers using HC1 and HC2 with link-local IPv6 addresses.

ImProved Header Compression (IPHC) (1/6)

- HC1 (HC2) compression: quite efficient for unicast link-local addresses
- Limited effect on global and multicast addresses
- Efficient compression techniques for global IPv6 addresses are needed for communication between nodes residing in different IP subnetworks (typically nodes in different PANs in the context of IEEE 802.15.4 networks), where routable addresses are required.
  → HC1 compression technique requires carrying the sources and destination IPv6 addresses in-line (non-compressed)
- IPHC requires:
  - 13 bits (5 bits of which are taken from the rightmost part of the dispatch byte);
  - an optional additional byte (8 bits) is added.

ImProved Header Compression (IPHC) (2/6)

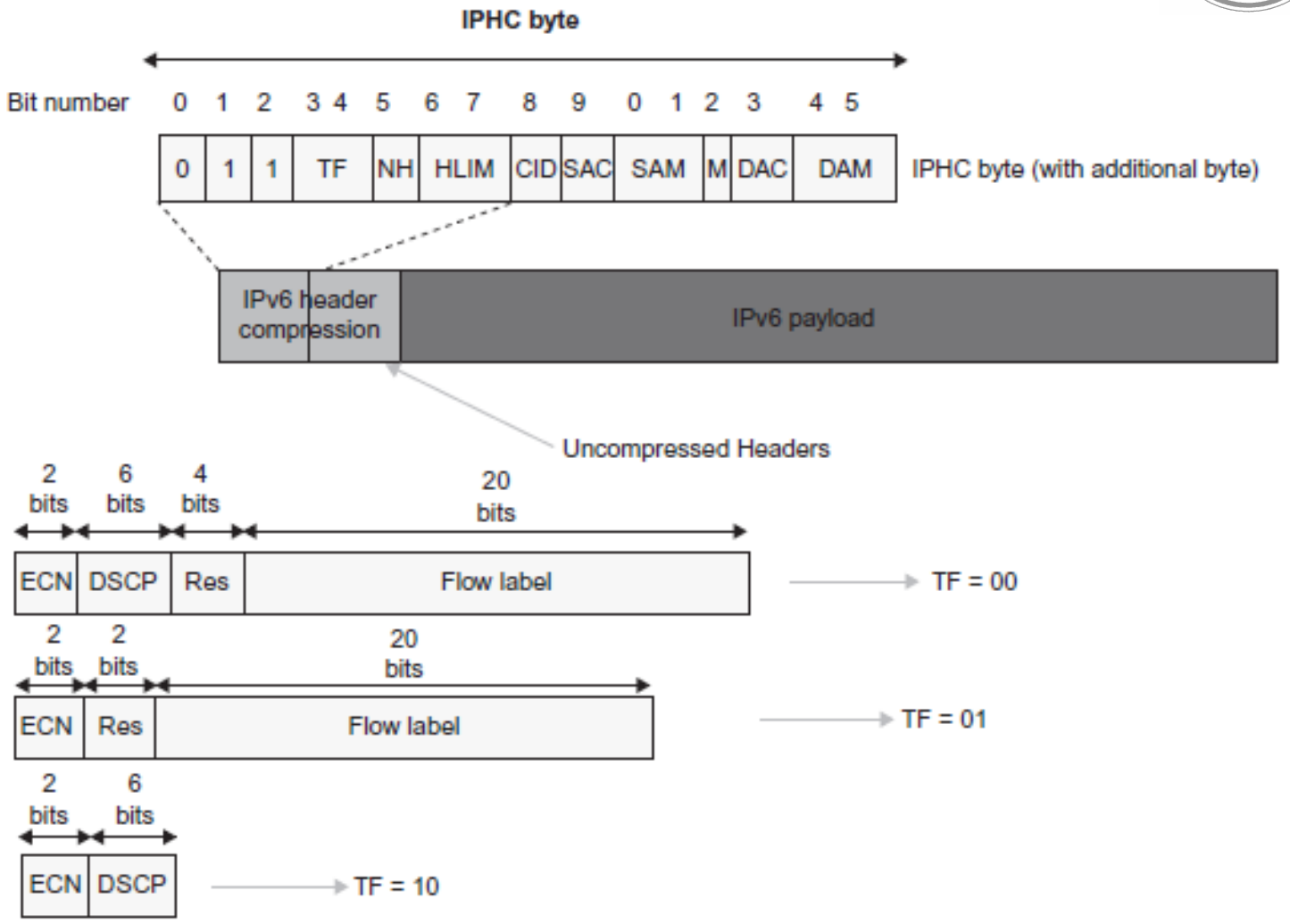
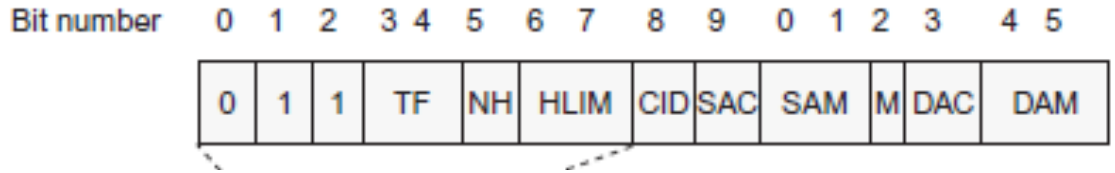


FIGURE 16.9

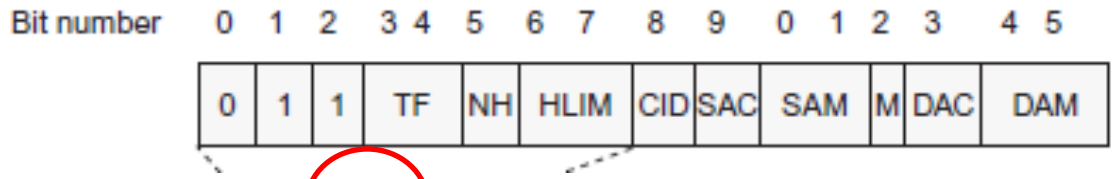
IPHC encoding: traffic and flow label compression.

ImProved Header Compression (IPHC) (3/6)



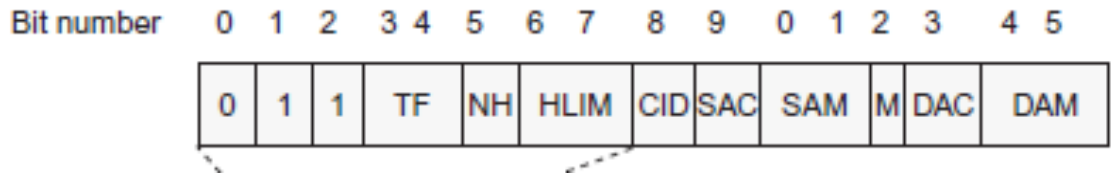
- Bits 3 and 4 (TF: Traffic Class): These bits allow for more granular compression of the TC and flow label IPv6 header fields.
  00: Both the TC and flow label fields are carried in line as shown in Figure 16.9. Note that 4 bits have been added for byte alignment.
  01: The TC field is compressed to 2 bits (ECN), as defined in [209] and the flow label field is uncompressed. Since the ECN bits are encoded using 2 bits and the flow label field is uncompressed, this leads to 22 bits to which 2 bits of padding are added for byte alignment.
  10: The flow label field is compressed (fully elided) and the TC field is carried in-line.
  11: The TC and flow label fields are compressed.
- Bit 5 (next header):
  0: The full 8-bit header of the next header is carried in-line.
  1: The IPv6 next header field is elided and another encoding mechanism (called next header coding, NHC) defined later is added.
- Bits 6 and 7 (HLIM, hop limit): In contrast with the HC1 compression technique, IPHC allows compression of the hop limit field of the IPv6 header.
  00: The hop limit field of the IPv6 packet is carried in-line.
  01: The hop limit field is elided and the hop limit is equal to 1.
  10: The hop limit field is elided and the hop limit is equal to 64.
  11: The hop limit field is elided and the hop limit is equal to 255.

ImProved Header Compression (IPHC) (4/6)



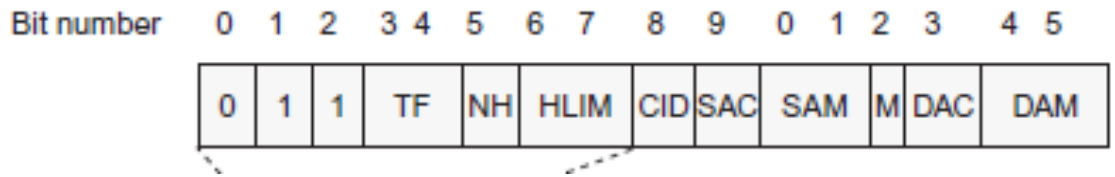
- Bit 8 (context identifier extension; CID).
  0: There is no use of additional context information.
  1: An additional 1-byte CID is added that immediately follows the destination address mode (DAM) field.
- Bit 9 (source address compression; SAD).
  0: Address compression is stateless.
  1: Address compression is stateful based on context.
- Bits 10 and 11 (source address mode; SAM).
  If SAC=0
  00: The full 128-bit address is carried in-line.
  01: The first 64 bits of the IPv6 address are elided and the value of the 64 bits is the link-local prefix, padded with zeros. The remaining 64 bits are carried in-line.
  10: The first 112 bits of the IPv6 address are elided and the value of those bits is the link-local prefix, padded with zeros. The remaining 16 bits are carried in-line.
  11: The address is fully elided. The first 64 bits are the link-local prefix. The remaining 64 bits are inferred from the IEEE 802.15.4 frame, similarly to HC1.
  If SAC=1
  00: The address is the unspecified address (::).
  01: 64 bits. The 64-bit prefix address is derived from the context information (see Section 16.2.3.4) and the remaining 64 bits are carried in-line.
  10: 16-bits. The 64-bit prefix address is derived from the context information (see Section 16.2.3.4) and the remaining 16 bits are carried in-line.
  11: 0 bits. The address is derived from the context information and potentially the link layer and no bits are carried in-line.

ImProved Header Compression (IPHC) (5/6)



- Bit 12 (multicast compression; M).
  0: The destination address is not a multicast address.
  1: The destination address is a multicast address.
- Bit 13 (destination address compression; DAC).
  0: The compression of the destination address is stateless.
  1: The compression of the destination address is stateful, based on the context.
- Bits 14 and 15 (DAM).
  If DAC=0
  00: 128 bits. The full 128-bit address is carried in-line.
  01: 64 bits. The first 64 bits of the IPv6 address are elided and the value of the 64 bits is the link-local prefix, padded with zeros. The remaining 64 bits are carried in-line.
  10: 16 bits. The first 112 bits of the IPv6 address are elided and the value of the 112 bits is the link-local prefix, padded with zeros. The remaining 16 bits are carried in-line.
  11: The address is fully elided. The first 64 bits are the link-local prefix and the remaining 64 bits are inferred from the IEEE 802.15.4 frame, similarly to HC1.
  If DAC=1
  00: Reserved.
  01: 64 bits. The prefix address is derived from the context information and the 64 bits are carried in-line.
  10: 16 bits. The prefix address is derived from the context information and the 16 bits are carried in-line.
  11: The address is derived from the context information and potentially the link layer and no bits are carried in-line.

ImProved Header Compression (IPHC) (6/6)



If M=1 and DAC=0

00: 128 bits. The full address is carried in-line.

01: 48 bits. The address is coded using 48 bits and has the form FFXX::00XX:XXXX:XXXX.

10: 32 bits. The address is coded using 32 bits and has the form FFXX::00XX:XXXX.

11: 8 bits. The address is coded using 8 bits and has the form FF02::00XX.

If M=1 and DAC=1

00: 48 bits. The address is coded using 48 bits and has the form FFXX::XXLL:PPPP:PPPP:PPPP:PPPP:XXXX:XXXX. X denotes nibbles carried in-line, P denotes nibbles to encode the prefix (i.e., given by the specific context), and L denotes nibbles used to encode the prefix length. The prefix information P and L is derived from the context itself. This format is compliant with the unicast-prefix-based IPv6 multicast addresses defined in [103] and [219].

01: Reserved

10: Reserved

11: Reserved

Context IDentifier (CID)

- IPHC relies on the notion of “shared context” between the sending node compressing the IPv6 packet and the receiving node expanding the received packet.
- The current specification does not describe how contexts are shared or maintained. At some point there might be protocol extensions or a new protocol for dynamic setup and negotiation of shared contexts.
- As indicated previously, the CID field in the LOWPAN_IPHC encoding indicates that an additional **byte** is added that follows the IPHC byte and precedes the IP header fields that are carried in-line:
  - the first 4 bits called the source context identifier or SCI is used for the source address;
  - the following 4 bits called the destination context identifier or DCI is used for the destination address).
- Since 4 bits are used to identify the context, up to 16 contexts are allowed.

Next Header Coding

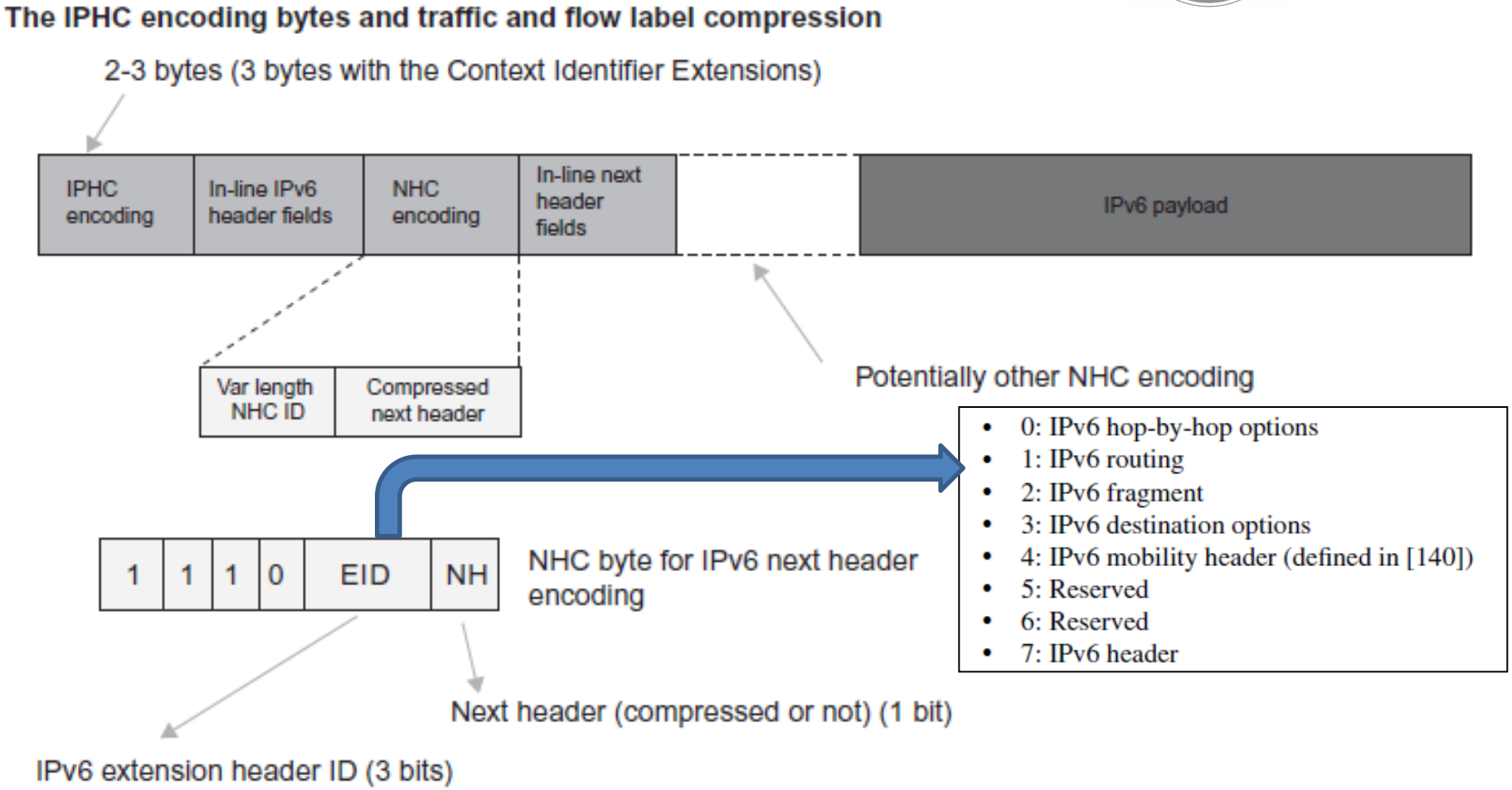


FIGURE 16.10

IPv6 packet using LOWPAN_IPHC and LOWPAN_NHC compression.

UDP Header Compression (1/3)

- HC_UDP: 4-bit short value for UDP address is used to extract the original UDP port using the formula 61616 (0xF0B0) + short_value.
- NHC introduces another UDP header compression technique with several improvements.
- A range of 16 contiguous well-known ports is specified in the form 0xF0Bx, thus identical to the HC_UDP compression technique for UDP ports → compatibility issues with applications already using these ports → Specification recommends using the Transport Layer Security Message Integrity Check (TLS MIC) to validate the content and its integrity.
- UDP checksum can be bypassed if and only if the upper layer permits

UDP Header Compression (2/3)

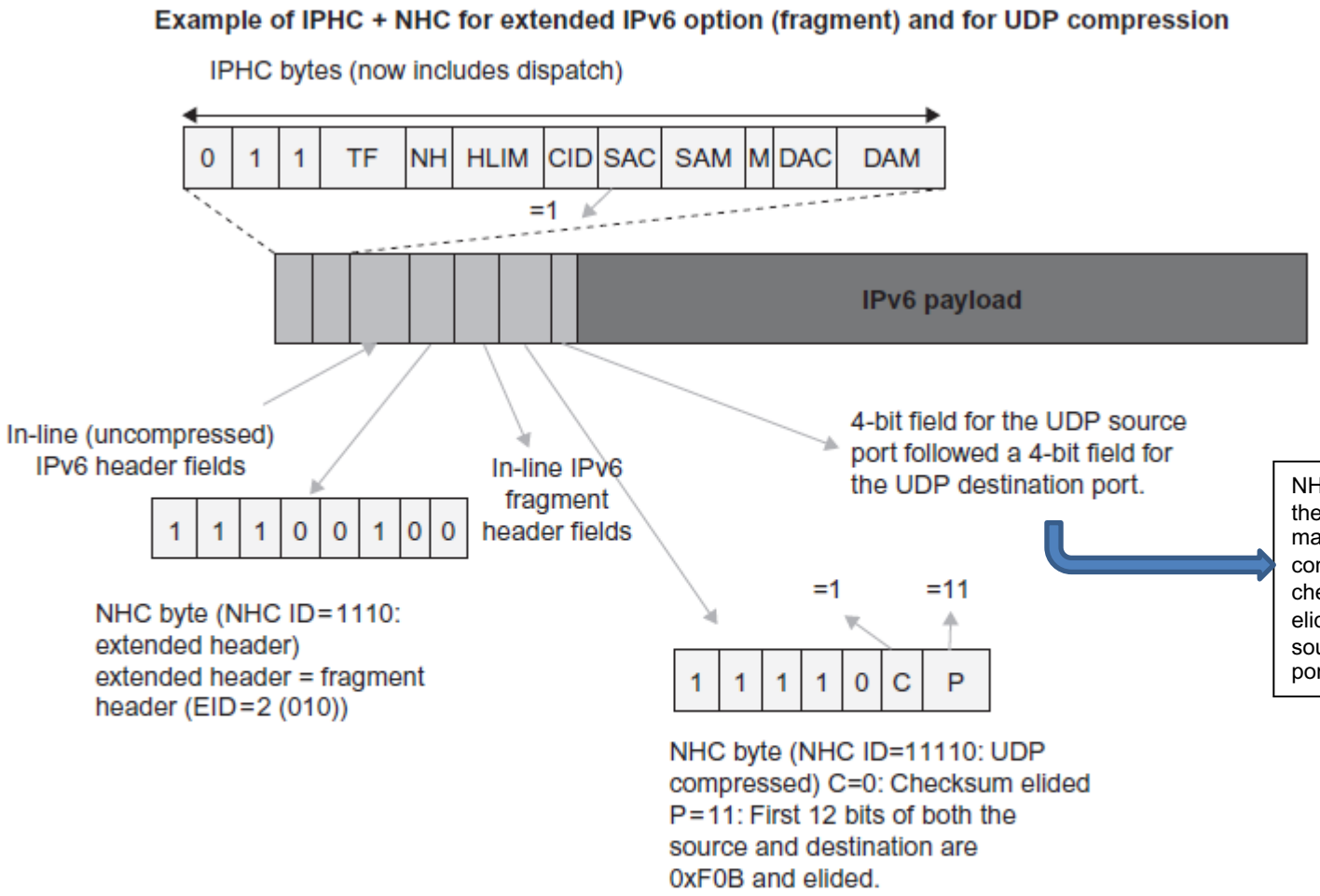
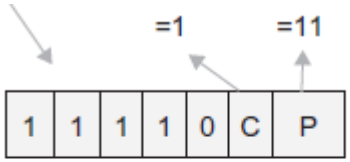


FIGURE 16.11

IPv6 packet using LOWPAN_IPHC and LOWPAN_NHC compression for extended IPv6 header compression and UDP header compression.

UDP Header Compression (3/3)



NHC byte (NHC ID=11110: UDP compressed) C=0: Checksum elided P=11: First 12 bits of both the source and destination are 0xF0B and elided.

- Bit 5 (Checksum; C).
  0: The 16-bit UDP checksum field is not compressed and carried in-line.
  1: The 16-bit UDP checksum is elided and recovered and recomputed by the 6LoWPAN termination point.

- Bits 6 and 7 (Ports; P).
  00: Both the source and destination 16-bit ports are carried in-line.
  01: The 16 bits of the source port are not compressed and carried in-line. The first 8 bits of the destination port are elided (and equal to F0) and the remaining 8 bits are carried in-line.
  10: The first 8 bits of the source port are elided (and equal to F0) and the remaining 8 bits are carried in-line. The 16 bits of the destination port are not compressed and carried in-line.
  11: The first 12 bits of the source and destination ports are elided and equal to 0xF0B0. The remaining 4 bits of both ports are carried in-line.

Best Compression Scenario

- With link-local unicast address, HC1 encoding allows compression of the IPv6 header **to 2 bytes** (the dispatch byte + the HC1 byte). When routing over multiple hops, the compression still performs quite well compressing the IPv6 header to **7 bytes** (the dispatch byte the HC1 byte + 1 byte for the IP hop limit field + 2 bytes for the sources address + 2 bytes for the destination address).
- With the use of global addresses, the IPv6 header can be compressed to **4 bytes** (2 bytes for the IPHC encoding + 1 byte for context identifier extension + 1 byte for the IP hop limit field + 0 bytes for the source and destination addresses with the interface IDs derived from the IEEE 802.15.4 link layer address). When using Context 0, the 1 byte for the context identifier is not needed. Potentially, additional bytes may be added if the TC and flow label field of the IPv6 field are not compressed.
- In the best case, the compressed UDP header only requires 2 bytes (1 byte for the NHC header + 1 byte for the compressed source and destination UDP ports) when the UDP checksum field is elided.

UNIVERSITÀ DI PARMA

IoT Lab

Routing in LLNs & RPL

Main sources

- J. P. Vasseur and A. Dunkels, `Interconnecting Smart Objects with IP,` Morgan Kaufmann, 2010. Chapter 17.

- Brandtr, Clausen, Dawson-Haggerty, Hui, Pister, Thubert, Winter, *RPL (ripple): Routing Protocol for Low Power and Lossy Networks – Walkthrough*, IETF `draft-dt-roll-rpl-01.txt`, IETF 75 – Roll WG – July 2009

From WSNs to RPL (1/2)

- Distance Vector routing (e.g., AODV) is widely used in WSNs
  - trade off between capabilities and resources
- Reduced states in nodes (vs. Link State)
- Simplicity => more control cost over time
- Improvements proposed
  - Loop detection
    - Packets received from a node that is on the way to the destination cause routing control
  - Trickle (adaptive adv period + suppression)
    - Limits the degradations due to node spatial density
  - Explicit link evaluation
    - Prior to selecting a parent as feasible

From WSNs to RPL (2/2)

- Expected Transmission Count (ETX) – MIT
  - Measure of the quality of a path between two nodes in a wireless packet data network
  - A lot more relevant than hop count in wireless
- Wikipedia:

"ETX is the number of expected transmissions of a packet necessary for it to be received without error at its destination. This number varies from 1 to infinity. An ETX of 1 indicates a perfect transmission medium, where an ETX of infinity represents a completely non-functional link. Because ETX is an *expected* transmission count for a future event, as opposed to an actual count of a past event, it represents a probability, and is therefore a real number, and not an integer. For example, if it took 1898 transmissions to transfer 1024 packets without error, the ETX on the link is 1898/1024, or approximately 1.85. Due to varying characteristics of the transmission medium, the number may vary widely."

Distance Vector (DV) protocol in IP Networks

- RIP (Routing Information Protocol): #hops as metric
  - Unfit for large, unstable & complex topologies
  - Sync effect, transient loops, count to infinity
  - Partial fixes by split horizon (poison reverse)
- More recent protocols
  - Trade RIP's loops for temporary black holes
  - Path hold-down then route poisoning
  - Quarantine path for which hop counts goes up

The link model

- LLN connectivity might not match the traditional wired link model
  - Links are not a given.
  - Statistical connectivity might cause transient unreachability from/to a rather good neighbor
  - and might enable to receive a packet from far far away from an improbable neighbor
  - Not a RPL-specific issue but a LLN-issue that the IETF ROLL WG has considered

Low-power and Lossy Networks (LLNs)

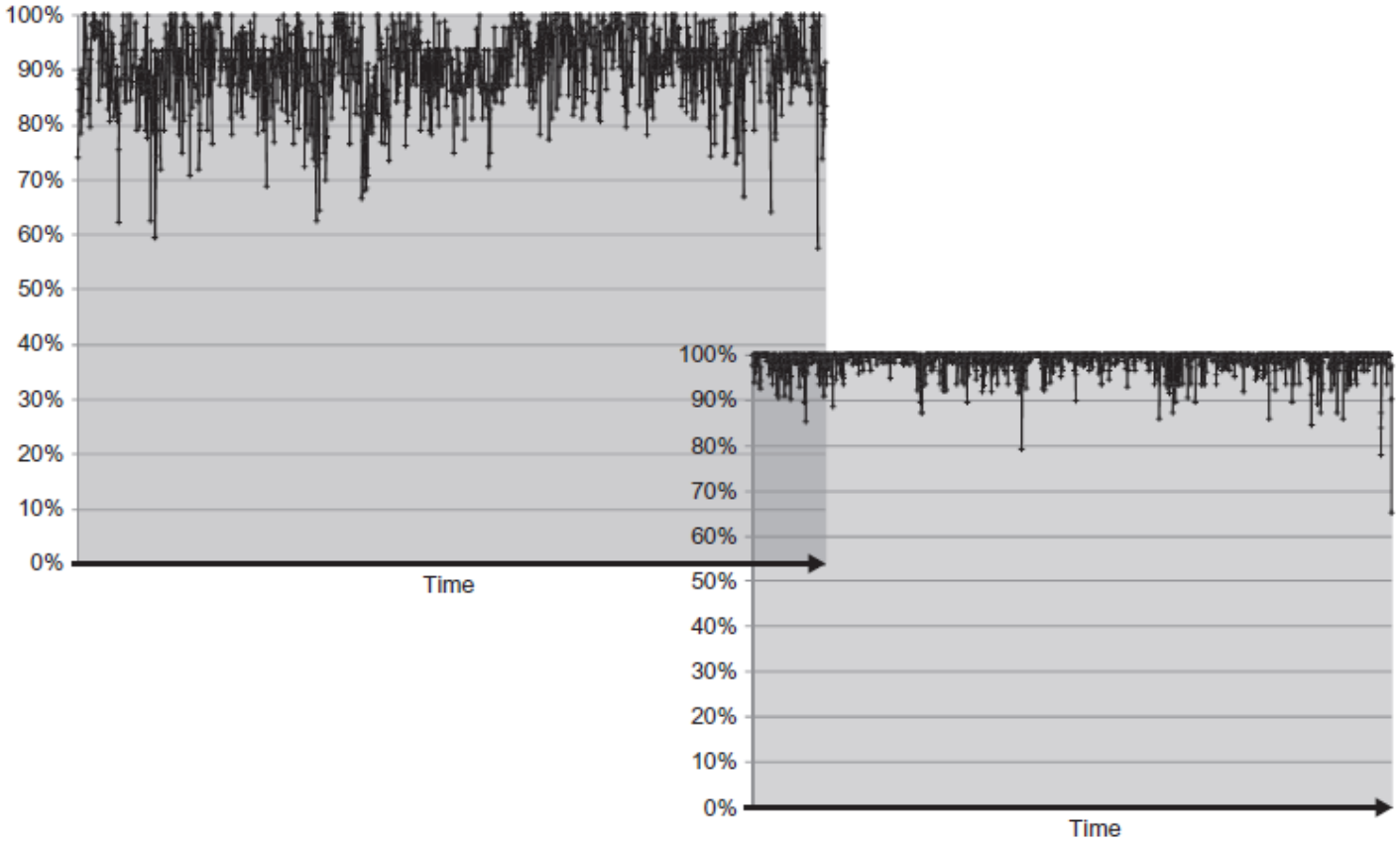


FIGURE 17.1

Packet Delivery Ratio for two IEEE 802.15.4 links.

Characterstics and Applications

- Common Characteristics
  - Low power and Lossy Networks (LLNs) consisting largely of constrained nodes.
  - Lossy and unstable links, typically supporting low data rates, relatively low packet delivery rates.
  - Traffic patterns are not simply point-to-point, but in many cases point-to-multipoint (P2MP) or multipoint-to-point (MP2P)
  - Potentially comprising up to thousands of nodes.
- Four main application scenarios
  - Urban LLNs (including smart grids) [RFC5548]
  - Industrial LLNs [RFC5673]
  - Home Automation LLNs [RFC5826]
  - Building Automation LLNs [RFC5867]

Requirements (1/2)

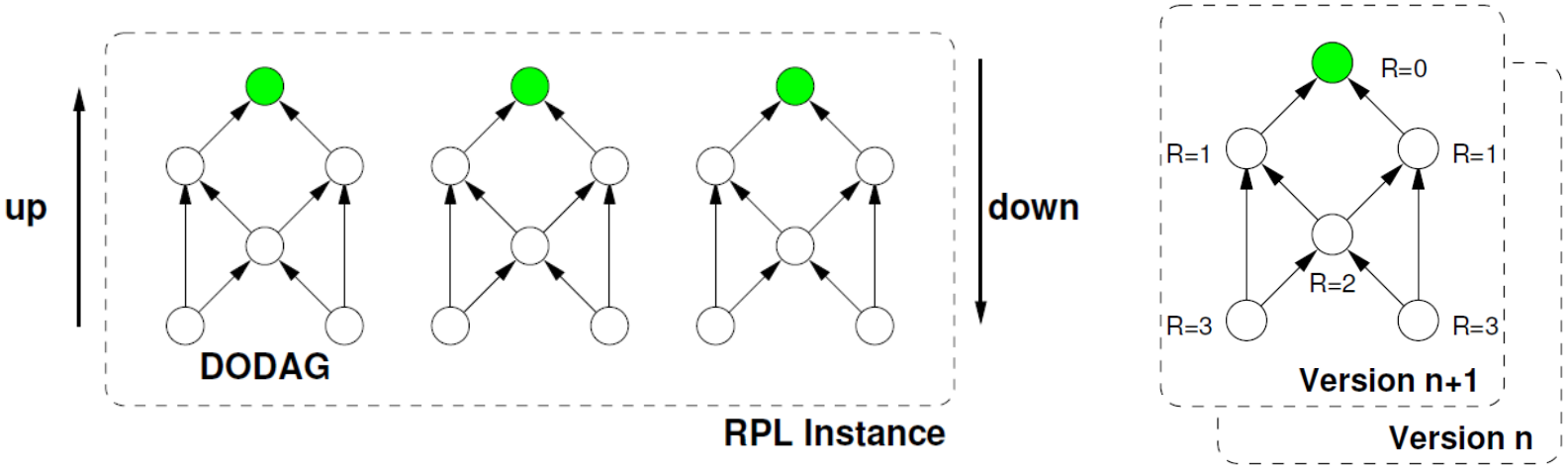
- Unicast/anycast/multicast
- Adaptive routing: new paths are dynamically and automatically recomputed as conditions change in the network (e.g., link/node failure, mobility, etc.)
- Constraint-based routing (energy, CPU, and memory as well as link attributes, such as link latency)
- Traffic characteristics (MP2P-data collection, P2MP-sw updates, P2P, computation of parallel paths)
- Scalability: from 250 to 10000 nodes
- Configuration and management: minimum or zero-configuration; isolation of a misbehaving node to limit/eliminate its impact on other nodes

Requirements (2/2)

- Node attribute: with sleeping nodes in the network (a frequent situation with battery-operated nodes), the routing protocol must discover the capability of a node to act as a proxy
- Performance: ‘‘tricky’’ requirement (e.g., a routing protocol must provide mobility with a convergence time below 0.5 s and it must converge within 0.5 s if no nodes have moved and within 2 s if the destination has moved)
- Security:
  - wide range (low to high);
  - the routing protocol must gracefully handle routing temporal security updates (e.g., dynamic keys) to sleeping devices on their ‘ *awake* ’ cycle to assure that sleeping devices can readily and efficiently access the network

RPL Approach (1/2)

- Design a **modular** routing protocol where the core component of the application would be specified by the RPL specification with optional features activated only where and when needed.
- For example, RPL specifies how to build a destination oriented directed acyclic graph (DODAG), but the characteristics of the DODAG are specified by an objective function.



RPL Approach (2/2)

- For the time being, think of a DODAG as a logical routing topology over a physical network that is built by the routing protocol to meet specific criteria.
- A node may join multiple DODAGs (if the application requires different objectives that must be realized through the use of multiple DODAGs) and mark the traffic according to the DODAG characteristics in support of Quality of Service (QoS) awareness and constrained-based routing.
- DODAG Properties
  - Many-to-one communication (MP2P): upwards
  - One-to-many communication (P2MP): downwards
  - Point-to-point communication (P2P): upwards-downwards

Routing Metrics in IP Nets

- Most of the IP routing protocols used in today's networks such as OSPF use static link metrics.
  - The network administrator is responsible for configuring the link metrics, which may refl ect the link bandwidth, delay, or combine several metrics.
  - Some Service Providers are combining up to three metrics (e.g., delays, bandwidth, cost) in the link metric.
  - Then the routing protocol computes the shortest path taking into account these static link metrics.
- Dynamic link metrics.
  - In ARPANET-2: studies to dynamically compute the link metric based on the averaged queue length to reflect the level of congestion.
  - These strategies were abandoned due to the difficulty in designing stable systems.
  - One of the main challenges with dynamic metrics is to carefully control the rate at which new metrics are advertised. Frequent link metric refreshers provide a high level of accuracy but may also lead to routing oscillation.

Routing in LLNs

- *Metric*: a scalar used to determine the best path according to some objective function (OF).
  - For example, if the link metric is representative of the link propagation delay, the path cost represents the total propagation delay to the destination and the OF may specify ‘’finding the shortest path based on the propagation delay.’’
  - Some metrics may not be additive; for example, the OF may be ‘’finding the path where the minimum link quality is maximized. ‘’
- *Constraint*: used to include or eliminate links or nodes that do not meet specific criteria (this is usually referred to as *constraint-based routing*).
  - For example, the OF may not select any path that traverses a node that is battery-operated or a link that does not provide link layer encryption.
- The OF may combine link/node metrics and constraints such as “ find the path with the minimum delay that does not traverse any non-encrypted link. ”

Objective Function (OF)

- Examples (IP nets):
  - the OF of RIP is to select the path with minimal hop count;
  - OSPF or IS-IS would compute the paths that provide the minimum cost where the path cost is simply the sum of the static link cost along the path.
  - With MPLS TE the OF may be slightly more complex: “ find the shortest path according to some metric such as the OSPF/ IS-IS
  - metric or the Traffic Engineering metric that satisfies some constraint such as the available reservable bandwidth or the type of recovery protection provided by the link. ”
- LLNs: strong interest in using several OFs because deployments greatly vary with different objectives and a single network may support traffic with very different requirements in terms of path quality.
  - Consider the case of a mixed network with battery- and main-powered nodes, a variety of high and low bandwidth links, and two main applications (telemetry and critical alarms).
  - Extremely useful for each node: supporting both applications to be able to use two paths.
    - One “ time-sensitive ” path for alarms where the objective is to have a short delay and a highly reliable path with no constraint on the type of nodes along the path to the destination
    - Another “ not time-sensitive path ” for the telemetry traffic where it is beneficial to not traverse any battery-operated node to preserve energy and where the objective would be to minimize hops to avoid traffic congestion in the network.
  - RPL addresses these requirements by building two DODAGs with each one having its own OF. The OF is used in conjunction with the routing metric to compute the path.

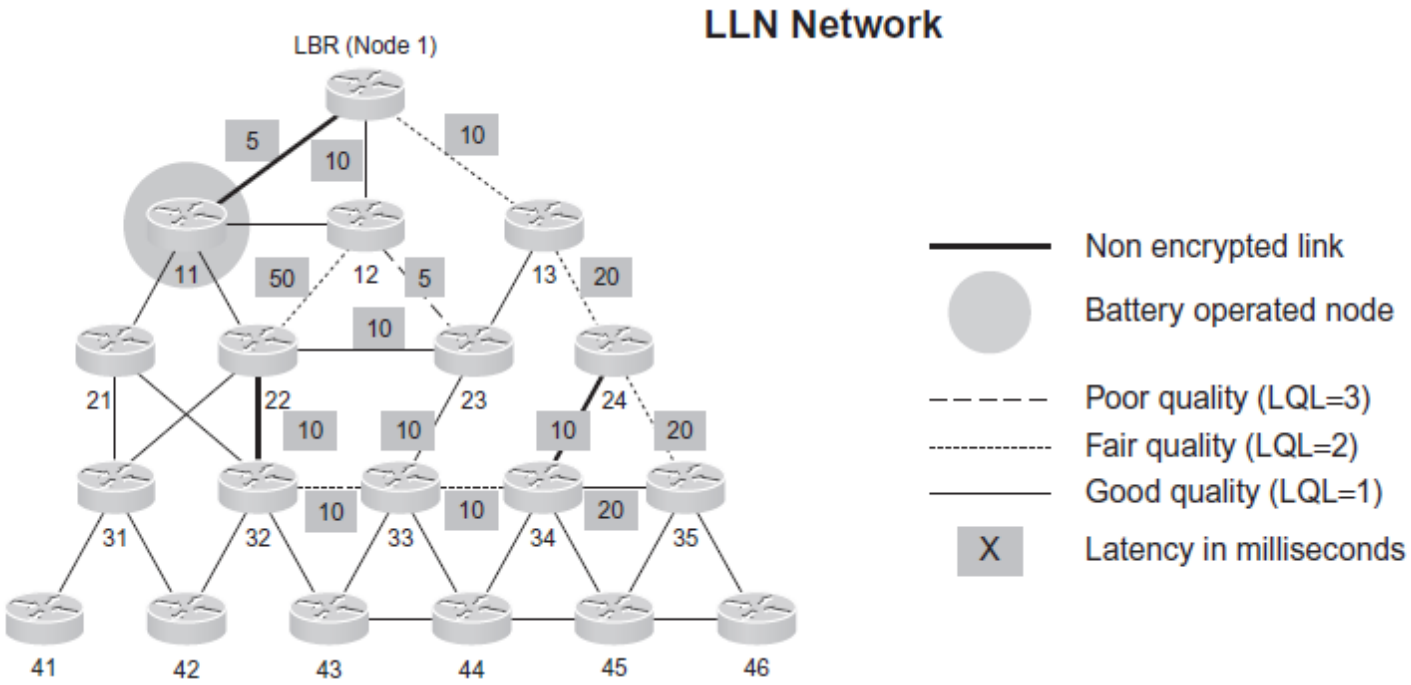
Routing Metrics (1/2)

- *Aggregated* (path cost is defi ned as the sum of the cost of all links along the path) versus *Recorded* (record each individual link metric)
- *Local* (not propagated along the DODAG) versus *Global*
- The Routing Metrics/Constraints Common Header (several flags → routing object: refers to a routing metric or a constraint; is local versus global; if the global metric is aggregated versus recorded; if a constraint is optional or mandatory, and if a metric is additive or reports a maximum/minimum.)
- Routing objects
  - The *Node State and Attributes (NSA) Object*: nodes may act as traffic aggregators (use many bits to describe the state); nodes have limited available resources → 1-bit fl ag set when a node sustainably experiences some level of congestion
  - *Node Energy Object*: several levels of granularity to characterize the node energy: (1) the node power mode; (2) the estimated remaining lifetime; and (3) potentially some detailed set of power-related metrics and attributes.
  - *Hop-count Object*
  - *Throughput Object*
  - *Latency Object*

Routing Metrics (2/2)

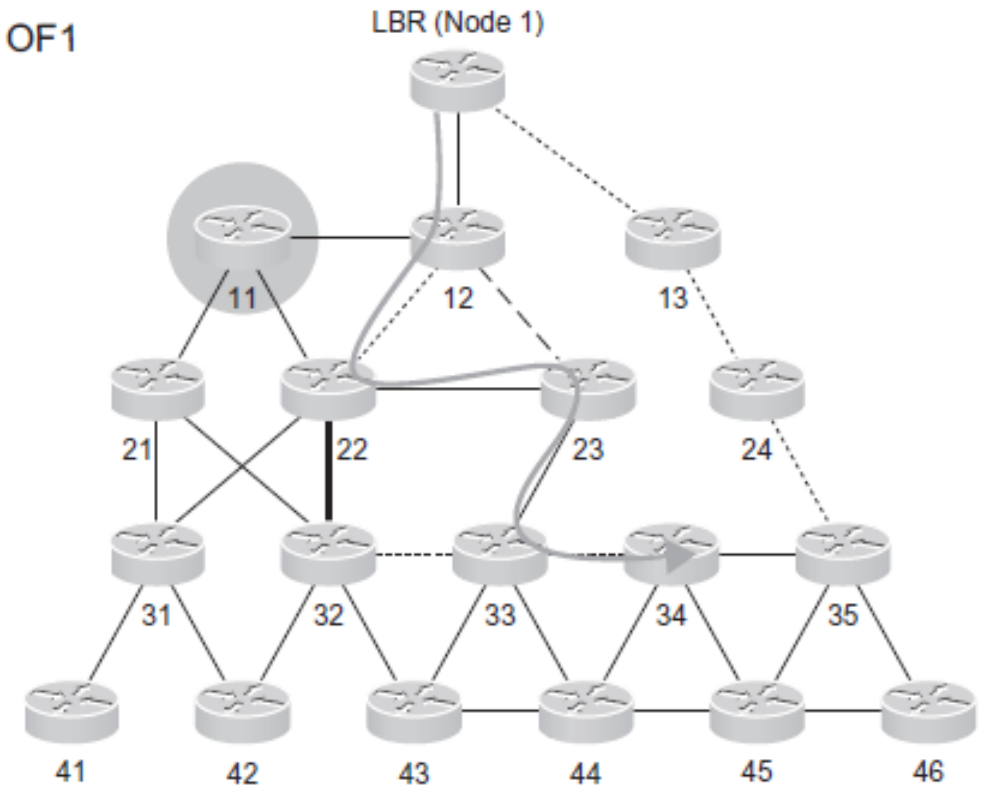
- Link Reliability Object: the most popular reliability metric thus far is the expected transmission (ETX) count metric
  - Characterizes the average number of packet transmissions required to successfully transmit a packet (see before).
  - Tightly coupled to the throughput along a path.
  - Several techniques have been proposed to compute ETX. → IETF does not specify the method for computing ETX values. The ETX is a link-specific quantity and the technique used to compute the ETX value should be independent of the link layer and not specified by the network layer that only carries it for routing protocol decisions.
- Link Colors Attribute: link color is encoded using a bit vector; the meaning of each color is left to the implementer.
  - RPL computes paths over a dynamically built DODAG. The DODAG root uses an OF required for each node along the path reporting path metrics to also report the set of colors of each link along the path.
  - For example, suppose that the color blue is used to indicate the support of the link layer encryption. Upon receiving the path metric, if link colors are recorded, a node may decide to elect as a parent the parent reporting paths with encrypted links (blue links) or with the maximum number of blue links in the absence of a path exclusively made of blue links.

An Example (1/3)



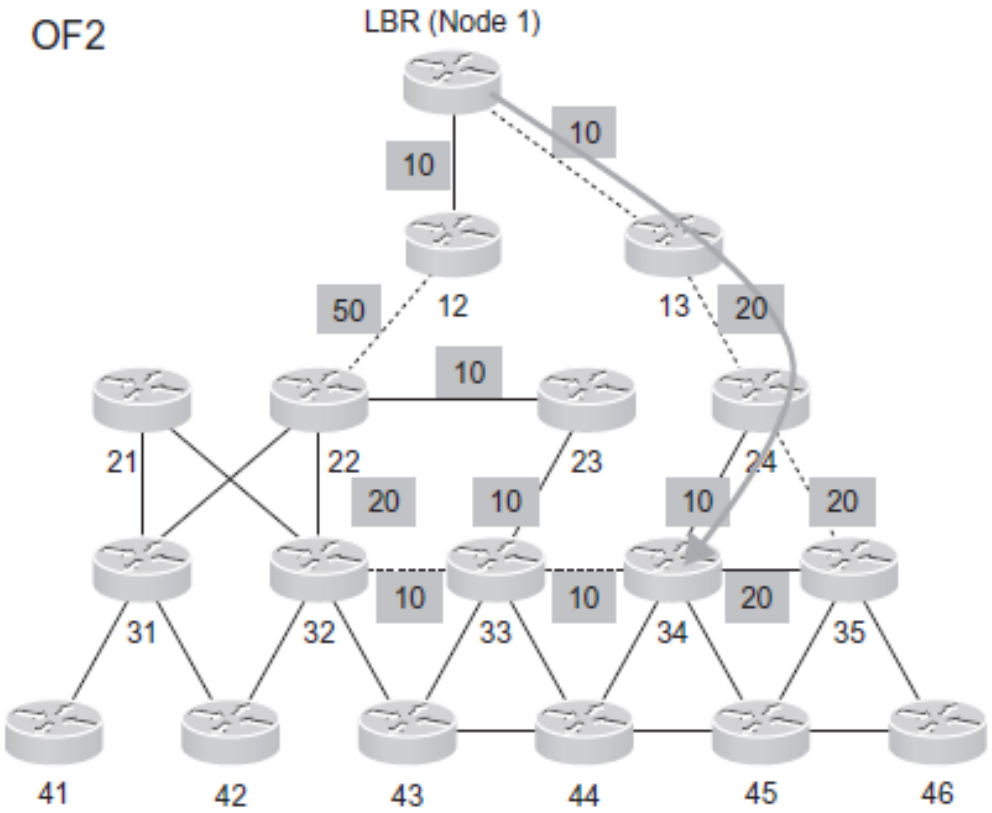
- Routing: from the low-power and lossy network border router (LBR) to node 34
- OF1: " Use the LQL as a global recorded metric and favor paths with the minimum number of low and fair quality links, use the link color as a link constraint to avoid non-encrypted links. "
- OF2: " Find the best path in terms of latency (link latency is used as a global aggregated metric), while avoiding poor quality links and battery-operated nodes. "

An Example (2/3)



- OF1: “ Use the LQL as a global recorded metric and favor paths with the minimum number of low and fair quality links, use the link color as a link constraint to avoid non-encrypted links. “
- Two paths are available with an equivalent aggregated LQL metric: 34-35-24-13-1 and 34-33-23-22-12-1. Since OF specifies using a recorded metric → path 34-33-**23-22**-12-1 is chosen since it only has two links of “ fair ” quality.

An Example (3/3)



- OF2: " Find the best path in terms of latency (link latency is used as a global aggregated metric), while avoiding poor quality links and battery-operated nodes. "
- Several paths have been pruned because they traverse battery-operated nodes (node 11) and traverse poor quality links (link 12-23). The best path (lowest latency) is 34-24-13-1.

RPL Basics

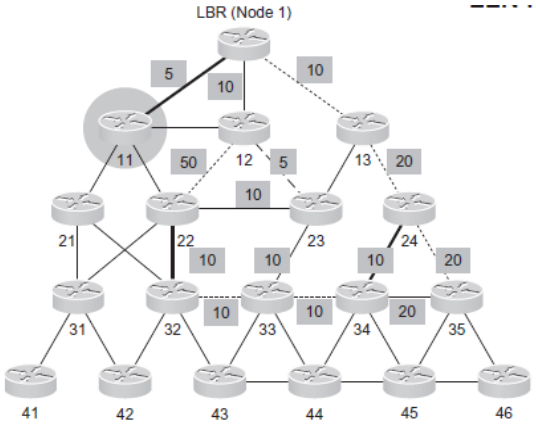
- RPL was designed to be **highly modular**: the core of the routing protocol would address the intersection of the application-specific routing requirements, and additional modules would be added as needed to address specific requirements.
- RPL was designed for LLNs where constrained devices are interconnected by (wireless and wired) lossy links → a node should carefully observe a link and start using it or determine whether to stop using it (thus triggering a global path recomputation in the network).
- Control traffic must be as tightly bounded as possible, ideally the control traffic should decrease as the routing topology stabilizes
- RPL is a distance vector protocol that builds a DODAG where paths are constructed from each node in the network to the DODAG root (typically a sink or an LBR)
- In contrast with tree topologies, DODAGs offer redundant paths, which is a MUST requirement for LLNs. Thus if the topology permits, RPL may provision more than one path between a node and the DODAG root and even other nodes in the network.

RPL Operations (1/5)

- First, one or more nodes are configured as DODAG roots by the network administrator.
- A node discovery mechanism based on newly defined ICMPv6 messages is used by RPL to build the DODAG.
- RPL defines two new ICMPv6 messages:
  - DODAG information object (DIO) messages; → UP
  - destination advertisement object (DAO) messages. → DOWN
- DIO messages (simply referred to as DIO) are sent by nodes to advertise information about the DODAG, such as: the DODAGID; the OF; the DODAG rank (depth); the DODAGSequenceNumber; along with other DODAG parameters such as a set of path metrics and constraints discussed before.
- When a node discovers multiple DODAG neighbors (that could become parents or sibling), it makes use of various rules to decide whether (and where) to join the DODAG. This allows the construction of the DODAG as nodes join.
- Once a node has joined a DODAG, it has a route toward the DODAG root (which may be a default route) in support of the MP2P traffic from the leaves to the DODAG root (in the up direction).

RPL Operations (2/5)

- RPL uses “ up ” and “ down ” directions terminology.
  - up direction from a leaf toward the DODAG root;
  - down direction refers to the opposite direction.
- The usual terminology of parents/children is used.
- RPL also introduces the “sibling”: two nodes are siblings if they have the same rank in the DODAG (note that they may or may not have a common parent).
- The parent of a node in the DODAG is the immediate successor within the DODAG in the up direction, whereas a DODAG sibling refers to a node at the same rank.
- In the side figure, 13 is a parent of 24, 22, and 23 are siblings, and 43 and 44 are children of 33.
- A DODAG is said to be:
  - *grounded* if it is connected to what RPL calls a “ goal, ” which can be a node connected to an external (non-LLN) private IP network or the public Internet;
  - a non-grounded DODAG is called a *floating* DODAG.



- RPL uses iterations controlled by the DODAG root to maintain the DODAG; the DODAGSequenceNumber is a counter incremented by the DODAG root to specify the iteration number of the DODAG .

RPL Operations (3/5)

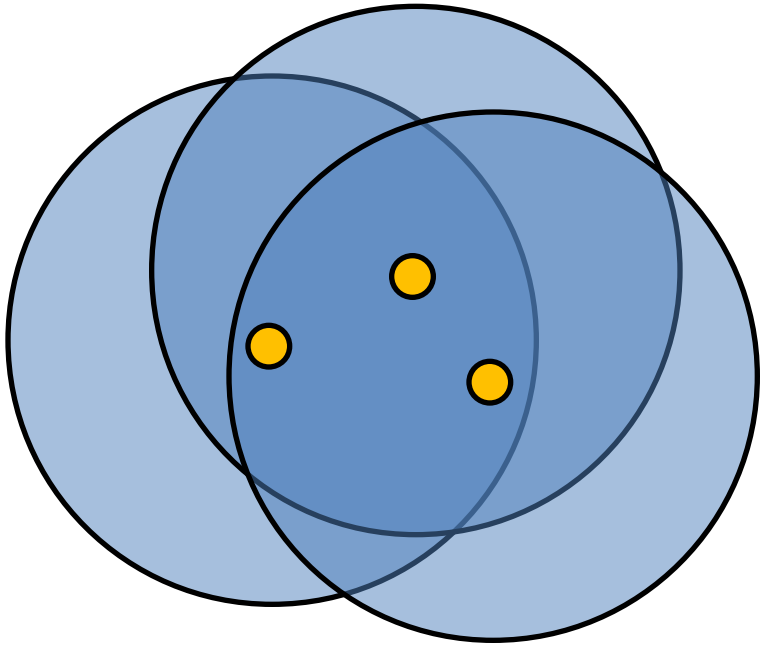
- DAO messages: provide routing information in the down direction (for the traffic from the route to the leaf) and for the P2P direction since the DODAG provides defaults routes to the DODAG root from each node in the network.
- DAO messages (simply referred to as DAO) are used to advertise prefi x reachability toward the leaves. DAOs carry prefix information along with a lifetime (to determine the freshness of the destination advertisement-DAs) and depth or path cost information to determine how far the destination is.
- Note that the path in the downward direction is dictated by the DODAG built by RPL in the other direction.
- In some cases, DAOs may also record the set of visited nodes → particularly useful when the intermediate nodes cannot store any routing states.
- If a parent receives destination advertisements that can be aggregated from multiple children, local policy may be used to perform prefix aggregation in an attempt to reduce routing table and the size of DAO messages.
- Note that redundant DAO messages are aggregated along the DODAG. An OF may be specifically designed to maximize prefix aggregation.

RPL Operations (4/5)

- RPL supports P2P traffic.
  - When node A sends a packet destined to node B, if B is not in direct reach, it forwards the packet to its DODAG parent.
  - From there, if the destination is reachable from one of its children, the packet is forwarded in the down direction.
  - In other words, the packet travels up to a common ancestor at which point it is forwarded in the down direction toward the destination.
- An interesting optimization periodically emits *link-local* multicast IPv6 DAOs - → if the destination is in direct range (one hop away), a node can send the packet directly to the destination without following the DODAG.
- Sending DIO and DAO messages is governed by the use of trickle timers. The trickle timers use dynamic timers that govern the sending of RPL control messages in an attempt to reduce redundant messages.
  - When the DODAG is unstable (e.g., the DODAG is being rebuilt) RPL control messages are sent more frequently (the DODAG becomes inconsistent).
  - On the other hand, as the DODAG stabilizes messages are sent less often to reduce the control plane overhead, which is very important in LLNs.

Trickle: idea

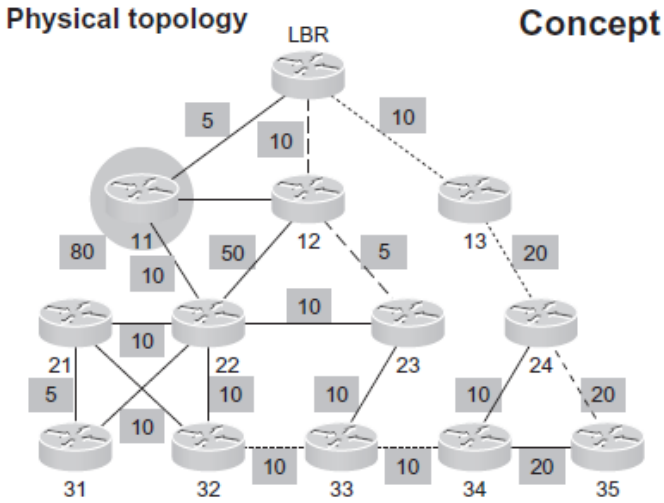
- Concerns
  - Broadcast is expensive
  - Wireless channel is a shared, spatial resource
- Idea
  - Dynamic adjustment of transmission period
  - Suppress transmissions that may be redundant
- Parameters:
  - T_min: Minimum advertisement period
  - T_max: Maximum advertisement period
  - k: Suppression threshold
- Period adjustment:
  - On receiving *inconsistent* route information, reset to T_min
  - Otherwise, double up to T_max
- Suppression:
  - Increment count when receiving *similar* advertisement
  - At end of period, transmit if c < k, set c = 0
- Proposal:
  - Carry T_min, T_max, and k in DIO

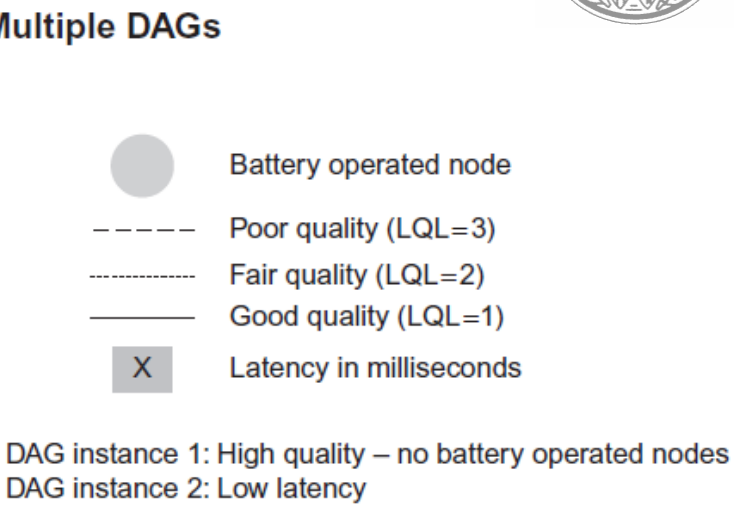


RPL Operations (5/5)

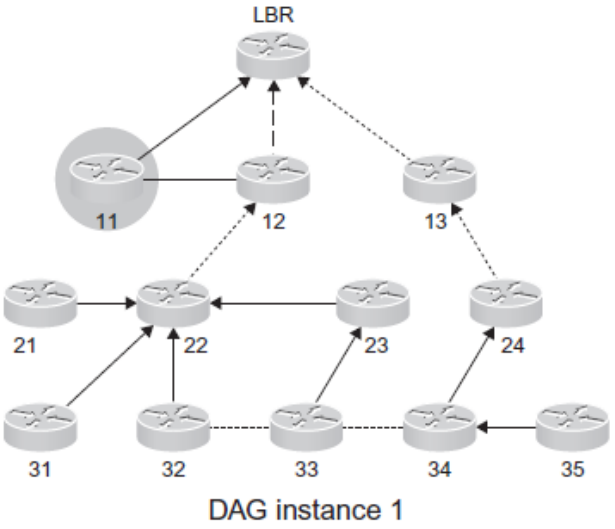
- Once the DODAG is built and routing tables are populated, routing is fully operational.
- As links and nodes fail, paths are repaired using local and global repair mechanisms.
  - Local repairs quickly fi nd a backup path without an attempt to globally reoptimize the DODAG entirely
  - Global repairs rely on a reoptimization process driven by the DODAG root.
- RPL also supports the concept of DODAG instances identifi ed by an Instance ID called the RPLInstanceID. It might be useful to form different topologies according to various sets of constraints and OFs. An RPL node may join multiple DODAG instances; for example:
  - one DODAG optimizes for high reliability
  - another DODAG optimizes for low latency.
- Data packets are then forwarded along the appropriate DODAG according to the application requirements.

Multiple DODAG Instances

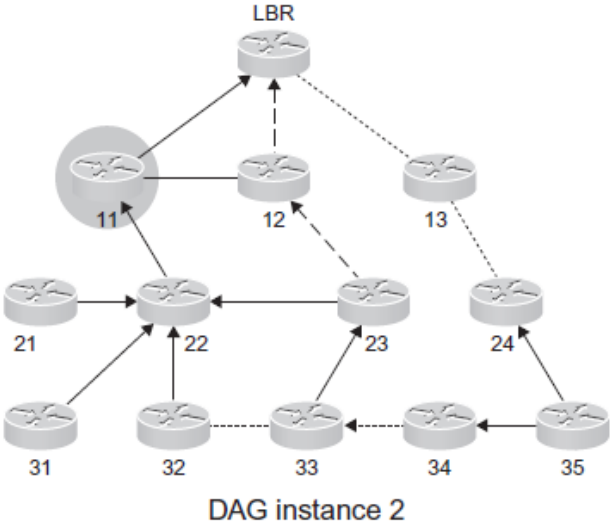




DAG instance 1: High quality – no battery operated nodes
DAG instance 2: Low latency



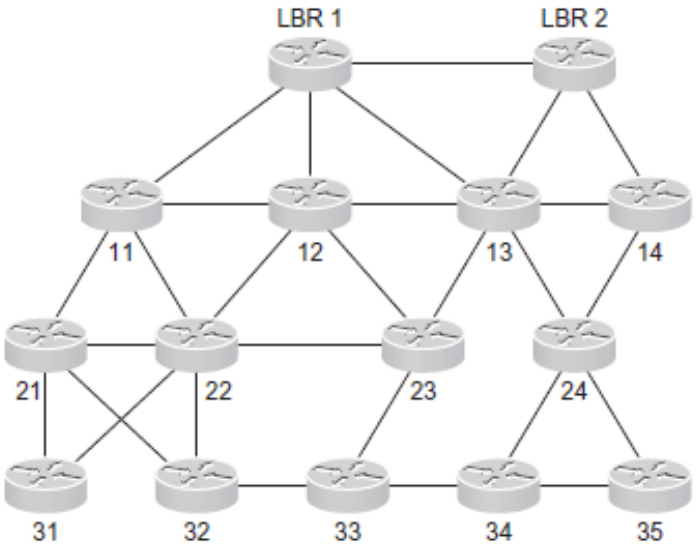
DAG instance 1



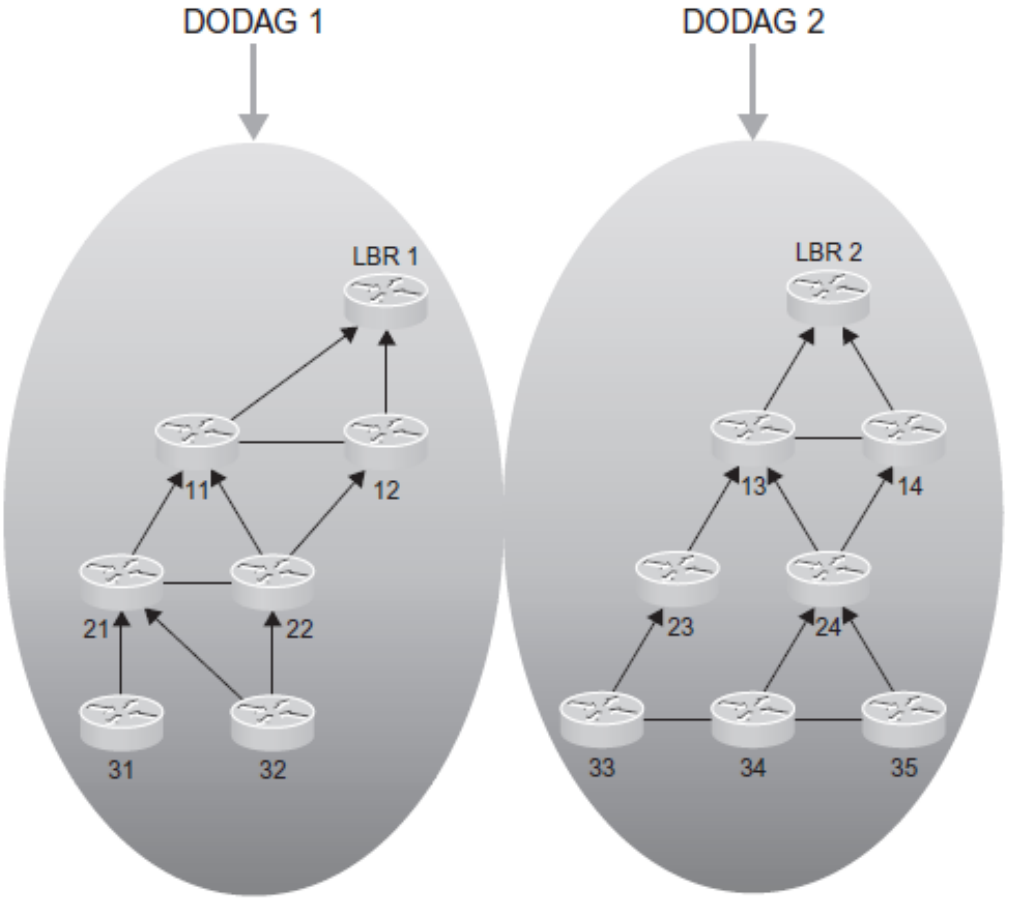
DAG instance 2

Multiple Disjoint DODAGs

Physical topology



Multiple DODAGs



DIO Messages

- DIO messages are sent by RPL nodes to advertise a DODAG and its characteristics → DIOs are used for DODAG discovery, formation, and maintenance.
- DIOs carry a set of mandatory information augmented with options.
- The following flags and fields are currently defined:
  - Grounded (G): Indicates whether the DODAG is grounded, in other words, the DODAG root is a goal for the OF (e.g., the DODAG root is connected to a non-LLN IP network such as a private network or the public Internet).
  - Destination Advertisement Trigger (T): The T bit is used to trigger a complete refresh of the routes in the down direction (downward routes).
  - Destination Advertisement Stored (S): The S bit is used to indicate that a non-root ancestor is storing routing table entries learned from DAO messages.
  - Destination advertisement supported (A flag): The A flag is set when the DODAG root supports the collection of prefix advertisements and enables the advertisement of prefixes in the DODAG.
  - DODAGPreference (Prf): The Prf is a 3-bit field set by the DODAG root to report its preference. It can be used to engineer the network and make some DODAGs more attractive to join.
- The DODAGSequenceNumber is the sequence number of the DODAG that characterizes the DODAG iteration and is exclusively controlled by the DODAG root.
- The RPLInstanceID is used to identify the DODAG instance and is provisioned at the DODAG root.
- The Destination Advertisement Trigger Sequence Number (DTSN) is an 8-bit integer set by the node sending the DIO → used to maintain the downward routes.
- The DODAGID is a 128-bit integer set by the DODAG root and that uniquely identifies the DODAG.
- The DODAG Rank is the rank of the node sending the DIO message
  - The rank determines the relative position of a node in the DODAG and is used primarily for *loop avoidance*.
  - The rank is computed according to the OF and is potentially subject to local node policy.
  - The rank (although potentially derived from routing metrics) is not a metric.
- When two nodes have the same rank, the nodes are said to be siblings (they are located at a similar level of optimality in the DODAG).

DAO Messages

- DAO messages are used to propagate destination information along the DODAG in the up direction to populate the routing tables of ancestor nodes in support of P2MP and P2P traffic.
- The DAO message includes the following information:
  - DAO sequence: A counter incremented by the node owning the advertised prefix each time a new DAO message is sent.
  - RPLInstanceID: The topology instance ID as learned from the DIO.
  - DAO rank: Corresponds to the rank of the node that owns the prefix.
  - DAO lifetime: It is expressed in seconds and corresponds to the prefix lifetime.
  - Route tag: 8-bit integer that can be used to tag “ critical ” routes. The priority could be used to indicate whether the route should be stored by the nodes with a lower rank (closer to the DODAG root), which could be useful if nodes have limited memory capacities and must be selective about which destination information to cache. Note that the size of that fi eld has been changed several times and is subject to further changes.
  - Destination Prefix: The Prefix Length field contains the number of valid leading bits in the prefix.
  - Reverse Route Stack (RRS): contains a number of RRCount (another field of the DAO message) IPv6 addresses used in LLNs with nodes that cannot store routing tables.

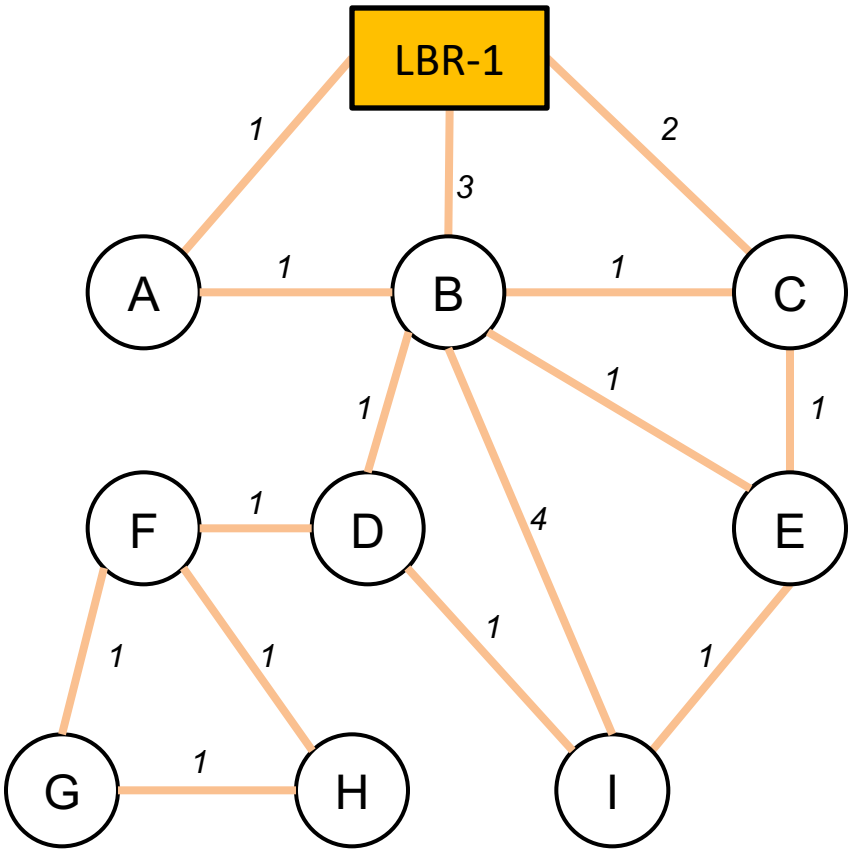
The “perfect” DAG?

- Since there no perfect wires with constant properties there’s hardly a perfect DAG
- Conditions change, batteries deplete
- Need to arbiter between conflicting metrics and goals
- Build a structure that is satisfactory for all
- Maintain with probably a slow evolution
- The characteristics of the network, including the metrics, are unpredictable. What we really want is that every node gets an acceptable result most of the time.
- There are many ways in which acceptable can be arranged (for some time).
- We do not look for the illusory “perfect DAG” so we could reach any of a large number of acceptable solutions.

An Open Issue: Carrying IPv6 Addresses

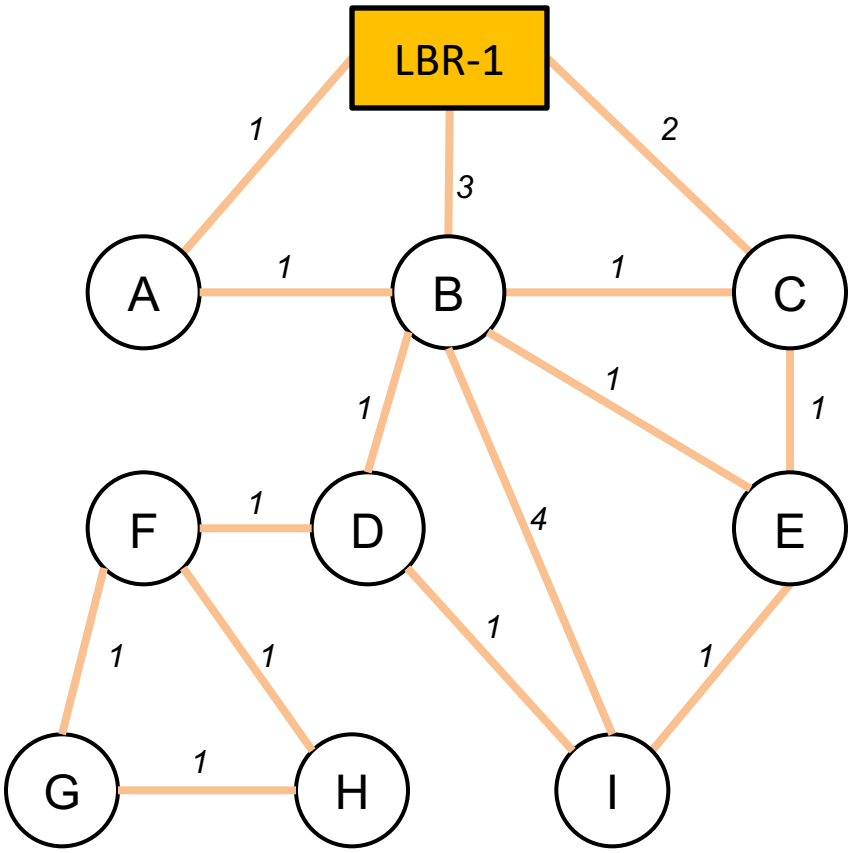
- High communication overhead
  - Carried in DIO, DAO, source routes
- High state overhead
  - Maintain destination routing state
- Options for compressing
  - Use a new namespace (e.g. labels)
  - Compress IPv6 addresses

DAG Construction



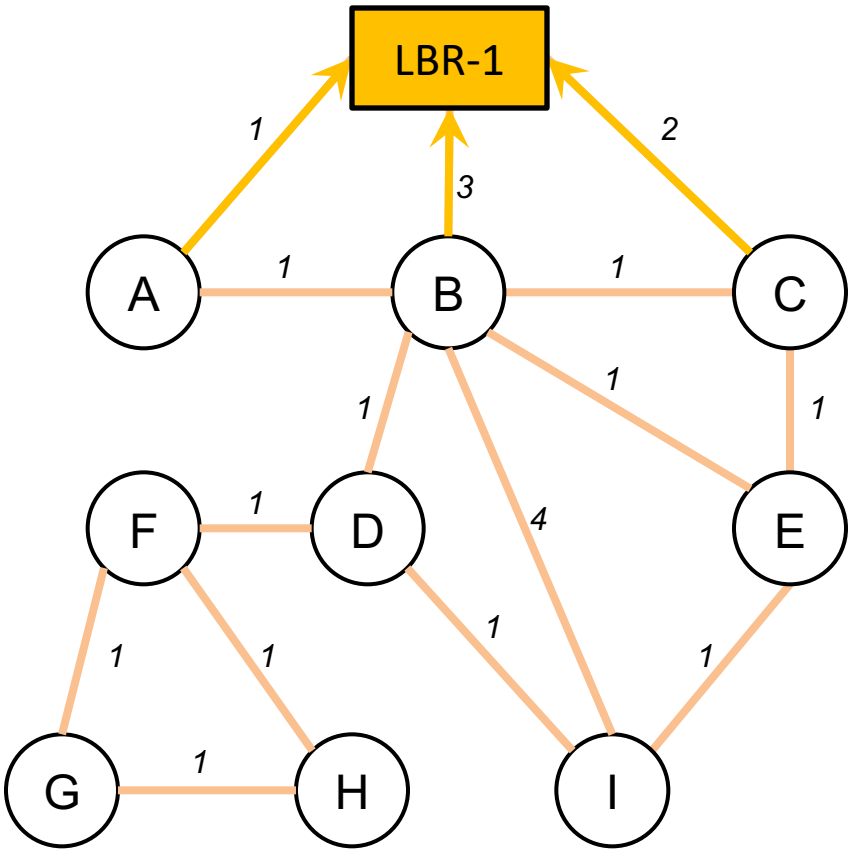
- LLN links are depicted
- Links are annotated w/ ETX
- It is expected that ETX variations will be averaged/filtered to be stable enough for route computation
  - Nodes are also to observe the metric and gain confidence before use

DAG Construction



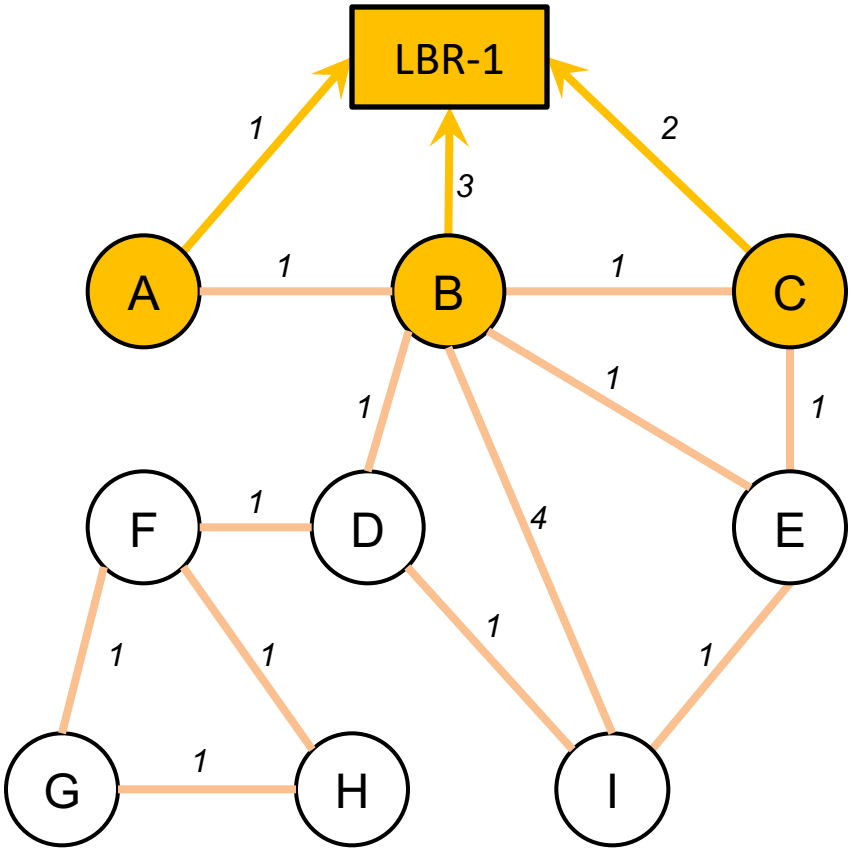
- Example: Objective Code Point (default OF)
- Main characteristics
  - Metric: ETX
  - Objective: Minimize ETX
  - Depth computation: **Depth ~ ETX**
    - Note that a practical computation may be more coarse

DAG Construction



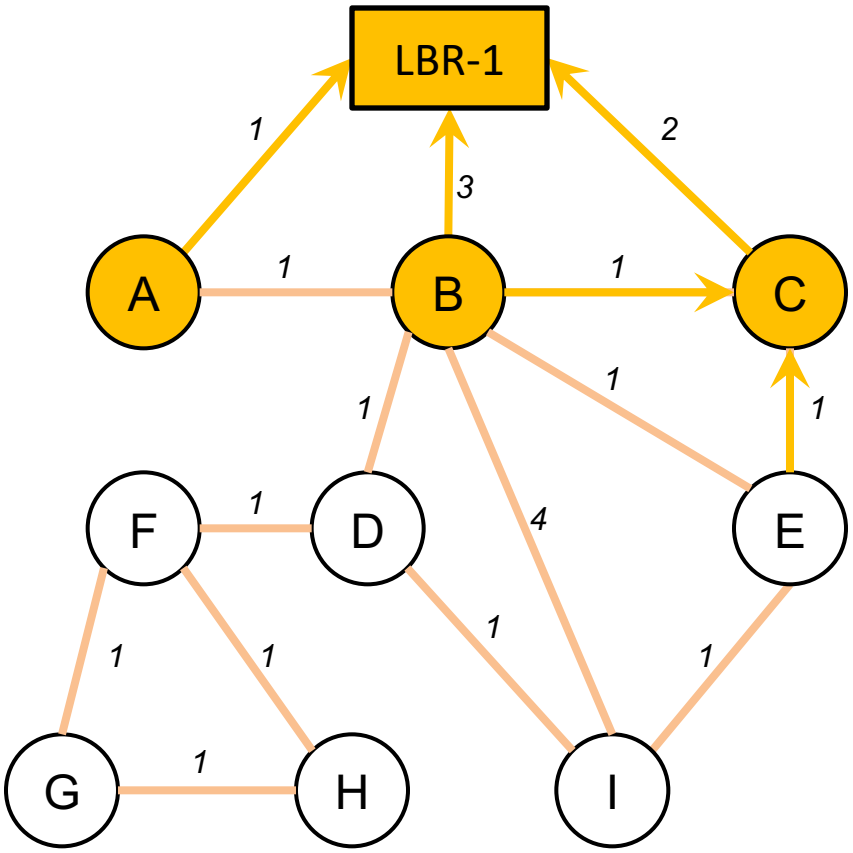
- LBR-1 multicasts RA-DIO
- Nodes A, B, C receive and process RA-DIO
- Nodes A, B, C consider link metrics to LBR-1 and the optimization objective
- The optimization objective can be satisfied by joining the DAG rooted at LBR-1
- Nodes A, B, C add LBR-1 as a DAG parent and join the DAG

DAG Construction



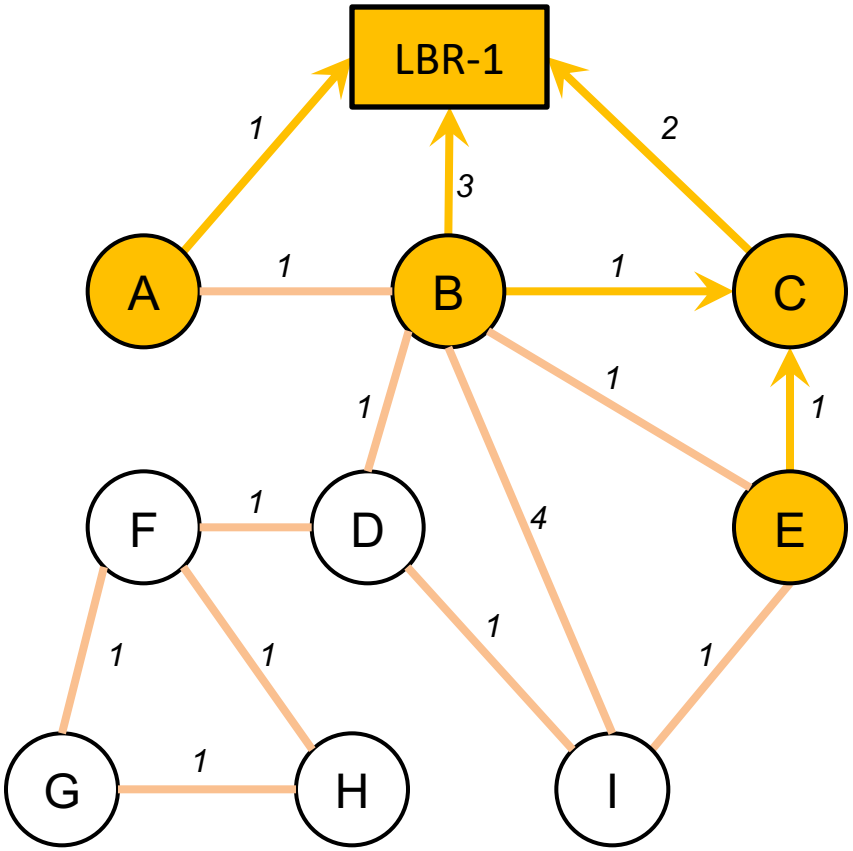
- Node A is at Depth 1 in the DAG, as calculated by the routine indicated by the example OCP (Depth ~ ETX)
- Node B is at Depth 3, Node C is at Depth 2
- Nodes A, B, C have installed default routes (::/0) with LBR-1 as successor

DAG Construction



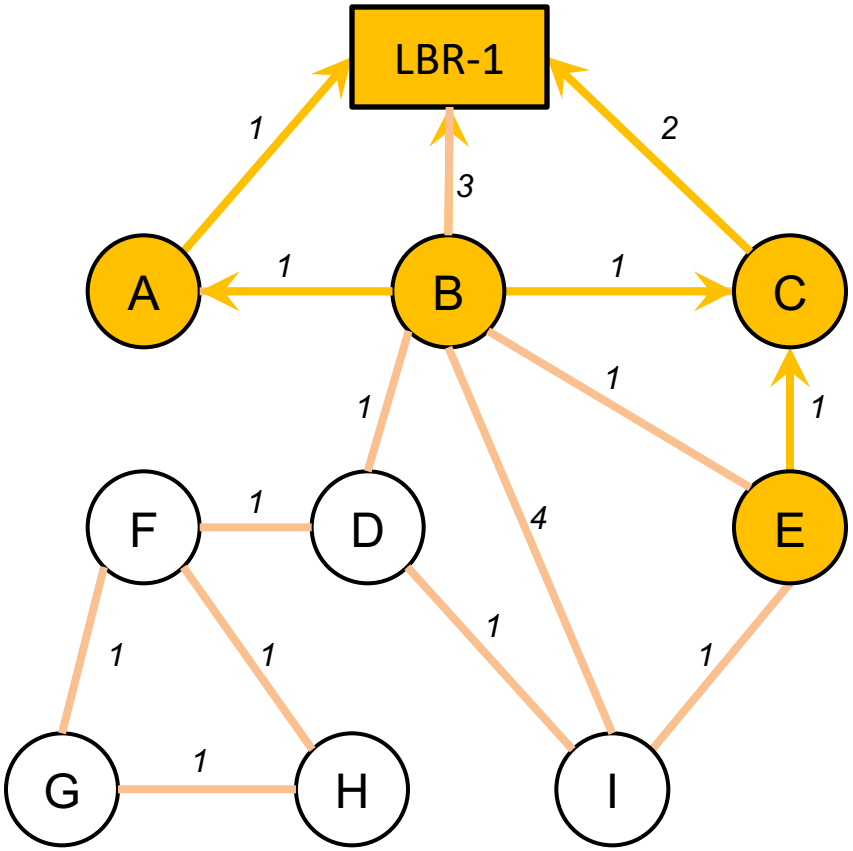
- The RA (Resource Advertisement) timer on Node C expires
- Node C multicasts RA-DIO
- LBR-1 ignores RA-DIO from deeper node
- Node B can add Node C as *alternate* DAG Parent, remaining at Depth 3
- Node E joins the DAG at Depth 3 by adding Node C as DAG Parent

DAG Construction



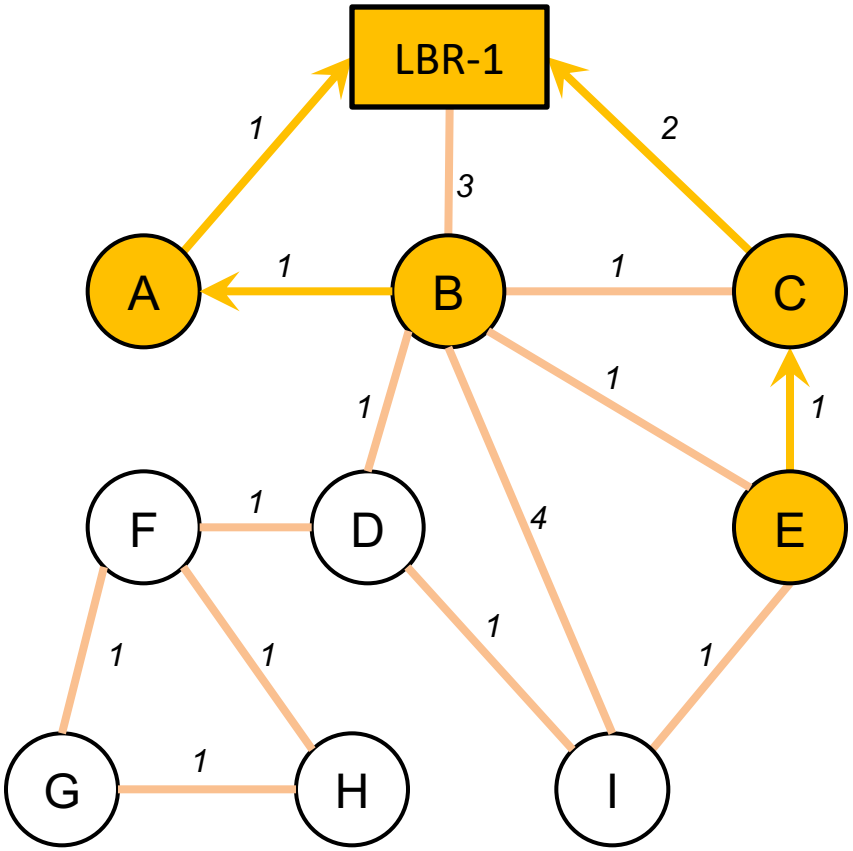
- Node A is at Depth 1, and can reach ::/0 via LBR-1 with ETX 1
- Node B is at Depth 3, with DAG Parents LBR-1, and can reach ::/0 via LBR-1 or C with ETX 3
- Node C is at Depth 2, ::/0 via LBR-1 with ETX 2
- Node E is at Depth 3, ::/0 via C with ETX 3

DAG Construction



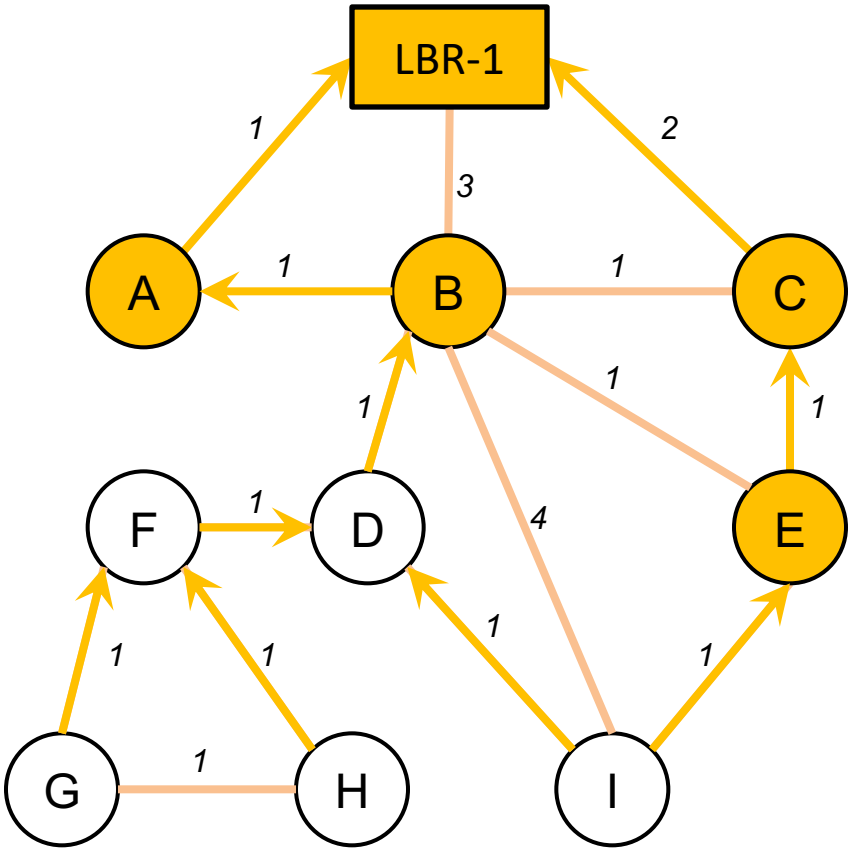
- The RA timer on Node A expires
- Node A multicasts RA-DIO
- LBR-1 ignores RA-DIO from deeper node
- Node B adds Node A
- Node B can improve to a more optimum position in the DAG
- Node B *removes* LBR-1, Node C as DAG Parents

DAG Construction



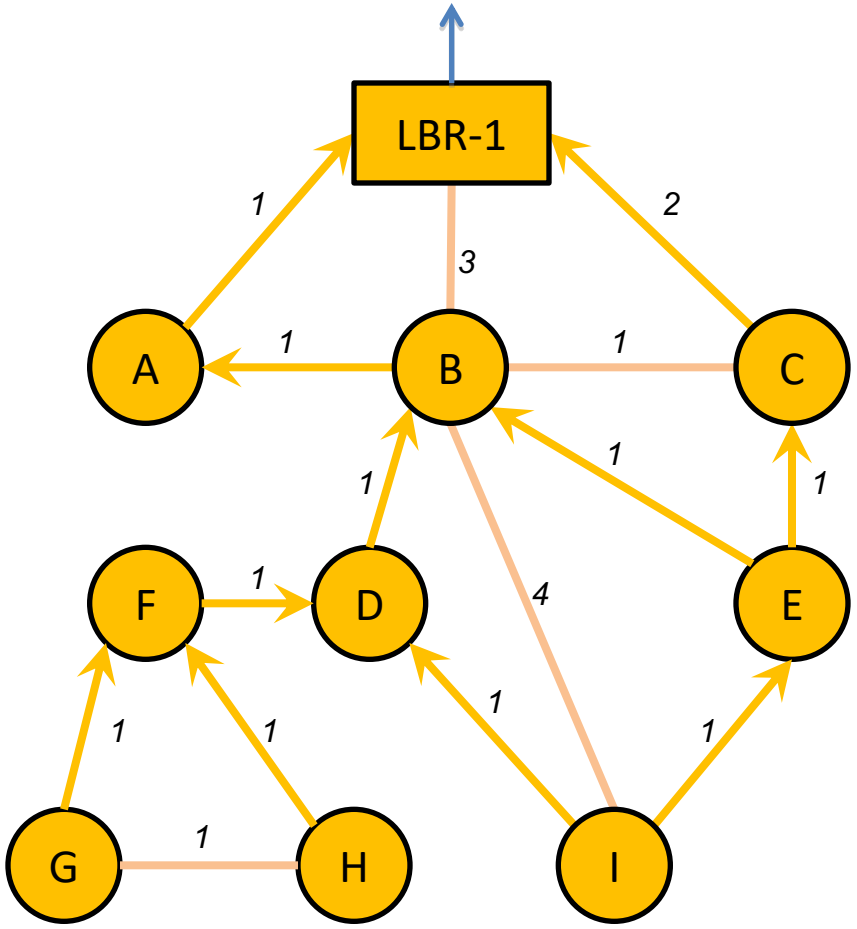
- Node A is at Depth 1, ::/0 via LBR-1 with ETX 2
- Node B is at Depth 2, ::/0 via A with ETX 2
- Node C is at Depth 2, ::/0 via LBR-1 with ETX 2
- Node E is at Depth 3, ::/0 via C with ETX 3

DAG Construction



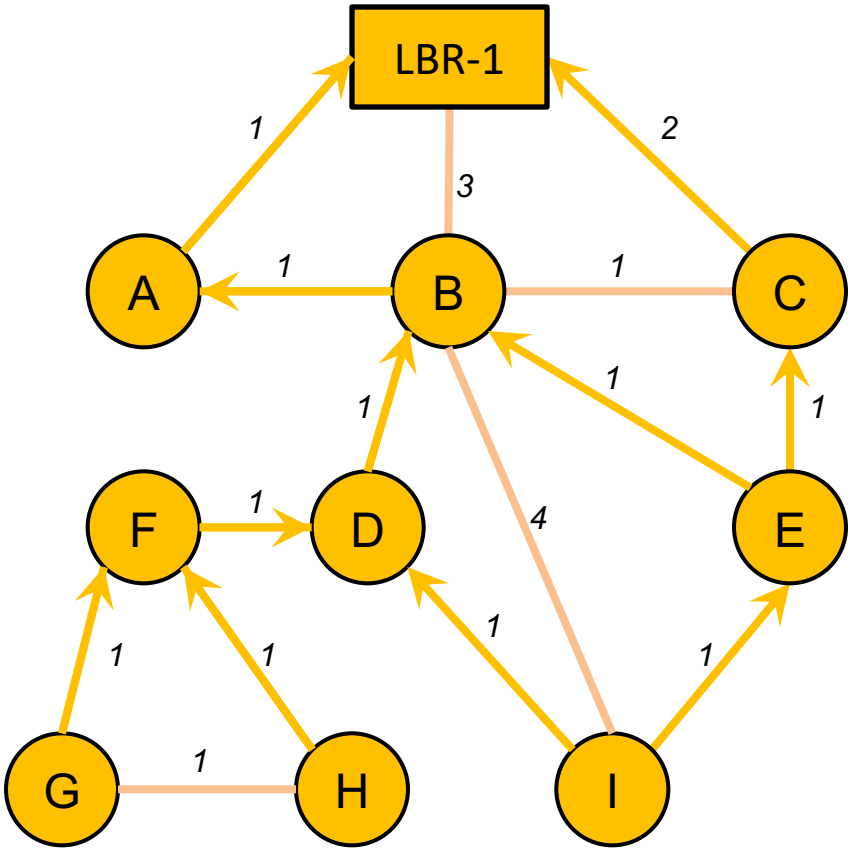
- DAG Construction continues…
- And is continuously maintained (control messages – RA-DIOs)

MP2P Traffic



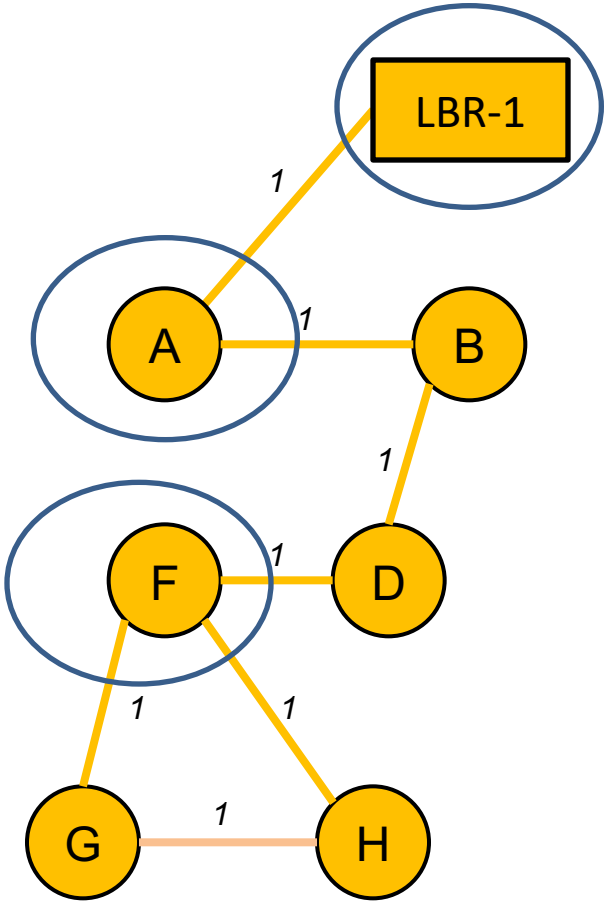
- MP2P traffic flows upwards along DAG, toward DAG Root
- DAG Root may also extend connectivity to other prefixes beyond the DAG root, as specified in the DIO
- Nodes may join multiple DAGs as necessary to satisfy application constraints

Destination Advertisements



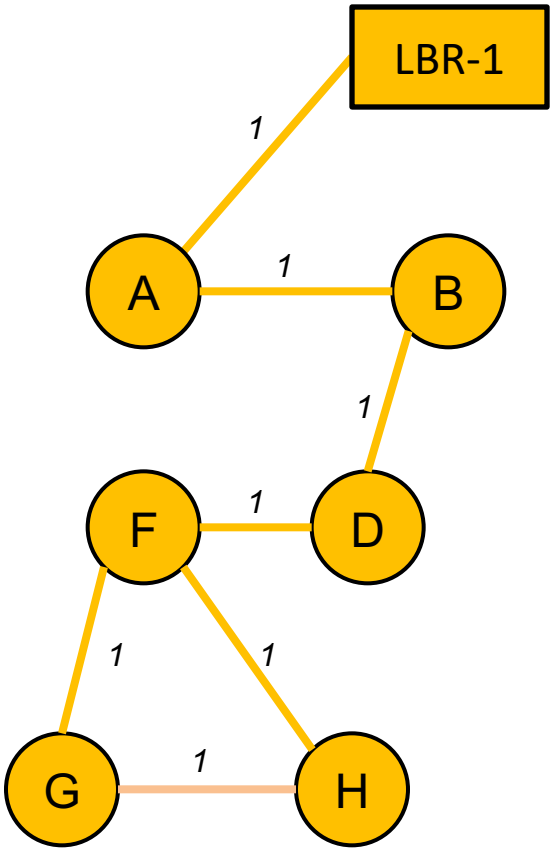
- Destination Advertisements (Das) build up routing state in support of P2MP traffic flows downward, from the sink to other nodes
- DAs use the same DAG
- For simplicity, we will focus on a subset of DAs in the example

Destination Advertisements



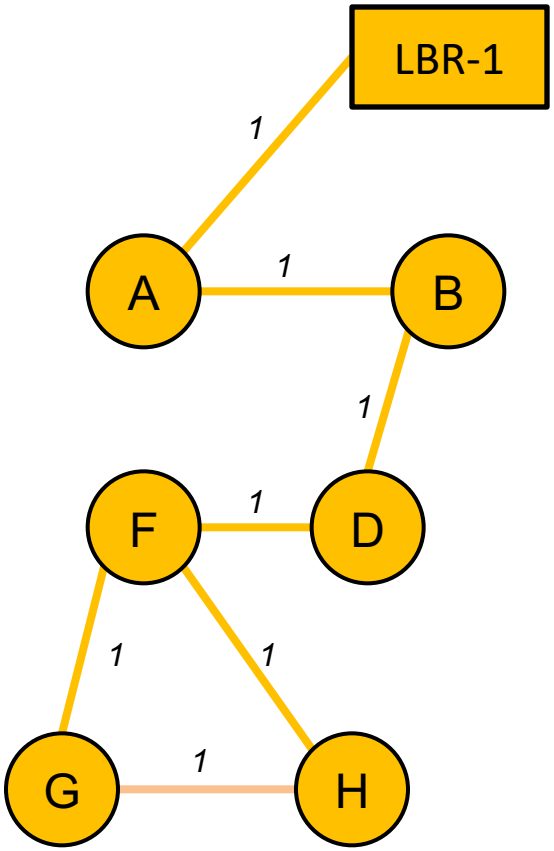
- Some nodes may be able to store routing state for downward flows (LBR-1, A, F)
- Some nodes may not (B, D)
- Some nodes may have a limited ability; DAs may indicate a priority for storage

Destination Advertisements



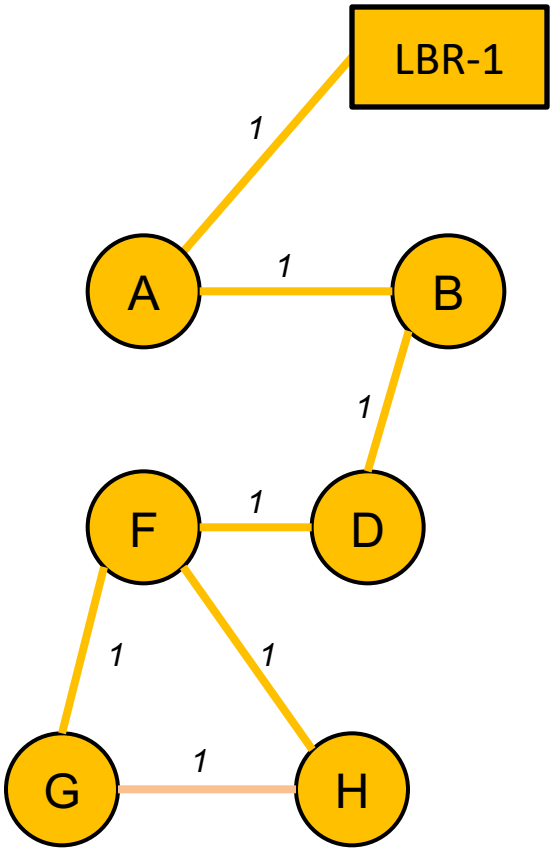
- DAs may be triggered by DAG root or node who detects a change
- DA timers configured such that DAs start at greater depth, and may aggregate as they move up

Destination Advertisements



- LBR-1 triggers Destination Advertisement mechanism in DIO
- G emits NA to F with DAO indicating reachability to destination prefix G::
- F stores G:: via G
- H emits NA to F for destination prefix H::
- F stores H:: via H

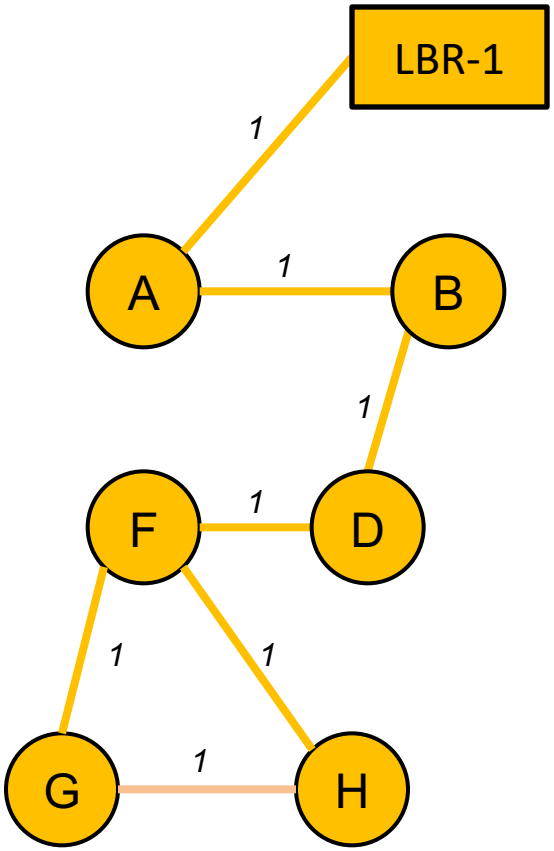
Destination Advertisements



- Suppose in this example F has a prefix F*:: capable of aggregating {F::, G::, H::}
  - The method to provision such a prefix is beyond the scope of RPL
- F emits NA to D with DAO indicating reachability to destination prefix F*::
- D cannot store…

(continued)

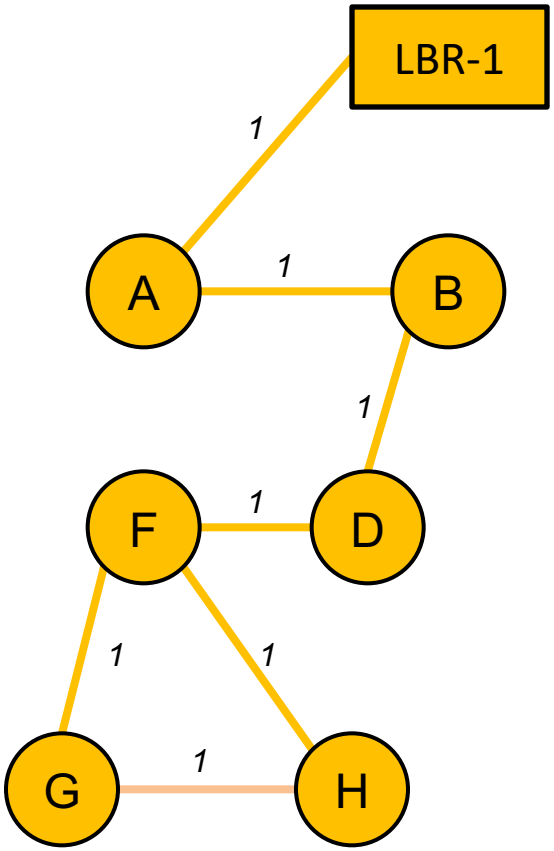
Destination Advertisements



- D adds F to the Reverse Route Stack in the DAO, and passes DAO on to B for F*:: [F]
- D also emits a DAO indicating prefix D:: to B
- B cannot store routing state…

(continued)

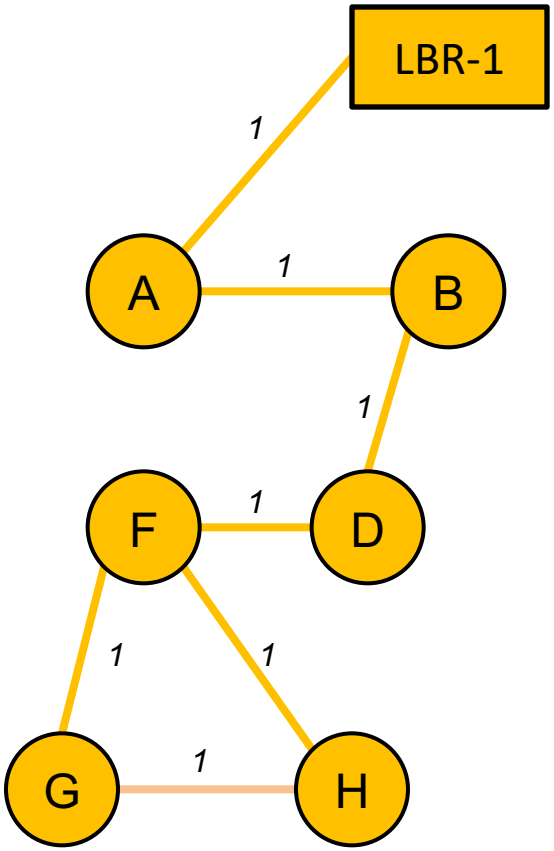
Destination Advertisements



- B adds D to the Reverse Route Stack in the DAO for D::, and passes DAO D:: [D] on to A
- A stores D:: via B, with the piecewise source route [D]
- B also emits a DAO indicating prefix B:: to A
- A stores B:: via B

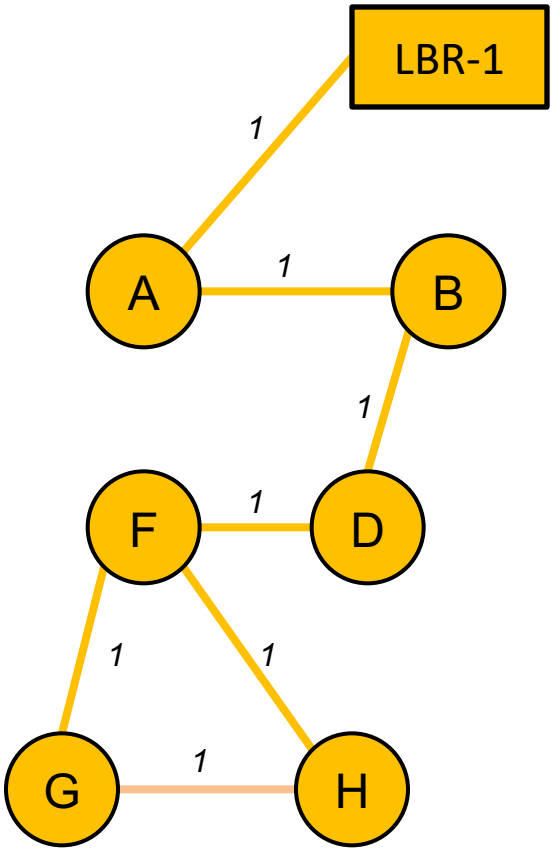
(continued)

Destination Advertisements



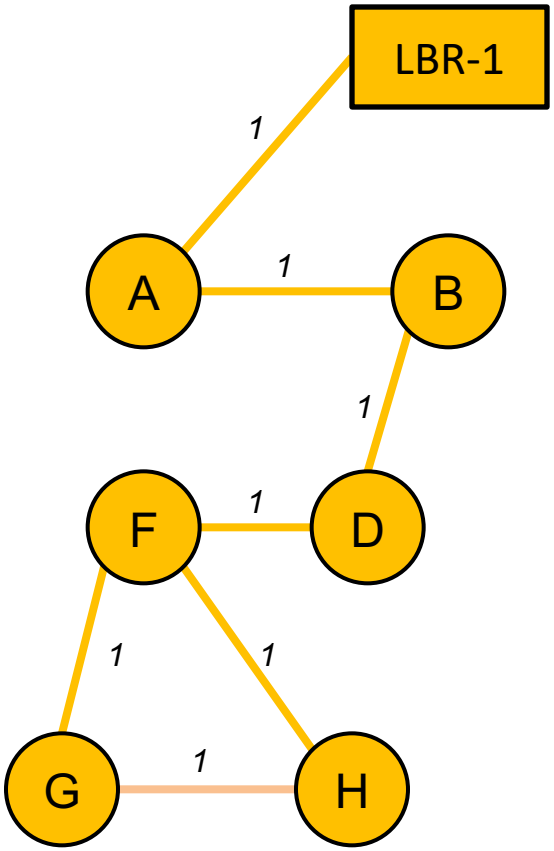
- A emits DAOs to LBR-1 for destination prefixes A::, B::, D::, and F*
- LBR-1 stores A:: via A, B:: via A, D:: via A, and F*:: via A
- A stored B:: via B, D:: via B [D], F* via B [D, F]
- B, D stored nothing
- F stored G:: via G, H:: via H

P2MP Traffic



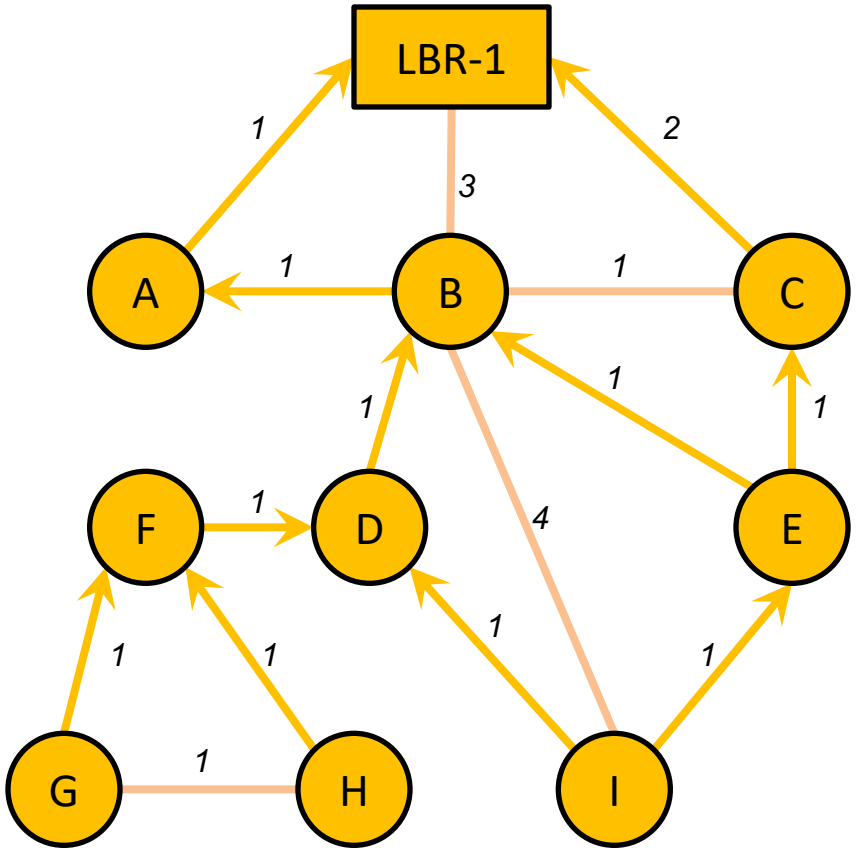
- The routing state setup by Destination Advertisement is used to direct P2MP traffic downward
- LBR-1 directs traffic for G (F*::) to A
- A adds source routing directive, [D, F], and forwards to B
- B uses source routing directive to forward to D...

P2MP Traffic



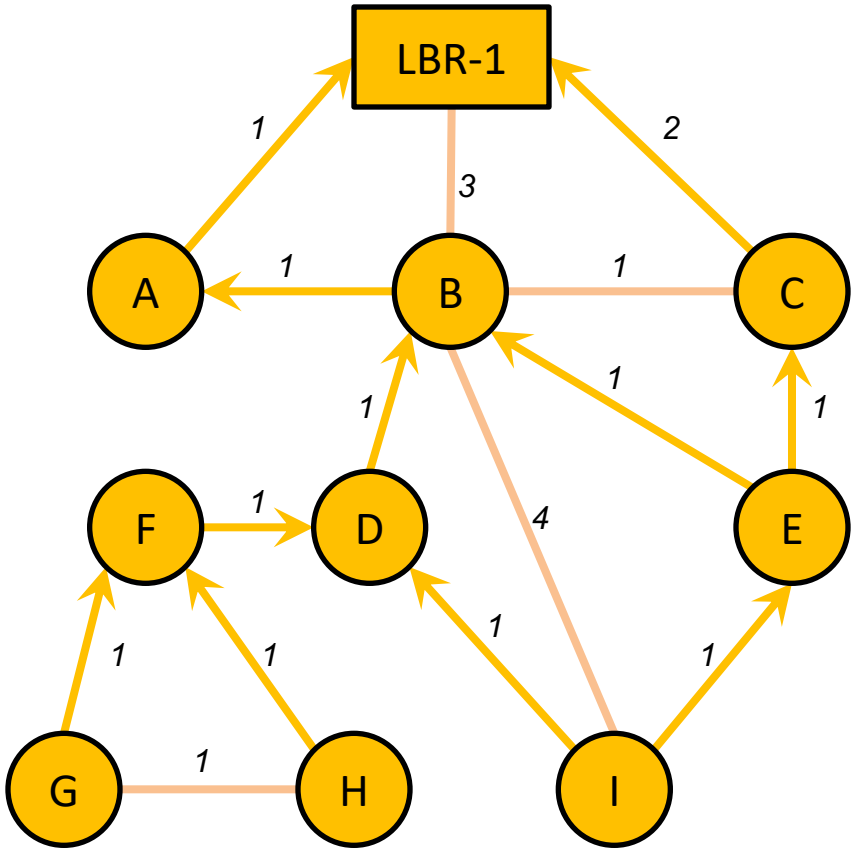
- D uses source routing directive to forward to F
- F uses routing state to forward to G
- Note the use of source routing to traverse the stateless region of the LLN
  - The details of the source routing mechanism are still to be defined

P2P Traffic



- By default, P2P traffic follows the default route until a node is encountered who has a more explicit route
- P2P traffic flows upwards along the DAG until a common node is reached to direct the traffic outwards

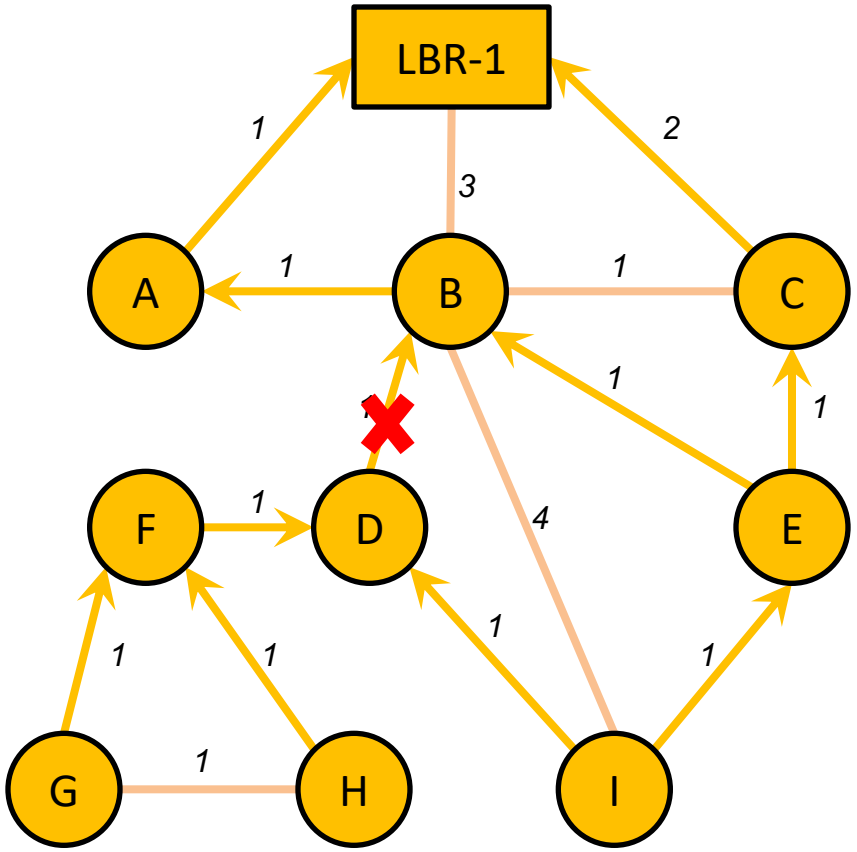
P2P Traffic



- The default scheme allows P2P traffic with minimal state required of nodes – but is not appropriate in all cases
- More optimal P2P routes may be computed and installed if the application requires
  - RPL does not specify nor preclude such a mechanism at this time

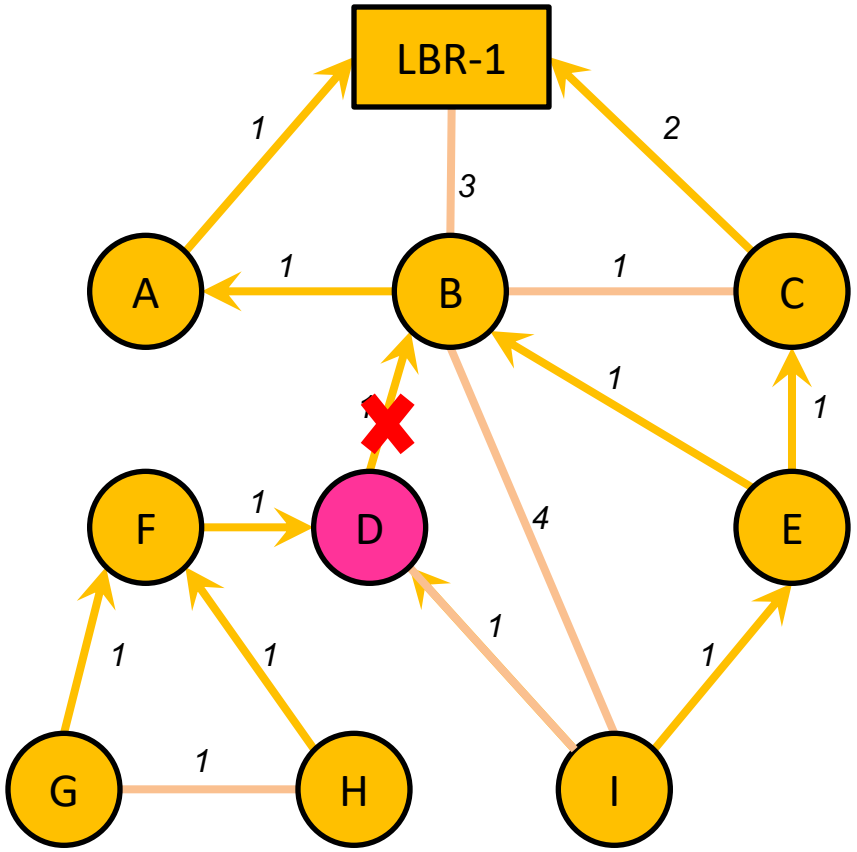
DAG Maintenance

Loop Avoidance



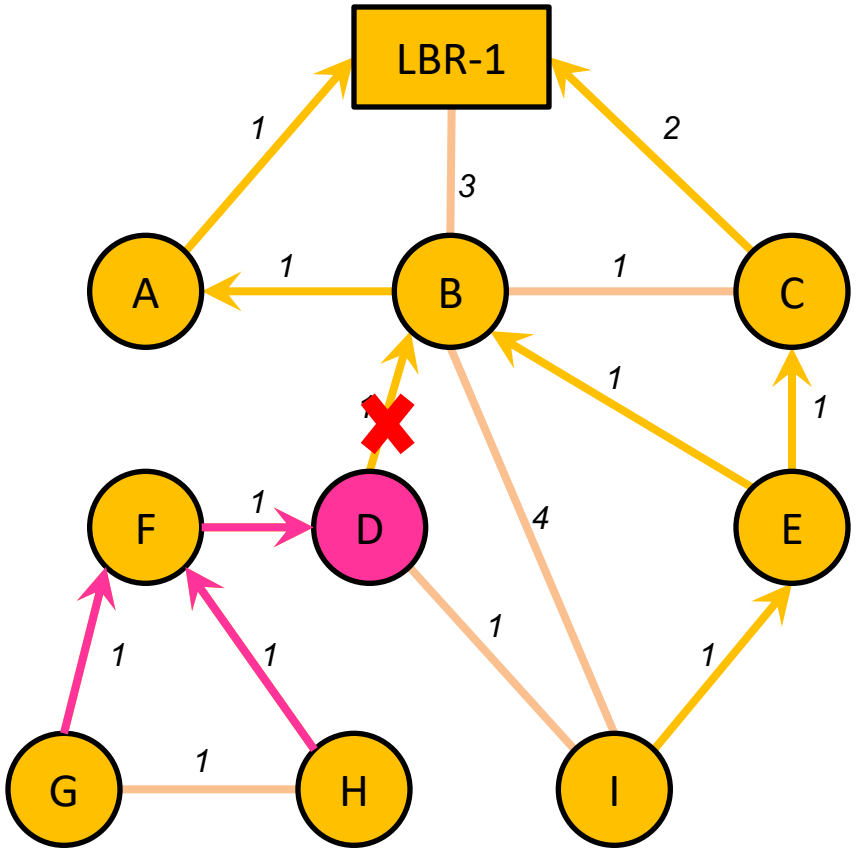
- Consider the case where the link B—D goes bad
- Node D will remove B from its DAG parent set
- Node D no longer has any DAG parents in the grounded DAG, so it will become the root of its own floating DAG

DAG Maintenance
Loop Avoidance



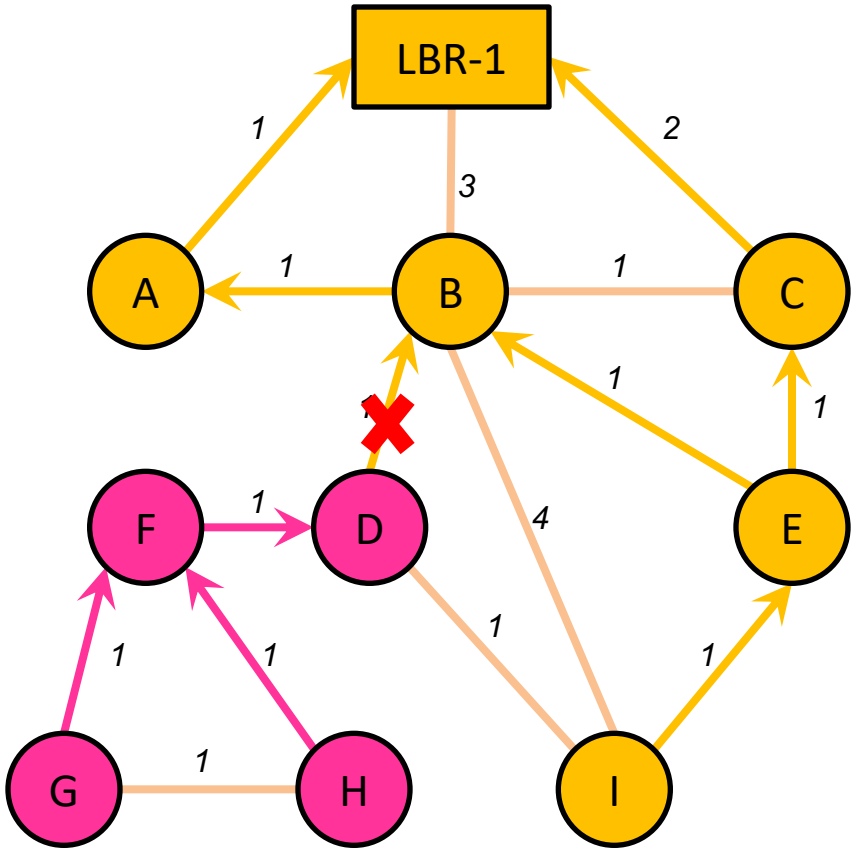
- Node D multicasts an RA-DIO to inform its sub-DAG of the change
- Node I has an alternate DAG Parent, E, and does not have to leave the DAG rooted at LBR-1.
- Node I removes Node D as a DAG Parent

DAG Maintenance
Loop Avoidance



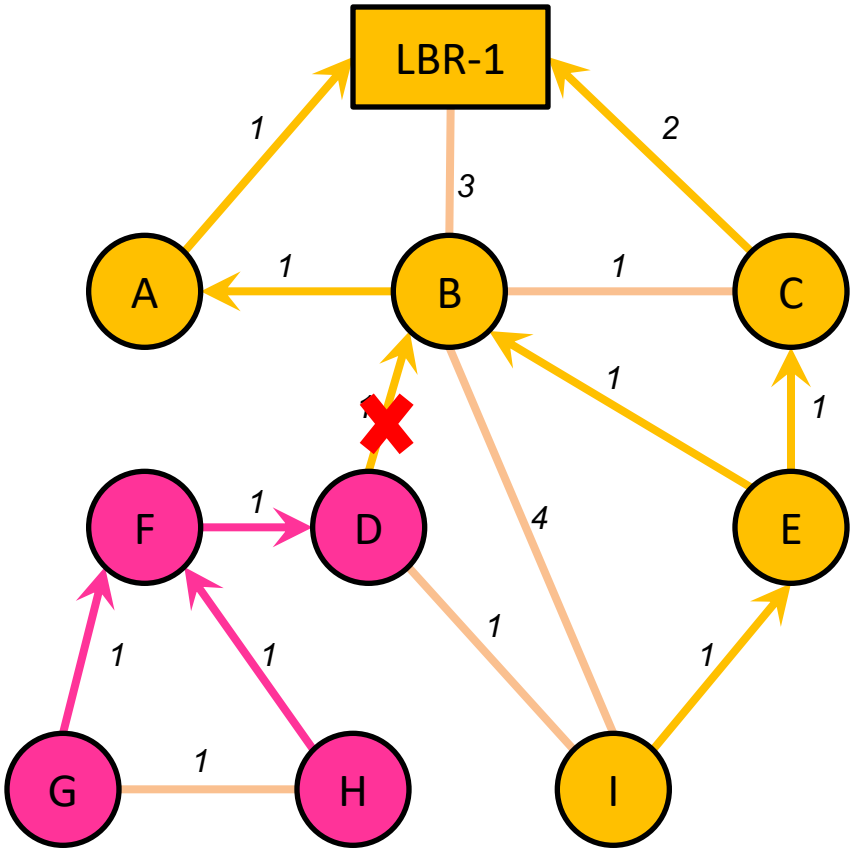
- Node F does not have an option to stay in the DAG rooted at LBR-1 (no alternate DAG Parents), so Node F follows Node D into the floating DAG
- Node F multicasts an RA-DIO
- Nodes G and H follow Node F into the floating DAG

DAG Maintenance
Loop Avoidance



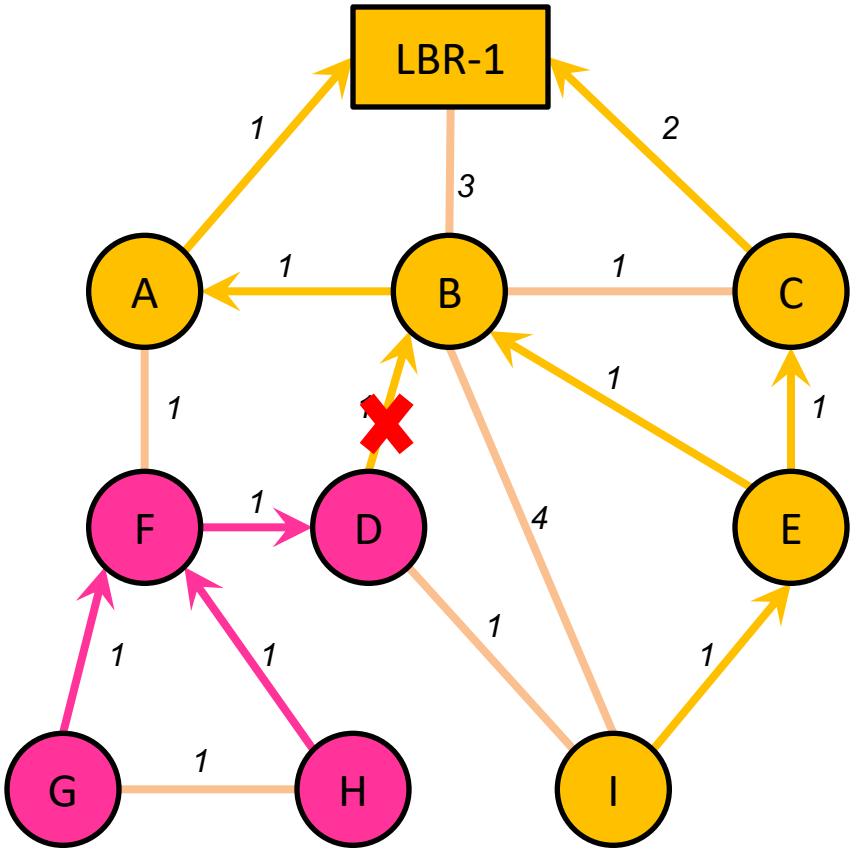
- The sub-DAG of node D has now been frozen
- Nodes contained in the sub-DAG have been identified, and by following node D into the floating DAG, all old routes to LBR-1 have been purged
- The floating DAG seeks to rejoin a grounded DAG…

DAG Maintenance
Loop Avoidance



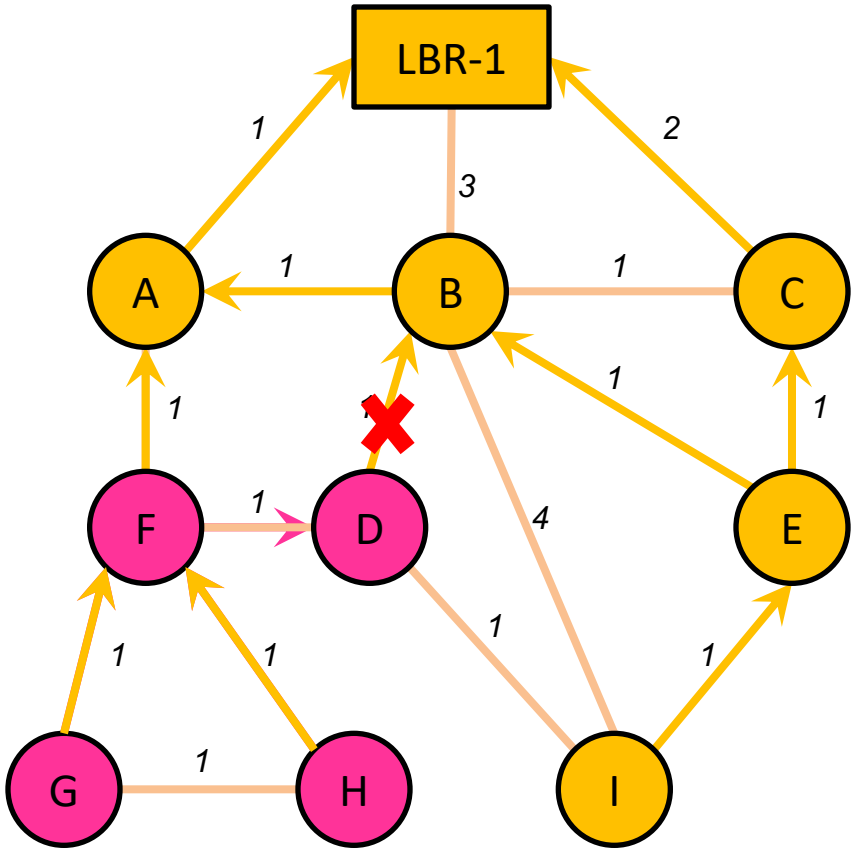
- Node I multicasts an RA-DIO
- Node D sees a chance to rejoin grounded DAG at depth 5 through Node I
- Node D starts a DAG Hop timer of duration α 4 associated with Node I

DAG Maintenance
Loop Avoidance



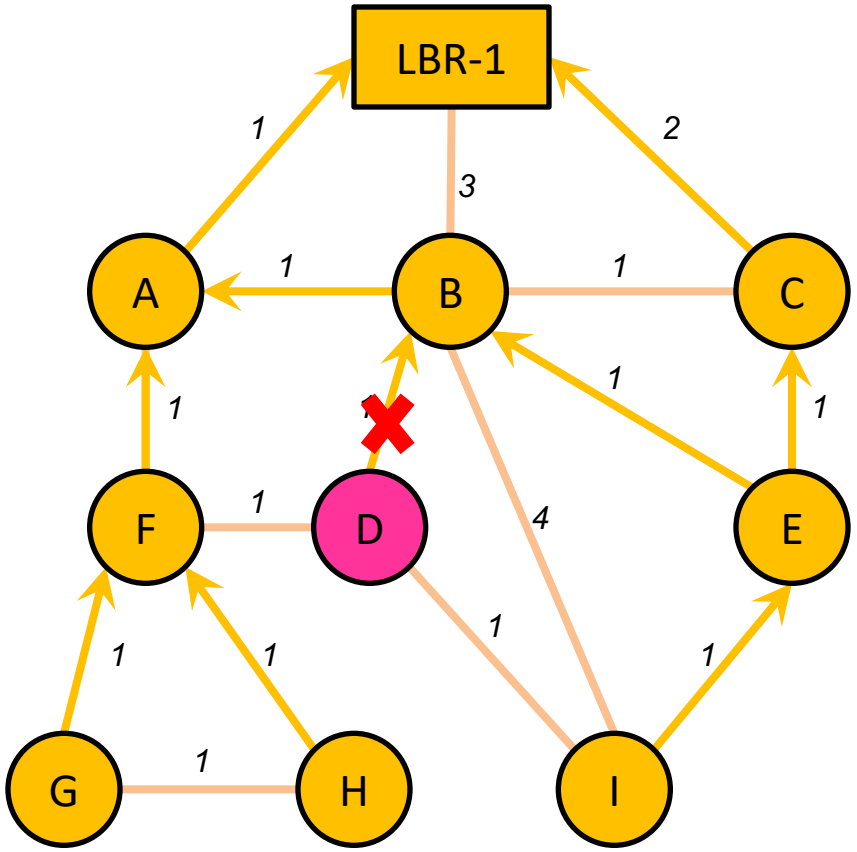
- Suppose a link A—F becomes viable
- Node A multicasts an RA-DIO
- Node F sees a chance to rejoin grounded DAG at depth 2 through Node A
- Node F starts a DAG Hop timer of duration α 1 associated with Node A

DAG Maintenance
Loop Avoidance



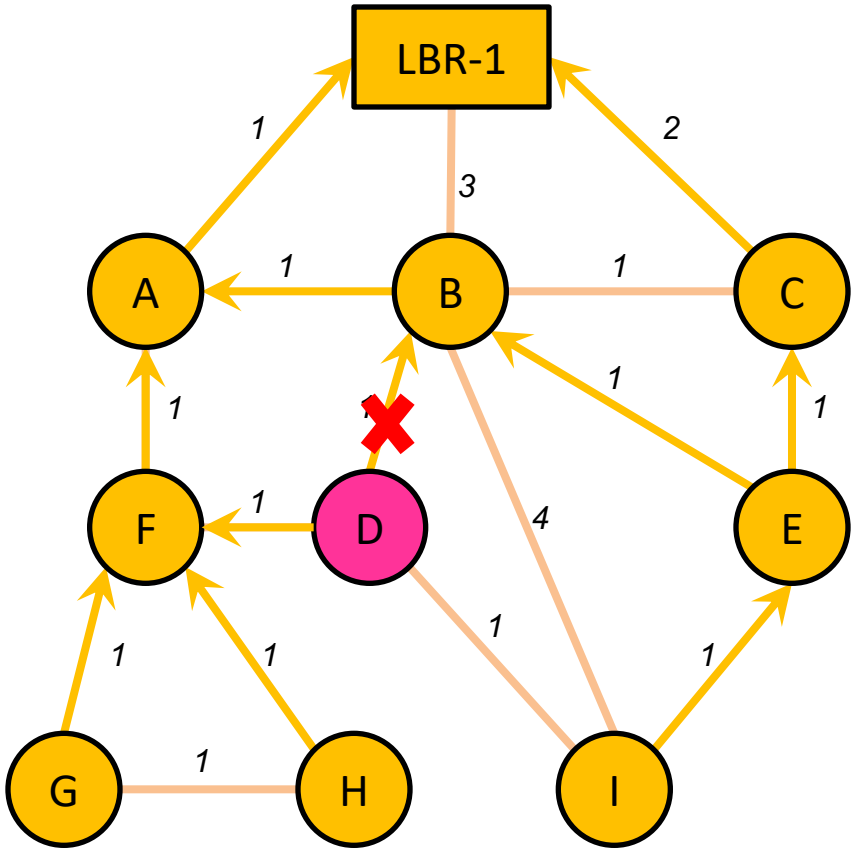
- Node F's DAG Hop Timer expires
- Node F joins to the grounded DAG at depth 2 by adding A as a DAG parent, and removing D
- Node F multicasts an RA-DIO
- Nodes G and H follow Node F to the grounded DAG

DAG Maintenance
Loop Avoidance



- Node D sees a chance to rejoin DAG LBR-1 at depth 3 through Node F
- Node D starts a DAG Hop timer of duration α 2 associated with Node F, in addition the DAG Hop timer already running with duration α 4 associated with Node I

DAG Maintenance
Loop Avoidance



- Node D's DAG Hop timer of duration α 2 tends to expire first
- Node D joins the grounded DAG at depth 3 by adding Node F as a DAG Parent
- The breaking-off and re-joining of the broken sub-DAG is thus coordinated with loop avoidance

UNIVERSITÀ
DI PARMA

IoT Lab

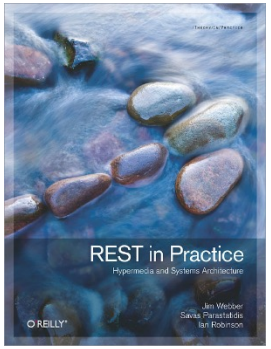
Communication paradigms for the IoT

Lecture Summary

- REST architectures
- Web of Things
- Publish/Subscribe

References

- Jim Webber, Savas Parastatidis, Ian Robinson, "**REST in Practice**," O'Reilly Media, September 2010 [http://shop.oreilly.com/product/9780596805838.do]



- "**Internet of Things Applications - From Research and Innovation to Market Deployment**", edited by O. Vermesan and P. Friess, pp. 287-313, The River Publishers Series in Communications, Alborg, Denmark, 2014. ISBN: 9788793102941 [http://riverpublishers.com/view_details.php?book_id=245]

- Matthias Kovatsch, "**CoAP: The Web-based Application-layer Protocol for the Internet of Things**," Lecture COLLECT at the Internet of Things and Smart Cities Ph.D. School 2014, Sep. 10th, 2014, Lerici (SP), Italy [http://phdschool.tlc.unipr.it/iot/2014/index.php/welcome/downloads.html]

Applications in the IoT

- Interconnected devices need to communicate using lightweight protocols, in order to minimize:
  - CPU processing
    - IoT devices have *limited computational capabilities* make it hard to process large messages
    - Lesser processing means *lower energy consumption*
  - Amount of data to be transmitted
    - Large messages result in more fragments (6LoWPAN), which introduce overhead
    - Transmitting more fragments can cause more retransmissions required (since devices operate in LLNs), which may result in higher delay and energy consumption

Applications in the IoT

- Protocols refer to specific communication paradigms:
  - request/response
  - publish/subscribe
- Each scenario might introduce requirements that drive the choice of suitable a communication paradigm and protocol... Which architectural style fits best in my scenario?
- While there is no final and “one-for-all” answer to this question, the IoT is being built around the **REST architectural style**, similarly to the Internet

Using a web-based approach for the IoT

- There are several reasons to use a web-based approach in the IoT
- The web has been around for decades and lots of experience has been gained in its development
- HTTP is by far the most used application-layer protocol
- Several software libraries implementing web protocols are available (web servers, HTTP clients, ...) for any programming language
- Patterns for building web applications are largely known and used: proficiency for developers
- The web has proved to scale extremely well: important for billions of connected devices

The web as a platform

- The web was born to be an easy-to-use, distributed, and loosely coupled system for sharing documents
- The architecture of the web is simple enough to make it easy to build applications and manage content
- The web is based on a small set of principles, yet it has proved to scale and evolve wonderfully
- The web has evolved to a **platform for building distributed systems**
- Distributed applications are built using HTTP (RFC 2616)

REpresentational State Transfer (REST)

- REST is the **architectural style** behind the World Wide Web
- Defined in 2000 in Roy Fielding's Ph.D. Dissertation "Architectural Styles and the Design of Network-based Software Architectures"*
- **REST defines a set of rules and principles** that all the elements of the architecture must conform to in order to build web applications that scale well, in terms of:
  - scalability (number of interacting clients)
  - robustness (long-term evolution of systems)

* R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," PhD thesis, University of California, 2000. Available: http://www.ics.uci.edu/~fielding/pubs/dissertation/ top.htm

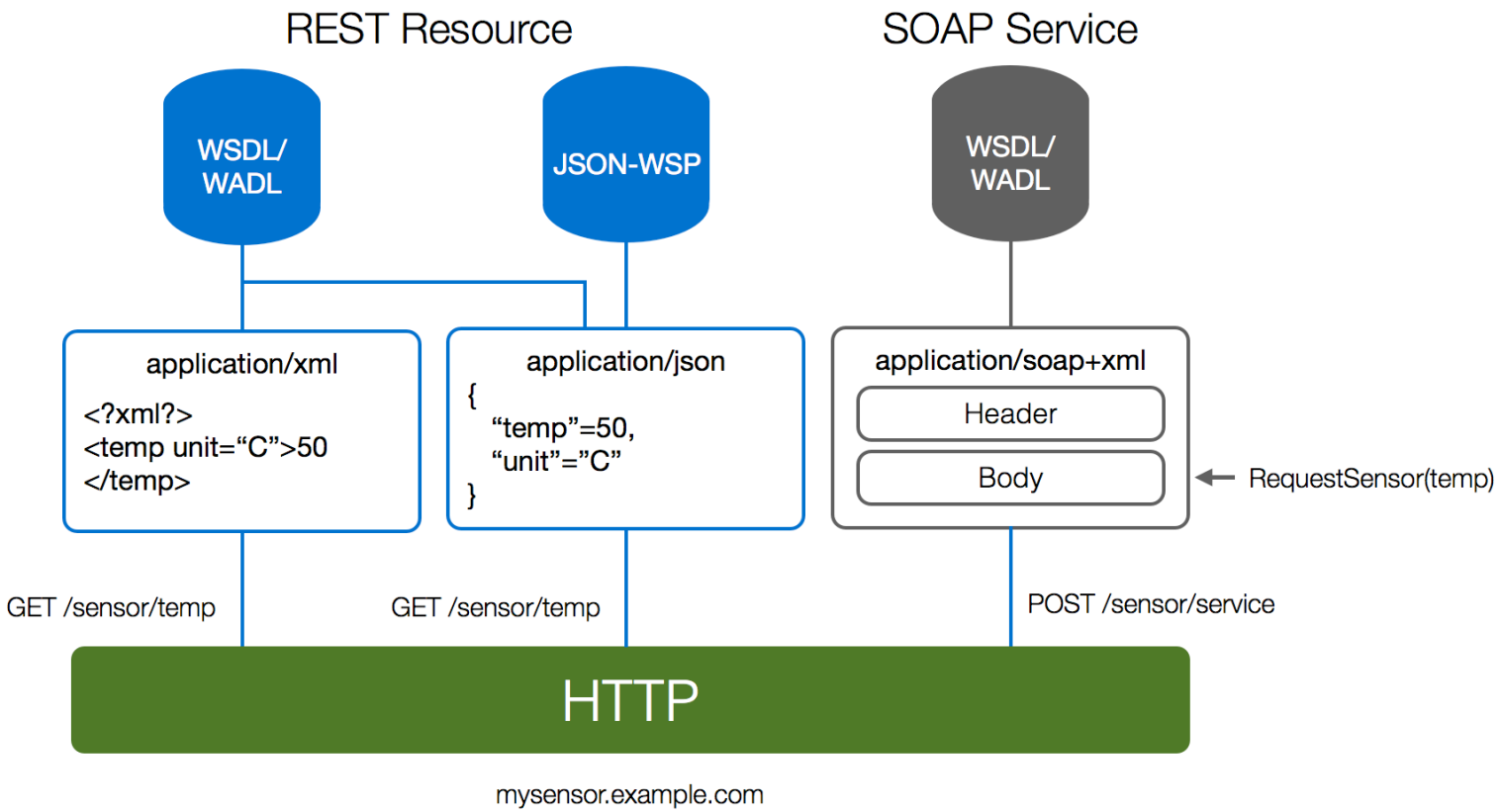
Resource-oriented architectures

- REST is based on the concept of **resource**
- A resource is any relevant entity in an application's domain and exposed in web (e.g., a webpage, a video, an order in an e-commerce website, ...)
- *A resource is anything with which a user interacts while progressing toward some goal*
- Anything can be mapped into a resource
- Resources are characterized by some data (e.g., the title of the page, the items in an order, ...)

Differences with Service-oriented architectures

- A Service-oriented architecture (SOA) refers to an architecture where two endpoints communicate through a pre-defined set of messaging contracts
  - a client interacts with a server by retrieving the list of available services and how these can be mapped to HTTP messages (WSDL)
  - Remote method calls are contained in SOAP (XML) included in the body of messages
  - The WSDL maps a message to a method call on the server
- SOA is typically used in legacy business-oriented systems
- The presence of a WSDL document is needed to add semantics to messages
- This is a weakness: if a server changes its services, a client needs to get access to the new WSDL or its functionalities are invalidated
- In a ROA, on the other hand, there is no endpoint exposing services, there are only resources that can be manipulated

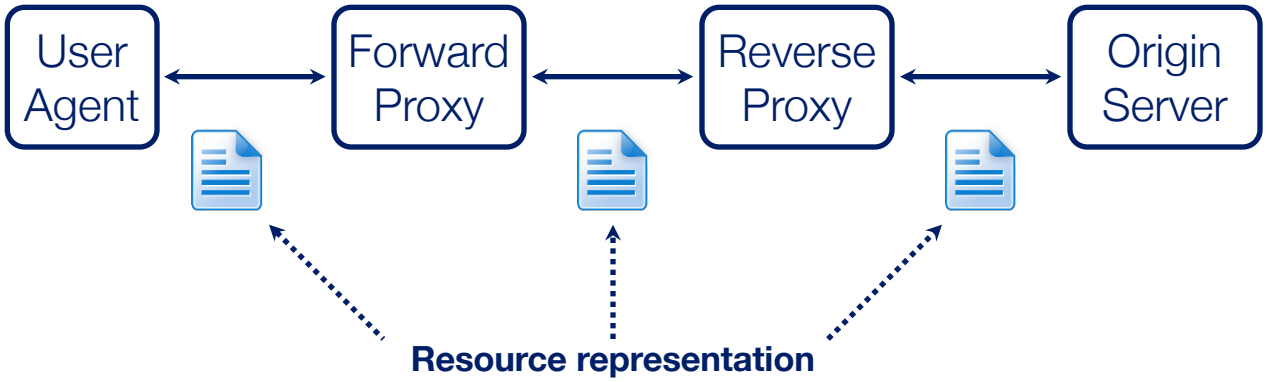
SOA vs REST


REST Resource
SOAP Service
WSDL/ WADL
JSON-WSP
WSDL/ WADL
application/xml
<?xml?>
<temp unit="C">50
</temp>
application/json
{
"temp"=50,
"unit"="C"
}
application/soap+xml
Header
Body
RequestSensor(temp)
GET /sensor/temp
GET /sensor/temp
POST /sensor/service
HTTP
mysensor.example.com

REpresentational State Transfer (REST)

- A REST architectures builds on:
  - clients (or *user agents*, such as browsers) are the application interface and initiate the interactions
  - servers (*origin servers*) host resources and serve client requests
- **Separation of concerns**
- Intermediaries act as clients and servers at the same time:
  - *forward proxies* (known to clients) are “exit points” for a request
  - *reverse proxies* appear as origin servers to a client, but actually relay requests
- Uniform interfaces: all connectors within the system must conform to this interface's constraints
  - identification of resources
  - manipulation of resources through representations
  - self-descriptive messages
  - hypermedia as the engine of application state

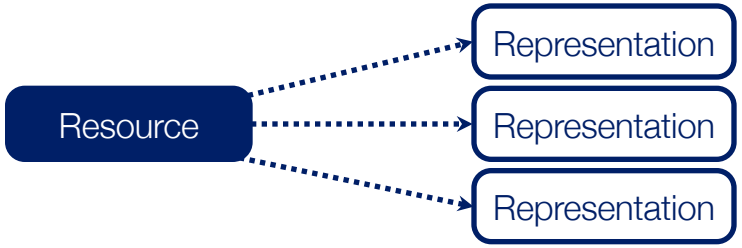
REpresentational State Transfer (REST)



- Resources are never exchanged directly by endpoints, but **representations of resources**
- A representation is a transformation or a view of a resource's state at an instant in time
- This view is encoded in one or more transferable formats, such as XHTML, Atom, XML, JSON, plain text, comma-separated values, MP3, or JPEG
- A resource must be serialized/deserialized properly at each endpoint

REpresentational State Transfer (REST)

- A resource (actually a representation of the resource) is exchanged back and forth between clients and servers
- The same resource can have many different representations (1:N relationship)



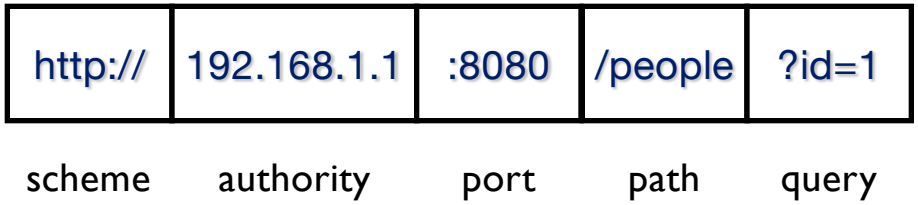
- For the sake of compactness, from now on we will refer to the representation of a resource just as a resource: always bear in mind the difference between them!!!

Resource identifiers

- In order to ensure that an application is handling the correct resource, there must a mechanism to identify a resource univocally in the network
- **Uniform Resource Identifiers** (URIs), defined in RFC 3986, serve this need:
  - A URI identifies a resource
  - A URI can be used to address a resource, so that it can be located, retrieved, and manipulated
- A resource can be mapped to multiple URIs, but a URI points exactly to one resource
- URIs can be of two kinds:
  - a URN specifies the name of a resource (e.g., `urn:ietf:rfc:2616`)
  - a URL specifies how to locate the resource, (e.g., `http://wasnlab.tlc.unipr.it/people/123`)

URI

- URIs have the form `scheme:scheme-specific-part`
- The scheme defines how the rest of the URI is to be interpreted
- URNs use the `urn` scheme
- URLs include all the information needed to successfully address the resource
- Typically, a URL is composed of the following parts:



- The authority can either be an IP address or a FQDN, which must be resolved through a DNS server to an IP address

URIs and representations

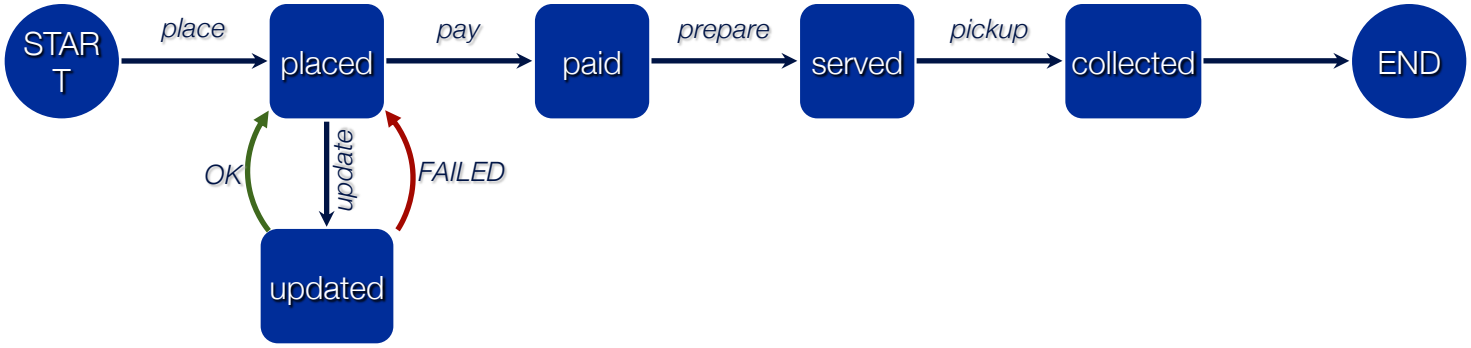
- URIs should be opaque to users and not contain any specific notion of the type of representation that will be used
- For example: `http://wasnlab.tlc.unipr.it/123.xml` and `http://wasnlab.tlc.unipr.it/123.json` are bad URIs
- Typically the type of representation is specified in one header of the message containing the resource (Content-Type)

REpresentational **S**tate **T**ransfer (RE**ST**)

- An important principle of REST is **statelessness**
- Requests must be stateless
- Each message should be self-descriptive
- Requests must contain all the information to understand the request so that servers can process it without context (the state of the client)
- There is no state maintained between clients and servers, everything is inside the message
- There is no concept of “session”... Cookies are bad!!!
- Basically, the state of the client is contained in the request, so the server is relieved from the burden of keeping track of client state

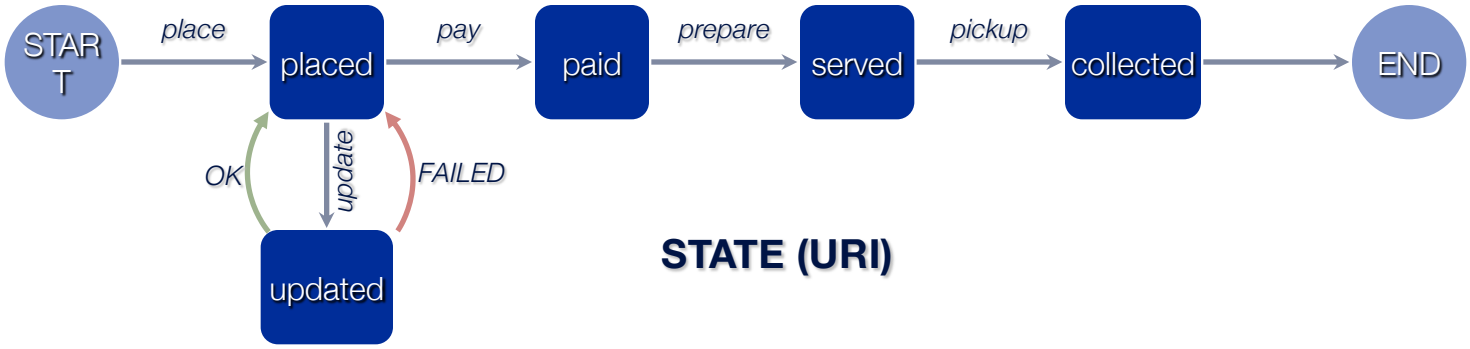
Applications as Finite State Machines

- RESTful applications makes forward progress by transitioning from one state to another, just like a state machineStates and the transitions between them are not known in advanceThe next possible transitions are discovered when the application reaches a new state (**gradual reveal**)Possible transitions are embedded in the representation of the resource as *links* (**hypermedia**)



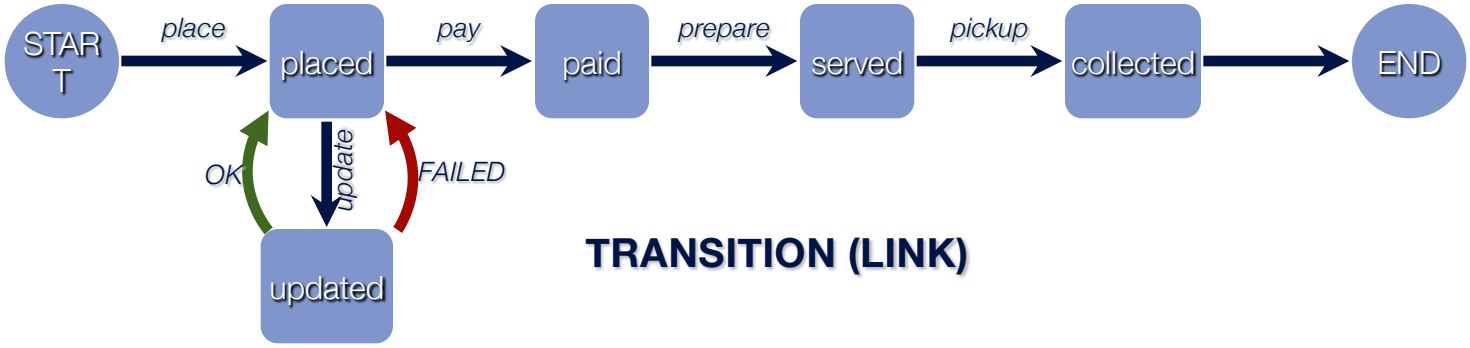
Applications as Finite State Machines

- RESTful applications makes forward progress by transitioning from one state to another, just like a state machineStates and the transitions between them are not known in advanceThe next possible transitions are discovered when the application reaches a new state (**gradual reveal**)Possible transitions are embedded in the representation of the resource as *links* (**hypermedia**)



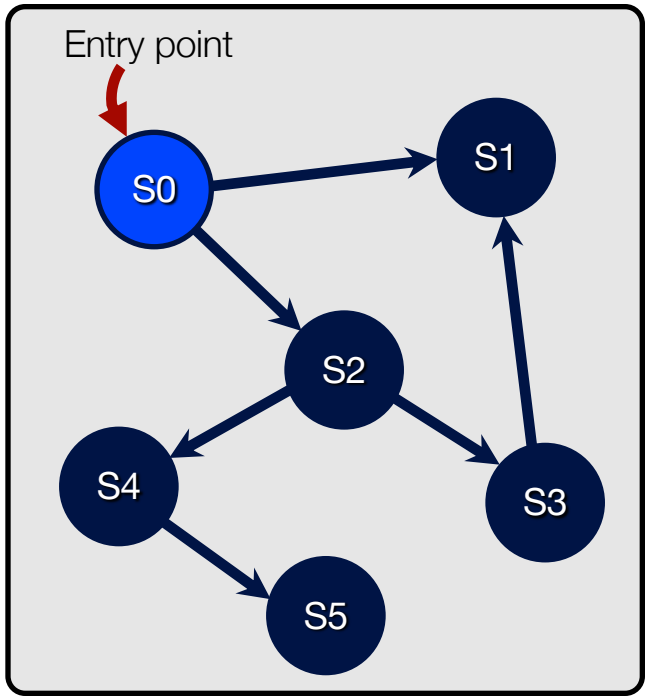
Applications as Finite State Machines

- RESTful applications makes forward progress by transitioning from one state to another, just like a state machine
- States and the transitions between them are not known in advance
- The next possible transitions are discovered when the application reaches a new state (**gradual reveal**)
- Possible transitions are embedded in the representation of the resource as *links* (**hypermedia**)



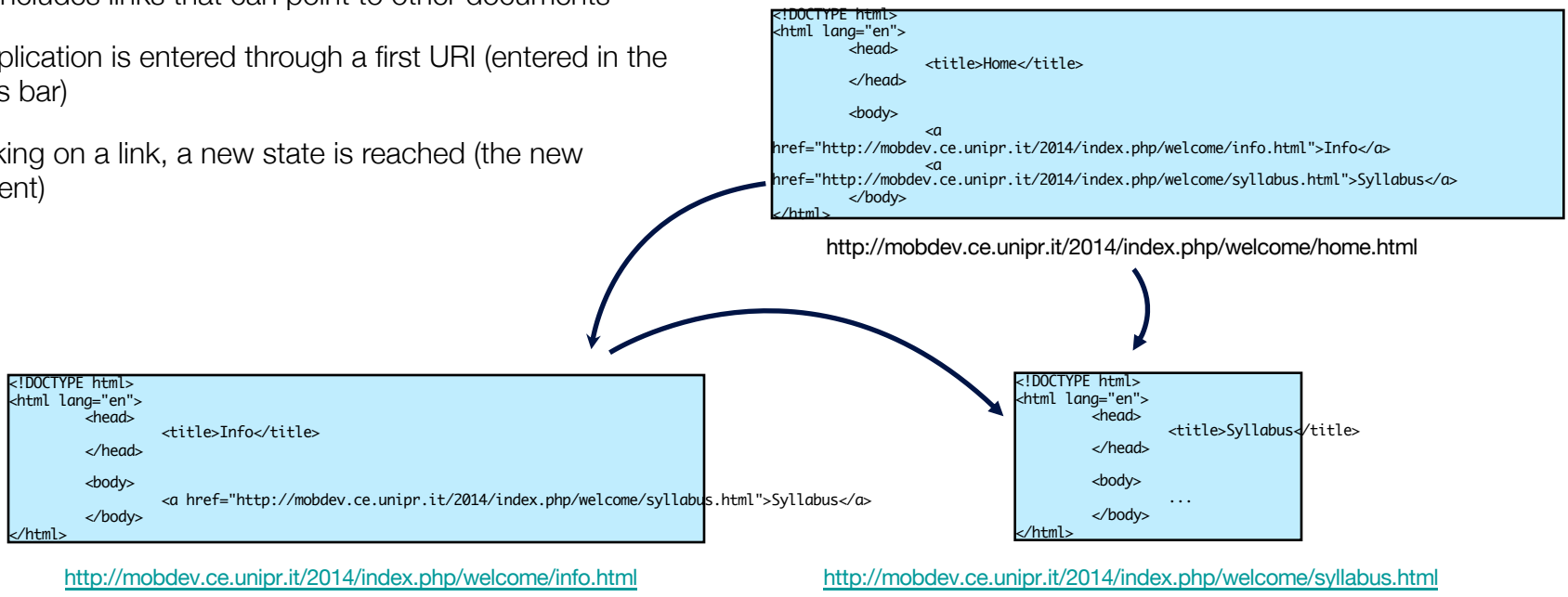
HATEOAS

- **H**ypermedia **A**s **T**he **E**ngine **O**f **A**pplication **S**tate
- Typically, RESTful applications progress according to the following steps:
  1. The client starts from an entry URI or a bookmark
  2. Response to a GET request includes a hypermedia representation
  3. The representation contains links that define the possible transitions to the next states of the FSM
  4. The client selects a link to follow and issues the next requests (i.e., triggers a transition)
     - URIs and possible transitions are never hardcoded into the client: the client "learns" the application on-the-fly through the media type and link relations
  5. The client can also go back



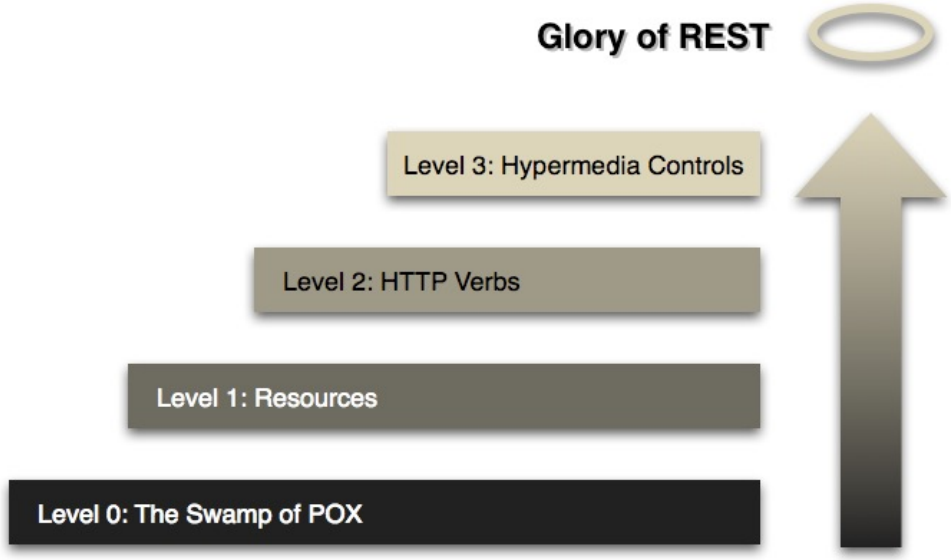
The Web as a Finite State Machine

- The web can be seen as a global distributed application
- The web browser is the client
- There are millions of servers
- Resources are documents (such as HTML pages)
- HTML includes links that can point to other documents
- The application is entered through a first URI (entered in the address bar)
- By clicking on a link, a new state is reached (the new document)



Richardson maturity model

- How REST-friendly is a web application?
- Level-based classification of web applications, based on the degree of compliance with the principles of REST
- The higher the level, the more RESTful an application is
- Basically, the higher the level, the less coupling exists between clients and servers
- Coupling should be avoided as much as possible in order to support **independent evolution of systems**



http://martinfowler.com/articles/richardsonMaturityModel.html

The Web of Things

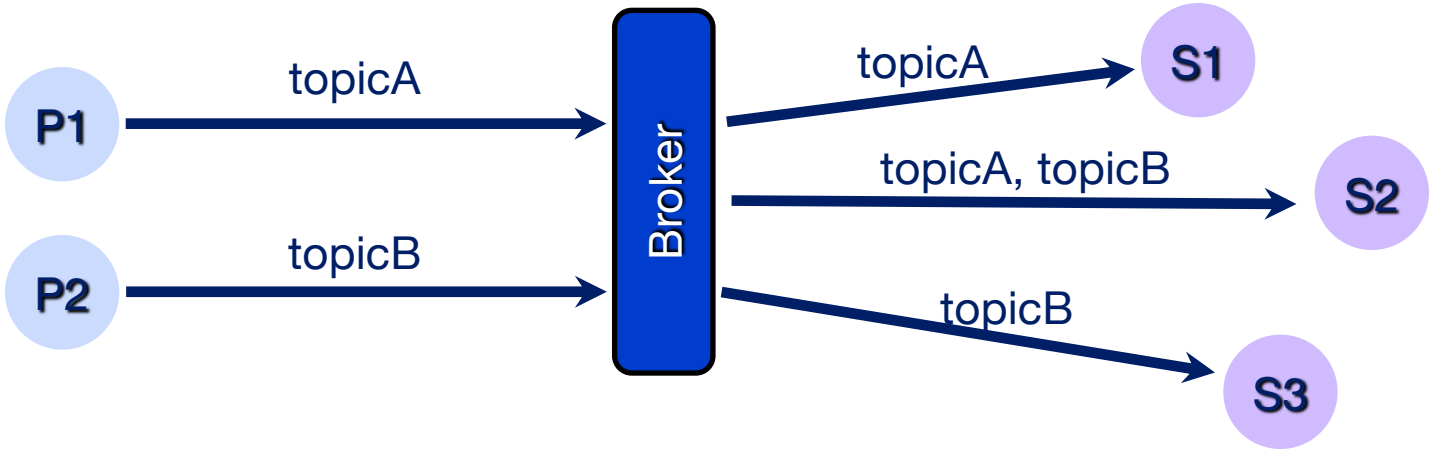
- Level 3 IoT ApplicationsHypermedia: CoRE Link Format (RFC 6690)
- attributes for links (better then relation types)Building the IoT around the REST paradigm and modeling it according to web concepts will allow to reuse all the experience gained in decades of building the web
- Web of Things (WoT) provides an Application Layer that simplifies the creation of the IoT
- The WoT will bring to the IoT the same usability of the IoT as the world wide web has done with Internet
- The WoT does not use HTTP (too heavyweight for constrained devices), but uses another lightweight REST-oriented web transfer protocol, named **CoAP** (RFC 7252)

Publish/Subscribe communications

- REST is not the only communication paradigm that can be used
- Another trending communication approach is Publish/Subscribe
- Pub/Sub architectures are composed by two kinds of entities:
- *publishers* generate information
- *subscribers* register to receive information, possibly filtered according to some policy
- Publishers do not send messages directly to specific receivers (similar to application-layer multicast)
- Messages are dispatched to subscribers according to their “class”
- A *broker* is responsible for dispatching messages
- publishers post messages to the broker
- subscribers register to the broker
- if a message matches a subscriber’s interest, the broker sends it to the subscriber

Publish/Subscribe communications

- Pub/Sub is often implemented with message-queue services
- Publishers send message into a message queue server (the broker), tagging the message with a **topic**
- Subscribers send registration requests to the broker for a given topic
- The broker routes messages (possibly also forking) to subscribers, based on the message topic

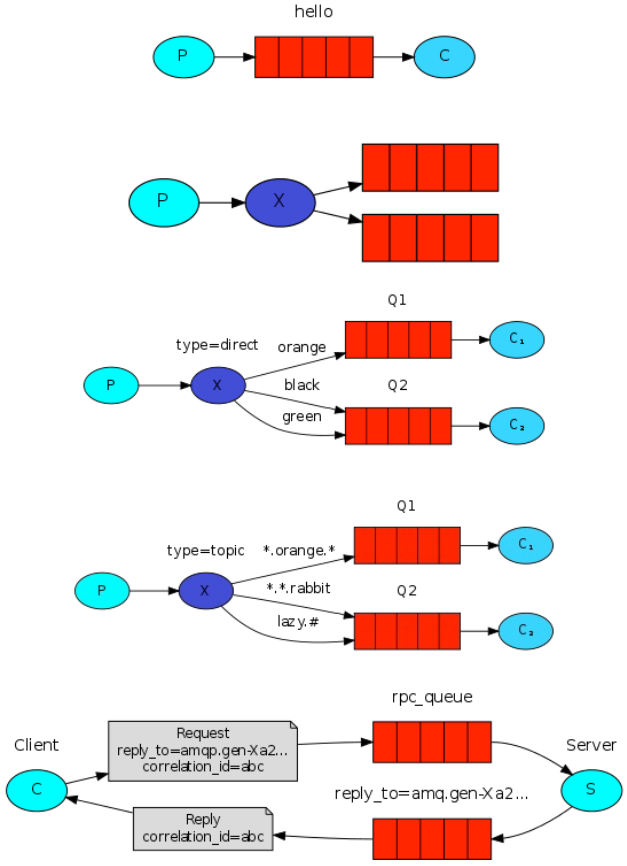


Publish/Subscribe communications

- Advantages of the Pub/Sub model:
  - **Loose coupling**: publishers need not know which subscribers receive messages or even if they exist (in a client/server paradigm a client must know the URI of the server and the server must exist)
  - **Scalability**: since brokers only need to route messages, they can be replicated easily to support higher volumes of data being transferred

Publish/Subscribe protocols

- Message Queue Telemetry Transport (MQTT): open-source, lightweight, TCP-based; supports hierarchical topics
  - http://mqtt.org
  - Organization for the Advancement of Structured Information Standards (OASIS) Standard
  - Endorsed by Eclipse Foundation
- Advanced Message Queue Protocol (AMQP): message-oriented, based on *topics*, *queues* (message storage facilities), and *exchanges* (routers for message dispatching)
  - https://www.amqp.org
  - Implementations: RabbitMQ, Apache ActiveMQ



UNIVERSITÀ
DI PARMA

IoT Lab

CoAP and MQTT

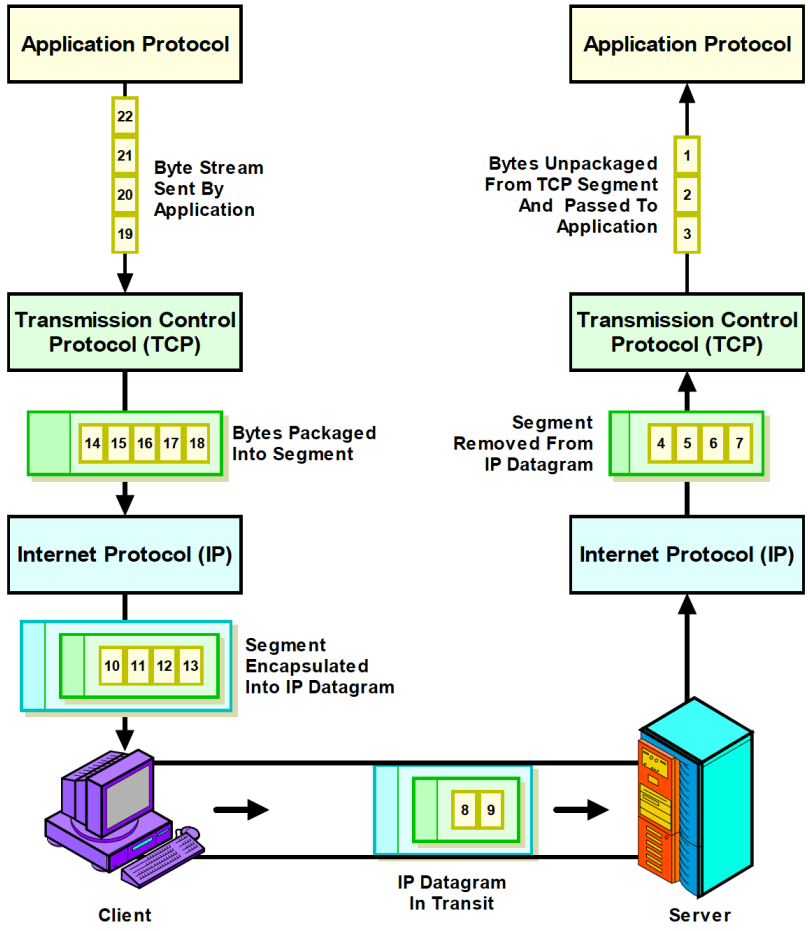
General ecosystem

Protocol	QoS	Communication Pattern	Target Devices
CoAP	YES	Req/Resp	Very constrained
MQTT	YES	Pub/Sub	Generic, small header
XMPP	NO	Req/Resp Pub/Sub	High memory consumption
HTTP	NO	Req/Resp	High performance
AMQP	YES	Pub/Sub	Ser-2-Ser communication
Web Socket	NO	Client/Server Pub/Sub	needs less power than HTTP still needs high power
AllJoyn	NO	Client/Server Pub/Sub	High computational power

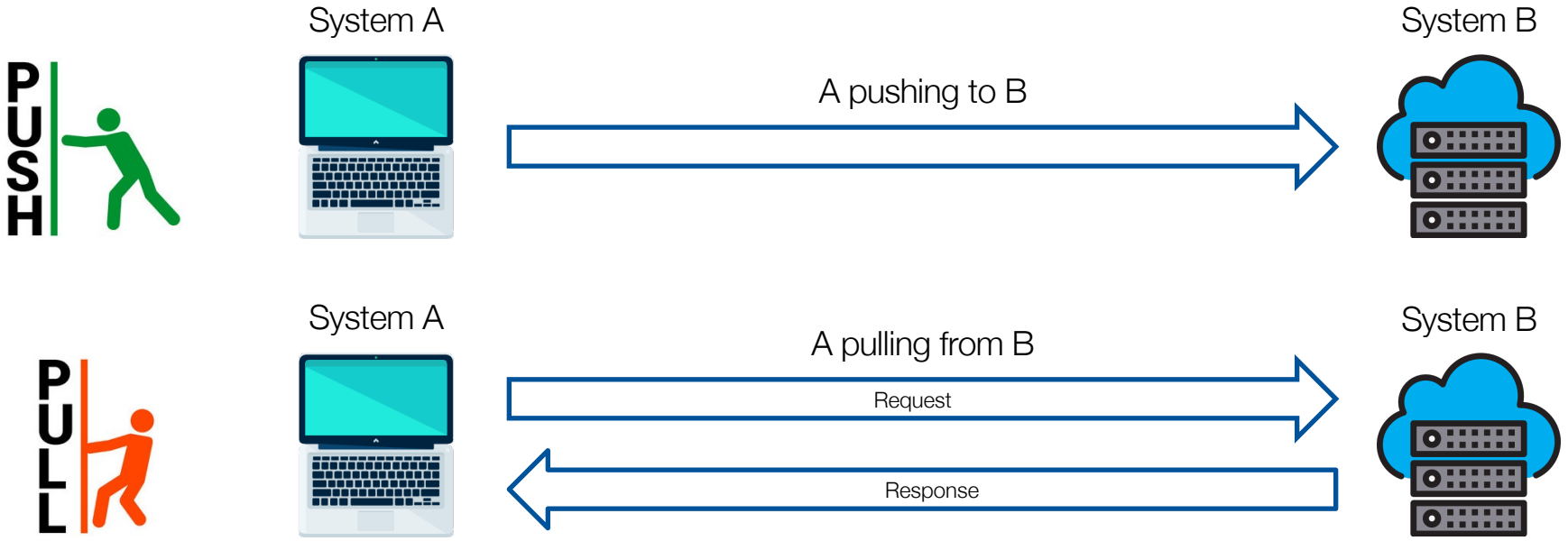
- CoAP: Constrained Application Protocol
- MQTT: Message Queuing Telemetry Transport
- XMPP: Extensible Messaging and Presence Protocol
- AMQP: Advanced Message Queuing Protocol
- WebSocket: Computer communications protocol
- Alljoyn: Full stack of protocols intended for Internet of Things (IoT)

From “byte streams” to “messages”

- The “old” vision of data communication was based on reliable byte streams (i.e., TCP)
- Nowadays messages interchange is becoming more common
  - E.g., Twitter, WhatsApp, Instagram, Snapchat, Facebook, …

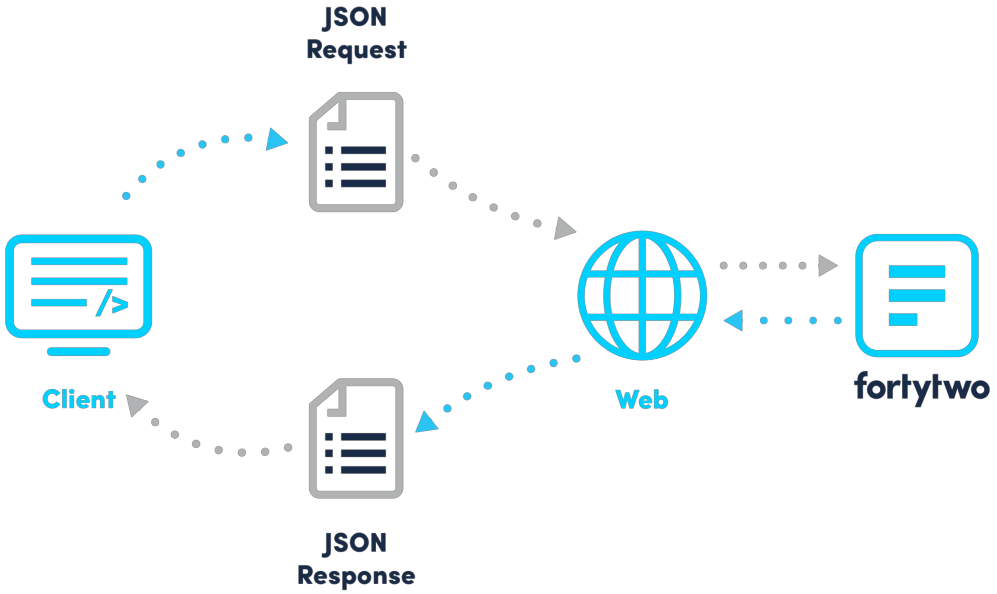


Ways to interchange "messages"



Request/response approach

- Representational State Transfer (REST)
- Widely used (especially in HTTP)
- Lighter version: CoAP (Constrained Application Protocol)



IoT Constrained Scenarios

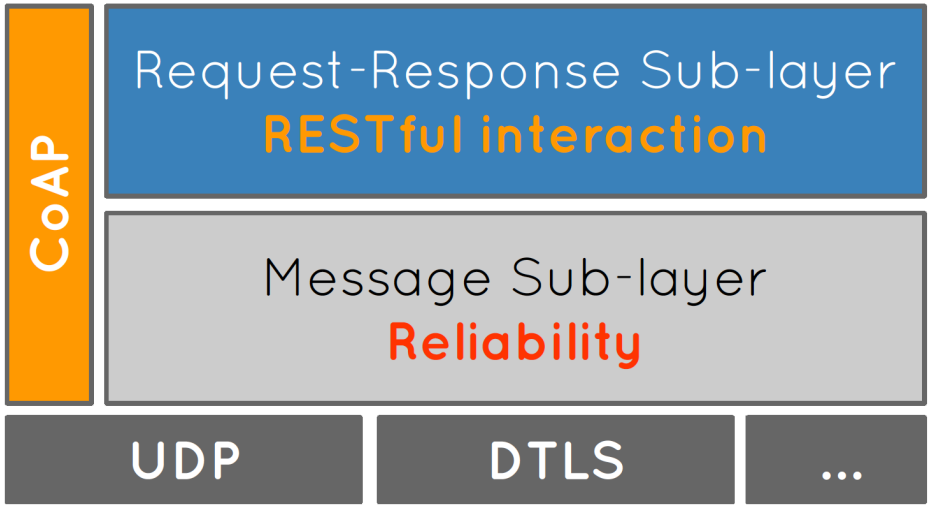
- Application layer protocols must be designed carefully in order to take into account the constraints deriving from lower layers and the nature of entities involved in such constrained scenarios (e.g., smart objects)
- HTTP and TCP are not well suited
  - HTTP introduces overhead (text-based and verbose protocol)
  - TCP (connection-oriented transport protocol)
    - Setting up and maintaining connections is a heavy task

    Duty cycling is not compatible with keeping connections alive

--> Solution: Constrained Application Protocol (CoAP)

Constrained Application Protocol (CoAP)

- RESTful application-layer protocol designed from scratch
- Transparent mapping to HTTP (to guarantee the integration with the WWW)
- Additional features for constrained applications
  - E.g., Internet of Things (IoT)

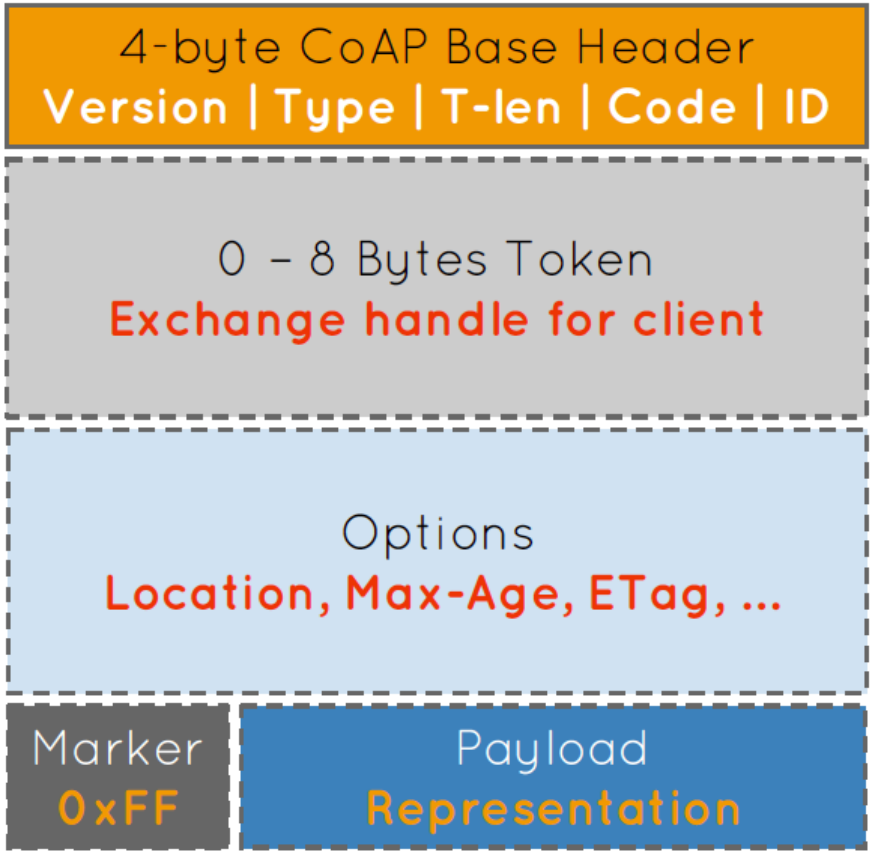


GET, POST, PUT, DELETE
URIs and Internet Media Types

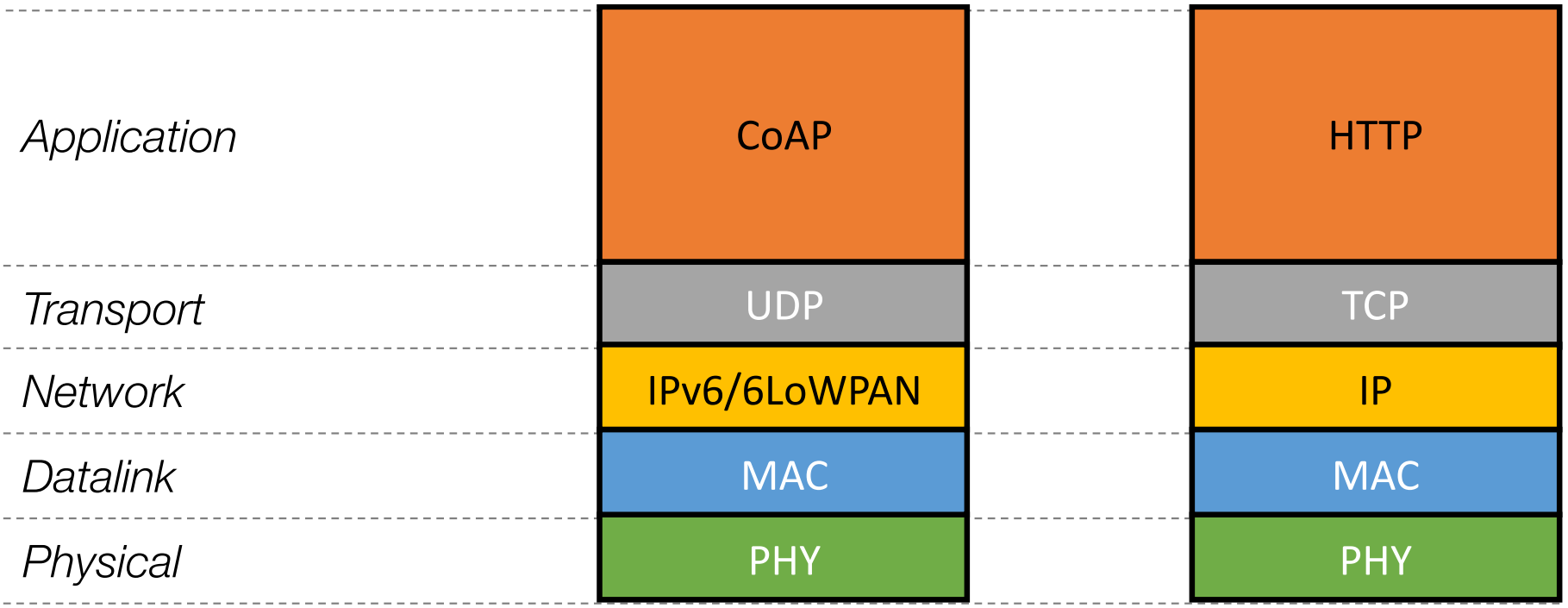
Deduplication
Optional retransmission

Constrained Application Protocol (CoAP) (cnt.)

- Runs on top of a lightweight transport protocol
  - E.g., User Datagram Protocol (UDP)
- Binary protocol
  - Low parsing complexity
  - Small message size
- Options
  - Type-Length-Value
  - Special option header marks payload if present



CoAP vs. HTTP


Application
Transport
Network
Datalink
Physical
CoAP
UDP
IPv6/6LoWPAN
MAC
PHY
HTTP
TCP
IP
MAC
PHY

CoAP Messaging

- CoAP messages are exchanged between UDP endpoints (default port is 5683)
- CoAP messages, as in HTTP, are either requests or responses
- Messages include a Message ID (16 bits) used to detect duplicates and for optional reliability (acknowledgements of received messages)
- Requests and responses are matched using a Token (0 to 8 bytes)
- Message IDs and Tokens are separate concepts and they actually work "orthogonally"

Resource Observation

- **The state of a resource on a CoAP server can change over time: how can clients always have the most recent state?**

- Polling: perform GET requests periodically
  - Polling is inefficient
  - Resource may not change state between two successive GET requests: unnecessary requests consume energy on the constrained CoAP server
  - The resource may have changed its state between two successive GET requests: for some time the client was not having the most updated state

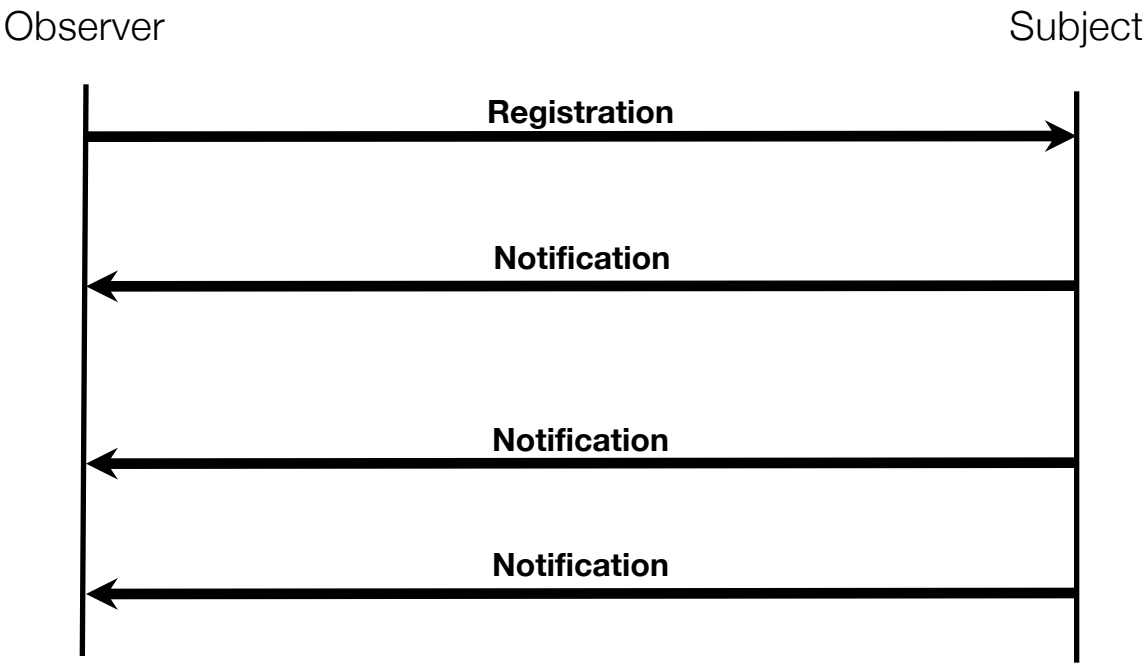
- `Observe` option: has been introduced to avoid polling (RFC 7641)
  - When included in a GET request, tells the server to send notifications to the client whenever the state of the resource changes
  - Similar to a Publish/Subscribe model, but slightly different: it is still request/response, but many responses are sent after a single request
  - Responses are linked to the request because they report the same Token

Resource observation

- The state of a resource on a CoAP server can change over time: how can clients always have the most recent state?
- Polling: perform GET requests periodically
- Polling is inefficient:
  - resource may not change state between two successive GET requests: unnecessary requests consume energy on the constrained CoAP server
  - the resource may have changed its state between two successive GET requests: for some time the client was not having the most updated state
- The **Observe option** has been introduced to avoid polling (RFC 7641)
- When included in a GET request, tells the server to send notifications to the client whenever the state of the resource changes
- Similar to a Pub/Sub model, but slightly different: it is still request/response, but many responses are sent after a single request
- Responses are linked to the request because they report the same Token

Resource observation

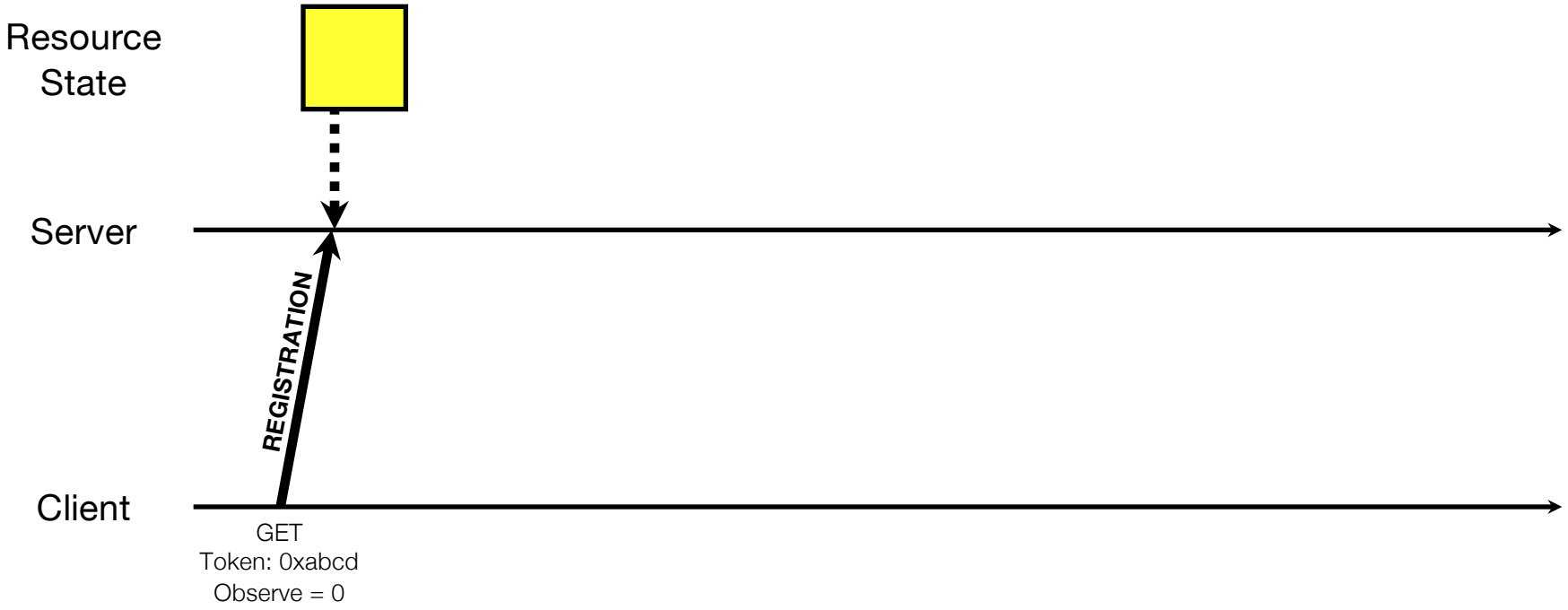
- The Observe Option implements the Observer Design Pattern



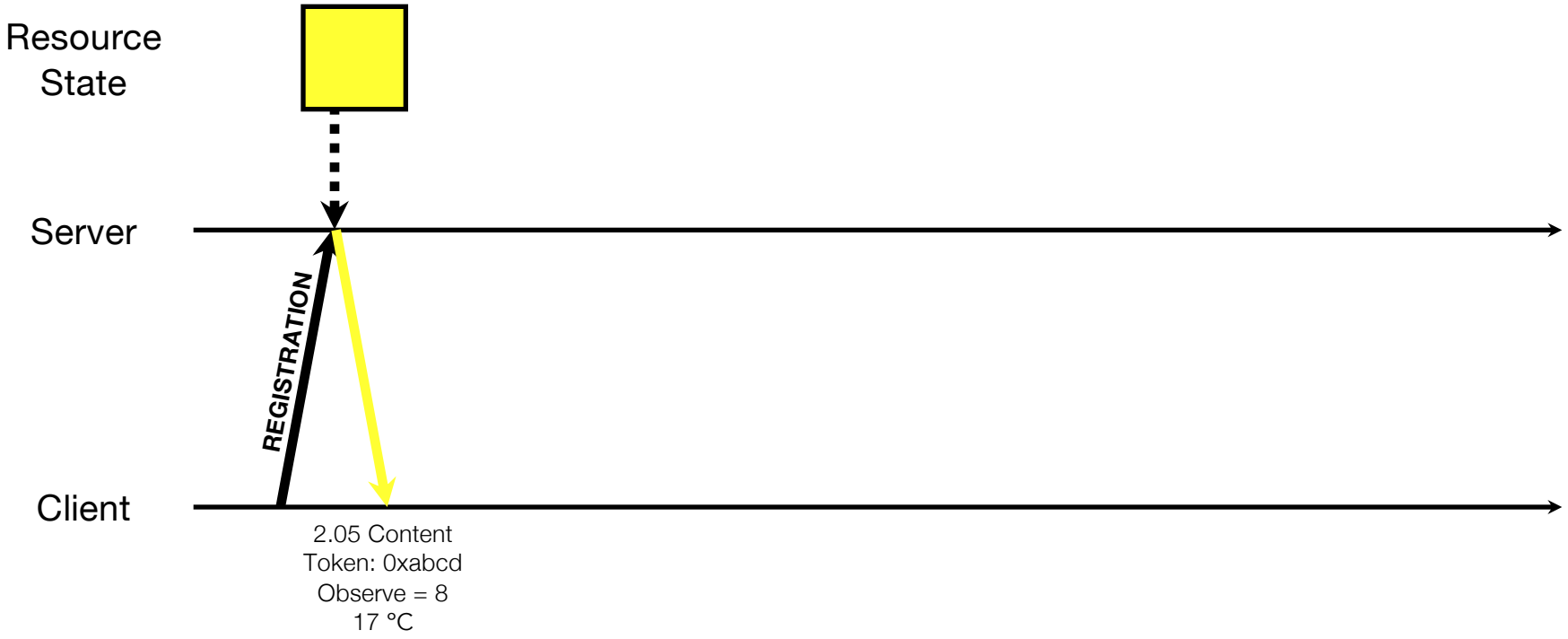
Resource observation

- The Observe Option has been assigned the option number 6
- The option length can be 0 to 3 bytes
- The option value is an unsigned integer:
  - 0 is used for registration (start observing)
  - 1 is used for deregistration (stop observing)
  - other values are used to sort notifications (since UDP is used, notifications may arrive in the wrong order), or to ignore older notifications if a fresher state has been received

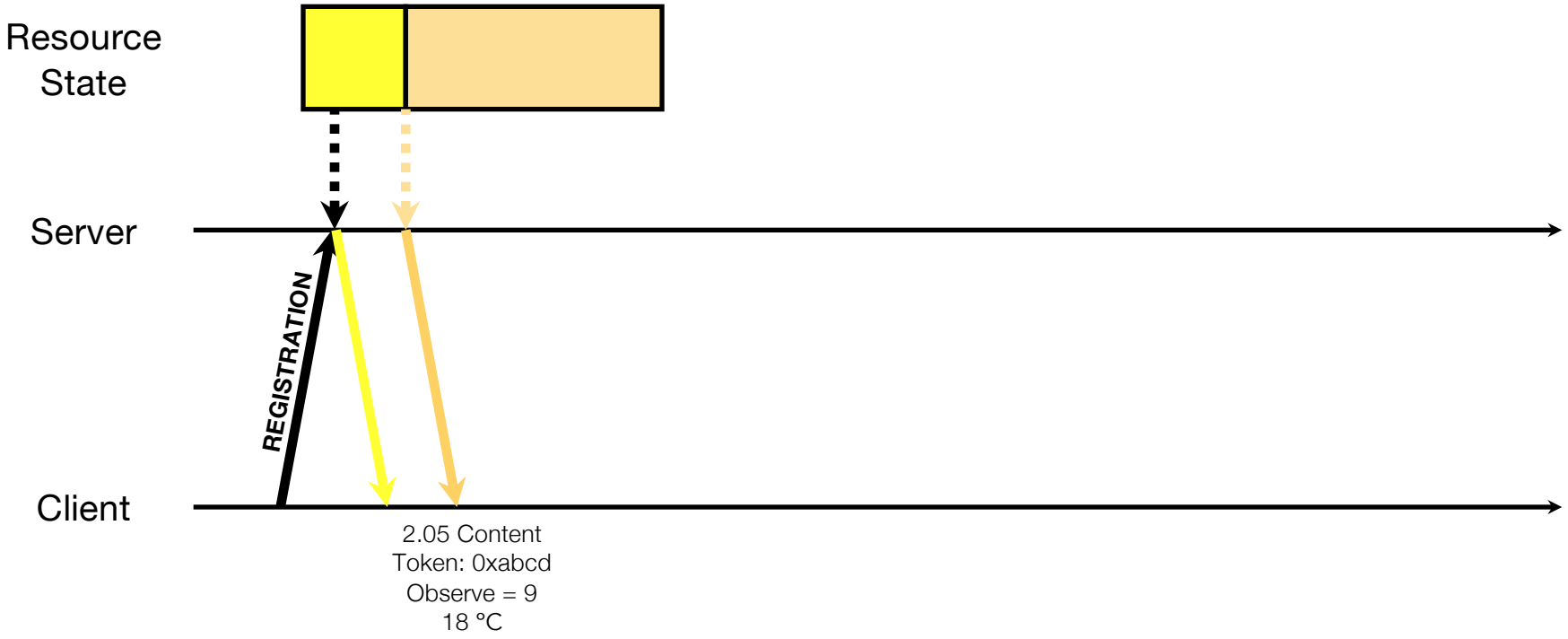
Resource observation



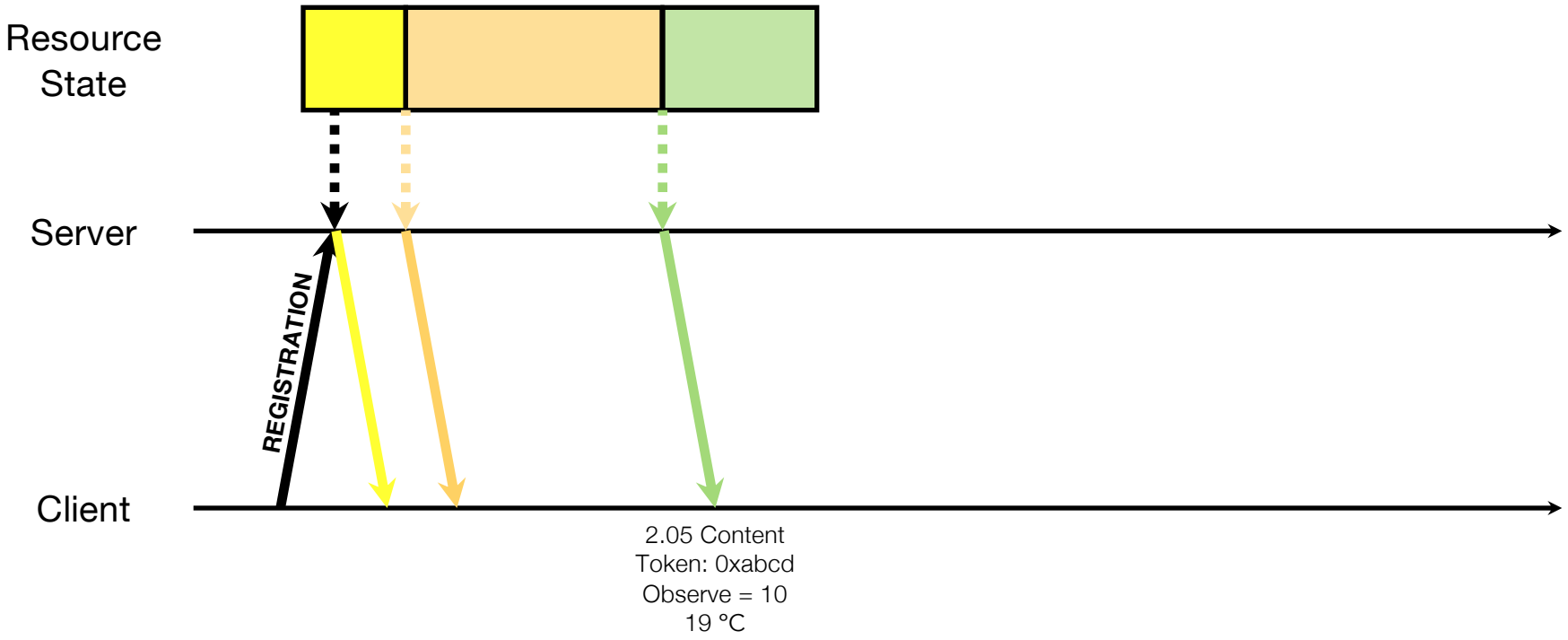
Resource observation



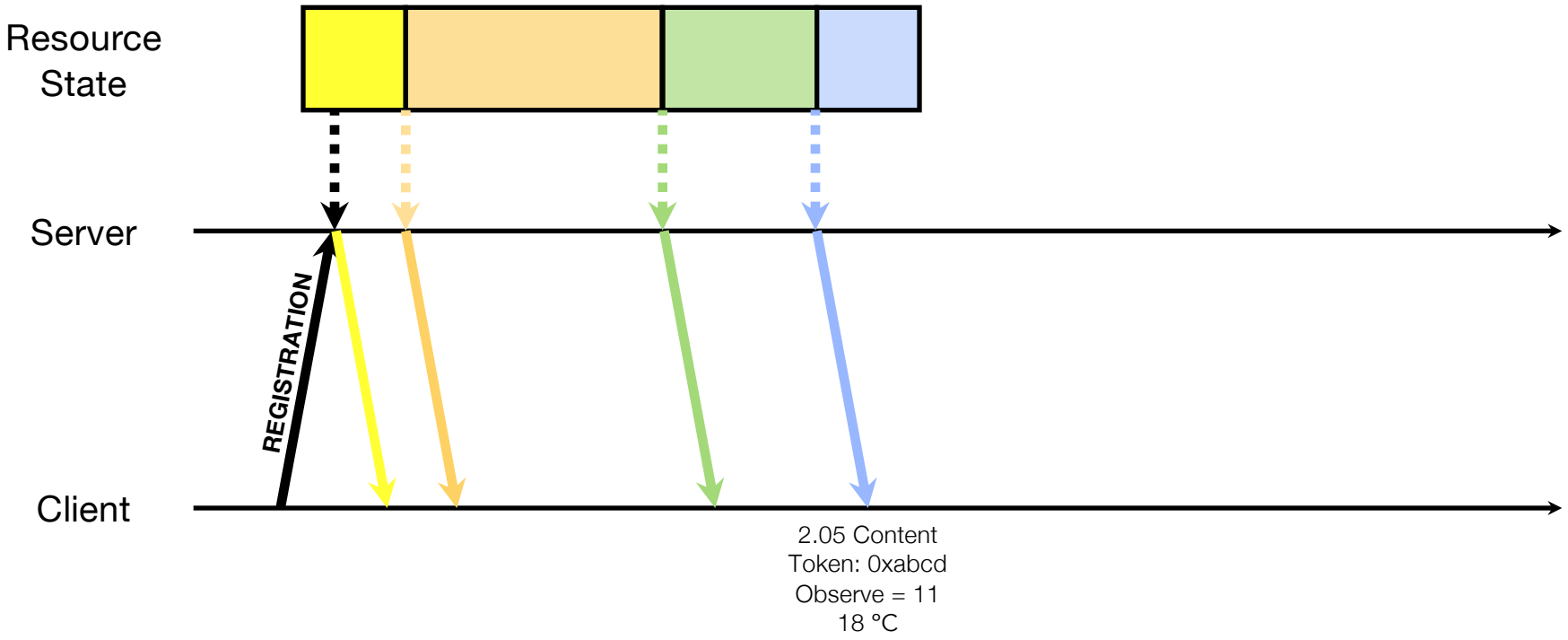
Resource observation



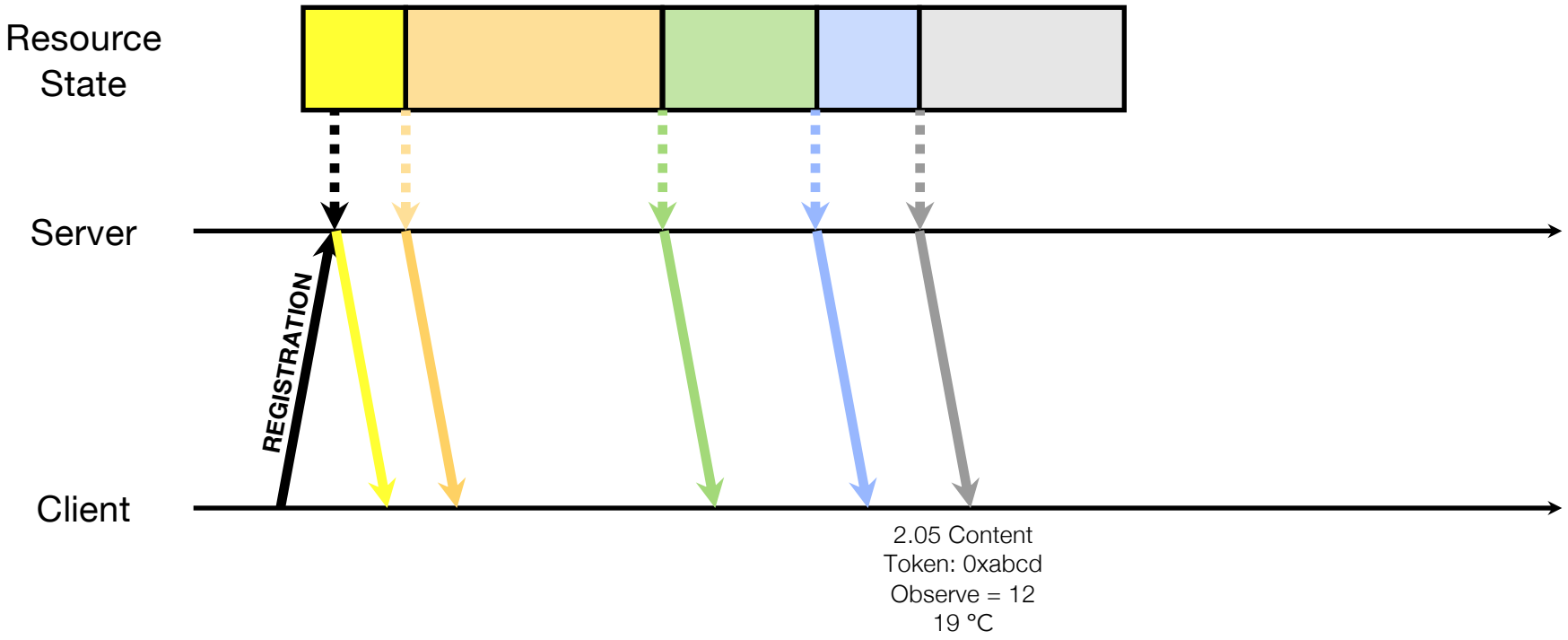
Resource observation



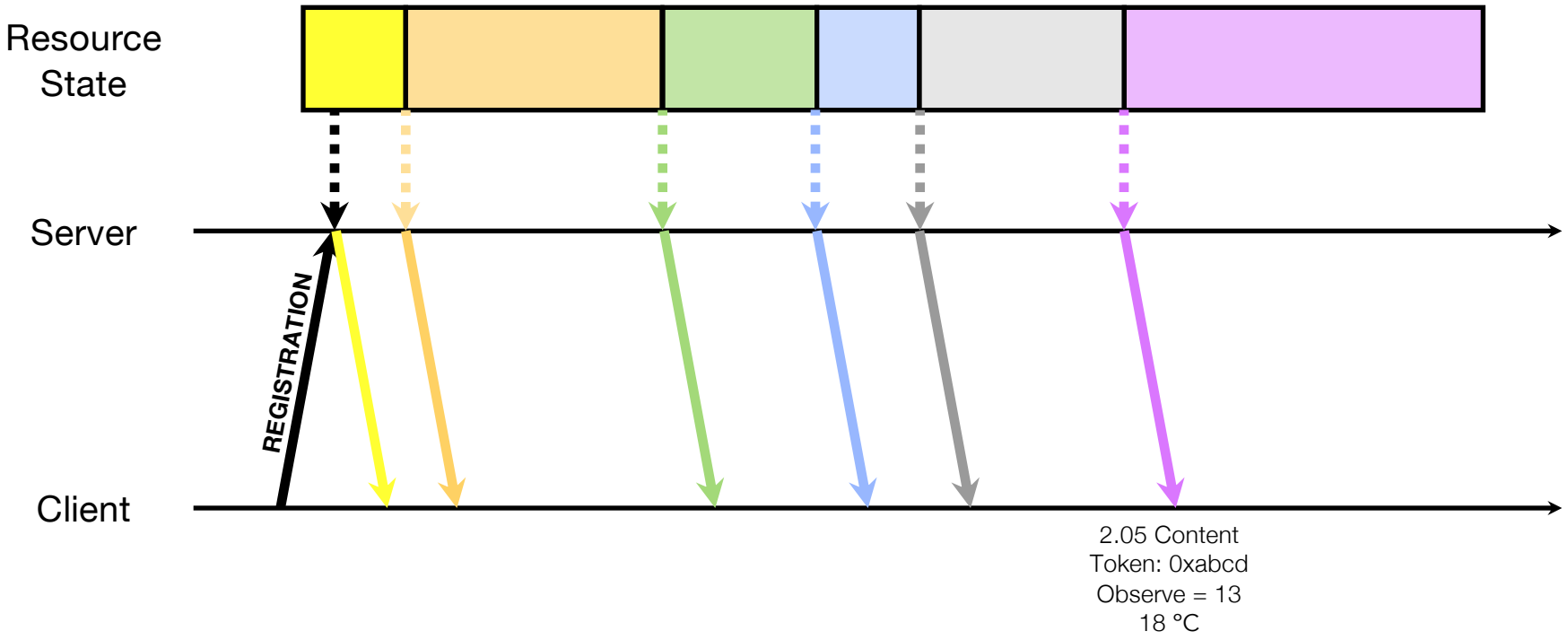
Resource observation



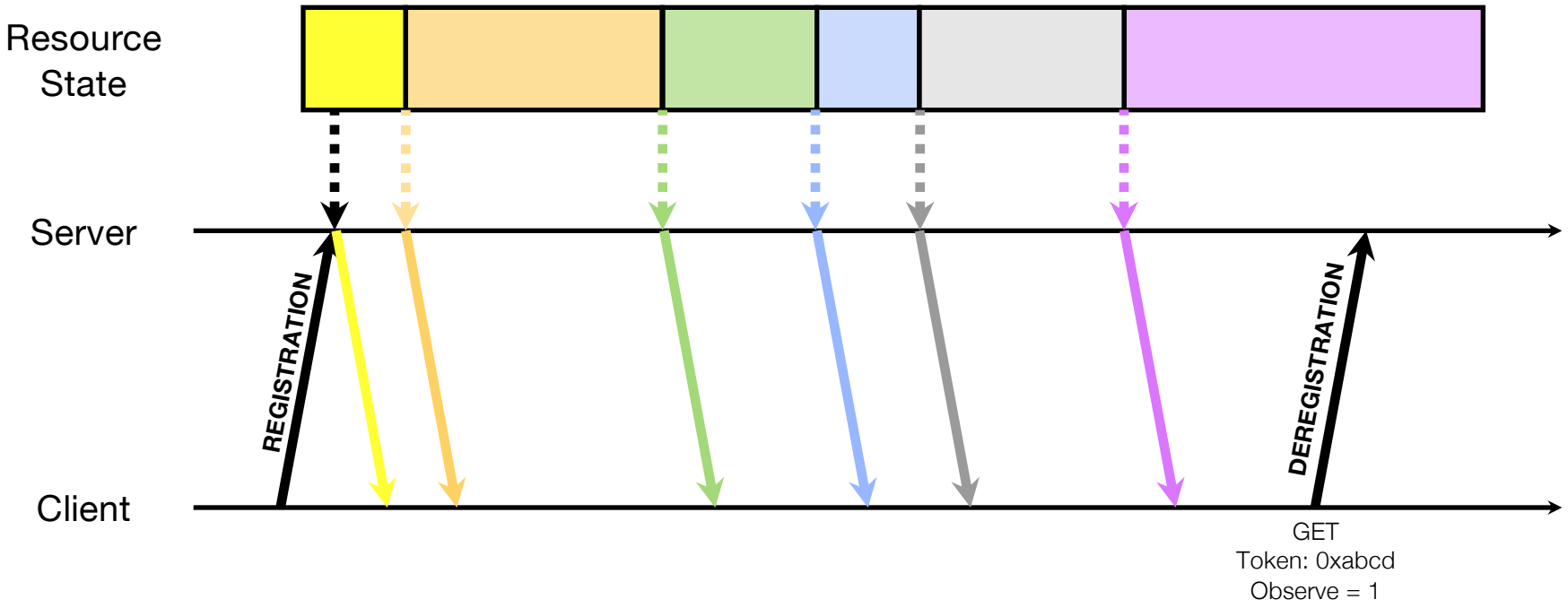
Resource observation



Resource observation

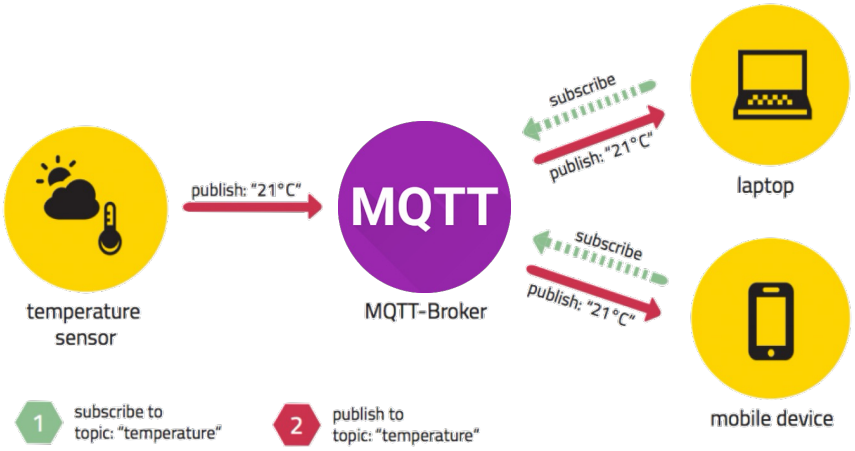


Resource observation

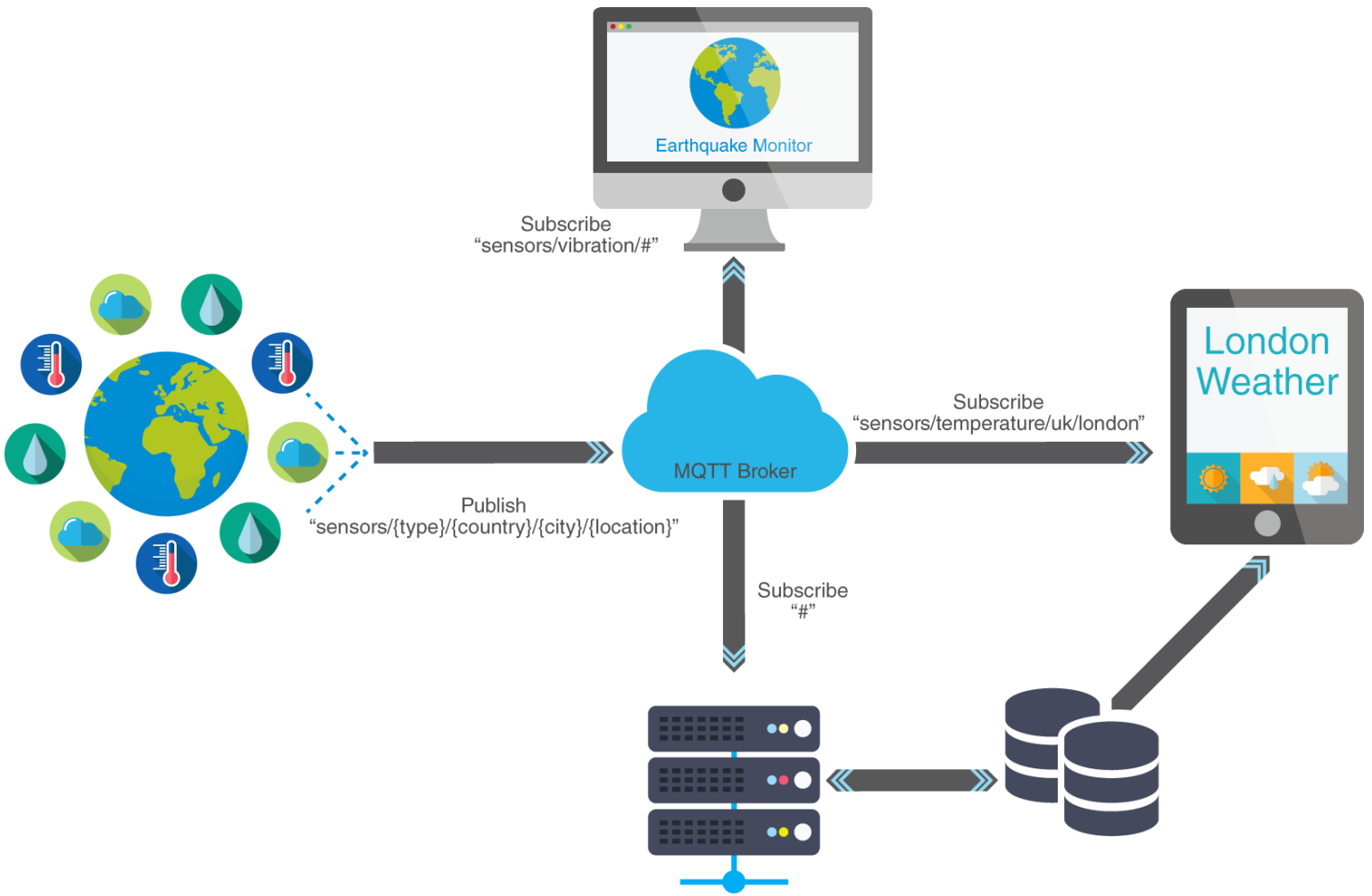


Publish/subscribe approach – Involved entities

- Publisher
  - Who is sending a message about a specific topic
- Subscriber(s)
  - Who is receiving the message (one or more clients)
- Broker
  - Known by both the publisher and subscriber, it filters the incoming messages and distributes them accordingly
  - Handle authentication/authorization of clients



Illustrative scenario



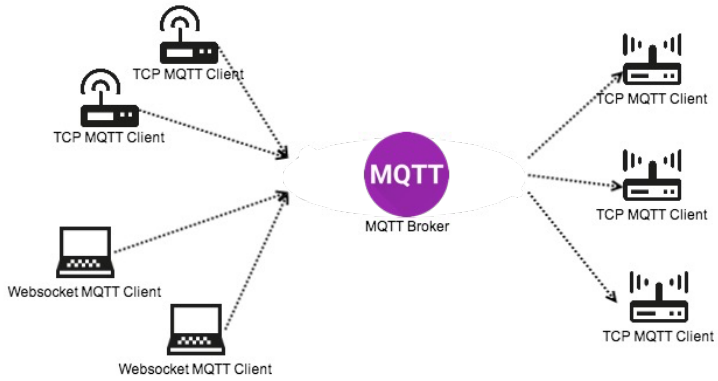
Message Queuing Telemetry Transport (MQTT)

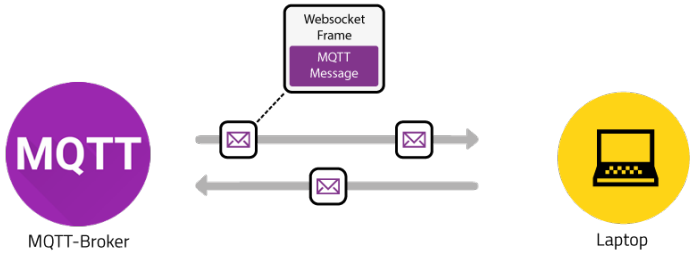
- Lightweight publish-subscribe messaging protocol for machine-to-machine (M2M) that can run on embedded devices and mobile platforms
  - Telemetry = Tele-Metering = remote measurements
  - Low power usage
  - Binary compressed headers
  - Maximum message size of 256 MB
    - Not really designed for sending large amounts of data
    - Better at a high volume of low size messages
  - Lightweight = low network bandwidth and small code footprint

- Documentation
  - http://mqtt.org/
  - https://github.com/mqtt/mqtt.github.io/wiki
  - http://www.hivemq.com/mqtt-essentials/

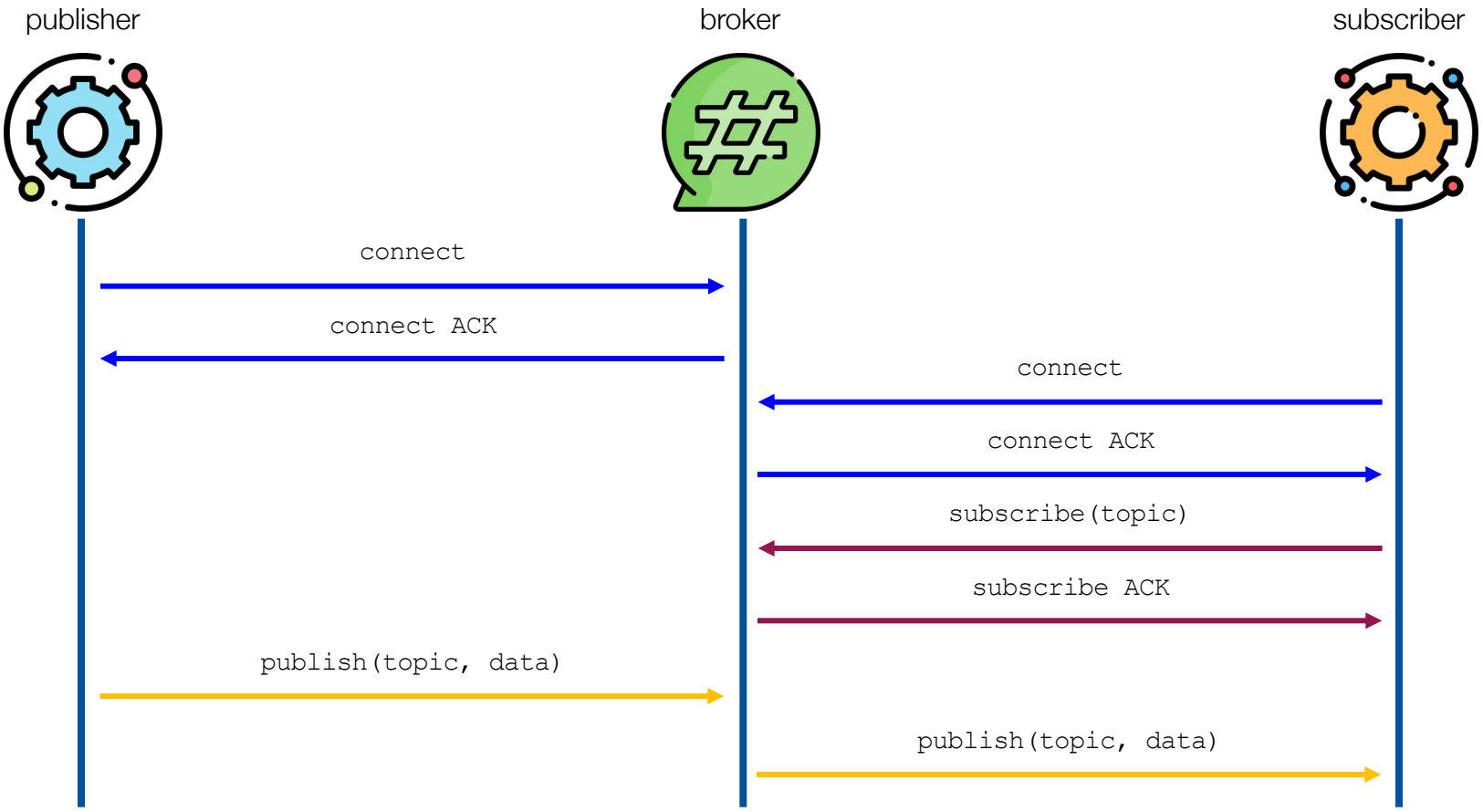
MQTT – Operating stack

- MQTT mainly works on top of TCP
  - Note: there exists MQTT for Sensor Networks (MQTT-SN), where TCP is replaced by UDP → TCP stack is too complex for WSN
- MQTT could work in cooperation with the WebSocket (WS) protocol
  - WS allow to receive MQTT data directly into a Web browser
- Both MQTT & WS can work on top of TLS



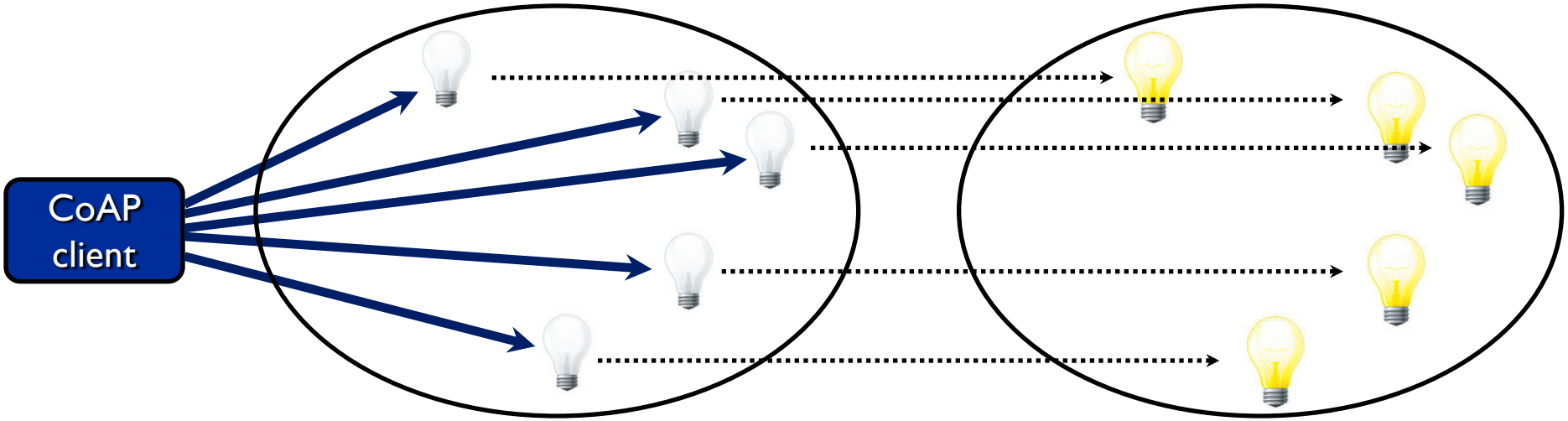


Publish/subscribe interactions sequence



Group communication

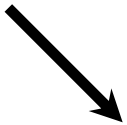
- CoAP supports multicast to target multiple endpoints with a single request
- Example: `POST All-Devices.floor1.west.bldg6.example.com/status/lights`
- draft-ietf-core-groupcomm-25 defines mechanisms to manage CoAP groups



Resource Discovery

- Resource discovery is based on discovering links to resources hosted by a CoAP service endpoint
- Standard mechanism based on link representations
  - Web Linking (RFC 5988)
  - CoRE Link Format (RFC 6690)
    - URI
    - List of attributes (e.g., `obs` means observable; `rt` indicates the resource type)
- A GET request to any CoAP server for the resource **/.well-known/core** results in a response that includes a list of links (formatted according to RFC 6690) to all resources hosted
- CoRE interfaces (`if` attribute): define the set of methods that the resource accepts

```
GET /.well-known/core
```



```
</sensors>;ct=40;title="Sensor
Index",</sensors/temp>;rt="temperature-
c";if="sensor",</sensors/light>;rt="light-
lux";if="sensor"</t>;anchor="/sensors/temp";rel="
alternate"
```

Resource Directory

- Sometimes direct discovery of resources through /.well-known/core is not practical
  - sleeping nodes
  - networks where multicast traffic is inefficient
- A resource directory (RD) is a CoAP endpoint that can be used as a registry to register, maintain, lookup and remove resources descriptions (draft-ietf-core-resource-directory-07)
- The RD contains CoRE Links to resources
  - it supports a function set to be populated
  - it supports a function set to be looked up (queries can be made to filter on the basis of certain attributes)

UNIVERSITÀ
DI PARMA

IoT Lab

IoT Security

Introduction

- Security is a very important aspect in current Internet and, probably more, in IoT scenarios
- Securing IoT communications is particularly complicated by the nature of nodes (smart objects):
  - **limited computational power**
  - **limited memory capabilities**
  - **limited communication resources**
  - **often battery-powered**
  - **often without user interface**

Threats

- Threats related to networked scenario:
  - **Eavesdropping attack**
    - if the communication channel is not adequately protected
    - e.g. if operational keying materials, security parameters, or configuration settings, are exchanged in clear using a wireless medium, or in case of session key compromise due to a long period of usage without key renewal
  - **Man-in-the-middle attacks**
    - particularly during key establishment and/or device authentication procedure
    - device authentication/authorization may be not automated and need support of a human decision process, since things usually do not have a priori knowledge about each other

Threats (cont.)

- **Routing attack**
  - routing information in IoT can be spoofed, altered, or replayed, in order to create routing loops, attract/repel network traffic, extend/ shorten source routes, etc.
  - relevant routing attacks may also include:
    - Sinkhole attack (or blackhole attack)
      - » an attacker declares himself to have a high-quality route/path to the base station, thus allowing him to do anything to all packets passing through it
    - Selective forwarding
      - » an attacker may selectively forward pkts or simply drop a packet
    - Sybil attack
      - » an attacker presents multiple identities to other things
- **Privacy threat**
  - an attacker can gather and track thing's information, deducing behavioral patterns
    - such information can subsequently be sold to interested parties

Threats (cont.)

- **Denial-of-Service attack**
  - typically, things have tight memory and limited computation, they are thus vulnerable to resource exhaustion attack
    - attackers can continuously send requests to be processed by specific things so as to deplete their resources
    - this is especially dangerous in the IoTs since an attacker might be located in the backend and target resource-constrained devices in an LLN (Low power and Lossy network)
  - DoS attack can be launched by physically jamming the communication channel
    - thus breaking down the T2T communication channel
  - network availability can also be disrupted by flooding the network with a large number of packets

Threats (cont.)

- Threats related to the physical nature of smart objects:
  - **Extraction of security parameters from physically unprotected devices**
    - if a group key is used and compromised this way, the whole network may be compromised as well
  - **Firmware Replacement attack**
    - e.g. when a thing is in operation or maintenance phase
    - its firmware or software may be updated to allow for new functionality or new features
  - **Cloning of smart things by untrusted manufacturers and/or Malicious Substitution of things during the installation**
    - e.g. with lower-quality things
    - untrusted manufacturer may change functionality of the cloned thing, resulting in degraded functionality

Countermeasures

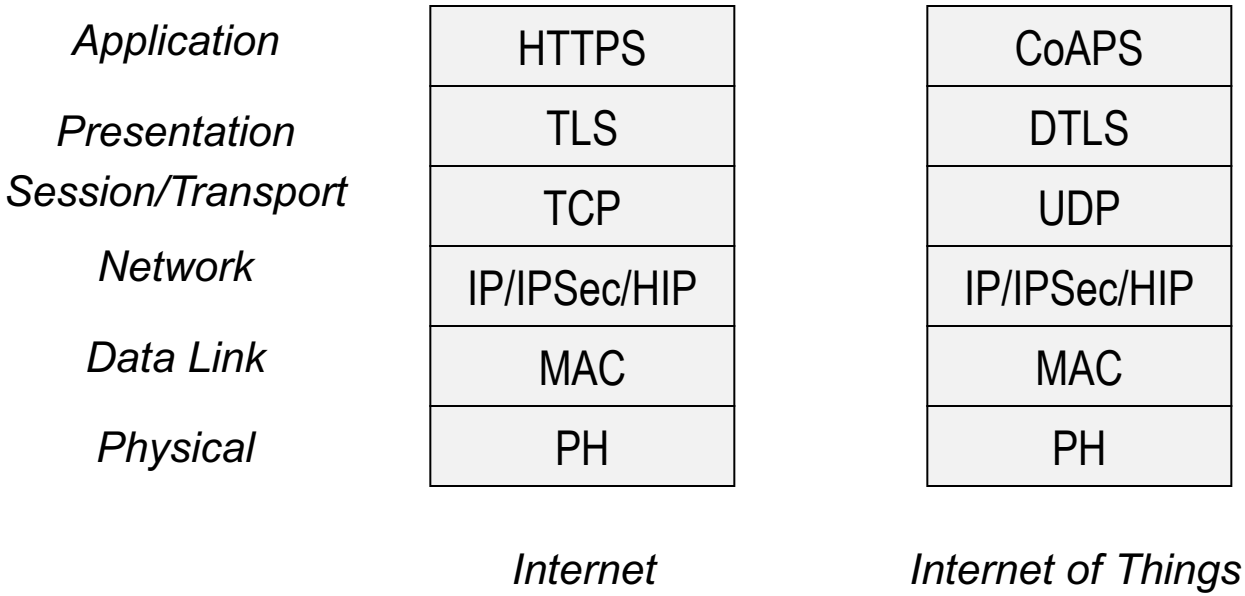
- Countermeasures against threats related to the physical nature of smart objects:
  - **safe supplying and installation measures, such as avoiding untrusted manufacturers and installers**
  - **protect smart objects in safe places**
- Countermeasures against threats related to constrained networked scenario:
  - **secure communication protocols and cryptographic algorithms**
    - to enforce the following basic security services:
      - peer authentication/authorization
      - data authentication/integrity
      - confidentiality
      - non-repudiation
      - availability
      - anonymity, and/or pseudonymity

Security services

- Security services can be implemented by a combination of
  - **cryptographic mechanisms, such as symmetric block ciphers, hash functions, asymmetric cryptographic or signature algorithms, and**
  - **non-cryptographic mechanisms, which implement authorization and other security policy enforcement aspects**
- For each of the cryptographic mechanisms, a solid key management infrastructure is fundamental to handling the required cryptographic keys

IP-based IoT stack with security protocols

- Example of IP-based IoT protocol stack with security enforcement vs. classical Internet protocol stack



IP-based IoT stack (cont.)

- Data authentication, integrity, and confidentiality might be provided at lower layers, such as PH or MAC (e.g., in IEEE 802.15.4 systems)
  - **hop-by-hop security**
  - **end-to-end security can be guaranteed only with a high level of trust on intermediate nodes**
  - **however, due to the highly dynamic nature of wireless multi-hop communications expected to be used to form the routing path between remote end nodes, this kind of security is not, in general, sufficient**
- For such reason, security mechanisms at network, transport, or application level should be considered instead of (or in addition to) PHY and MAC level mechanisms

Selecting security protocols

- Some important aspects:
  - **Avoiding security service duplication**
    - it has impact on the computation and transmission performance
    - e.g. end-to-end security can be guaranteed through IPSec at the network layer or TLS/DTLS at the transport layer
  - **Interoperability**
    - many of the Internet security protocols are independent by the used security algorithms
      - used algorithms and parameters sometime can be negotiated between the communication parties
    - in order to guarantee full interoperability, and to reduce the number of implemented cipher suites, it is important to define a set of mandatory algorithms (targeted for constrained environments) that all objects must implement for minimal support
    - however it is not easy to agree on a set of common protocols and algorithms that can be supported by all devices in such a very heterogeneous environment

IPSec

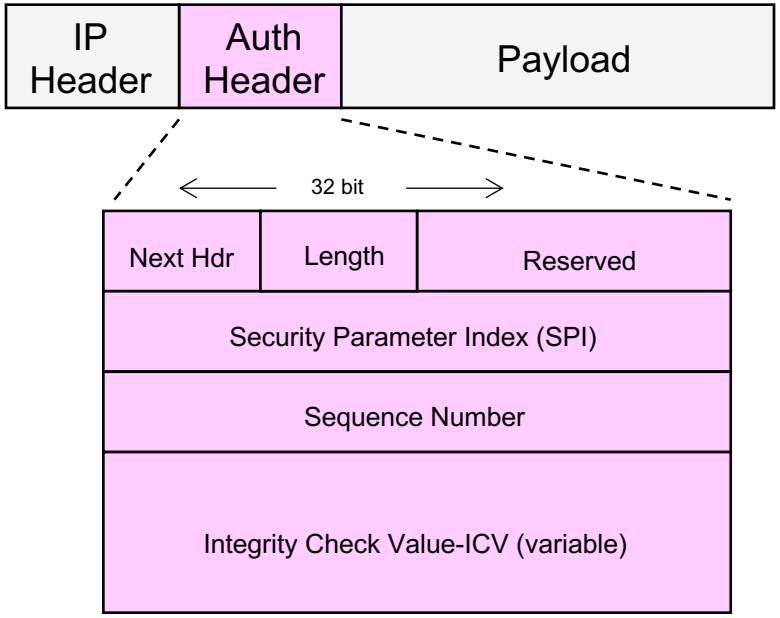
- At network layer, an IoT node can secure data exchange in a standard way by using the Internet Protocol Security (IPSec)
- IPSec (RFC 4301) is the Security Architecture for the IP protocol
  - **It was originally developed for IPv6 but found widespread deployment, first, as an extension in IPv4, into which it was back-engineered**
  - **was an integral part of the base IPv6 protocol suite, but has been made optional**
- IPSec can provide:
  - **confidentiality**
  - **integrity**
  - **data-origin authentication**
  - **protection against replay attacks**

IPSec Security Services

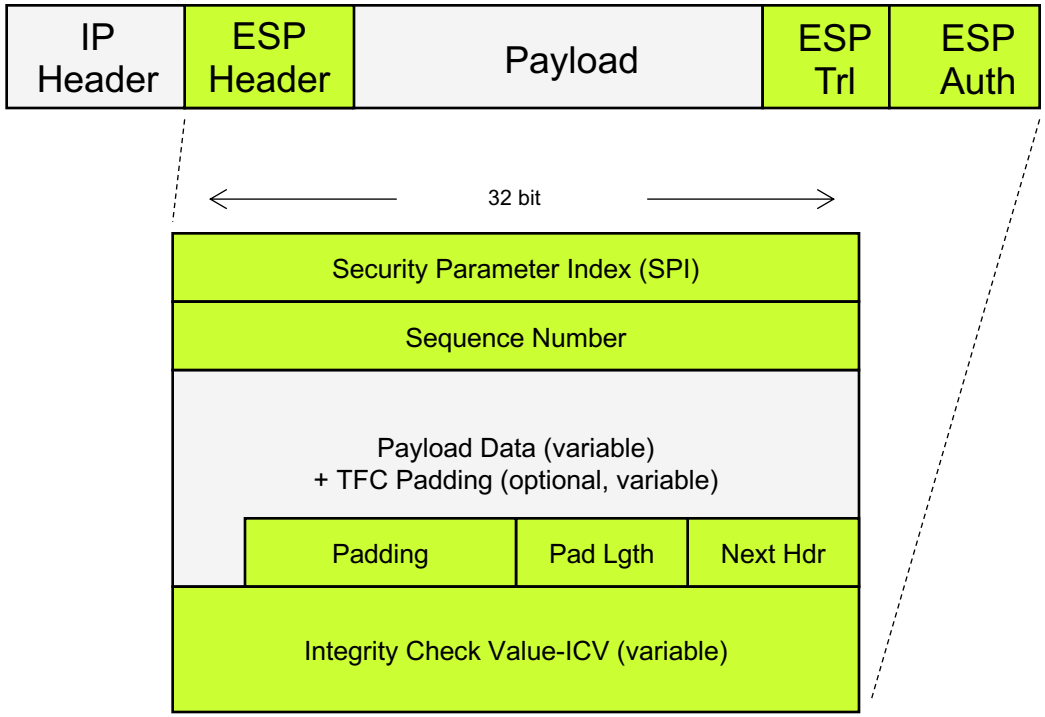
- Such security services are implemented by two IPSec security protocols:
  - **Authentication Header (AH)**
    - provides data-origin authentication, integrity, and optionally anti-replay
  - **Encapsulated Security Payload (ESP)**
    - provides confidentiality, data-origin authentication, integrity, and anti-replay capabilities

IPSec AH and ESP packet formats

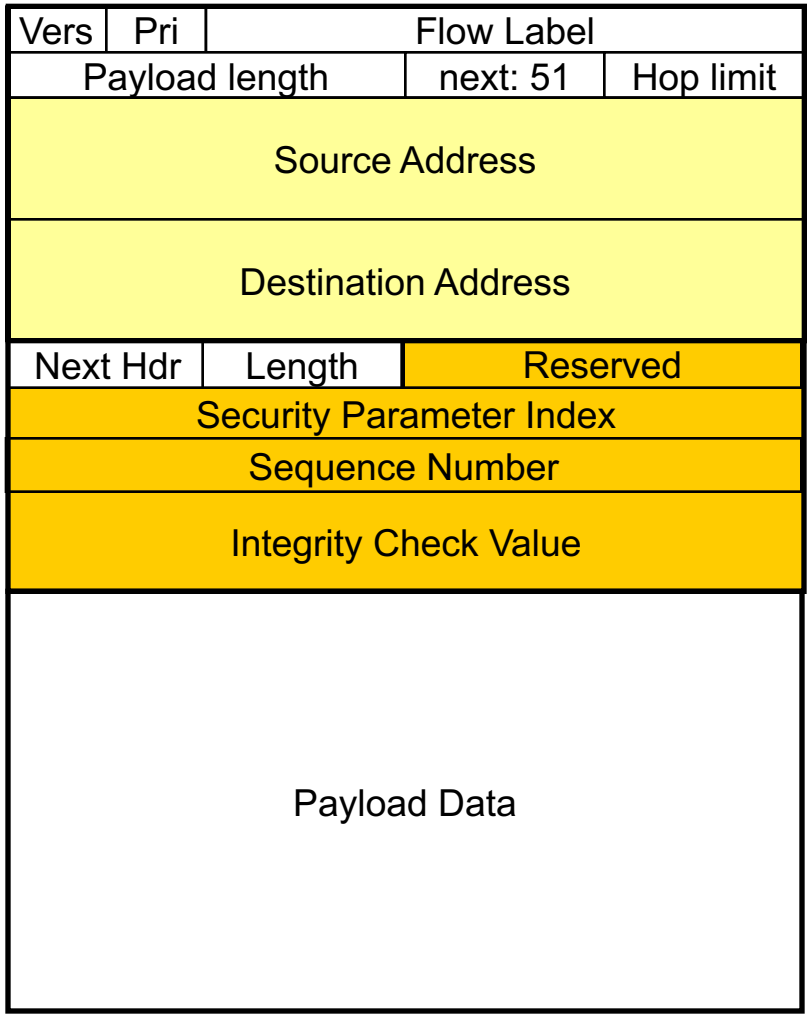
- IPSec AH packet format



- IPSec ESP packet format

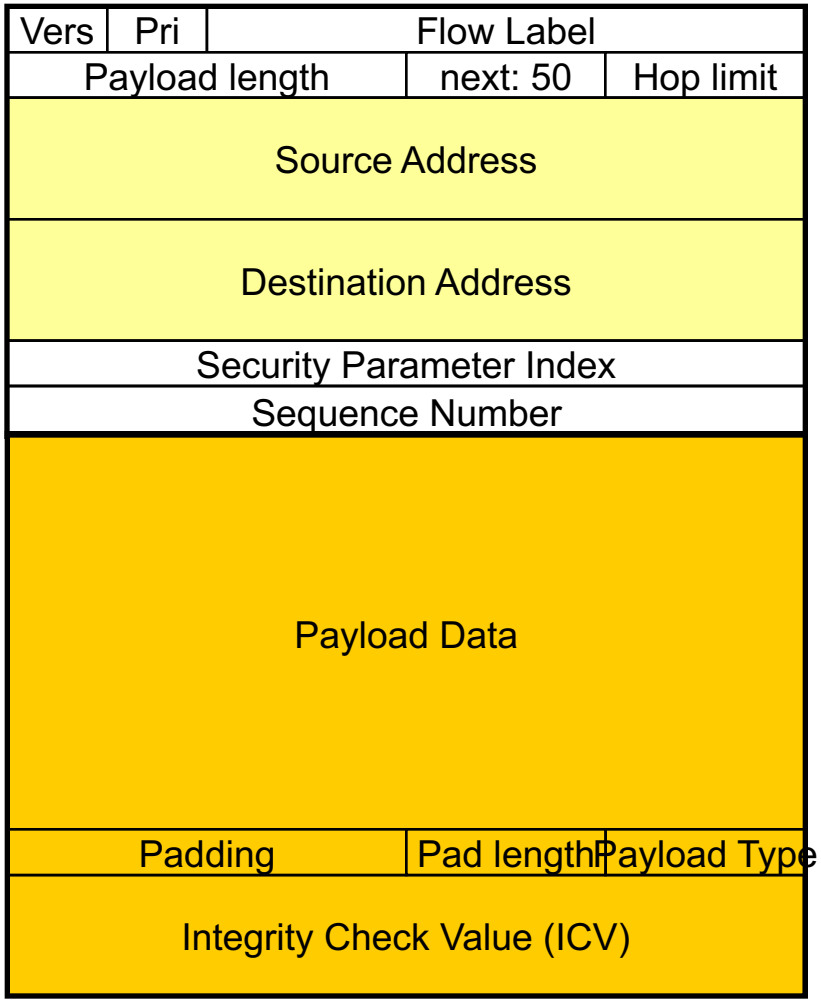


Authentication Header



- Next header
  - **8-bit field that identifies the type of the next payload after the AH**
- Payload Length
  - **8-bit field specifies the length of AH in 32-bit words (4-bytes), minus 2**
- Reserved
  - **16-bit field is reserved for future use, and must be set to 0 by the sender**
- Security Parameter Index (SPI)
  - **32-bit value used to identify the used SA**
- Sequence number
  - **32-bit counter value that increases by one for each packet sent**
- Integrity Check Value (ICV)
  - **variable-length field (multiple of 32 bits) that contains the ICV for this packet**

Encapsulating Security Payload



- Security Parameter Index (SPI)
- Sequence number
- Payload Data
  - **may including the cipher Initialization Vector**
  - **IPsec implementation can add also Traffic Flow Confidentiality (TFC) padding**
- Padding and Pad length
  - **If the encryption algorithm requires the plaintext to be a multiple of some number of bytes, e.g., the block size of a block cipher**
  - **or to ensure that the resulting ciphertext terminates on a 4-byte boundary**
- Next header
- Integrity Check Value (ICV)

IPSec (cont.)

- IPSec AH and ESP define only the way IPSec control information are encapsulated
  - **the effective security algorithms are specified separately algorithms can be selected amongst a set of available cipher suites**
  - **this makes IPSec usable also in the presence of very resource-constrained devices, if a proper algorithm that guarantees both usability and sufficient security level is selected**
- The keying material and the selected cryptographic algorithms used by IPSec for securing a communication (Security Association) can be:
  - **pre-configured (specifying a priori a pre-shared key, hash function and encryption algorithm) or**
  - **dynamically negotiated by the IKE protocol**
- Unfortunately the IKE protocol uses asymmetric cryptography, which may be computationally too heavy for very small devices
  - **proper IKE extensions should be considered using lighter algorithms**

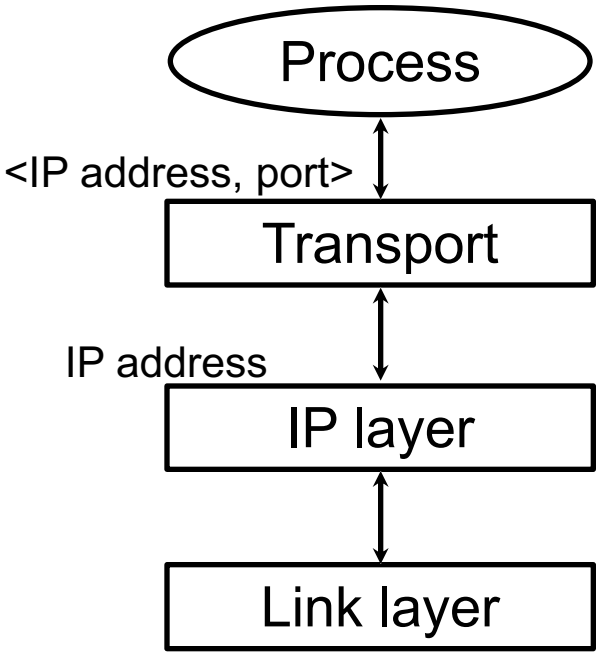
IPSec (cont.)

- Some problems related to the implementation of IPSec in constrained IoT nodes are
  - **data overhead**
    - introduced by the extra header encapsulation of IPSec AH and/or ESP
    - this can be limited by
      - using only one protocol between AH and ESP
      - selecting the “transport mode” encapsulation when possible
      - implementing header compression techniques, similarly to what is done in 6LoWPAN for the IP header
  - **configuration, and practical implementation aspects**
    - IPSec works at network layer is often designed for VPNs, thus making it difficult for them to be dynamically configurable and controlled by an application

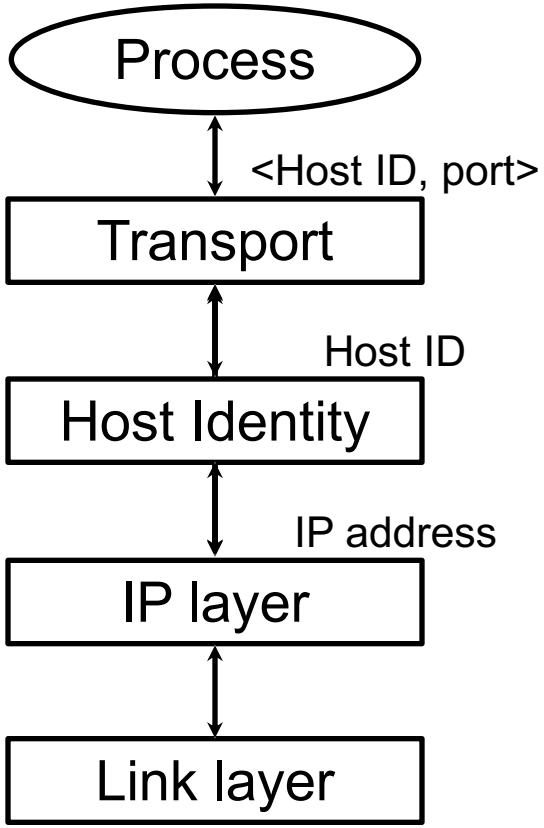
Host Identity Protocol (HIP)

- Host Identity Protocol (HIP)
  - **decouple the two functions of host locators (for routing purposes) and host identifiers (for actual host identification) currently both performed by IP addresses**
  - **when used with cryptographic identity, it simplifies the establishment of a security association**
  - **Host Identity (HI)**
    - public keys can be used (cryptographic identity); in this case the private key is owned by the host and the public key is used as HI
  - **introduces a new cryptographic exchange, called the HIP Base Exchange (BEX) for authentication and key agreement**
    - based on Diffie-Hellman
    - alternative to using IKE (for IPSec)
- When used together with a security protocol like ESP, it provides integrity protection and encryption for upper-layer protocols
  - **the payload traffic between two HIP hosts is protected with ESP**

Host Identity (HI)



IP addresses at network and transport layer



Host Identity

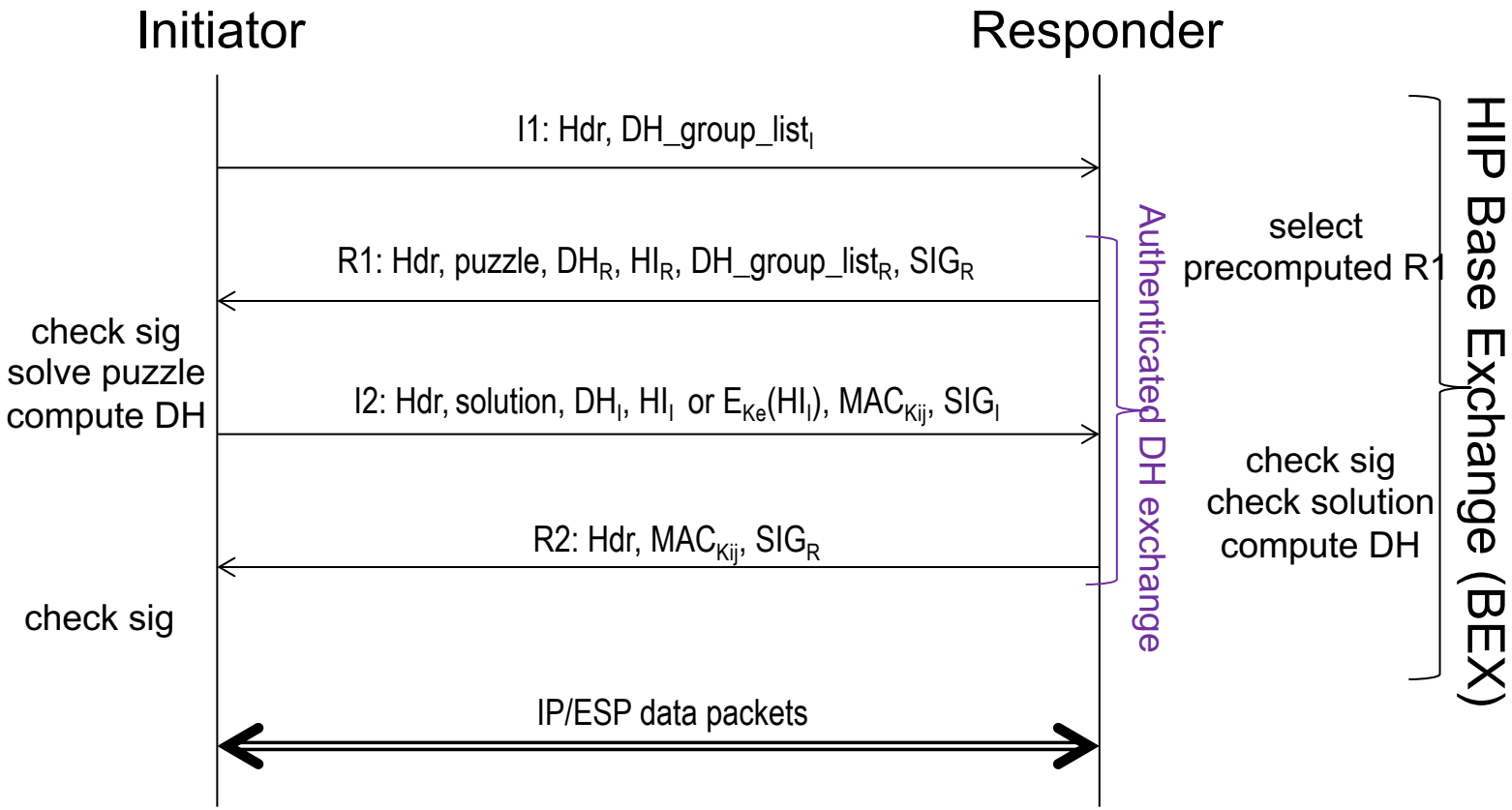
HIP (cont.)

- The HIs may be stored in various DNS or other directories
- The public key of a private-public key pair is normally chosen as the HI
  - **correspondingly, the host itself is defined as the entity that holds the private key of the key pair**
  - **HIs can either be public (e.g. published in the DNS), or unpublished**
- HIP implementations must support the RSA public key algorithm and should support the DSA and ECDSA algorithms, for generating the HI
- HIP also defines an Host Identity Tag (HIT)
  - **a 128-bit representation of the HI based on the hash of HI plus other information**
  - **can be used for example as unique host identifier at the existing IPv6 API and by application protocols**

Host Identity Tag (HIT)

- The Host Identity Tag (HIT) is a 128-bit hashed encoding of the HI
  - **same length as an IPv6 address and can be used in fixed address-sized fields in APIs and protocols**
  - **self-certifying**
    - i.e., given a HIT, it is computationally hard to find a Host Identity key that matches the HIT
  - **the probability of a HIT collision between two hosts is very low**
    - it is infeasible for an attacker to find a collision with a HIT that is in use
- The HIT can be an Overlay Routable Cryptographic Hash Identifier (ORCHIDv2), formed by:
  - **Prefix: constant 28-bit-long bitstring value (2001:20::/28) to distinguish HIT from other IPv6 addresses**
  - **ORCHID generation algorithm (OGA) ID: a 4-bit encoding of the algorithms used for generating the HI and the HIT**
  - **HI Hash: a 96-bit hashed representation of the HI = Hash(HIP Context ID || HI)**
    - Def.: HIP Context ID = 0xF0EF F02F BFF4 3D0F E793 0C3C 6E61 74EA

HIP Base Exchange (BEX)

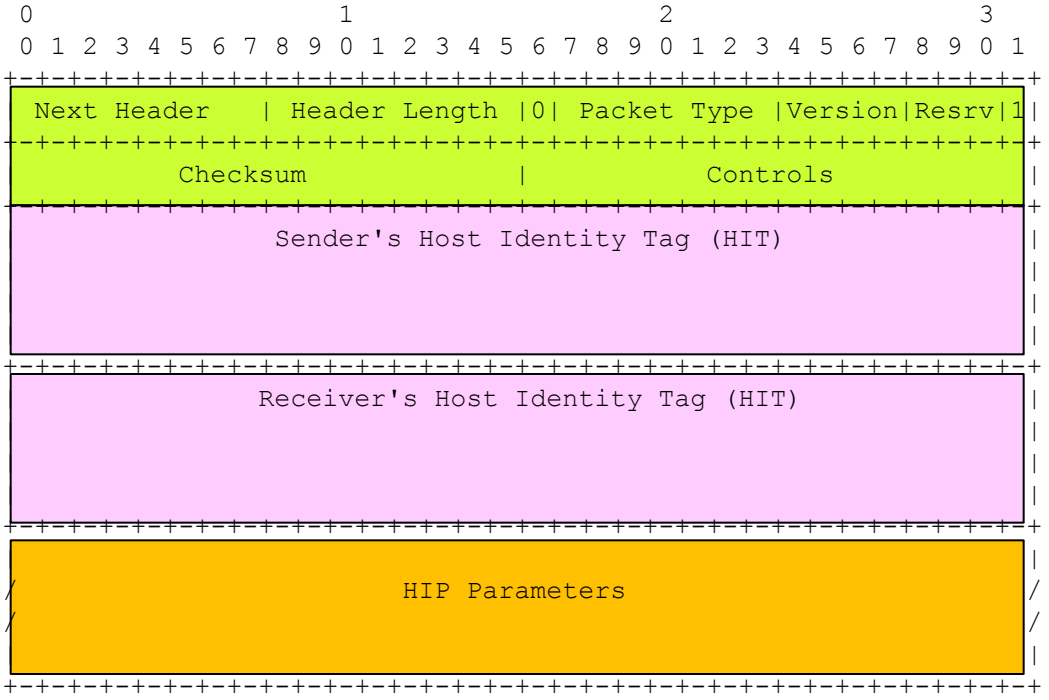


HIP Puzzle Mechanism

- The purpose of the HIP puzzle mechanism is to protect the Responder from a number of denial-of-service threat
  - **the puzzle allows the Responder to use a fairly cheap calculation to check that the Initiator is "sincere“**
    - in the sense that it has churned enough CPU cycles in solving the puzzle
  - **it allows the Responder to delay state creation until receiving the I2 packet**
- Puzzle procedure
  - **The Responder supplies a random number #I, and requires the Initiator to find a number #J**
  - **to select a proper #J, the Initiator must create the concatenation of #I, the HITs of the parties, and #J, and calculate a hash over this concatenation. The lowest-order #K bits of the result must be zeros**
    - the value #K sets the difficulty of the puzzle
    - to generate a proper number #J, the Initiator will have to generate a number of #Js until one produces the hash target of zeros

HIP BEX Packet Format

- An HIP BEX packet is formed by a HIP Header and HIP parameters
- It is encapsulated within an IPv6 packet as IPv6 extension header
  - **Current version does not consider the possibility of an IPv6 payload when HIP header is present**
    - Next Header is 59 (IPPROTO_NONE) that is the IPv6 'no next header' value



Transport Layer Security

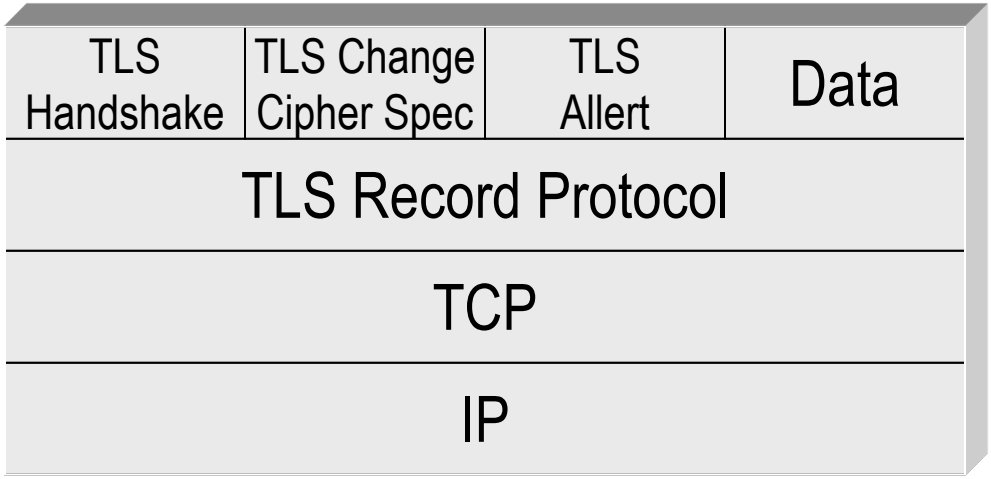
- In the standard Internet, data exchange between applications can be secured at transport layer through
  - **Transport Layer Security (TLS)**
    - widest used secure protocol, running on top of the TCP providing to the application layer the same connection and stream-oriented interface of TCP
  - **Datagram Transport Layer Security (DTLS)**
    - introduced more recently in order to provide a security service similar to TLS on top of UDP
    - can be used to secure datagram traffic for client/server applications

Transport Layer Security (TLS)

- Transport Layer Security (TLS) protocol (RFC 5246, version 1.2)
  - **widest used secure protocol, running on top of the TCP**
  - **same connection and stream-oriented interface of TCP**
- Provides privacy and data integrity between two communicating client/server applications
  - **peer's identity authentication**
    - using asymmetric cryptography (e.g., RSA, DSA, etc.)
  - **negotiation of encryption algorithm and key exchange**
    - protected against eavesdropping, modifying and replay attacks
  - **confidentiality**
    - symmetric cryptography is used
  - **data authentication and integrity**
    - message transport includes a message integrity check using a keyed MAC (e.g. based on SHA-1 hash)
- Uses X.509 certificates

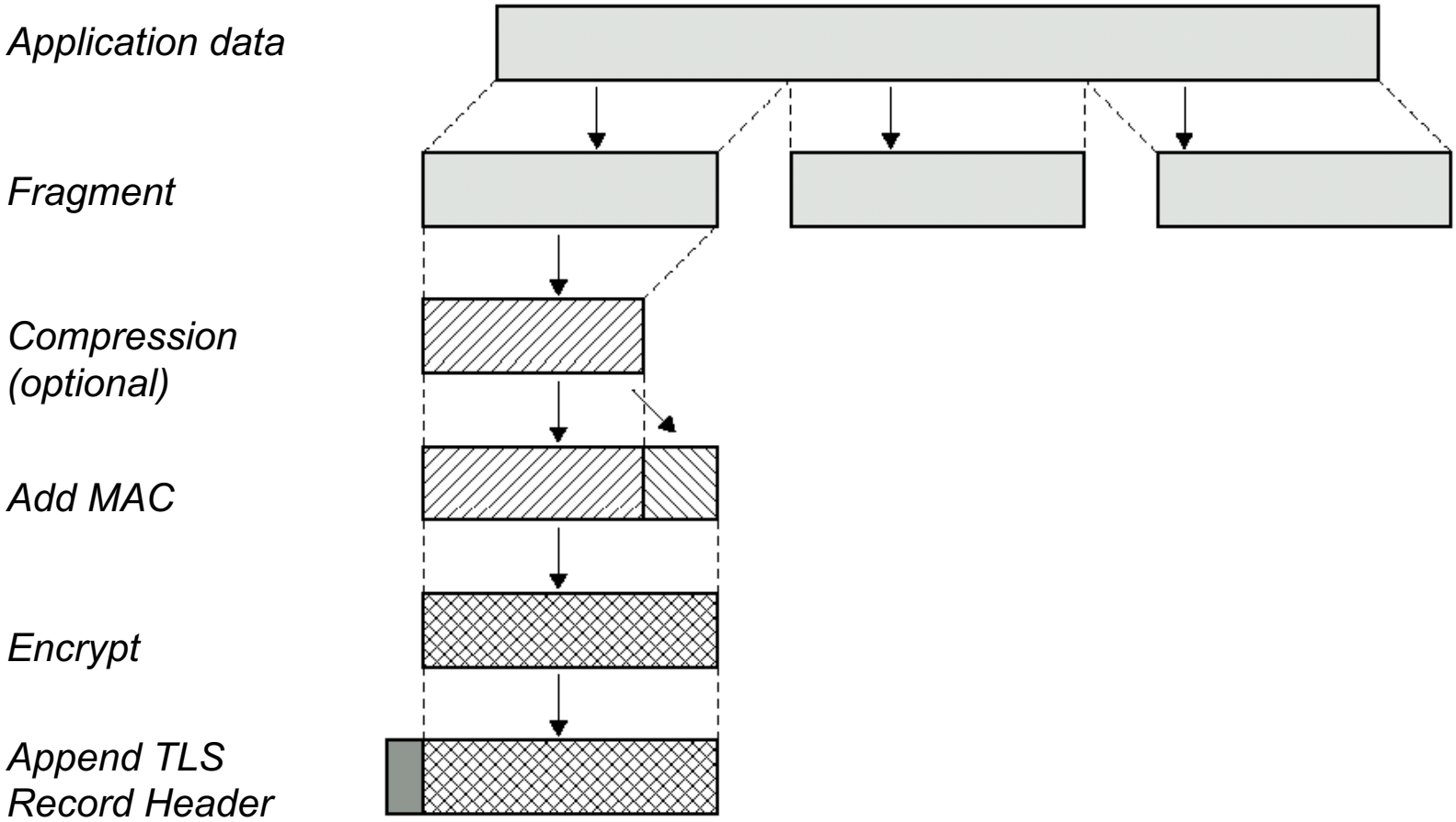
TLS protocol stack

- The protocol is composed of two layers:
  - **TLS Handshake Protocol**
  - **TLS Record Protocol**



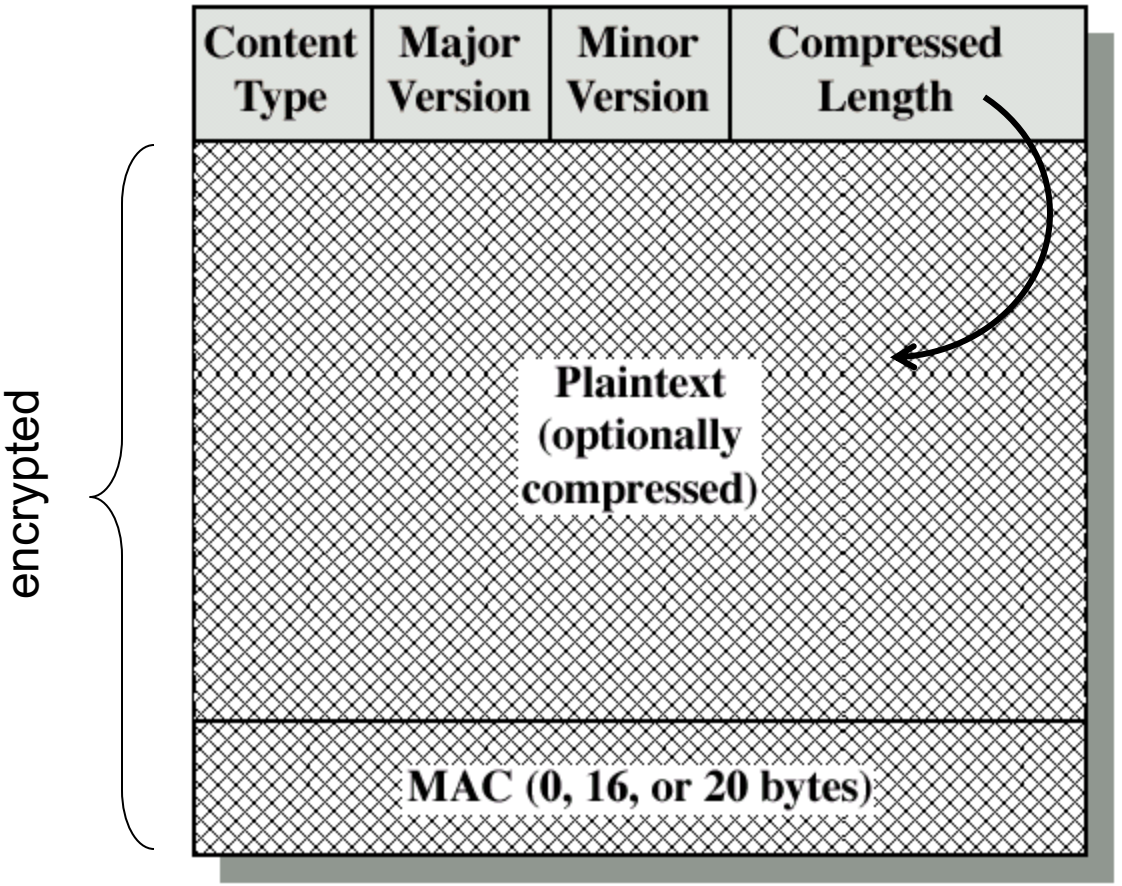
TLS Record Protocol (cont.)

- TLS-RP operation:



TLS Record Protocol (cont.)

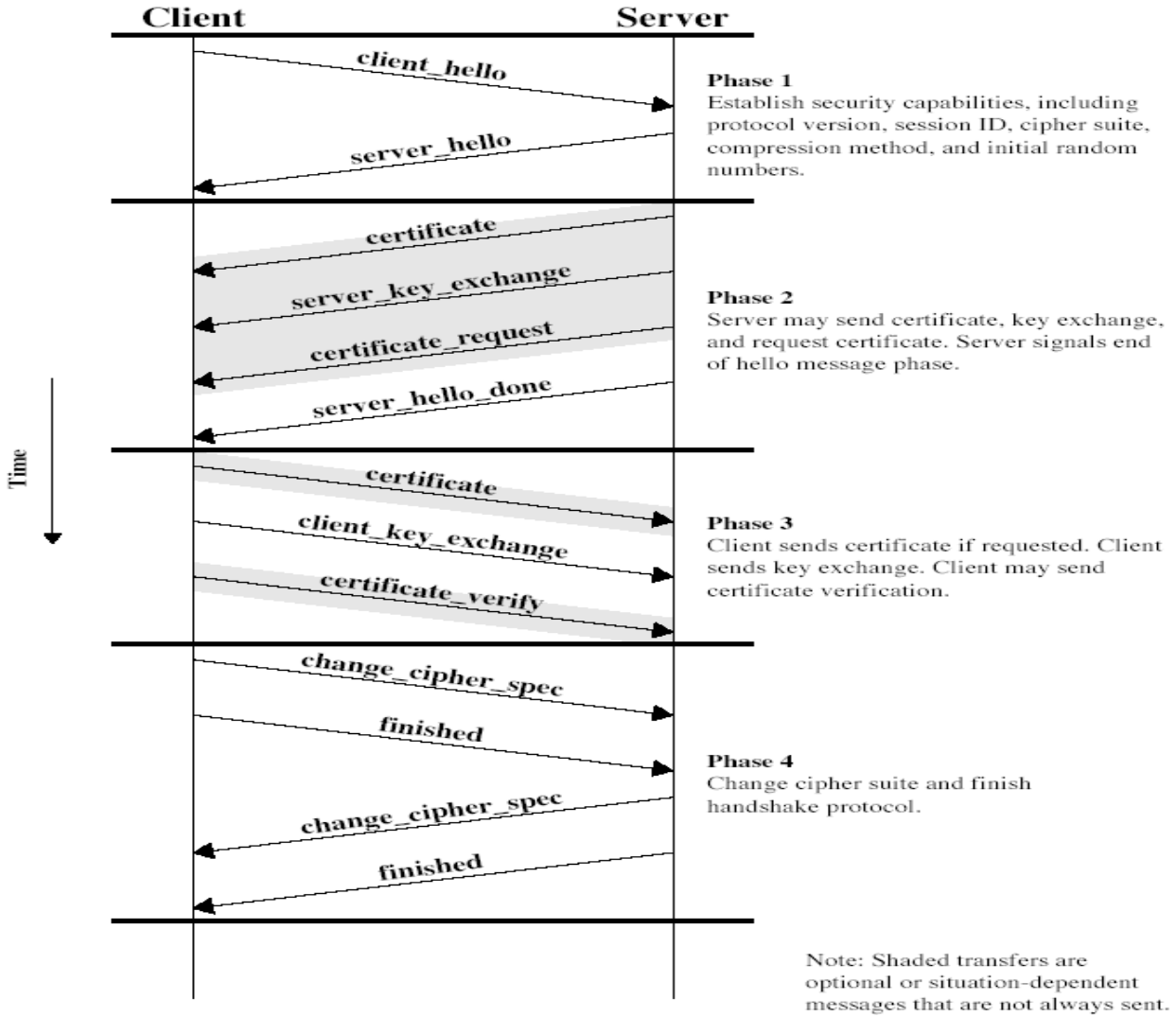
- TLS-RP format



TLS Handshake Protocol

- Peer-entity authentication and key exchange is provided by the TLS Handshake phase, performed at the beginning of the communication
- TLS Handshake Protocol allows the server and client to authenticate each other and to negotiate an encryption algorithm and cryptographic keys before the application protocol transmits or receives its first byte of data
  - **the peer's identity can be authenticated using asymmetric, or public key, cryptography (e.g., RSA, DSA, etc.)**
  - **secure negotiation of a shared secret**
    - unavailable to eavesdroppers, or to attackers who can place themselves in the middle of the connection

TLS Handshake Protocol (cont.)



TLS and DTLS

- TLS cannot be directly used with UDP, since packets may be lost or reordered (unreliability)
  - **TLS does not allow independent decryption of individual records**
    - because the integrity check depends on the sequence number, if record N is not received, then the integrity check on record N+1 will be based on the wrong sequence number and thus will fail
    - prior to TLS 1.1, there was no explicit IV and so decryption would also fail
  - **TLS handshake layer assumes that handshake messages are delivered reliably**
    - breaks if those messages are lost
- Datagram Transport Layer Security (DTLS) Version 1.2
  - **TLS over datagram transport (UDP)**
  - **DTLS makes only the minimal changes to TLS required to fix the problem**

Datagram Transport Layer Security (DTLS)

- Introduced in order to provide a security service similar to TLS on top of UDP
- Although it is still poorly supported in standard Internet nodes, it is currently the reference security protocol for IoT systems
  - **it uses UDP as transport**
  - **It does not suffer from the problems originated by the use of TCP in network-constrained scenarios**
    - due to the extremely variable transmission delay and lossy links

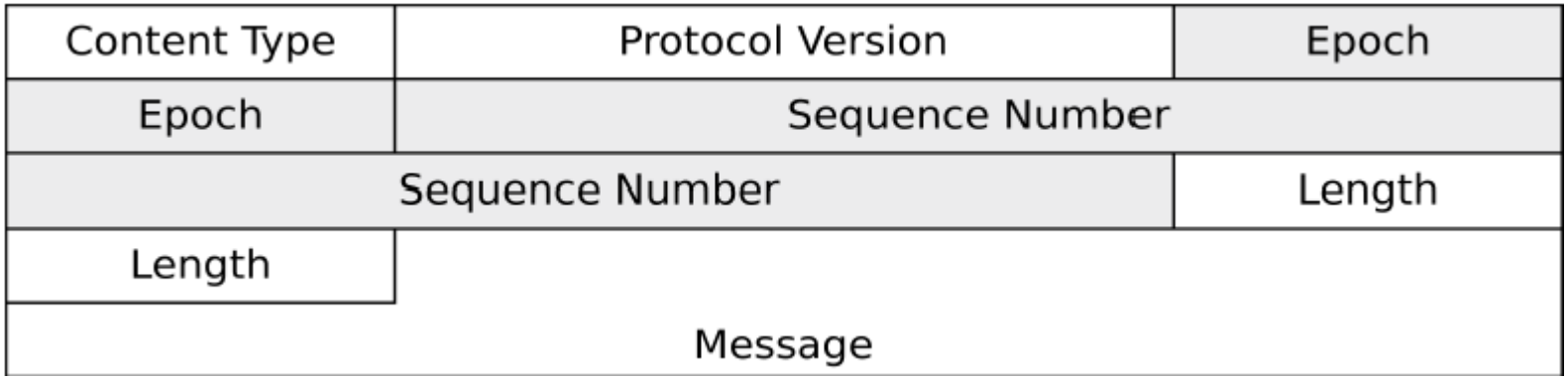
DTLS (cont.)

- Loss-Insensitive Messaging
  - **In TLS Record Layer records are not independent**
    - Cryptographic context of stream cipher key stream is retained between records
    - Anti-replay and message reordering protection are provided by a MAC that includes a sequence number, but the sequence numbers are implicit in the records
  - **DTLS solves the first problem by banning stream ciphers**
  - **DTLS solves the second problem by adding explicit sequence numbers**

DTLS (cont.)

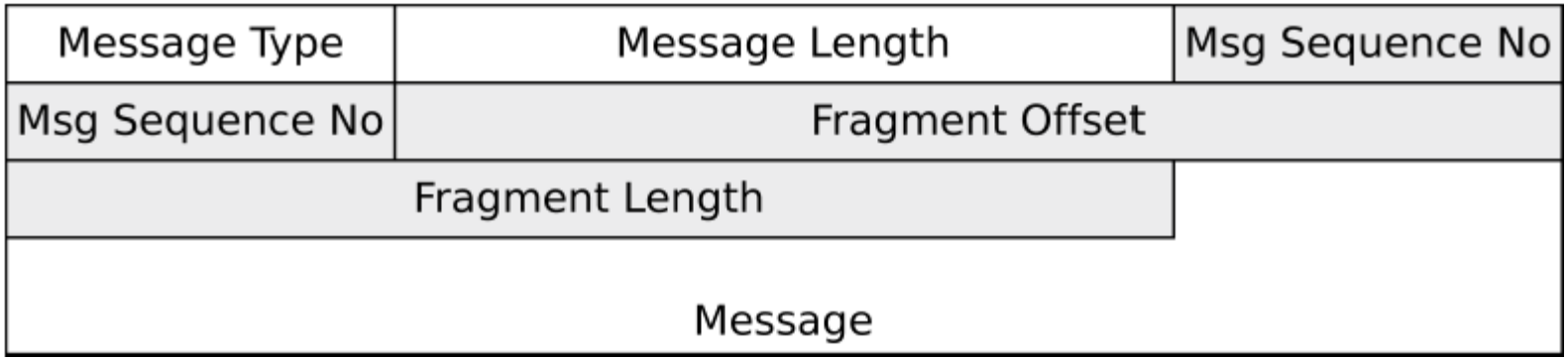
- Providing Reliability for Handshake
  - **the TLS handshake is a lockstep cryptographic handshake**
    - messages must be transmitted and received in a defined order; any other order is an error
      - this is incompatible with reordering and message loss
  - **in DTLS, each handshake message is assigned a specific sequence number within that handshake**
    - when a peer receives a handshake message, it can quickly determine whether that message is the next message it expects
  - **DTLS uses a simple retransmission timer to handle packet loss**
- TLS handshake messages are potentially larger than any given datagram, thus creating the problem of IP fragmentation
  - **In order to compensate for this limitation, each DTLS handshake message may be fragmented over several DTLS records, each of which is intended to fit in a single IP datagram**
    - each DTLS handshake message contains both a fragment offset and a fragment length

DTLS Record Protocol Format



- Epoch
  - **a counter value that is incremented on every cipher state change**
- Sequence Number
  - **the sequence number for this record**

Handshake Message Format



- Message Sequence Number (MsgSeqNum)
  - **The first message each side transmits in each handshake always has MsgSeqNum = 0**
  - **Whenever each new message is generated, the MsgSeqNum value is incremented by one**
- Handshake Message Fragmentation and Reassembly
  - **When transmitting the handshake message, the sender divides the message into a series of N messages, all with the same MsgSeqNum value as the original handshake message**

DTLS vs. IPSec

- Both IPSec and DTLS provide the same security features with their own mechanisms at different stack layers
  - **also the IPSec IKE key agreement reflects almost the same DTLS Handshake function**
- The main advantage of securing communications at transport layer with DTLS consists in allowing applications to directly and easily select which, if any, security service has to be set up
  - **another practical advantage is that the adoption of DTLS can allow for the reuse of the large experience and implementations that came with TLS**
- DTLS has recently received significant attention for constrained networked applications
  - **it has been standardized as the security protocol for CoAP associated to "coaps" URIs**

Hop-by-hop and E2E security

- From the security point of view, one problem of using DTLS or IPSec is that end-to-end (E2E) communication is not guaranteed when intermediate nodes such as proxies or application level gateways are introduced
  - **both IPSec and DTLS provide secure communications at IP and transport layers respectively, and, in presence of a multi-hop application-level communications, they can assure security only within each hop**
- The presence of intermediate nodes may arise for
  - **application reasons**
    - e.g. caching, data aggregation, application level routing, filtering
  - **Security reasons**
    - as security gateway acting as access controller and granting access to the internal network only to trusted nodes
      - may protect the constrained nodes against flooding or replay attacks carried on by full-powered host attached to the Internet

Hop-by-hop and E2E security (cont.)

- When communication is relayed at application level by intermediate nodes (e.g., proxies), end-to-end security is guaranteed through transport-level or IP-level mechanisms only if:
  - **communication is secured on each hop**
  - **all intermediate systems are considered trusted**
    - this requirement is complicated by the handling of hop-by-hop trust relationship

Application-level security

- A different approach for providing complete end-to-end security is to enforce security directly at application level
  - **simplifies the requirements for underlying layers, and probably reduces the cost, in term of packet size and data processing**
    - since only application data have to be secured and per-data and not per-packet overhead is introduced
  - **multicast communications, and in-network data aggregation is easier to be implemented at application level**
- The main disadvantages are:
  - **the complications introduced for application development**
  - **the overall code size due to poor reuse of software codes**
- This is mainly due to the lack of well defined and adopted secure protocols at application level
  - **examples of standards that can be used for this purpose are S/MIME and SRTP**

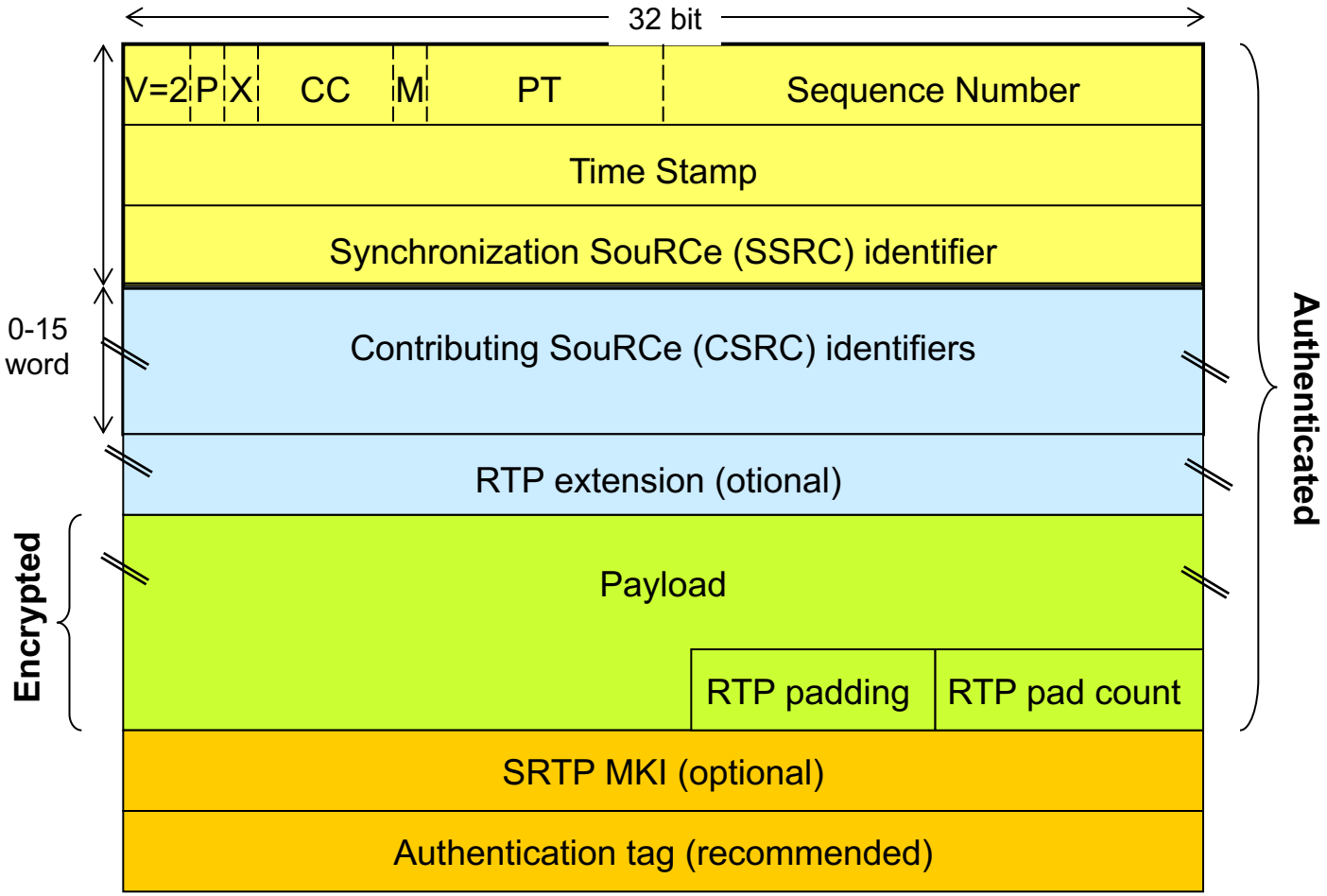
S/MIME

- Secure/Multipurpose Internet Mail Extension (S/MIME) is a standard for providing:
  - **authentication, message integrity, non-repudiation of origin, and confidentiality for application data**
- Originally developed for securing MIME data between mail user agents
  - **it is not restricted to mail and can be used for securing any application data**
  - **and encapsulated within any application and transport protocols**

Secure Real-time Transport Protocol (SRTP)

- Security extension of the Real-time Transport Protocol (RTP)
  - **developed for handling real-time data communications (e.g., voice or video communication), but can be re-used also in other application scenarios**
- Provides the following security services:
  - **confidentiality of the RTP payload**
    - default use of AES128
  - **message authentication and integrity of the entire RTP packet**
    - default use of HMAC-SHA1
  - **replay protection**
- It works in a per-packet fashion and is usually encapsulated in UDP

SRTP packet format



Key Agreement and Distribution

- Key distribution and management is a crucial issue that needs to be addressed when security mechanisms have to be adopted
- Asymmetric (public-key) cryptographic algorithms are often the basis for key agreement protocols
  - e.g. Diffie–Hellman
- In IoT environments particular attention must be paid, due to processing constrains
  - **type of used algorithm**
  - **frequency of key exchanges**
- Alternative approaches may involve
  - **key pre-distribution**
  - **use of a third party Key Distribution Center (KDC)**

Shared group key establishment

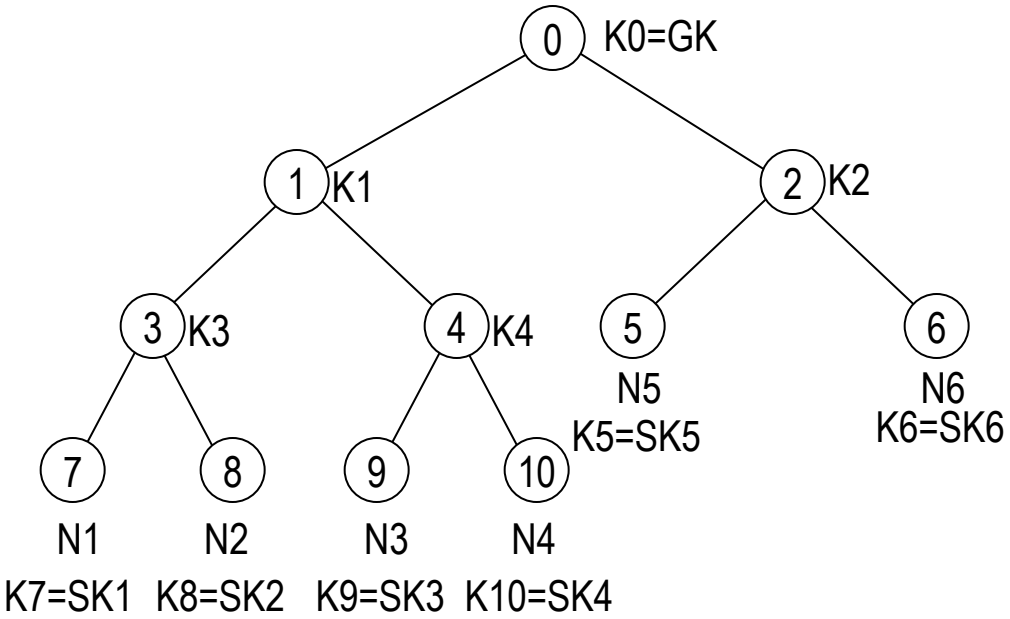
- Well known key establishment protocols such IKE and TLS Handshake only apply to scenarios where communication occurs between two parties
  - **unicast (point-to-point) communications**
- A lot of IoT scenarios may have advantages in using also point-to-multipoint (multicast) communications
- At present, the most effective way is to encrypt data with a secret key shared between all multicast group members
  - **called group key (or multicast key)**
  - **requires a proper group key establishment mechanism**

Shared group key establishment (cont.)

- In order to consider the possibility that the group membership changes during the time (new members may join and current members may leave)
- and in order to guarantee both:
  - **forward secrecy**
    - a former member cannot access successive communication
  - **backward secrecy**
    - a new member cannot access previous communication
- Proper rekeying operation is required at any membership change
- Several group key management mechanisms for multicast communication and for WSN have been proposed
  - **they may apply also to IoT scenarios**

Shared group key establishment (cont.)

- Up to now, probably the most efficient group key management mechanism is the LKH (Logical Key Hierarchy)



Security bootstrapping

- All key agreement protocols require that some credentials have been previously installed and configured on nodes, so that the key agreement procedure can occur securely
- Bootstrapping refers to the processing operations required before the network can operate
  - **it is a very important phase in the lifecycle of smart objects and can affect the way they behave in operational conditions**
  - **in smart objects it can be complicated by the absence of UIs**
  - **in this case, the bootstrapping may exploit:**
    - hardware pre-configured settings
    - physical environment where the node is bootstrapping or operates

Lightweight Cryptography (LWC)

- Smart objects are tiny computing devices, featuring constrained resources, such as
  - **low computation capabilities, little memory, and limited battery**
- Security mechanism/algorithms with smart objects in resource-constrained environments must necessarily take into account all these hard limitations
  - **Lightweight Cryptography (LWC)**

Lightweight Cryptography (LWC)

- Interesting research area, aiming at the design of new ciphers that might meet the requirements set by the use of smart objects
  - **cryptography tailored to (extremely) constrained devices**
  - **algorithms with smaller footprint, low energy consumption, and low computational power needs**
- “lightweight" should not be mistaken with weak (in terms of cryptographic protection)
  - **aim at providing sufficient security in the environment of restricted resources as can be found in many ubiquitous devices**
  - **trade-offs between security level, cost, and performance**
- Not intended for all-powerful adversaries

Symmetric LWC

- Tiny Encryption Algorithm (TEA)
  - **block cipher renowned for its simplicity of description and implementation, typically a few lines of code**
  - **operates on two 32-bit unsigned integers (could be derived from a 64-bit data block) and uses a 128-bit key**
    - it relies only on arithmetic operations on 32-bit words and uses only addition, XORing, and shifts
  - **it uses a large number of iterations, rather than a complicated program, in order to avoid preset tables and long setup times**
- Extended TEA (XTEA)
  - **redesign of TEA by changing the key schedule, and slightly faster**
- Other modifications of the TEA are:
  - **XXTEA, Block TEA, Speed TEA, and Tiny XTEA**

Symmetric LWC (cont.)

- HIGh security and light weigHT (HIGHT)
  - **generalized Feistel network with a block size of 64 bits, 128-bit keys and 32 rounds**
  - **designed with an eye on low-resource hardware performance**
  - **HIGHT uses very simple operations, such as XORing, addition mod 28, and bitwise rotation**

Symmetric LWC (cont.)

- Scalable Encryption Algorithm (SEA)
  - **targeted for small embedded applications**
    - design considers a context with very limited processing resources and throughput requirements
  - **flexibility**
    - the plaintext size n, key size n, and processor (or word) size b are design parameters
- PRESENT
  - **ultra-lightweight block cipher algorithm based on a Substitution-Permutation Network (SPN)**
  - **designed to be extremely compact and efficient in hardware**
  - **operates on 64-bit blocks and with keys of either 80 or 128 bits**
  - **works in 31 rounds, each comprising three stages:**
    - key-mixing, through XOR operation and a rotation key schedule
    - substitution layer
    - permutation layer

Comparison of different symmetric-key cryptographic algorithms

	Cipher	Key size	Block size	Rounds	GE (hardware impl.)	Code size (software impl.)
SOFTWARE CIPHERS	AES	128	128	10	3400	2606
	TEA	128	64	32	3490	1140
	$SEA_{96,8}$	96	8	$\geq 3n/4$	3758 3925 2547	2132
HARDWARE CIPHERS	PRESENT	80	64	32	1570	936
	HIGHT	128	64	32	3048	5672

Public-Key Cryptography

- RSA (Rivest, Shamir, and Adleman)
  - **the most known and widely used public-key scheme**
  - based on exponentiation on finite fields
  - **unsuitable for adoption in constrained devices due to the need to operate on large numbers and the fact that long keys are required to achieve sufficient security**
  - **moreover, both key generation and encryption/decryption are demanding procedures that result in higher energy consumption**
- Elliptic Curve Cryptography (ECC)
  - **public-key cryptography based on the algebraic structure of elliptic curves over finite fields**
    - depends on point multiplication on elliptic curves
  - **may provide higher security and a better performance than the first generation public-key techniques (RSA and Diffie–Hellman)**

Energy consumption

- Energy cost of digital signature algorithms:

Algorithm	Key size ($bits$)	Key generation (mJ)	Sign (mJ)	Verify (mJ)
RSA	1,024	270.13	546.50	15.97
DSA	1,024	293.20	313.60	338.02
ECDSA	163	226.65	134.20	196.23
ECDSA	193	281.65	166.75	243.84
ECDSA	233	323.30	191.37	279.82
ECDSA	283	504.96	298.86	437.00
ECDSA	409	1034.92	611.40	895.98

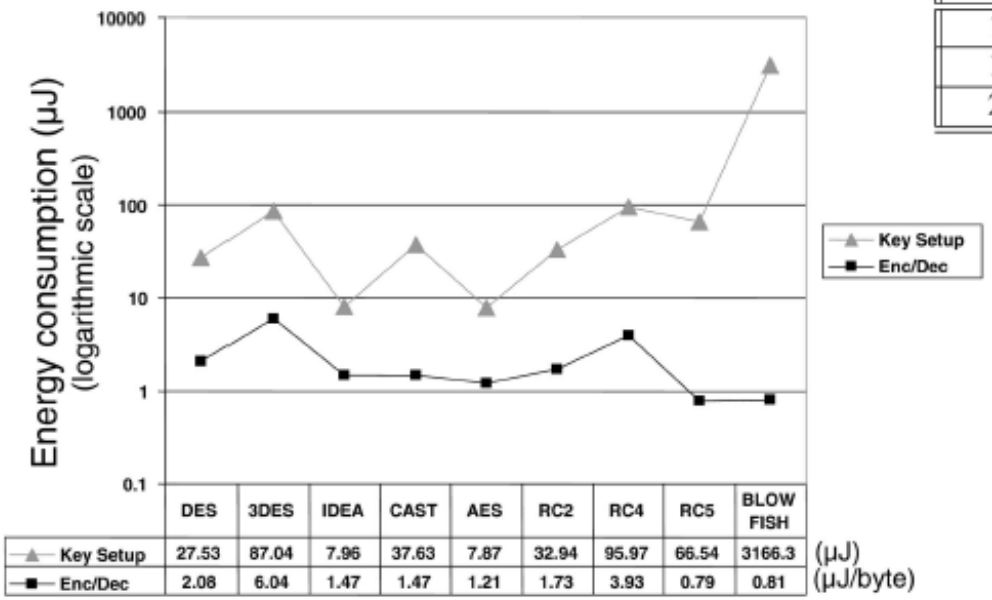
- Energy cost of key exchange algorithms:

Algorithm	Key size ($bits$)	Key generation (mJ)	Key exchange (mJ)
DH	1,024	875.96	1,046.5
ECDH	163	276.70	163.5
DH	512	202.56	159.6

[4] N. R. Potlapally, S. Ravi, A. Raghunathan, and N. K. Jha, "A Study of the Energy Consumption Characteristics of Cryptographic Algorithms and Security Protocols", IEEE Transactions on Mobile Computing, vol. 5, no. 2, February 2006

Energy consumption (cont.)

- Energy consumption for various symmetric ciphers:



	DES	3DES	IDEA	CAST	AES	RC2	RC4	RC5	BLOW FISH	
Key Setup	27.53	87.04	7.96	37.63	7.87	32.94	95.97	66.54	3166.3	(µJ)
Enc/Dec	2.08	6.04	1.47	1.47	1.21	1.73	3.93	0.79	0.81	(µJ/byte)

- Energy cost of AES variants:

Key size (bits)	Key setup (μJ)	ECB (μJ/B)	CBC (μJ/B)	CFB (μJ/B)	OFB (μJ/B)
128	7.83	1.21	1.62	1.91	1.62
192	7.87	1.42	2.08	2.30	1.83
256	9.92	1.64	2.29	2.31	2.05

[4] N. R. Potlapally, S. Ravi, A. Raghunathan, and N. K. Jha, "A Study of the Energy Consumption Characteristics of Cryptographic Algorithms and Security Protocols", IEEE Transactions on Mobile Computing, vol. 5, no. 2, February 2006

TLS energy consumption

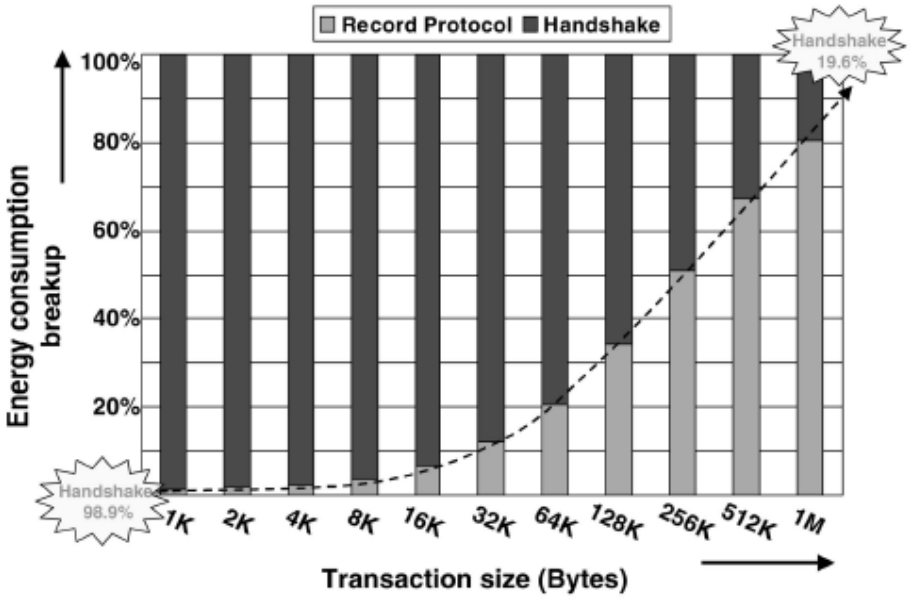
- Communication energy overhead in the client due to TLS security processing:

Transaction size (*bytes*)	Energy needed without SSL overhead (*uJ*)	Action	Data exchanged when using SSL (*bytes*)	Energy spent with SSL overhead (*uJ*)	Energy overhead of SSL usage (*uJ*)
1K	2,703.36	Tx Rx	236 2,748	8,047.68	5,344.32
10K	27,033.6	Tx Rx	236 12,092	32,715.84	5,682.24
100K	270,336.00	Tx Rx	236 105,660	279,735.36	9,399.36
1M	2,768,240.64	Tx Rx	236 1,066,620	2,816,669.76	48,429.12

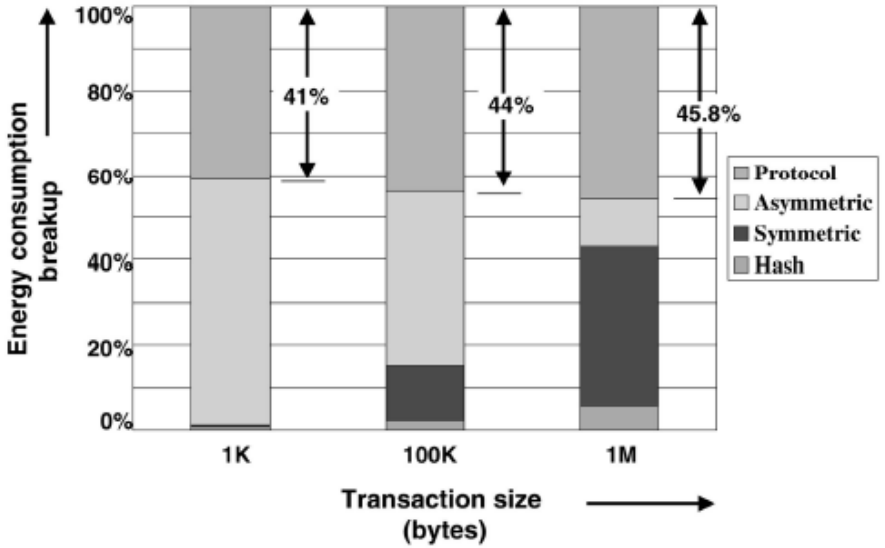
[4] N. R. Potlapally, S. Ravi, A. Raghunathan, and N. K. Jha, "A Study of the Energy Consumption Characteristics of Cryptographic Algorithms and Security Protocols", IEEE Transactions on Mobile Computing, vol. 5, no. 2, February 2006

TLS energy consumption (cont.)

- Handshake vs. RP energy consumption:



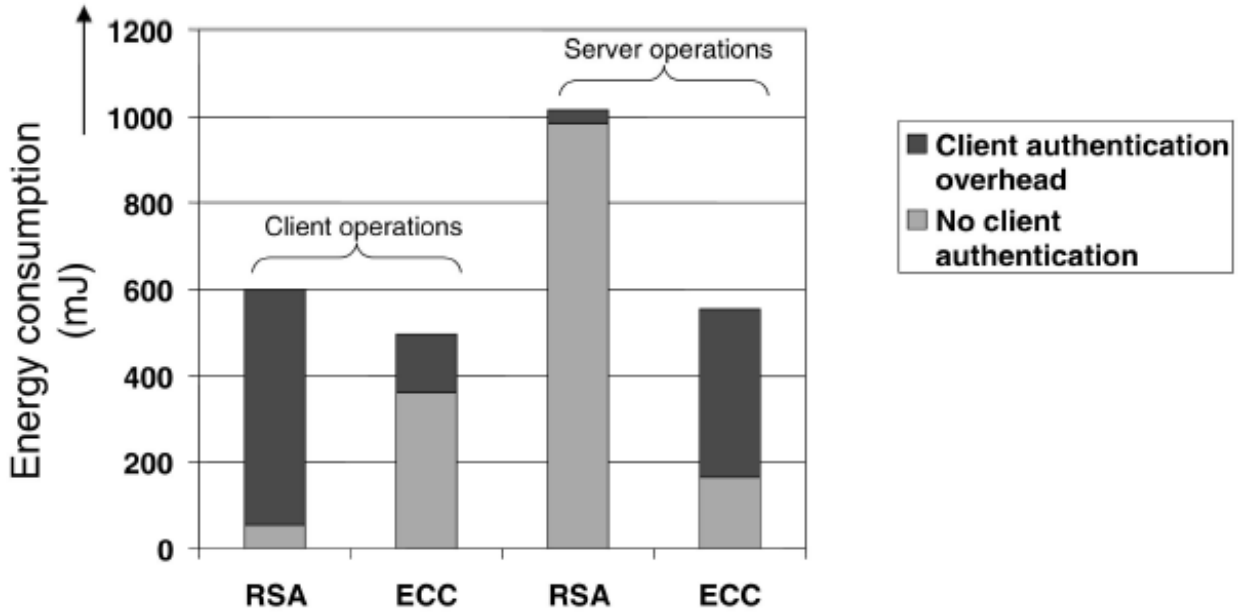
- TLS energy consumption into cryptographic and non-cryptographic components:



[4] N. R. Potlapally, S. Ravi, A. Raghunathan, and N. K. Jha, "A Study of the Energy Consumption Characteristics of Cryptographic Algorithms and Security Protocols", IEEE Transactions on Mobile Computing, vol. 5, no. 2, February 2006

TLS energy consumption (cont.)

- TLS Client and Server energy consumption:



[4] N. R. Potlapally, S. Ravi, A. Raghunathan, and N. K. Jha, "A Study of the Energy Consumption Characteristics of Cryptographic Algorithms and Security Protocols", IEEE Transactions on Mobile Computing, vol. 5, no. 2, February 2006

LWC

- Comparison of hardware implementation of symmetric ciphers:

Cipher	Key bits	Block bits	Cycles per block	Throughput at 100 kHz (Kbps)	Area (GEs)
Block ciphers					
Present	80	64	32	200.00	1,570
AES	128	128	1,032	12.40	3,400
Hight	128	64	34	188.20	3,048
Clefia	128	128	36	355.56	4,993
mCrypton	96	64	13	492.30	2,681
DES	56	64	144	44.40	2,309
DESXL	184	64	144	44.40	2,168
Stream ciphers					
Trivium[5]	80	1	1	100.00	2,599
Grain[5]	80	1	1	100.00	1,294

[5] T. Eisenbarth, S. Kumar, C. Paar, A. Poschmann, and L. Uhsadel, "A Survey of Lightweight-Cryptography Implementations", IEEE Design & Test of Computers, Vol.24, pp522--533, 2007

LWC (cont.)

- Comparison of software implementations of symmetric ciphers:

Cipher	Key size (bits)	Block size (bits)	Encryption (cycles/ block)	Throughput at 4 MHz (Kbps)	Decryption (cycles/ block)	Relative throughput (% of AES)	Code size (bytes)	SRAM size (bytes)	Relative code size (% of AES)
Hardware-oriented block ciphers									
DES	56	64	8,633	29.6	8,154	38.4	4,314	0	152.4
DESXL	184	64	8,531	30.4	7,961	39.4	3,192	0	112.8
Hight	128	64	2,964	80.3	2,964	104.2	5,672	0	200.4
Present	80	64	10,723	23.7	11,239	30.7	936	0	33.1
Software-oriented block ciphers									
AES	128	128	6,637	77.1	7,429	100.0	2,606	224	100.0
IDEA	128	64	2,700	94.8	15,393	123.0	596	0	21.1
TEA	128	64	6,271	40.8	6,299	53.0	1,140	0	40.3
SEA	96	96	9,654	39.7	9,654	51.5	2,132	0	75.3
Software-oriented stream ciphers									
Salsa20	128	512	18,400	111.3	NA	144.4	1,452	280	61.2
LEX	128	320	5,963	214.6	NA	287.3	1,598	304	67.2

[5] T. Eisenbarth, S. Kumar, C. Paar, A. Poschmann, and L. Uhsadel, "A Survey of Lightweight-Cryptography Implementations", IEEE Design & Test of Computers, Vol.24, pp522--533, 2007

Reference

- S. Cirani, G. Ferrari and L. Veltri, “Enforcing Security Mechanisms in the IP-Based Internet of Things: An Algorithmic Overview”, Algorithms Journal, 2013